book 5

java 27

a complete language manual

Raeliosol

version 1.0, compiled 10 2026

Contents

1. toolchain and the jdk 8
   1.1. the pin 8
   1.2. the whole toolchain 9
   1.3. run without ceremony 10
   1.4. the compile discipline 10
   1.5. the runner 11
   1.6. the tests, vendored once 11
   1.7. the checks that gate every commit 12
2. from java 8 to java 27 13
   2.1. java 8, the floor 13
   2.2. the module decade, 9 through 17 13
   2.3. the cadence era, 18 through 27 14
   2.4. the lts ladder 15
   2.5. withdrawn: string templates 16
   2.6. preview versus final 16
   2.7. the migration, worked 16
   2.8. the ladder, asserted 18
3. lexical structure 21
   3.1. identifiers 21
   3.2. keywords, 51 reserved and 17 contextual 22
   3.3. operators and precedence 23
   3.4. separators 23
   3.5. literals 23
   3.6. comments 25
   3.7. semicolons and blocks for c readers 25
4. types and values 27
   4.1. the eight primitives 27
   4.2. integer arithmetic wraps 27
   4.3. float and double at the bit level 28
   4.4. boolean, the type that converts to nothing 29
   4.5. char, the utf-16 code unit 29
   4.6. String, immutable and pooled 30
   4.7. boxing and the cache 30
   4.8. var, inference for locals 31
   4.9. records, the data shape 32
   4.10. sealed, the closed hierarchy 33
   4.11. null and Optional 33
   4.12. the conversion lattice 33
5. classes and objects 35
   5.1. the four member kinds 35
   5.2. construction and initialization order 35
   5.3. inheritance: overriding, hiding, dispatch 37
   5.4. interfaces 38
   5.5. records as the fixed data shape 39
   5.6. why contextual keywords matter 40
6. generics 42
   6.1. type parameters 42
   6.2. erasure and bridges 42

6.3. variance: arrays covariant, generics invariant ... 44
6.4. bounds ... 45
6.5. heap pollution and `@SafeVarargs` ... 46
6.6. generics, records, and patterns ... 47
7. enums, annotations, and nested types ... 48
7.1. an enum is a class ... 48
7.2. annotations ... 50
7.3. nested types ... 51
8. functional java ... 54
8.1. the function package ... 54
8.2. the lambda compile story ... 55
8.3. method references ... 55
8.4. capture and the effectively final rule ... 56
8.5. Optional as a return contract ... 57
8.6. the stream pipeline ... 59
9. pattern matching ... 61
9.1. instanceof patterns ... 61
9.2. record patterns ... 61
9.3. switch patterns and exhaustiveness ... 62
9.4. guards, null, dominance ... 63
9.5. unnamed variables and patterns ... 64
9.6. what is still preview ... 65
10. concurrency: threads to virtual threads ... 66
10.1. the platform thread model ... 66
10.2. synchronized and the monitor story ... 66
10.3. volatile and happens-before ... 67
10.4. executors and CompletableFuture ... 68
10.5. ThreadLocal and its costs ... 69
10.6. virtual threads ... 69
10.7. structured concurrency, the shape of the future ... 71
10.8. scoped values ... 71
11. memory and runtime ... 73
11.1. the heap and generations ... 73
11.2. the collectors ... 73
11.3. compact object headers ... 73
11.4. finalization is dead ... 74
11.5. java flight recorder ... 75
11.6. the jit at a walk level ... 75
11.7. the measurement discipline ... 76
12. collections and streams ... 78
12.1. the hierarchy ... 78
12.2. sequenced collections ... 78
12.3. factories since 9 ... 79
12.4. the stream api ... 80
12.5. stream gatherers ... 81
12.6. the swing from imperative ... 82
13. text, numbers, and time ... 84
13.1. String as the modern api ... 84
13.2. text blocks ... 85
13.3. BigInteger and BigDecimal ... 86
13.4. java.time ... 88

14. io and networking 91
14.1. streams and try-with-resources 91
14.2. path and files 91
14.3. channels and buffers 92
14.4. tcp sockets 93
14.5. the http client 94
14.6. websockets, client and peer 94
15. modules and jlink 98
15.1. the classpath world and the unnamed module 98
15.2. the descriptor 98
15.3. services 99
15.4. jdeps and jlink 100
15.5. module import declarations 100
16. reflection and method handles 102
16.1. class objects 102
16.2. invoking and constructing 102
16.3. annotations at runtime 103
16.4. records and the reflective contract 103
16.5. final means final 103
16.6. dynamic proxies 104
16.7. method handles 105
17. security and cryptography 108
17.1. digests and macs 108
17.2. secure random 108
17.3. key derivation 108
17.4. aes-gcm 110
17.5. tls end to end 110
17.6. the security manager’s exit 112
18. testing and idioms 113
18.1. the ok contract 113
18.2. asserts and tables by hand 113
18.3. interleavings and contention 114
18.4. the junit lane 116
18.5. java 27 idioms, the wrap-up 117
19. http kernel 119
19.1. the server and the exchange 119
19.2. the route table 120
19.3. the error envelope 122
19.4. json by hand 124
19.5. the decode ladder 126
19.6. the skeleton and its lifecycle 127
19.7. testing through a real socket 129
20. middleware 131
20.1. filters are the chain 131
20.2. request id 131
20.3. recover, the panic lane 132
20.4. the access log, no wrapper needed 133
20.5. timing on the unwind 133
20.6. wiring order and why 134
20.7. testing the stack 135
21. the user resource 137

21.1. the representation ... 137
21.2. create, validation, and the 422 versus 409 boundary ... 138
21.3. the store seam ... 139
21.4. the password seam ... 140
21.5. keyset pagination ... 141
21.6. get and the etag ... 142
21.7. handler tests over http ... 143
22. authn ... 145
22.1. the token, three segments and one primitive ... 145
22.2. verification, order and constant time ... 146
22.3. login, prove then issue ... 147
22.4. the filter, and the routing question it must ask first ... 148
22.5. the wiring ... 151
22.6. the tests ... 151
23. authz and rbac ... 153
23.1. authentication versus authorization ... 153
23.2. policies as functions ... 153
23.3. object-level authorization ... 154
23.4. roles as stored data ... 155
23.5. the table and deny by default ... 157
23.6. the wiring ... 158
23.7. the rbac matrix ... 159
24. store ... 161
24.1. the frame ... 161
24.2. replay, the recovery story ... 162
24.3. the transaction, append then mutate ... 164
24.4. the read view ... 166
24.5. compaction ... 166
24.6. the user store over the engine ... 168
24.7. the wiring and the reboot proof ... 169
25. concurrency control ... 170
25.1. thread-per-request removes a cap that was doing work ... 170
25.2. one hundred thousand threads, measured ... 170
25.3. executor sizing, the question that came back ... 171
25.4. admission control ... 171
25.5. where the pair sits in the stack ... 173
25.6. pinning, one paragraph ... 173
25.7. structured task scope, waiting to land ... 173
25.8. compare-and-swap, the promise chapter 21 made ... 174
26. caching ... 180
26.1. ttl from first principles ... 180
26.2. bounded: the eviction decision ... 180
26.3. reads without locks ... 181
26.4. the sliding window, the same table serving reuse ... 181
26.5. single flight ... 182
26.6. measured: the stampede and the hit rate ... 185
26.7. invalidation on write ... 185
26.8. the etag interplay ... 185
26.9. what not to cache ... 185
27. rate limiting ... 187
27.1. why limits exist ... 187

27.2. the token bucket from first principles ... 187
27.3. the 429 contract ... 188
27.4. what to key on ... 189
27.5. burst and the sustained rate ... 189
27.6. bounded per-key state ... 189
27.7. the admission contrast ... 190
27.8. limits under test ... 190
28. observability ... 192
28.1. the two series ... 192
28.2. the text exposition ... 193
28.3. the ops port ... 194
28.4. the filter at the edge ... 195
28.5. the custom jfr event ... 195
28.6. p99 done honestly ... 197
28.7. one request fully observed ... 198
29. load and p99 ... 200
29.1. the two loops ... 200
29.2. tested without sleeping ... 201
29.3. the report ... 202
29.4. the ladder, measured ... 204
29.5. the cliff ... 205
29.6. the production budget under load ... 205
29.7. what the run leaves behind ... 206
30. ship it ... 207
30.1. what a java service ships as ... 207
30.2. the image, two stages ... 207
30.3. compose, one command ... 209
30.4. the docker lane as a gate ... 210
30.5. the shipped checklist ... 211
31. the test suite ... 213
31.1. what the suite holds ... 213
31.2. the one lane gate ... 213
31.3. junit without a build tool ... 214
31.4. the deterministic seams ... 214
31.5. the adversarial history ... 215
31.6. what the suite does not hold ... 216
31.7. what the suite proves ... 216
32. capstone: line clone chat with mini apps ... 217
32.1. the shape in thirty seconds ... 217
32.2. the wire we own ... 218
32.3. the frame codec’s edge table ... 221
32.4. the connection, a reader and a writer ... 222
32.5. the hub ... 224
32.6. message history ... 228
32.7. the mini app runtime ... 228
32.8. the four mini apps ... 229
32.9. the react frontend ... 232
32.10. one process, two protocols ... 233
32.11. measured, and why ... 235
32.12. accepted design, stated ... 235
32.13. what the capstone proves ... 236

33. appendices: coverage and sources 238
33.1. keyword coverage 238
33.2. stdlib coverage 240
33.3. pinned sources 241

1. toolchain and the jdk

Java arrives as one download, the jdk, and everything this book does runs on the six programs its `bin` directory puts on the path: `javac`, `java`, `jshell`, `jar`, `jlink`, and `jdeps`. There is no second package manager, no project file format, no build daemon. The book pins oracle jdk 27, the ga build `27+35-2325` released 2026-09-15, and every sample lives under `books/java/samples/src` in the default package, one public class per file, compiled and run by that jdk alone.

27 is a feature release, not an lts one. The current lts is 25 (ga 2025-09-16), 21 before it, and oracle intends future lts releases every two years, so 29 lands september 2027. Chapter 2 walks the whole ladder from java 8 to here. This chapter owns the toolchain that the ladder's top rung hands you.

1.1. the pin

The machine this repo builds on has jdk 26.0.2 installed, so the book does not trust the machine. `tools/build-jdk27.sh` fetches the oracle archive zip, gates it on a sha256, and unpacks it into `tools/jdk27/build/jdk-27`:

```
url="https://download.oracle.com/java/27/archive/jdk-27_windows-x64_bin.zip"
sha=c75756490b95f44db07a21d4491b712399107d50fc041c2ecf0a6848662bfbd7
unzip -q -o jdk-27.zip -d tools/jdk27/build
```

fragment, illustrative, not executed

A fresh clone is red until that script runs, by design and loudly: the sample runner prints the missing path and the command that builds it rather than silently falling back to whatever `java` happens to resolve to. Pinned bytes or no build is the repo's law for vendored tooling, the same stance book 4, ch 1 (go 1.27, toolchain and modules) takes when it refuses to compile samples against anything but the module's declared version.

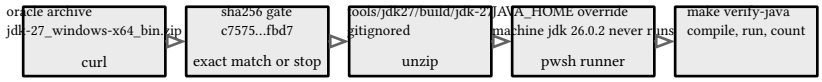


figure 1: the hermetic chain, from oracle archive to make verify-java

The sample that states the pin's identity is 33 lines and answers every version question a chapter might raise. Its checks ran green before this paragraph was written:

```
  public static void main(String[] args) throws Exception {
    ok(System.getProperty("java.version").equals("27"), "java.version is 27 exactly");
    ok(System.getProperty("java.vm.version", "").startsWith("27+35-2325"), "the ga build 27+35-2325");
    ok(System.getProperty("java.vm.name").contains("HotSpot"), "the hotspot vm");
    ok(System.getProperty("java.runtime.name").startsWith("Java(TM) SE"), "the oracle se runtime");
    ok(Runtime.version().feature() == 27, "runtime.version().feature() is 27");
    // the compact source file shape: this class needs no package, the
    // runner compiles each chapter directory as one default-package unit
    ok(Version.class.getPackageName().isEmpty(), "the default package");
    // virtual threads exist and run under the instance main's carrier
    try (var exec = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
      var tid = exec.submit(() -> Thread.currentThread().threadId()).get();
      var tid2 = exec.submit(() -> Thread.currentThread().isVirtual()).get();
      ok(tid > 0, "a virtual thread reports a thread id");
      ok(tid2, "the thread is virtual");
    }
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27");
  }
}
```

listing 1: the pinned jdk describes itself, eight machine checked facts (source: java/samples/src/Ch01/Version.java, lines 15-33)

`java.version` is 27 exactly, `java.vm.version` carries the build `27+35-2325`, the vm is hotspot, the runtime is oracle se, and `Runtime.version().feature()` reads 27, the same call the ladder chapter asserts on. The last two checks spawn work on a virtual thread executor and read the thread's id and `isVirtual`, which pins the concurrency story of chapter 10 to measured facts on day one.

1.2. the whole toolchain

Six programs, and all six answer `--version` with 27 on this build, measured 2026-10-04:

program	role	first seen
`javac`	compiles source to class files, the book's every build	1.0
`java`	runs class files, and since 11 launches `.java` source directly	1.0
`jshell`	read eval print loop, one expression at a time	9
`jar`	packages class files and resources into the archive format	1.0
`jlink`	assembles modules into a trimmed runtime image	9
`jdeps`	reads class dependencies, the input jlink works from	8

NOTE: no gradle, no maven, no dependencies

This book compiles everything with `javac` and runs everything with `java`, and no sample links a library outside the jdk. Two reasons. The platform under a build tool is still the platform: the class file model, the module path, and the launcher are what those tools wrap, so the book teaches the layer that survives fashion. And the service part of chapters 19 through 31 genuinely needs nothing more, the jdk ships an http server, a json tokenizer, a websocket client, and the cryptography the service part uses. When chapter 18 needs a test framework, it vendors one jar behind a sha256 and nothing else.

The module-aware half of that table earns its keep in chapter 15, but two measured facts belong here. The full jdk lists 66 modules, and an ordinary classpath program resolves 61 of them into its boot layer. `java.se`, the aggregate module covering the whole specification, is not among the resolved: aggregation is opt in, `jshell --add-modules` or a jlink image asks for it explicitly, and jlink in 27 refuses to guess, it errors on a missing `--add-modules` rather than defaulting the image to `java.se`.

1.3. run without ceremony

Since java 11 the launcher compiles for you. `java Hello.java` reads the source, compiles in memory, and runs, no `javac` step and no class file left behind. Java 22 widened it to multi file programs, JEP 458: the launcher follows source imports from the entry file and compiles the graph. The book's runner still uses `javac` explicitly, because the `-Xlint:all -Werror` discipline below is a compile time contract, but for a single file nothing beats it:

```
$ java HttpV.java
[HTTP_1_1, HTTP_2, HTTP_3]
```

fragment, illustrative, not executed

Java 25 removed the last of the boilerplate, JEP 512's compact source files. A source file may declare no class at all, the launcher wraps the statements in an implicit one, `main` becomes an instance method with whatever parameter list it wants, and a simple `IO` class handles console output. The pinned build runs this verbatim, measured 2026-10-04:

```
void main() {
  IO.println("implicit class, instance main, jdk " + Runtime.version().feature());
}
```

fragment, illustrative, not executed

The file can be named anything, `greet.java` lower case and mismatched is fine, because a compact source file is not declaring a public type the launcher has to find by name. This is the shape of every quick probe in this book: the chapter states a claim, a compact source file or a jshell session checks it, and the claim carries the date. The `Version` sample above stays in the classical shape, one public class matching its filename, because the sample suite compiles whole directories with `javac` and that contract wants declared types.

jshell is the other probe surface, an instantaneous check for a one line question:

```
$ jshell
|  Welcome to JShell -- Version 27
jshell> java.util.List.of(1, 2, 3).reversed()
$1 ==> [3, 2, 1]
```

fragment, illustrative, not executed

One measured wrinkle from this build: jshell's default execution runs through a remote agent and resolves everything this book needs, but `--execution local` failed to load nested enum classes like `HttpClient$Version` with a `ClassNotFoundException`. The flag looks like a performance win and is not one. The default agent is the mode the probes in this book ran under.

1.4. the compile discipline

Every sample in the book compiles under `javac -Xlint:all -Werror`. The first flag turns every lint on, raw types, unchecked calls, serialization, reflection on internals. The second turns warnings into errors, so the suite never ships with a yellow compile. Measured on this build 2026-10-04, a four line class using a raw `List` produces three warnings, and with `-Werror`:

```
error: warnings found and -Werror specified
1 error
3 warnings
```

fragment, illustrative, not executed

Exit code 1, no class file. The same source without `-Werror` compiles clean with three warnings scrolling past, which is exactly the failure mode the flag exists to prevent: warning fatigue normalizes the noise until the real one is missed. Chapters justify the rare suppression inline with a scoped `@SuppressWarnings` and a comment, never a file wide silence.

1.5. the runner

`tools/run-java-samples.ps1` is the whole harness. It resolves the pinned jdk, exports `JAVA_HOME` over it, walks every `ChNN` directory under `books/java/samples/src`, compiles each directory as one unit under the lint discipline, runs every class with a `main`, and counts `ok N name` lines:

```
static int checks = 0;

static void ok(boolean condition, String name) {
  checks++;
  if (!condition) {
    System.err.println("FAIL " + name + " (check " + checks + ")");
    System.exit(1);
  }
  System.out.println("ok " + checks + " " + name);
}
```

listing 2: the ok helper, the corpus contract every sample embeds (source: java/samples/src/Ch01/Version.java, lines 4-13)

A check that fails prints `FAIL` to stderr and exits 1, which the runner turns red. A class that prints no `ok` lines is equally red: silence is not success. The count is the book's unit of verification, the same contract the c and python sample runners use, so `Ch01/Version 8 checks` and `Ch02/Ladder 19 checks` are comparable statements. `pwsh tools/run-java-samples.ps1 -Chapter ChNN` scopes a run to one chapter while iterating, and `make verify-java` runs the whole lane inside `make verify`.

1.6. the tests, vendored once

The sample suite proves the platform claims. The test chapter, 18, needs a real framework, and its discipline belongs here because it is a toolchain decision: the book vendors `junit-platform-console-standalone 6.1.3`, one jar, sha256 `e62b96ac475dbcde8599ea905d088f65d90778f86e259b856a49fa5c4ea256ec`, behind the same gate the jdk zip sits behind. The standalone jar is subcommand shaped, 6.x dropped the bare flag style of the 1.x era:

```
java -jar junit-platform-console-standalone-6.1.3.jar \
  execute --select-class books.java.suite.Ch18Tests
exit 0: green, 1: failing tests, 2: the run never happened
```

fragment, illustrative, not executed

Test classes take the `*Tests` suffix so `--select-class` never points at a helper, and the exit codes are the whole ci contract: 0 green, 1 red, 2 for a discovery or usage failure that produced no verdict at all. Chapter 18 builds the suite. Nothing before it needs the jar.

1.7. the checks that gate every commit

A chapter is prose over evidence, and the repo gates the evidence. Every listing in this book is a live slice of a real sample file, read at compile time, so the pdf cannot drift from the code it quotes: `check-listings` hashes each pinned line range into `pins.lock.json` and fails when a source edit moves a slice. `check-headings` fails a chapter with two level 1 titles, `check-figures` rasterizes every figure page and fails ink in the guard band, and the harmony floor holds the book to the corpus word count. All of it rides in one target:

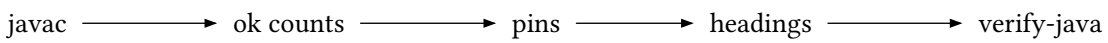


figure 2: the commit gate: pinned jdk, lint discipline, ok counts, listing pins, heading and figure checks, one pipeline into make verify

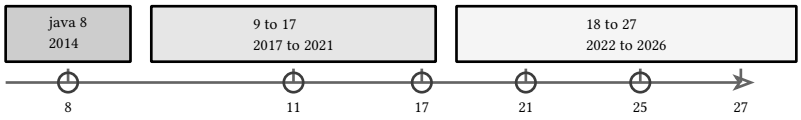
VERIFY: the claim discipline for this book

Every chapter ends with a sources line naming the pages it was written against and the date each was fetched. Behavior claims carry a measurement: which sample check or which jshell probe answered the question on the pinned build. When something could not be verified, the chapter says so instead of guessing.

sources: openjdk.org jdk 27 project page, ga 2026-09-15 and the jep list, accessed 2026-10-04. oracle java se support roadmap, lts cadence and the 29 in september 2027 note, accessed 2026-10-04. Toolchain behavior verified live on `tools/jdk27/build/jdk-27` measured 2026-10-04: the Version sample's 8 checks under `pwsh tools/run-java-samples.ps1 -Chapter Ch01`, `javac java jshell`
`jar jlink jdeps --version` all printing 27, `java greet.java` running the compact source form above, `java HttpV.java` printing the HttpClient version enum, `java -XX:+PrintFlagsFinal -version` for the flag table, the raw type class failing under `-Werror`, jshell resolving `List.of(1, 2, 3).reversed()` under the default execution and failing `HttpClient$Version` under `--execution local`, `java --list-modules` counting 66, and the classpath boot layer counting 61. The junit console jar's subcommand shape and exit codes are the wave's measured facts, recorded 2026-10-04 for chapter 18.

2. from java 8 to java 27

Every feature in this book carries the release that shipped it: “var infers since 10”, “records arrived final in 16”, “virtual threads went final in 21”. This chapter is where those numbers come from. Twenty releases run from the java 8 floor most production fleets still stand on to the 27 this book pins, and the road divides cleanly into three eras: java 8 alone, the module decade of 9 through 17, and the cadence era of 18 through 27 where a release lands every march and september and only some are lts.



ringed: lts. 8 declared retroactively, 29 rings in september 2027

figure 3: twenty releases, three eras, five of them lts with 29 pending

The chapter reads top down: the eras, one row per release, then the lts story, the one withdrawal, the preview machinery, and a worked migration that carries one domain from 2007 idioms to a form no earlier release compiles.

2.1. java 8, the floor

Java 8 (march 2014) rewrote the working style of the language in one release: lambda expressions, method references, and the stream pipeline over the collections, `Optional` as the explicit maybe, the `java.time` date and time api that succeeded joda time, default methods that let interfaces grow, and `CompletableFuture` for composable async. Nothing before it matters to a reader starting today except as history, and a great deal of the world still runs on it: the module transition was expensive enough that 8 was declared lts after the fact, and shops that skipped 9 through 17 built a decade of muscle memory on 8 lambdas and 8 streams.

That floor is why this book starts its ladder there rather than at 1.0. The 8 programmer already knows the lambda arrow and the stream pipeline. Everything after is: better type inference (10), better data carriers (16), closed hierarchies (17), pattern matching (16 through 21), and a concurrency model that finally matches the promise (19 through 21).

2.2. the module decade, 9 through 17

Java 9 arrived 3.5 years after 8 with project jigsaw, the platform module system that reorganized the jdk itself into modules, put every class behind a named module with exports, and encapsulated the internal sun APIs frameworks had reached into for years. It also brought `jshell` and the collection factory methods, made G1 the default collector, and replaced the version string scheme. The adoption pain was real, so the era’s story is the industry parking on 11 while the language work continued underneath: the Amber features (`var`, switch expressions, text blocks, records, sealed types, pattern matching) each previewed in an interim release and finalized in time for an lts landing.

release	language	platform	remembered for
9, 2017-09	module declarations, jigsaw syntax, the ten contextual keywords (`module`, `open`, `requires`, `transitive`, `exports`, `opens`, `to`, `uses`, `provides`, `with`)	JPMS, jshell, collection factories, G1 default, compact strings	the jdk restructured, internal apis closed
10, 2018-03	`var` for locals	time-based versioning, parallel full GC in G1, root certificates	first six month release, `var`
11 lts, 2018-09	`var` in lambda parameters	HttpClient standard, single-file launch, Java EE and	the lts the 9 skippers waited for

release	language	platform	remembered for
		CORBA modules removed, TLS 1.3, Flight Recorder, ZGC experimental	
12, 2019-03	switch expressions preview	Shenandoah experimental, microbenchmark suite, default CDS archives	Amber's first preview
13, 2019-09	text blocks preview, switch second preview	ZGC uncommits memory, legacy socket api reimplemented	text blocks arrive
14, 2020-03	switch expressions final, records preview, instanceof patterns preview	helpful NullPointerExceptions, CMS removed	NPEs that name the variable
15, 2020-09	text blocks final, sealed classes preview	ZGC and Shenandoah production, hidden classes, Nashorn removed	the low pause collectors go production
16, 2021-03	records final, instanceof patterns final	strong encapsulation by default, vector api incubator, development moves to github	records day
17 lts, 2021-09	sealed classes final, switch patterns preview	foreign function and memory incubator, macOS aarch64, security manager deprecated	the lts that emptied java 8

The era's grammar: a feature previews in one or two interim releases, finalizes in the next lts or the one after, and the lts rows above are where the industry actually met it.

2.3. the cadence era, 18 through 27

After 17 the cadence settled into culture. Every release ships in march or september, previews graduate on their own clock, and from 17 onward an lts lands every two years: 21 in 2023, 25 in 2025, 29 planned for september 2027. The language work converged on pattern matching and data oriented programming while Loom's concurrency went preview in 19, second preview in 20, final in
21. Panama's foreign function and memory api took longer, incubating

from 14 through 17, previewing from 19, final in 22.

release	language	platform	remembered for
18, 2022-03	switch patterns second preview	UTF-8 the default charset, simple web server, finalization deprecated	the charset default flips
19, 2022-09	virtual threads preview, record patterns preview, switch patterns third preview	structured concurrency incubator, RISC-V port	Loom's debut
20, 2023-03	second previews: record patterns, switch patterns, virtual threads	scoped values incubator, all seven JEPs preview or incubator	the all preview release
21 lts, 2023-09	record patterns final, switch patterns final, string templates preview	virtual threads final, sequenced collections, generational ZGC, key encapsulation api	virtual threads for everyone

release	language	platform	remembered for
22, 2024-03	unnamed variables final, statements before `super()` preview	foreign function and memory final, multi-file source launch, stream gatherers preview	the underscore returns as a discard
23, 2024-09	markdown doc comments, module import declarations preview, implicit classes third preview	ZGC generational by default, sun.misc.Unsafe memory access deprecated	markdown javadoc, string templates withdrawn
24, 2025-03	primitive patterns second preview, flexible constructor bodies third preview	stream gatherers final, class-file api final, compact object headers experimental, virtual threads stop pinning, security manager disabled, ML-KEM and ML-DSA	synchronized virtual threads
25 lts, 2025-09	compact source files final, module imports final, flexible constructor bodies final, scoped values final	compact object headers opt-in, AOT class caching and method profiles, generational Shenandoah	the simple java lts
26, 2026-03	structured concurrency sixth preview, lazy constants second preview	HTTP/3 in the client, prepare to make final mean final, applet api removed, AOT object caching with any GC	HTTP/3, final hygiene
27, 2026-09	structured concurrency seventh preview, primitive patterns fifth preview, lazy constants third preview	G1 the default in all environments, post-quantum TLS by default, compact object headers by default, JFR data redaction	the footprint and crypto release, this book's pin

Chapter 1 measured this build against the 27 row: the HttpClient version enum reads `HTTP_1_1, HTTP_2, HTTP_3`, and `java -XX:+PrintFlagsFinal -version` shows `UseCompactObjectHeaders = true` as a default, the flag 24 introduced experimentally and 25 made opt-in.

2.4. the lts ladder

The lts badge is a vendor promise, not a language property: openjdk ships every release, and the vendors choose which ones they will patch for years. In practice the line is unambiguous. Java 8 was declared lts retroactively when the module transition proved expensive, 11 was the first planned one on the three year cadence, and from 17 onward oracle has committed to a new lts every two years, so 21 and 25 followed at two year intervals and 29 comes in september 2027.

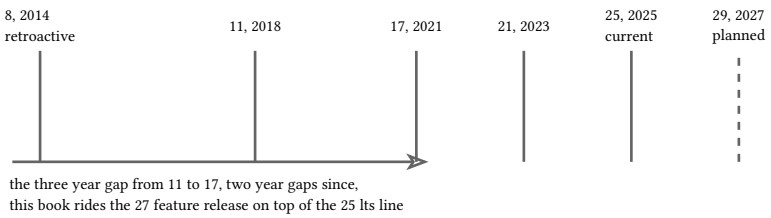


figure 4: the lts ladder, years on the x axis, one rung per supported line

This book rides a feature release, 27, rather than the current lts, 25. The two differ by the table's last two rows, and chapter 1's pin makes the choice safe: the samples prove on the exact ga build every claim the chapters make. A team building this book's service part for production deployment would run the same code on 25 with two rows of preview features subtracted, nothing the service part uses.

2.5. withdrawn: string templates

The cadence has one full withdrawal on its record. String templates, the `STR."hello \{name}"` interpolation form, previewed in 21 as JEP 430 and re-previewed in 22 as JEP 459. The third preview, JEP 465, was pulled before 23 shipped: the design's template processor model and the `\{}` embedding did not sit right after two rounds of real use, and the Amber group chose withdrawal over iteration. Java 23 contains no string templates, and code that used the preview had to be rewritten. Interpolation may return in another shape, no JEP carries it today.

NOTE: what withdrawal teaches

A preview feature is a live experiment on real codebases, and the contract runs both ways. Users accept churn, and in exchange the platform accepts the possibility of saying: this design was wrong, it is gone. String templates are the one time since the cadence began that the platform spent that option on a language feature.

2.6. preview versus final

The ladder tables above say "preview" often enough that the word needs precise meaning. A preview language feature compiles and runs only with `--enable-preview` on both `javac` and `java`, and the class files it produces are stamped for that exact release: 21's preview class files do not run on 22, the code must recompile against each new release. An incubator api is one step earlier, a module named `jdk.incubator.something` that the classpath does not see until `--add-modules` asks for it, and incubator apis may change shape outright between releases. The foreign memory api lived that life for eight releases before finalizing as `java.lang.foreign` in 22.

This book's marking follows the platform's: a feature is taught as working code only when final, previews appear in prose and in the tables with their preview release named, and the sample suite compiles without `--enable-preview` entirely. The one nuance worth knowing: "final" acquired teeth in 26, when JEP 500 began making final mean final, closing the reflective writes to final fields that frameworks had abused for years.

2.7. the migration, worked

One domain, weather readings off three stations, carried from the idioms a 2007 codebase would use to the 27 form, each step naming its release. The sample asserts the whole way down that the forms agree.

The 2007 form hand rolls the function types guava shipped and the loops everyone wrote, over a data carrier class with fields, a constructor, getters, and no equals:

```
// the pre-record data carrier: fields, constructor, getters, and the
// equals and hashCode nobody wrote, so == and contains lied for years
static final class LegacyReading {
  final String station;
  final double fahrenheit;
  LegacyReading(String station, double fahrenheit) {
    this.station = station;
    this.fahrenheit = fahrenheit;
  }
  String getStation() { return station; }
  double getFahrenheit() { return fahrenheit; }
}

// the guava-era shim every pre-lambda codebase carried, one interface
// per arity, instantiated through anonymous inner classes
interface Fn<I, O> {
  O apply(I input);
}
```

listing 3: the pre-lambda world, a 10 line carrier plus one interface per arity (source: java/samples/src/Ch02/Migration.java, lines 17-34)

```
// the java 7 form: two anonymous classes, an explicit accumulator loop,
// an anonymous comparator, getters down every line
static List<String> warmLegacy(List<LegacyReading> readings) {
  Fn<LegacyReading, Boolean> warm = new Fn<>() {
    public Boolean apply(LegacyReading r) {
      return r.getFahrenheit() ≥ 68;
    }
  };
  Fn<LegacyReading, String> render = new Fn<>() {
    public String apply(LegacyReading r) {
      long c = Math.round((r.getFahrenheit() - 32) * 5 / 9);
      return r.getStation() + " " + c + "C" + (r.getStation().equals("sapa") ? " summit" : "");
    }
  };
  List<String> out = new ArrayList<>();
  for (LegacyReading r : readings) {
    if (warm.apply(r)) {
      out.add(render.apply(r));
    }
  }
  out.sort(new Comparator<String>() {
    public int compare(String a, String b) {
      return a.compareTo(b);
    }
  });
  return out;
}
```

listing 4: anonymous classes at every use site, an explicit accumulator loop, an anonymous comparator (source: java/samples/src/Ch02/Migration.java, lines 46-72)

Lambda expressions (8) delete the shim and the anonymous class ceremony, and streams (8) move the loop into the pipeline. Then `var` (10) stops restating the type the right hand side already states. Records (16) collapse the carrier to a signature, with equals and hashCode generated, and `Stream.toList` (16) finishes the pipeline. Sealing (17) closes the hierarchy, which is what lets the pattern switch (21) be exhaustive with no default arm. Text blocks

(15) hold the expected output. The virtual thread executor (21) replaces the fixed pool the java 5 era would have sized by hand:

```
// the java 27 form: var (10), text blocks (15), records (16), streams
// with toList (16), sealed (17), pattern switch (21)
static List<String> warmModern(List<Reading> readings) {
  return readings.stream()
      .filter(r -> r.fahrenheit() ≥ 68)
      .map(r -> switch (r) {
        case Plain(var station, var f) ->
            station + " " + Math.round((f - 32) * 5 / 9) + "C";
        case Summit(var station, var f) ->
            station + " " + Math.round((f - 32) * 5 / 9) + "C summit";
      })
      .sorted()
      .toList();
}
```

listing 5: the 27 form, sealed plus records plus pattern switch, the whole pipeline (source: java/samples/src/Ch02/Migration.java, lines 74-87)

```
  var expected = """
      dalat 20C
      khoa 31C
      sapa 20C summit
      """;
  ok(String.join("\n", warmModern(modernData)).equals(expected.strip()),
      "the rendered lines match the text block (15)");
  ok(warmLegacy(legacyData).equals(warmModern(modernData)),
      "the 2007 form and the 27 form agree");
  // the concurrency leg of the same migration: a fixed pool was the java 5
  // answer, virtual threads (21) are the current one
  try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    var futures = modernData.stream()
        .map(r -> exec.submit(() -> warmModern(List.of(r))))
        .toList();
    var collected = new ArrayList<String>();
    for (var f : futures) {
      collected.addAll(f.get());
    }
    collected.sort(Comparator.naturalOrder());
    ok(collected.equals(warmModern(modernData)),
        "one virtual thread per reading renders the same lines (21)");
  }
```

listing 6: text block expectations, equivalence, and one virtual thread per reading (source: java/samples/src/Ch02/Migration.java, lines 98-120)

The equivalence check is the migration's contract: the 2007 form and the 27 form render the same three lines, and rendering them again spread over virtual threads still does. The 27 form is clearer because the domain is visible in it, and the collapse from 13 lines of carrier and shim to 6 lines of sealed hierarchy is the decade in one number.

2.8. the ladder, asserted

The ladder chapter closes by running itself. `Ladder` opens with `import module java.base` (25) and asserts one rung at a time, every check naming its release, 19 checks green on the pinned build 2026-10-04:

```
    ok(Runtime.version().feature() == 27, "the ladder's top rung is 27");
    // java 9: every class carries a module, the boot layer resolves them
    ok(String.class.getModule().isNamed()
        && String.class.getModule().getName().equals("java.base"),
        "classes know their module (9)");
    ok(ModuleLayer.boot().modules().size() == 61,
        "this build resolves 61 modules into the boot layer (9)");
    // java 9 collection factories plus java 10 var in one local
    var names = List.of("ada", "grace");
    ok(names.size() == 2, "List.of plus var (9 and 10)");
    // java 11: String.strip and the HttpClient that replaced HttpURLConnection
    ok(String.class.getMethod("strip") != null, "String.strip exists (11)");
    ok(Class.forName("java.net.http.HttpClient") != null,
        "java.net.http.HttpClient exists (11)");
    // java 14: switch expressions
    int len = switch ("b") {
      case "a" -> 1;
      case "b" -> 2;
      default -> 0;
    };
    ok(len == 2, "switch expressions yield values (14)");
    // java 15: text blocks
    String block = """
        {"user": "gabriel"}
        """;
    ok(block.strip().startsWith("{\"user\""), "text blocks hold json (15)");
```

listing 7: rungs 9 through 15, modules, factories, var, strip, HttpClient, switch expressions, text blocks (source: java/samples/src/Ch02/Ladder.java, lines 48-73)

```
    // java 16: records plus the instanceof pattern
    Object p = new Point(1, 2);
    ok(p instanceof Point pt && pt.x() == 1,
        "instanceof binds a pattern variable (16)");
    // java 17: sealed types report their permits
    ok(Shape.class.isSealed() && Shape.class.getPermittedSubclasses().length == 2,
        "sealed types know their permits (17)");
    // java 21: sequenced collections, virtual threads, pattern matching
    ok(List.of(1, 2, 3).reversed().equals(List.of(3, 2, 1)),
        "lists reverse (21)");
    ok(describe(new Circle(new Point(0, 0), 2)).startsWith("circle"),
        "record patterns destructure in switch (21)");
    var virtual = new boolean[1];
    Thread.ofVirtual().start(() -> virtual[0] = Thread.currentThread().isVirtual()).join();
    ok(virtual[0], "virtual threads run (21)");
    // java 22: unnamed variables, the final foreign function and memory api
    int counted = 0;
    for (var _ : names) {
      counted++;
    }
    ok(counted == 2, "the underscore discards (22)");
    ok(Class.forName("java.lang.foreign.MemorySegment") != null,
        "java.lang.foreign landed final (22)");
```

listing 8: rungs 16 through 22, records, sealed, the 21 cluster, the underscore, java.lang.foreign (source: java/samples/src/Ch02/Ladder.java, lines 74-96)

```
    // java 25: scoped values, flexible constructor bodies, module imports
    var seen = new String[1];
    ScopedValue.where(USER, "gabriel").run(() -> seen[0] = USER.get());
    ok(seen[0].equals("gabriel"), "scoped values bind (25)");
    ok(new Reading("F", 212).celsius == 100.0,
        "statements before this() compute first (25)");
    // the first line of this file imports module java.base, final since 25
    ok(ModuleLayer.boot().findModule("java.base").isPresent(),
        "import module java.base resolves (25)");
    // java 26: http/3 joined the client's version enum
    ok(Arrays.toString(java.net.http.HttpClient.Version.values()).contains("HTTP_3"),
        "the http client speaks http/3 (26)");
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27");
```

listing 9: rungs 25 through 27, scoped values, flexible constructors, module imports, HTTP/3 (source: java/samples/src/Ch02/Ladder.java, lines 97-109)

Two rungs are stated rather than asserted because they are launcher and flag facts, not classpath facts: compact source files with the instance main need `java file.java` not `javac`, which chapter 1 demonstrated on `greet.java`, and compact object headers are a VM flag default, `UseCompactObjectHeaders = true`, measured on this build via `-XX:+PrintFlagsFinal`. The 61 module boot layer and the 66 module jdk counts in chapter 1 are this chapter's module rung, restated.

sources: openjdk.org per release project pages, jdk9 through jdk27, feature lists and GA dates, all accessed 2026-10-04. oracle java se support roadmap via search summary, the lts every two years intent and the 29 in september 2027, accessed 2026-10-04. javaalmanac.io and the withdrawn JEP 465 page, string templates previewed 21, re-previewed 22, withdrawn before 23, accessed 2026-10-04. Java in a nutshell 8th edition, the brief history section (print pages 16 to 18) and the beyond java 17 appendix (print pages 431 to 434), read from ref/java via pdftotext, used for the era shape and rewritten here. The ladder sample and migration sample verified live under `pwsh tools/run-java-samples.ps1 -Chapter Ch02`, 22 checks, and the `UseCompactObjectHeaders` and `HttpClient.Version` facts measured on tools/jdk27/build/jdk-27, 2026-10-04.

3. lexical structure

Every java program is unicode text before it is anything else, and the lexer cuts that text into five token kinds: identifiers, keywords, literals, operators, and separators, with comments and whitespace dropped on the way. Chapter 2 walked the ladder release by release, and this chapter walks the surface it delivered: what a name may look like, which words the language has frozen, how every literal is spelled, the one precedence table worth memorizing, and the comment forms including the markdown doc comments of 23. The Ch03 samples run every claim through `javax.lang.model.SourceVersion`, the compiler's own token predicates, so the taxonomy below is measured on the pinned 27 rather than recited.

3.1. identifiers

Java has accepted unicode in source since 1, and an identifier is any length of unicode letters and digits with a letter in front, where letter and digit are decided by `Character.isJavaIdentifierStart` and `isJavaIdentifierPart`. Vietnamese, greek, and kanji spellings are legal, a digit cannot lead, and the dollar sign and other currency symbols are legal but reserved by convention for generated code, the compiler names its own artifacts with them. The underscore may appear inside a name but has not been a name on its own since 9.

```
    // identifiers: unicode since 1, currency symbols reserved for generators
    int año = 2026, 歲 = 42;
    String μέτρο = "meter", kawauso = "otter";
    ok(año == 2026 && 歲 == 42 && μέτροUnknown(μέτρο, kawauso),
      "identifiers draw on all of unicode, letters and digits beyond ascii");
    ok(Character.isJavaIdentifierStart('A') && Character.isJavaIdentifierStart('$')
      && !Character.isJavaIdentifierStart('5') && !Character.isJavaIdentifierStart('#'),
      "letters and currency symbols may start a name, digits and # may not");
    ok(Character.isJavaIdentifierPart('5') && !Character.isJavaIdentifierPart(' '),
      "digits may continue a name, a space may not");
```

listing 10: unicode identifiers, and the classifier the lexer actually uses (source: java/samples/src/Ch03/Tokens.java, lines 60-69)

Identifiers shadow the contextual keywords freely, which is the point of contextuality: code written before `var` or `record` meant anything still compiles. All 16 identifier-shaped contextual words can name locals in one scope, `var` still infers beside a local actually named `var`, and since 22 the underscore declares an unnamed variable that discards its initializer, the migration JEP 456 gave every catch and lambda parameter.

```
    // contextual keywords as working identifiers
    int var = 1, record = 2, yield = 3, sealed = 4, permits = 5, module = 6;
    int open = 7, requires = 8, exports = 9, opens = 10, provides = 11;
    int uses = 12, with = 13, to = 14, transitive = 15, when = 16;
    ok(var + record + yield + sealed + permits + module + open + requires
      + exports + opens + provides + uses + with + to + transitive + when == 136,
      "all 16 identifier-shaped contextual words name locals in one scope");
    var context = "var";
    ok(context.equals("var"), "var infers String since 10 even beside a local named var");

    // the unnamed variable since 22: declaration compiles, value discarded
    int _ = Integer.parseInt("42");
    ok(true, "int _ = 42 compiles on 27, an unnamed variable discarding its value");
```

listing 11: contextual words as working identifiers, and the unnamed variable (source: java/samples/src/Ch03/Tokens.java, lines 71-83)

One rule cuts the other way for type names: a class cannot be called `var`, `record`, `sealed`, `permits`, or `yield`, the `TypeIdentifier` production in the spec, and a method invocation of `yield` must be qualified so the parser can tell it from the statement.

3.2. keywords, 51 reserved and 17 contextual

The spec for 27 reserves 51 character sequences. The list froze at 51 when the underscore joined in 9: `enum` arrived with 5, `const` and `goto` have been reserved and unused since 1.0 as error-message insurance for c++ programmers, and `strictfp` survives as an obsolete word that 17 turned into a no-op when the platform went uniformly IEEE 754. Alongside the reserved list sit three literal tokens, `true`, `false`, and `null`, reserved like keywords but classified as literals, so the honest sentence is that 54 character sequences cannot name anything.

```
String[] reserved = {"abstract", "assert", "boolean", "break", "byte",
  "case", "catch", "char", "class", "const", "continue", "default",
  "do", "double", "else", "enum", "extends", "final", "finally",
  "float", "for", "goto", "if", "implements", "import", "instanceof",
  "int", "interface", "long", "native", "new", "package", "private",
  "protected", "public", "return", "short", "static", "strictfp",
  "super", "switch", "synchronized", "this", "throw", "throws",
  "transient", "try", "void", "volatile", "while", "_"};
int reservedCount = 0;
for (String word : reserved) {
  if (javax.lang.model.SourceVersion.isKeyword(word)) {
    reservedCount++;
  }
}
ok(reservedCount == 51,
  "the JLS se27 reserved list is 51 words and javac reserves all 51");
ok(javax.lang.model.SourceVersion.isKeyword("_"),
  "the underscore is a reserved keyword since 9, named variables since 22");
ok(!javax.lang.model.SourceVersion.isName("_"),
  "so _ alone is not a name, isName says false");
int contextualFalse = 0;
for (String word : new String[] {"var", "record", "sealed", "permits",
  "yield", "when", "module", "open", "requires", "exports", "opens",
  "provides", "uses", "with", "to", "transitive"}) {
  if (!javax.lang.model.SourceVersion.isKeyword(word)) {
    contextualFalse++;
  }
}
ok(contextualFalse == 16,
  "the 16 identifier-shaped contextual keywords pass isKeyword false, "
    + "the 17th, non-sealed, carries a hyphen and cannot be a name");
ok(javax.lang.model.SourceVersion.isKeyword("true")
  && !javax.lang.model.SourceVersion.isName("true"),
  "true, false, null are literals, reserved like keywords but not keywords");
```

listing 12: the reserved list counted through the compiler's own isKeyword (source: java/samples/src/Ch03/Tokens.java, lines 25-58)

Contextual keywords are the escape valve that kept the frozen list frozen. Each one is only a keyword where the grammar expects it, and the spec adds a guard worth knowing: the sequence is not a keyword when a letter or digit touches it, so `varfilename` is one identifier, never `var` followed by `filename`.

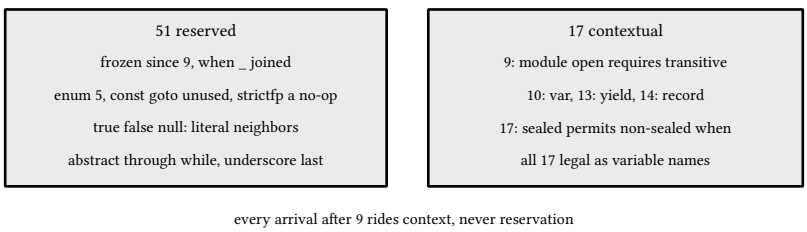


figure 5: the keyword taxonomy: 51 reserved and frozen, 17 contextual and dated

The dates on the right are the Amber story in miniature: the module system minted 10 words in 9, `var` inferred in 10, `yield` arrived with the 13 switch-expression preview and went final with the expressions in 14, `record` previewed in 14 and finalized in 16, and the sealed trio plus the `when` guard landed in 17, with `when` waiting for the 21 pattern switch to carry it final. Chapter 9 is where the sealed set and `when` earn their keep.

3.3. operators and precedence

Operators associate left to right except where the table says otherwise, and assignment is the broadest right-associative form. The levels, tightest first:

level	operators	notes
postfix	`expr++ expr--`, `.`, `[]`, method call	binds tightest, left to right
unary	`++expr --expr +expr -expr ~ !`	right to left, `~` flips bits, `!` flips booleans
multiplicative	`* / %`	left to right
additive	`+ -`	left to right, `+` also concatenates strings
shift	`<< >> >>>`	`>>>` is the logical shift, no sign extension
relational	`< > ≤ ≥ instanceof`	left to right
equality	`== ≠`	compares references on objects, values on primitives
bitwise and	`&`	unparenthesized `&` before `^` before `|`
bitwise xor	`^`	
bitwise or	`|`	
logical and	`&&`	short circuits
logical or	`||`	short circuits
ternary	`? :`	right to left, chapter 4 measures its var typing
assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`	right to left, an expression, not a statement
arrow	→	lambda and switch case labels, not an operator by precedence

The two rows c readers trip on are `>>>`, which java has and c lacks, and the fact that `&` and `|` on booleans do not short circuit, which is occasionally exactly what an order-independent side effect wants. Precedence answers ambiguity, but the working habit is parentheses: the corpus style adds them the moment a shift meets an additive operator.

3.4. separators

Twelve separator tokens: `( ) { } [ ] ; , . ... @ ::`. The ellipsis is varargs since 5, the double colon method reference since 8, and the at-sign annotates declarations. They carry no precedence, they only bracket. The semicolon and brace get their own grounding at the end of this chapter because java inherits c's statement discipline exactly.

3.5. literals

Integer literals come in four bases. Hexadecimal runs `0x` or `0X`, octal is a leading zero, and binary `0b` and the underscore digit separator both arrived with 7's project coin. Octal is legacy, kept because file permissions and pre-java history read naturally in it, and it is the one base that silently changes meaning when a padding zero lands in front of a decimal.

```
    // integer bases: decimal, hex, octal, binary (binary since 7)
    ok(31 == 0x1F && 31 == 037 && 31 == 0b11111,
      "0x1F, 037, and 0b11111 are the same 31 in three bases");
    ok(0644 == 420 && 0377 == 255,
      "legacy octal reads file permissions: 0644 is 420, 0377 is 255");

    // underscores between digits since 7
    ok(1_000_000 == 1000000 && 0xCAFE_BABE == 0xCAFEBABE,
      "underscores group digits since 7 and vanish at parse time");
    ok(0b1010_1010 == 170,
      "binary literals want grouping most, 0b1010_1010 is 170");

    // int by default, long with L, hex literals past int range need it too
    long fromHex = 0x7FFFFFFF; // the largest int, 2147483647
    ok(fromHex == 2147483647L && 0xFFFFFFFFL == 4294967295L,
      "literals are int by default, 0xFFFFFFFF is past the range and needs L");
```

listing 13: four bases, underscores since 7, and int by default with L for long (source: java/samples/src/Ch03/Literals.java, lines 18-33)

A decimal literal is an `int` unless it ends in `L` or `l`, and the uppercase form wins on readability because lowercase `l` reads as `1`. Floating literals default to `double`, take `f` or `F` for `float`, and scale with `e` or `E`. The exotic form is hexadecimal floating point, significand in hex with a `p` binary exponent, in the language since 5 and read back by `Double.toHexString` since the same release: `0x1.8p1` is one-and-a-half times two, exactly 3.0.

```
    // floating forms: suffix, exponent, and hexadecimal floats since 5
    float f = 1.5f;
    double d = 1.2345E02;
    ok(f == 1.5 && d == 123.45,
      "f makes a float, the default is double, e notation scales by 10");
    ok(0x1.8p1 == 3.0 && 0x1p-1 == 0.5,
      "hex floats since 5: 0x1.8p1 is 1.5 times 2, 0x1p-1 is a half");
```

listing 14: suffixes, exponents, and the hexadecimal float form (source: java/samples/src/Ch03/Literals.java, lines 35-41)

Char literals quote one utf-16 code unit and share their escapes with string literals. The five control escapes cover the ascii whitespace set, octal escapes run one to three digits capped at `\377`, and `\uXXXX` takes exactly four hex digits with any number of extra `u` characters allowed in front, a spec quirk the sample pins. The `\s` escape, an explicit space that survives text-block trailing-space stripping, came with 14′s second text block preview.

```
    // char escapes: the classic set, octal escapes to 377, \s since 15
    ok('\n' == 10 && '\t' == 9 && '\r' == 13 && '\b' == 8 && '\f' == 12,
      "the five control escapes: \\n \\t \\r \\b \\f");
    ok('\'' == 39 && '\"' == 34 && '\\' == 92,
      "quoting the delimiters: \\' \\\" \\\\");
    ok('\101' == 'A' && '\0' == 0 && '\77' == '?' && '\377' == 255,
      "octal escapes run 1 to 3 digits, capped at 377");
    ok('\101' == '\uu0041' && '\uu0041' == 'A',
      "octal, escaped, and plain forms agree, extra u's are legal before the 4 hex digits");
    ok('\s' == ' ',
      "\\s pins an explicit space since 15, the text block escape set");
```

listing 15: every escape family, octal to unicode, plus the space escape (source: java/samples/src/Ch03/Literals.java, lines 43-53)

String literals sit in double quotes on one source line. Text blocks, previewed in 13 and 14 then final in 15, span lines with `"""`, and at runtime they are ordinary `String` values, indented by the compiler's minimum-strip algorithm: chapter 13 pins that algorithm line by line, so this chapter only claims the lexical shape. One compile-time fact belongs here: literal concatenation is a constant expression, folded and pooled before the program runs.

```
    // string literals: same escapes, one source line
    String quoted = "a\"b\\c";
    ok(quoted.length() == 5 && quoted.indexOf('\t') < 0 && "x\ty".indexOf('\t') == 1,
      "string literals share the char escapes inside double quotes");

    // text blocks since 15: free quotes, real newlines, same String
    String block = """
        alpha
        beta""";
    ok("alpha\nbeta".equals(block),
      "a text block compiles to the same String, indent algorithm in ch13");

    // constant expressions fold and intern at compile time
    ok(("ja" + "va") == "java",
      "literal concatenation folds at compile time and interns");
```

listing 16: one-line strings, the text block shape, and compile-time folding (source: java/samples/src/Ch03/Literals.java, lines 55-69)

3.6. comments

Four forms. // runs to end of line, /* */ spans lines and nests nothing, /** */ is the javadoc doc comment, and since 23 the /// line form carries doc comments in markdown instead of html, JEP 467, the form this book's samples use. The compiler only strips them, and javadoc is the tool that reads the doc forms: chapter 1 set up the jdk toolchain those ride on.

One lexical trap hides in the preprocessing stage that runs before the lexer. Unicode escapes are translated into their characters first, so an escape inside a comment is code as far as the compiler is concerned:

```
// looks like a comment, ends the line:
// the escape below is a real newline \u000A int nowCode = 1;
```

fragment, illustrative, not executed

The `\u000A` becomes a line terminator before tokenization, the // comment ends, and the words after it must compile or javac rejects the file. Unicode escapes also cannot name a \u they have already consumed: the translation is one pass, earliest leftmost first.

3.7. semicolons and blocks for c readers

Java keeps c's punctuation habits whole. Semicolons terminate statements, braces group them and open a scope, whitespace between tokens means nothing, and there is no preprocessor: no `#define`, no `#include`, no textual substitution of any kind, the nearest equivalents being `static final` constants and the `import` that resolves names rather than pasting text. Statements that look like c statements read like c statements, and the one habit worth unlearning is the declaration-on-top style the old c compilers needed, since java declares at first use and `var` rewards it.

sources: docs.oracle.com/javase/specs/jls/se27/html/jls-3.html accessed 2026-10-04 for the 51 reserved words, the 17 contextual keywords and their recognition rule, true false null as literal classifications, const and goto reservation, and the obsolete strictfp note, openjdk.org/jeps/213 accessed 2026-10-04 for the underscore warning in 8 and removal in 9, openjdk.org/jeps/354 for yield in the 13 preview, openjdk.org/projects/jdk/14 and /jdk/15 accessed 2026-10-04 for records preview 14 and sealed preview 15 alongside the ladder-chapter spine (records final 16, sealed final 17, switch expressions final 14, text blocks final 15, markdown doc comments 23), JEP 456 for unnamed variables in 22, docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html accessed 2026-10-04 for underscores in numeric literals since se 7, docs.oracle.com/javase/7/docs/technotes/guides/language/binary-literals.html for

binary literals since 7, and docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Double.html for the since 1.5 marker on hexadecimal float notation. Grounding on Java in a Nutshell 8th edition chapter 2 (lexical structure, primitive data types) for the token inventory, the contextual keyword list at 17, and the separator set, rewritten here. Everything behavioral verified live on tools/jdk27/build/jdk-27 by the Ch03 samples under pwsh tools/run-java-samples.ps1 -Chapter Ch03: 26 checks, Tokens 11 and Literals 15, covering the 51 reserved words through SourceVersion, the 16 identifier-shaped contextual words as locals, the underscore as unnamed variable, unicode identifier classification, the four integer bases, underscores, suffixes, hexadecimal floats, every escape family, the text block shape, and compile-time literal folding, all dated 2026-10-04.

4. types and values

Java values come in exactly two kinds, and the split explains most of the language's daily friction. A primitive value is its own bits, one of eight fixed types. A reference value is a pointer the program can never dereference by hand, null or an object. Assignment and argument passing always copy the value, the bits or the reference, never the object contents, so java is pass by value with references among its values. This chapter walks the primitives and their arithmetic, `boolean` and `char` as the two odd members, the `String` and boxing story on the reference side, `var` inference, records and sealed hierarchies as the modern data shapes, null and `Optional`, and the conversion lattice that ties the numeric types together. Chapter 2 walked the ladder, and most of what follows arrived on its rungs: boxing in 5, `var` in 10, records final in 16, sealed in 17.

4.1. the eight primitives

type	kind	bits	range and default
`byte`	signed integer	8	−128 to 127, default 0
`short`	signed integer	16	−32768 to 32767, default 0
`int`	signed integer	32	−2147483648 to 2147483647, default 0
`long`	signed integer	64	−9223372036854775808 to 9223372036854775807, default 0
`float`	ieee 754 binary32	32	1.4E-45 to 3.4028235E38, default 0.0
`double`	ieee 754 binary64	64	4.9E-324 to 1.7976931348623157E308, default 0.0
`char`	unsigned utf-16 code unit	16	0 to 65535, default NUL
`boolean`	truth value	jvm-dependent	true or false, default false

Widths are fixed by the specification, not by the machine the code runs on, which is the portability promise c never made: a `long` is 64 bits everywhere and `Integer.MAX_VALUE` is the same constant on every vm. The two counts the sample pins first.

```
    // widths are fixed by the spec, not by the machine
    ok(Byte.SIZE == 8 && Short.SIZE == 16 && Character.SIZE == 16
      && Integer.SIZE == 32 && Long.SIZE == 64 && Float.SIZE == 32
      && Double.SIZE == 64, "seven numeric widths, 8 to 64 bits, everywhere");
    ok(Integer.MAX_VALUE == 2147483647 && Integer.MIN_VALUE == -2147483648
      && Long.MAX_VALUE == 9223372036854775807L,
      "the ranges: int to 2147483647, long to 9223372036854775807");
```

listing 17: widths through the SIZE constants, ranges through MIN and MAX (source: java/samples/src/Ch04/Primitives.java, lines 18-24)

4.2. integer arithmetic wraps

Integer overflow is not an error in java, it is the two's-complement wrap the hardware already does. The compiler says nothing, the runtime says nothing, and the one value with no positive counterpart, `Integer.MIN_VALUE`, negates to itself through the same wrap, which is why `Math.abs` can return a negative number.

```
    // integer overflow wraps in silence
    ok(Integer.MAX_VALUE + 1 == Integer.MIN_VALUE,
      "max plus 1 wraps to min, no flag, no trap: " + (Integer.MAX_VALUE + 1));
    ok(Math.abs(Integer.MIN_VALUE) == Integer.MIN_VALUE,
      "negating min wraps back to min, abs included");
    ok((byte) (127 + 1) == -128, "byte arithmetic promotes to int, the cast wraps to -128");
    try {
      Math.addExact(Integer.MAX_VALUE, 1);
      ok(false, "addExact should have thrown");
    } catch (ArithmeticException e) {
      ok("integer overflow".equals(e.getMessage()),
        "Math.addExact refuses instead: " + e.getMessage());
    }
    try {
      Math.multiplyExact(Integer.MAX_VALUE, 2);
      ok(false, "multiplyExact should have thrown");
    } catch (ArithmeticException e) {
      ok("integer overflow".equals(e.getMessage()), "multiplyExact too, since 8");
    }
```

listing 18: the wrap measured, and the exact family that refuses instead (source: java/samples/src/Ch04/Primitives.java, lines 26-44)

When silence is the bug, java 8 shipped the escape hatch: `addExact`, `subtractExact`, `multiplyExact`, and `incrementExact` check and throw `ArithmeticException` on overflow, the message a plain `integer overflow`. Anything that accumulates counts from the outside world, money, sequence ids, timestamps summed over a queue, belongs on the exact forms. Division has its own discipline: it truncates toward zero, `modulo` takes the dividend's sign, and integer division by zero is the one arithmetic error that always throws.

```
    // division truncates toward zero, modulo follows the dividend
    ok(7 / 2 == 3 && -7 / 2 == -3 && 7 % 2 == 1 && -7 % 2 == -1,
      "integer division truncates toward zero, -7/2 is -3 not -4");
    try {
      int zero = "x".length() - 1; // runtime 0, not a constant for the divzero lint
      int boom = 1 / zero;
      ok(false, "division by zero should have thrown");
    } catch (ArithmeticException e) {
      ok("/ by zero".equals(e.getMessage()), "integer division by zero throws: " + e.getMessage());
    }
```

listing 19: truncation toward zero on both signs, and the one throwing operation (source: java/samples/src/Ch04/Primitives.java, lines 46-55)

4.3. float and double at the bit level

Both floating types are ieee 754 with a fixed layout: one sign bit, 8 or 11 exponent bits, 23 or 52 stored mantissa bits, the biased exponents 127 and 1023 for 1.0. The representation is why 0.1 has no exact resident and why adding money in doubles corrupts it: book 2, ch 1 (mathematics for programmers, floating point anatomy) takes the format apart bit by bit and owns the depth, this chapter only needs the measured behavior and the special values.

```
    // float and double at the bit level: 1 sign, 8 or 11 exponent, 23 or 52 mantissa
    ok(Float.floatToIntBits(1.0f) == 0x3F800000,
      "1.0f is bits 0x3F800000, sign 0 exponent 127 mantissa 0");
    ok(Double.doubleToLongBits(1.0) == 0x3FF0000000000000L,
      "1.0 is bits 0x3FF0000000000000L, exponent 1023");
    ok(0.1 + 0.2 != 0.3,
      "binary floats cannot hold tenths: " + (0.1 + 0.2));
```

listing 20: the bit patterns of 1.0 in both widths, and the tenths that do not add (source: java/samples/src/Ch04/Primitives.java, lines 57-63)

Floating arithmetic never throws. Overflow runs to infinity, underflow runs to a signed zero that equals zero under == but keeps its sign through division, and the illegal operations produce NaN, the only value in the language unequal to itself, so NaN tests go through `Double.isNaN`. One naming trap: `Float.MIN_VALUE` is the smallest positive value, not the most negative, the opposite convention from `Integer.MIN_VALUE`.

```
    // the special values: two zeros, two infinities, one NaN
    ok(1.0 / 0.0 == Double.POSITIVE_INFINITY && -1.0 / 0.0 == Double.NEGATIVE_INFINITY,
      "overflow runs to infinity instead of throwing");
    ok(-0.0 == 0.0 && 1.0 / -0.0 == Double.NEGATIVE_INFINITY,
      "negative zero equals zero under == but the sign survives division");
    double nan = 0.0 / 0.0;
    ok(nan != nan && Double.isNaN(nan), "NaN is the only value not equal to itself");
    ok(Float.MIN_VALUE == 1.4E-45f && Float.MIN_VALUE > 0f,
      "Float.MIN_VALUE is the smallest positive, unlike Integer.MIN_VALUE");
```

listing 21: infinities, the two zeros, NaN, and the MIN_VALUE naming trap (source: java/samples/src/Ch04/Primitives.java, lines 65-73)

4.4. boolean, the type that converts to nothing

`boolean` has exactly two values and no arithmetic, and it is the only primitive that converts to and from no other type, not int, not anything. The c idiom of testing an integer's truth is a compile error, which forces the comparison into the source and is generally counted a win:

```
int count = 1;
if (count) { }        // error: int cannot be converted to boolean
if (count != 0) { }  // the explicit form the language demands
```

fragment, illustrative, not executed

4.5. char, the utf-16 code unit

`char` is a 16-bit unsigned integer that also spells a character, so it widens into arithmetic and reads as a number when cast. The same 16 bits walk through `char` unsigned and through `short` signed, one of the quiet asymmetries worth seeing once. Unicode outgrew the 16 bits long ago: code points past U+FFFF, the supplementary planes, ride a surrogate pair of two chars, a high followed by a low, so `length` counts units while `codePointCount` counts characters, and the emoji in the sample is two of one and one of the other.

```
    // char is an unsigned 16-bit utf-16 code unit
    ok((int) 'A' == 65 && 'a' + 1 == 98,
      "char widens to int and joins arithmetic, 'a' plus 1 is 98");
    char allBits = (char) 0xFFFF;
    ok((int) allBits == 65535 && (short) allBits == -1,
      "the same 16 bits, unsigned through char, signed through short");
    String otter = "otter 🦦";
    ok(otter.length() == 8 && otter.codePointCount(0, 8) == 7,
      "the otter emoji is 2 chars and 1 code point, length counts units");
    ok(otter.codePointAt(6) == 0x1F9A6 && Character.isHighSurrogate(otter.charAt(6)),
      "supplementary code points ride a surrogate pair, high then low");
```

listing 22: char as number, the signed and unsigned readings, and the surrogate pair (source: java/samples/src/Ch04/Primitives.java, lines 75-85)

The working rule that follows: apis that index text by `int` code point, `codePointAt`, `codePoints`, `offsetByCodePoints`, are the safe vocabulary on text that might hold anything beyond the basic multilingual plane, and chapter 13's string api sits on top of it.

4.6. String, immutable and pooled

`String` is a reference type with literal syntax, immutable by design, and every method returns a new string rather than moving the original, which is what makes sharing one across threads free of races and keys safe in hash structures. The pool is the other half: string literals are interned into one shared copy per value at class load, constant expressions fold and intern at compile time as chapter 3 measured, and runtime-built strings join the pool only through an explicit `intern()`.

```
    // the pool and immutability
    String fresh = new String("java");
    ok(fresh != "java" && fresh.equals("java"),
      "new String builds a distinct object, equals compares content");
    ok(fresh.intern() == "java", "intern returns the pooled literal");
    String runtime = "ja";
    runtime = runtime + "va";
    ok(runtime != "java" && runtime.equals("java"),
      "runtime concatenation is not a constant, it stays out of the pool");
    ok("abc".toUpperCase() != "ABC",
      "every method returns a new string, the original never moves");
```

listing 23: the pool through intern, and the new object every method returns (source: java/samples/src/Ch04/References.java, lines 42-52)

4.7. boxing and the cache

Every primitive has a wrapper class, `Integer` for `int` and so on, and since 5 the conversion runs automatically in both directions. Autoboxing calls `Integer.valueOf`, which serves small values from a cache that runs −128 to 127 by default, its top measured at 127 on this jvm, and constructs fresh objects above it, so == on boxed integers is a reference comparison that happens to work in the cache band and fails right past it. The constructors are deprecated since 9, measured on the pinned build with `forRemoval` false, and `valueOf` is the only spelling worth keeping. The reverse direction has its own trap: unboxing a null wrapper throws `NullPointerException`, because there is no such thing as a null int.

```
    // autoboxing: valueOf, the cache, and the == trap
    Integer lowA = 127, lowB = 127, highA = 128, highB = 128;
    ok(lowA == lowB, "127 boxes through the cache, one object");
    ok(highA != highB && highA.equals(highB),
      "128 boxes fresh, == fails where equals holds: the classic trap");
    int cacheTop = -1;
    for (int i = 0; i < 400; i++) {
      if (Integer.valueOf(i) != Integer.valueOf(i)) {
        cacheTop = i - 1;
        break;
      }
    }
    ok(cacheTop == 127, "measured on this jvm: the Integer cache runs -128 to " + cacheTop);
```

listing 24: the cache boundary measured, 127 true at == and 128 not (source: java/samples/src/Ch04/References.java, lines 54-66)

```
    try {
      Constructor<Integer> legacy = Integer.class.getConstructor(int.class);
      Deprecated marker = legacy.getAnnotation(Deprecated.class);
      System.out.println("new Integer(int): since " + marker.since()
        + ", forRemoval " + marker.forRemoval());
      ok(marker != null, "new Integer(int) carries a Deprecated annotation, since 9");
    } catch (NoSuchMethodException e) {
      ok(false, "constructor lookup failed: " + e);
    }
    Integer absent = null;
    try {
      int unwrap = absent;
      ok(false, "unboxing null should have thrown");
    } catch (NullPointerException e) {
      ok(true, "unboxing null throws NullPointerException, no such thing as a null int");
    }
```

listing 25: the deprecation read reflectively, and unboxing null (source: java/samples/src/Ch04/References.java, lines 67-82)

PITFALL: the == that works until it does not

Boxed small integers compare equal under == inside the cache band and stop the moment a value crosses 127, which is why the failure surfaces in production and not in the test that used small fixtures. The same holds for `String` and every wrapper: reference equality is identity, `equals` is content, and the cache is an implementation detail the spec permits, not a promise to rely on.

4.8. var, inference for locals

`var` since 10 infers a local's type from its initializer. It works wherever the initializer pins a type: constructors, factory calls, generic calls with real arguments, lambda parameters since 11. It cannot name a field or a parameter or any member, it cannot stand alone or hold a bare `null`, array brackets do not survive it, and a lambda or method reference needs a target type to give `var` something to read.

```
class C {
  var count = 1;            // error: fields carry declared types
  void send(var text) { }   // error: parameters too, lambdas since 11 only
  void init() {
    var nothing = null;     // error: null has no type to infer
```

```
    var pair[] = {1, 2};    // error: brackets belong on the type
  }
}
```

fragment, illustrative, not executed

One interaction surprises people: the conditional expression, which normally waits for a target type, turns standalone under `var` and merges its arms, so an `int` arm beside a `double` arm widens to `double` on the spot.

```
// var: local inference since 10, and the ternary that stops being one
List<String> names = new ArrayList<>();
names.add("first");
ok(names.size() == 1 && names.get(0).equals("first"),
  "var infers ArrayList<String> from the initializer, reads the same");
var width = true ? 1 : 2.0;
ok(String.valueOf(width).equals("1.0"),
  "under var the ternary is standalone, int and double merge to double: " + width);
```

listing 26: inference from the constructor, and the ternary that widened to double (source: java/samples/src/Ch04/References.java, lines 84-91)

4.9. records, the data shape

A record, final since 16, is a transparent carrier for a fixed set of components: the header names them, and the compiler generates the accessor per component, `equals`, `hashCode`, `toString`, and the canonical constructor. Validation rides the compact constructor, the form without parentheses that runs before the fields settle. Chapter 5 takes records deep, the reflection contract included, this chapter only needs the shape as the default way to write a value type:

```
// records since 16: the data shape, deep treatment in chapter 5
record Interval(int lo, int hi) {
  Interval {
    if (lo > hi) {
      throw new IllegalArgumentException("lo " + lo + " greater than hi " + hi);
    }
  }
}
```

listing 27: a record with a validating compact constructor (source: java/samples/src/Ch04/References.java, lines 22-29)

```
// records: components, generated members, compact constructor
Interval window = new Interval(2, 3);
ok(window.lo() == 2 && window.hi() == 3 && window.equals(new Interval(2, 3)),
  "accessors, equals, and hashCode arrive generated, lo() hi()");
ok(window.toString().equals("Interval[lo=2, hi=3]"),
  "toString spells the shape: " + window);
try {
  Interval bad = new Interval(5, 1);
  ok(false, "the compact constructor should have thrown");
} catch (IllegalArgumentException e) {
  ok(true, "the compact constructor validates before fields settle: " + e.getMessage());
}
```

listing 28: the generated members measured, and validation throwing (source: java/samples/src/Ch04/References.java, lines 93-104)

4.10. sealed, the closed hierarchy

A sealed type, since 17 after previews in 15 and 16, lists exactly which types may implement or extend it, and every permitted subtype must itself be `final`, `sealed`, or marked `non-sealed` to reopen the branch. The closure is what makes exhaustiveness checkable, the reason chapter 9's pattern switch can prove it covered every case with no default clause, and the runtime knows the shape through `isSealed` and `getPermittedSubclasses`.

```
// sealed since 17: the closed hierarchy, pattern matching in chapter 9
sealed interface Shape permits Circle, Square {
}

record Circle(double radius) implements Shape {
}

record Square(double side) implements Shape {
}
```

listing 29: a sealed interface permitting two records (source: java/samples/src/Ch04/References.java, lines 31-39)

```
// sealed: closed by declaration, open to reflection
ok(Shape.class.isSealed() && Shape.class.getPermittedSubclasses().length == 2,
  "Class knows the sealing since 17, permits Circle and Square");
Shape shape = new Circle(1.5);
ok(shape instanceof Circle c && c.radius() == 1.5,
  "instanceof patterns since 16 dispatch over the closed set, chapter 9");
```

listing 30: the sealing visible to Class, and pattern dispatch over the closed set (source: java/samples/src/Ch04/References.java, lines 106-111)

4.11. null and Optional

The null literal belongs to every reference type, the null type sits below them all as their bottom, and `instanceof` answers false for it without throwing. The pointer is unchecked at the type level, so 8 shipped `Optional` as the explicit maybe for returns: a container that forces the caller to acknowledge the empty case, not a general null replacement but a return contract. Chapter 8 owns the full discipline and its stream interactions.

```
// null: the bottom of every reference type
String none = null;
ok(none == null && !(none instanceof String),
  "null belongs to every reference type and instanceof answers false");
ok(Optional.<String>empty().isEmpty(),
  "Optional carries the explicit maybe, chapter 8 owns the contract");
```

listing 31: null's instanceof, and the empty Optional carrier (source: java/samples/src/Ch04/References.java, lines 113-118)

4.12. the conversion lattice

Between the numeric primitives sits one small graph. Widening conversions, the arrows, are implicit and always safe on range. Everything else is a narrowing cast, explicit by rule, and the cast is where bits die: a truncating double to int, a masking byte cast that turns 130 into −126, and the quiet one, int or long to float, which is widening by range yet lossy on precision because 24 effective mantissa bits cannot hold a 32-bit integer's low digits. `boolean` stands outside the lattice entirely.

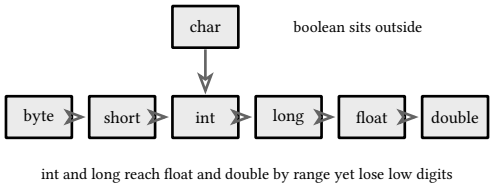


figure 6: the conversion lattice: arrows widen implicitly, everything else casts

The sample walks the widening chain for free, then measures each loss: the byte cast that wraps, the double truncation toward zero, and the 123456789 that comes back from a float as 123456792.

```
// the conversion lattice: widening free, narrowing explicit and lossy
byte fits = 13;
long widened = fits;
double widenedMore = widened;
ok(fits == 13 && widened == 13L && widenedMore == 13.0,
  "byte to long to double widens implicitly, always safe for range");
ok((byte) 130 == -126, "the narrowing cast truncates bits: 130 becomes -126");
ok((int) 13.756 == 13 && (int) -13.756 == -13,
  "double to int truncates toward zero, both signs");
ok((int) (float) 123456789 == 123456792,
  "int to float is widening yet lossy: 123456789 reads back 123456792");
```

listing 32: widening free and narrowing lossy, each loss measured (source: java/samples/src/Ch04/Primitives.java, lines 87-97)

VERIFY: the lossy widening, read as range not precision

The lattice calls int to float a widening conversion because every int value stays inside float's range, and the same page's rules let low digits fall off, 123456789 to 123456792 measured here. Any integer above 2^24 crossing into a float, or above 2^53 into a double, needs either the exact integer types or an honest rounding decision, which is chapter 13 territory.

sources: Java in a Nutshell 8th edition chapter 2 (primitive data types, primitive type conversions, string literals) and chapter 5 (java values, pass by value) for the type inventory, the conversion table, the string and boxing stories, grounded and rewritten. docs.oracle.com/javase/specs/jls/se27/html/jls-3.html accessed 2026-10-04 for the literal grammar behind the arithmetic. openjdk.org/projects/jdk/14 and /jdk/15 accessed 2026-10-04 anchor records preview 14 final 16 and sealed preview 15 final 17 alongside the ladder spine, jeps/286 for var in 10 and the ladder chapter's own row for lambda parameters in 11, and the oracle javadoc for Math and Integer read on the pinned build for the exact family since 8 and the measured deprecation. Everything behavioral verified live on tools/jdk27/build/jdk-27 by the Ch04 samples under pwsh tools/run-java-samples.ps1 -Chapter Ch04: 42 checks, Primitives 24 and References 18, covering widths and ranges, the overflow wrap and Math.addExact's exact message, truncating division and the throwing zero, the bit patterns of 1.0 in both floats, 0.1 plus 0.2, the special values and both zeros, NaN, the MIN_VALUE trap, char signed and unsigned readings, the surrogate pair and code point count, intern and pool identity, the Integer cache boundary measured at 127, the deprecation read reflectively as since 9 forRemoval false, the unboxing null throw, var inference and the standalone ternary widening, the record's generated members and compact constructor validation, isSealed with two permitted subclasses, null's instanceof, Optional.empty, and every conversion loss in the lattice, all dated 2026-10-04.

5. classes and objects

Every Java program is classes: all statements live in methods, all methods live in types, and the two workhorse type forms are the class and the interface. Chapter 4 covered the value side, primitives and their boxes, this chapter covers the machinery: the four member kinds, construction order, inheritance and dispatch, the abstract and interface forms, and records as the fixed data shape. Chapter 2 walked the version ladder, here each feature appears with the release it arrived in.

5.1. the four member kinds

Members sit on two axes. Static or instance decides what the member belongs to, the class itself or each object. Field or method decides whether it is state or behavior. Four combinations, and a class field is one copy shared by every instance while an instance field is one slot per object:

```
static class Gauge {
  // class field: one copy per class, not per instance
  static int made;
  // class method: no this, cannot touch instance state
  static int nextId() {
    return ++made;
  }
  // instance field: one per object
  final int id = nextId();
  // instance method: this is the receiver
  int doubled() {
    return this.id * 2;
  }
}
```

listing 33: one class with all four member kinds (source: java/samples/src/Ch05/Members.java, lines 14-27)

The static method has no `this`: it runs without a receiver, so it cannot read `id` or call `doubled()`, and the sample proves the sharing by watching `Gauge.made` climb as two objects build themselves. The instance method receives the current object as an implicit `this` parameter, which is why `this.id` inside `doubled()` means the receiver's slot. Access outside the class qualifies by class name for statics and by expression for instances, `Gauge.nextId()` versus `g.doubled()`.

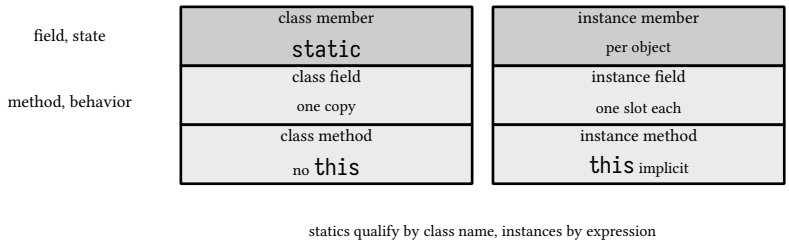


figure 7: the two member axes: what it belongs to, and what it is

5.2. construction and initialization order

Object creation is a fixed sequence, and the sample records it in a trace list: memory is allocated and zeroed, the superclass chain constructors run top down from `Object`, then field initializers and instance initializer blocks run in source order, then the rest of the constructor body. Class initializers run once, at first use, before any instance exists, base class before subclass:

```
  static class Car extends Engine {
    // instance initializer and field initializers run after super(),
    // in source order, before the rest of the constructor body
    final int wheels = addTrace("car field initializer");
    final String badge;
    final String shouted;

    static { trace.add("car static init"); }
    { trace.add("car instance init block"); }

    Car(int cylinders, String badge) {
      // java 25 prologue: statements before super(), which may validate
      // arguments and assign this-class fields that lack initializers
      if (badge == null || badge.isEmpty()) {
        throw new IllegalArgumentException("badge required");
      }
      this.badge = badge;
      this.shouted = badge.toUpperCase();
      super(cylinders);
      trace.add("car ctor body");
    }
```

listing 34: the initialization order the trace asserts, and the java 25 prologue (source: java/samples/src/Ch05/Construction.java, lines 32-52)

The measured trace is exact: `engine ctor body`, then `car field initializer`, then `car instance init block`, then `car ctor body`, with both static initializers already done. Before java 25 none of that constructor body could run before the `super(...)` call, so argument validation had to choose between throwing after the superclass construction work and a static factory. Flexible constructor bodies (JEP 513, final in java 25) lifted the rule: a prologue may validate arguments and even assign fields of the same class that lack initializers, and the explicit constructor invocation comes last:

```
    Car(int cylinders, String badge) {
      // java 25 prologue: statements before super(), which may validate
      // arguments and assign this-class fields that lack initializers
      if (badge == null || badge.isEmpty()) {
        throw new IllegalArgumentException("badge required");
      }
      this.badge = badge;
      this.shouted = badge.toUpperCase();
      super(cylinders);
      trace.add("car ctor body");
    }
```

listing 35: prologue validation and field assignment before super() (source: java/samples/src/Ch05/Construction.java, lines 42-52)

The failure path is the point. `new Car(4, "")` throws before any constructor body anywhere has run, and the trace shows only the two static initializers. The prologue runs in an early construction context: it may not read `this` as an object, only assign the blank final fields, and it may not reference the instance being built otherwise. `this(...)` delegation keeps the same shape, one prologue, then the delegated call.

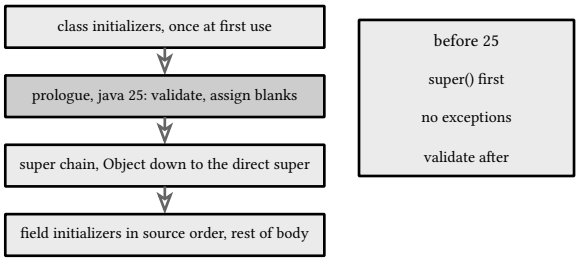


figure 8: one construction, top to bottom

Records follow the same sequence and add two forms: the canonical constructor the header implies, and a compact constructor that omits the parameter list, validates, and lets the compiler do the field assignments. Chapter 6 revisits the generic forms.

5.3. inheritance: overriding, hiding, dispatch

`extends` gives one superclass, every class but `Object` has one, and a subclass inherits the accessible members. Fields and static methods hide, instance methods override, and the difference decides everything about polymorphism:

```
static class A {
  int i = 1;
  int f() { return i; }
  static char g() { return 'A'; }
  char viaMember() { return g(); }
}

static class B extends A {
  int i = 2; // hides A.i, distinct storage
  int f() { return -i; } // overrides A.f
  static char g() { return 'B'; } // hides A.g
  char viaSub() { return g(); }
}

static class C extends B {
  @Override int f() { return super.f() + 100; }
}
```

listing 36: the same names, three different relations: hide, override, one-step super (source: java/samples/src/Ch05/Dispatch.java, lines 15-31)

Hiding follows the static type. `b.i` reads the subclass slot, the cast view reads the superclass slot, both slots exist in one object. Overriding follows the runtime type. `asA.f()` calls `B.f()` through an `A` variable, because instance method invocation is a virtual lookup on the receiver's actual class, and that is the entire basis of substitutability: a `Circle[]` may hold any shape and each element computes with its own method. `super.f()` is the one exception, it starts the lookup one class up and runs exactly one step, `C.f()` reaching `B.f()` but never `A.f()` directly.

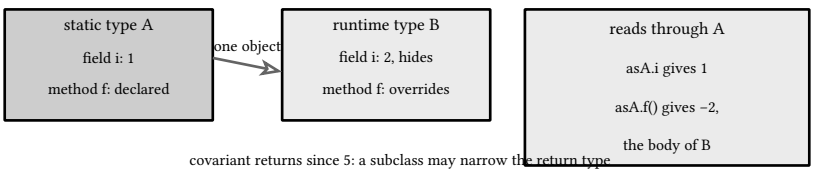


figure 9: one receiver, two views: hiding by static type, dispatch by runtime type

A class can also be `final`, closed to extension, the default stance for most classes. `abstract` is the opposite move, the class declares signature without body and cannot be instantiated, a concrete subclass must implement every

abstract method. Sealed types, the third possibility between open and final, arrived final in java 17 (JEP 409, after previews in 15 and 16) and chapter 9 owns them, because their whole value is pattern exhaustiveness.

5.4. interfaces

An interface is a reference type of signatures. A class gets one superclass but implements any number of interfaces, which is how Java splits single inheritance of state from multiple inheritance of behavior. The member rules evolved in three steps and the releases are worth remembering: before java 8 an interface held only public abstract methods and constants, java 8 added `default` and `static` methods so the collections could grow lambda-facing methods without breaking every implementation, java 9 added private methods for code shared between defaults:

```
interface Scaled {
  double factor();

  // a private interface method, since java 9, exists to serve defaults
  private double clamp(double v) {
    return Math.max(0.0, Math.min(1.0, v));
  }

  default double apply(double v) {
    return clamp(v) * factor();
  }

  // a static interface method, since java 8, lives on the interface
  static Scaled identity() {
    return () -> 1.0;
  }
}
```

listing 37: one abstract, one private since 9, one default, one static since 8 (source: java/samples/src/Ch05/Interfaces.java, lines 17-33)

A default method is an inherited implementation the class may keep or override, and an override always wins. Two interfaces bringing the same default signature is the diamond, and the language refuses to guess: the class must override, and `Vocal.super.call()` picks a side explicitly. Interfaces hold no instance state, defaults can only call the interface's other methods, so the multiple inheritance is of behavior alone.

```
interface Vocal {
  default String call() { return "hello"; }
}

interface Caller {
  default String call() { return "dial"; }
}

// two clashing defaults: the class must resolve or the compile fails
static class Person implements Vocal, Caller {
  @Override public String call() { return Vocal.super.call() + "/" + Caller.super.call(); }
}

// a class override always beats any inherited default
static class Loud implements Vocal {
  @Override public String call() { return "HELLO"; }
}
```

listing 38: diamond clash resolved in code, and a plain override beating a default (source: java/samples/src/Ch05/Interfaces.java, lines 35-51)

C#'s near-identical machinery lives in book 6, ch 3 (c# 15, f# 11, and .net 11, the type system), where default interface methods arrived only in c# 8, five years after java.

5.5. records as the fixed data shape

A record is a class that promises its header is the whole state. The compiler generates the canonical constructor, the accessors named after each component, and `equals`, `hashCode`, and `toString` derived from the components. Records previewed in 14 and 15 and arrived final in java 16 (JEP 395), with the deeper type treatment in chapter 4. What remains here is the contract the generated methods carry, because handwritten classes still carry it too:

```
  static class Dot {
    private final int x;
    private final int y;

    Dot(int x, int y) {
      this.x = x;
      this.y = y;
    }

    @Override public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Dot)) return false;
      Dot that = (Dot) o;
      return x == that.x && y == that.y;
    }

    @Override public int hashCode() {
      return 31 * x + y;
    }

    @Override public String toString() {
      return "Dot[x=" + x + ", y=" + y + "]";
    }
  }
```

listing 39: the handwritten trio every value class owes Object (source: java/samples/src/Ch05/Contracts.java, lines 19-42)

`equals` defines value equality against == reference identity: reflexive, symmetric, transitive, consistent, and `null` is equal to nothing, which the `instanceof` test gives for free since it is false on `null`. `hashCode` must agree with it: equal objects carry equal hash codes, or every hash table misbehaves. The sample measures the failure mode directly:

```
    Map<BadHash, String> bad = new HashMap<>();
    bad.put(new BadHash(7), "seven");
    ok(bad.get(new BadHash(7)) == null,
        "equals without matching hashCode loses the entry, measured");
```

listing 40: an equals with no matching hashCode loses the entry, measured (source: java/samples/src/Ch05/Contracts.java, lines 88-91)

That class also trips the `[overrides]` lint, javac knows the smell, and the sample suppresses it in exactly that one place to run the experiment. The same trio from a record needs no writing at all:

```
record Rec(int x, int y) {}

record Ranged(int lo, int hi) {
  Ranged { // compact constructor: validate, then assign
    if (hi < lo) throw new IllegalArgumentException(hi + " < " + lo);
  }
}
```

listing 41: a record pair, and a compact constructor that validates (source: java/samples/src/Ch05/Contracts.java, lines 59-65)

The compact constructor assigns the fields implicitly after the body runs, so validation is its only job. `isRecord()` distinguishes the shapes at runtime, and the accessor rule is strict: component `x` produces method `x()`, never `getX()`.

PITFALL: records are shallowly immutable

A record's fields are final, but a component of reference type can still point at mutable state, `record Log(ArrayList<String> lines)` is a record holding a mutable list. Deep immutability is a design property you bring, chapter 12 returns to it with `List.of` copies.

5.6. why contextual keywords matter

Java froze its reserved word list at java 1 and kept it frozen, and every later language word is contextual: meaningful in one syntactic position, an ordinary identifier everywhere else. `record` and `sealed` arrived in 16 and 17, `permits` in 17, `yield` in 14 when switch expressions did, `var` in 10, and code written before any of them keeps compiling:

```
public static void main(String[] args) {
  int record = 3;
  var sealed = "still an identifier";
  int permits = 5;
  int yield = 6;
  ok(record == 3 && permits == 5 && yield == 6,
      "record, permits, and yield all name ordinary locals");
  ok(sealed.equals("still an identifier"),
      "var infers a String for a variable named sealed");
  Token token = new Token("record");
  ok(token.word().equals("record") && token instanceof Token,
      "a record class and a variable named record coexist in one file");
  ok(withYield(0) == 99 && withYield(1) == 99,
      "yield works as both a local name and the switch keyword");
  System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27");
```

listing 42: four new words as four ordinary locals, one record in the same file (source: java/samples/src/Ch05/Contextual.java, lines 29-43)

The one hyphenated exception proves the design discipline: `non-sealed`, needed by sealed types in java 17, is the first keyword spelled with a hyphen, because `nonsealed` would have stolen an identifier outright.

NOTE: why contextual matters

A frozen reserved set is why old code never breaks on new language. When you meet a new word in java, check whether it is contextual. If it is, the word is also a legal identifier, which is the tell that it arrived in a later release. C# runs the same policy from the other direction, book 6, ch 2 (c# 15, f# 11, and .net 11, lexical structure) counts its contextual table growing past 48 words.

sources: openjdk.org/jeps/395, /jeps/409, and /jeps/513, statuses and release attributions checked 2026-10-04. Java in a Nutshell 8th edition, chapters 3 and 5, object lifecycle, inheritance, and the common method contracts, pages

107 to 216. Every behavioral claim in this chapter is asserted by `java/samples/src/Ch05`, 33 checks green under `pwsh -NoProfile -File tools/run-java-samples.ps1 -Chapter Ch05` on the pinned build 27+35-2325, measured 2026-10-04.

6. generics

Generics arrived in java 5 (JSR 14) to let a container say what it holds, turning a whole family of runtime cast failures into compile errors. The design constraint that shaped everything after it was migration compatibility: millions of lines of nongeneric java 1 through 4 code had to keep running and even interoperate with the new generic collections. The mechanism that bought that compatibility, type erasure, is also the source of every rough edge this chapter measures.

6.1. type parameters

A generic declaration introduces a type parameter, `List<E>`, and each use supplies a type argument, `List<String>`. The parameter stands for a reference type only, primitives are excluded and autoboxing bridges the gap. The diamond, `new ArrayList<>()`, arrived in java 7 to stop the doubled spelling the compiler could already infer. A generic method declares its own parameter without the class carrying one:

```
static <T> T pick(T a, T b, boolean first) {
  return first ? a : b;
}
String s = pick("a", "b", false);  // T inferred as String
```

fragment, illustrative, not executed

Inference reads the arguments and the expected return type. Mixed arguments infer the least upper bound, `pick(1, 2L, true)` is typed `Number` even though the value returned is the `Integer`. Chapter 2 walked the ladder, `var` since 10 rides on the same inference engine.

6.2. erasure and bridges

`javac` checks every type argument, then removes the parameters from the bytecode it emits. One `List` class exists at runtime, not one per argument, and the sample proves both halves:

```
static Class<?> classOf(List<String> strings, List<Integer> ints) {
  return strings.getClass() == ints.getClass() ? strings.getClass() : null;
}
```

listing 43: one runtime class behind every use (source: java/samples/src/Ch06/Erasure.java, lines 20-23)

```
public static void main(String[] args) {
  ok(classOf(new ArrayList<String>(), new ArrayList<Integer>()) == null,
      "List<String> and List<Integer> share one runtime class");
  ok(new ArrayList<String>().getClass().getName().equals("java.util.ArrayList"),
      "the runtime name carries no type arguments");
```

listing 44: the names agree and no arguments survive (source: java/samples/src/Ch06/Erasure.java, lines 41-45)

Erasure explains the compile errors that otherwise look arbitrary. Two overloads that differ only in a type argument erase to the same raw signature and the language refuses them:

```
// will not compile: both erase to totalOrders(Map)
int totalOrders(Map<String, List<String>> byName);
int totalOrders(Map<String, Integer> byCount);
```

> fragment, illustrative, not executed

When a class implements a parameterized interface, the erased call sites need a target, so the compiler writes a synthetic bridge. The sample counts the bridge directly in the class file's reflection view:

```
static class Node implements Comparable<Node> {
  final String key;

  Node(String key) {
    this.key = key;
  }

  @Override public int compareTo(Node other) {
    return key.compareTo(other.key);
  }
}
```

listing 45: compareTo(Node) is the real method (source: java/samples/src/Ch06/Erasure.java, lines 24-34)

```
  int bridges = 0;
  int plain = 0;
  for (Method m : Node.class.getDeclaredMethods()) {
    if (m.getName().equals("compareTo")) {
      if (m.isBridge() && m.isSynthetic()
          && m.getParameterTypes()[0] == Object.class) {
        bridges++;
      } else {
        plain++;
      }
    }
  }
  ok(bridges == 1 && plain == 1,
      "erasure forces one synthetic bridge compareTo(Object) beside the real one");
  Node a = new Node("alpha");
  Node b = new Node("beta");
  Comparable<Node> viaInterface = a;
  ok(viaInterface.compareTo(b) < 0, "the bridge forwards the erased call correctly");
```

listing 46: one synthetic bridge compareTo(Object) beside it, measured (source: java/samples/src/Ch06/Erasure.java, lines 46-63)

`Comparable<Node>` through an interface variable dispatches the bridge, which casts and forwards. The one idiom that buys a type back is the class token, `Class<T>` is reified because it names one class:

```
// the token idiom: Class<T> carries a type the erased method lacks
static <T> T as(Object o, Class<T> type) {
  return type.cast(o);
}
```

listing 47: the token idiom, a reified type carried as a value (source: java/samples/src/Ch06/Erasure.java, lines 36-39)

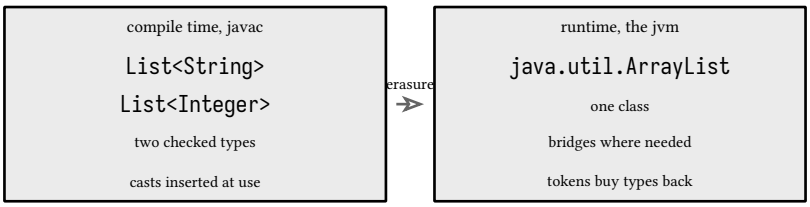


figure 10: what javac sees, what the jvm sees

6.3. variance: arrays covariant, generics invariant

Java 1 made arrays covariant, `String[]` assigns to `Object[]`, so pre-generic methods like a sort for `Object[]` could serve every reference array. The hole is measurable: the array remembers its real element type and checks every store at runtime:

```
    // covariance predates generics: Object[] aliases String[]
    String[] words = {"alpha", "beta"};
    Object[] asObjects = words;
    ok(asObjects[0].equals("alpha"), "array covariance needs no cast, java 1 design");
    try {
      asObjects[0] = 42;
      ok(false, "the wrong element store throws at runtime");
    } catch (ArrayStoreException e) {
      ok(true, "the wrong element store throws ArrayStoreException at runtime");
    }
```

listing 48: covariance compiles, the runtime check pays for it (source: java/samples/src/Ch06/Variance.java, lines 29-38)

When generics arrived the same question came back, is `List<String>` a subtype of `List<Object>`, and the answer was no. If it were, the same store failure would move into collections with no runtime check to catch it, since the element type is erased:

```
// will not compile: generics are invariant
List<Object> objects = new ArrayList<String>();
// the error: incompatible types
```

fragment, illustrative, not executed

Invariance is safe but stiff, so use-site variance arrived with wildcards, and the mnemonic is PECS, producer extends, consumer super. A list you read from is `List<? extends T>`, a list you write to is `List<? super T>`:

```
static <T> int copyInto(List<? super T> dst, List<? extends T> src) {
  int n = 0;
  for (T item : src) {
    dst.add(item);
    n++;
  }
  return n;
}
```

listing 49: one signature with both variance directions (source: java/samples/src/Ch06/Variance.java, lines 19-26)

```
    // the wildcard pair replaces that runtime check with static rules
    List<Integer> ints = List.of(1, 2, 3);
    List<Number> nums = new ArrayList<>();
    ok(copyInto(nums, ints) == 3 && nums.size() == 3,
        "List<Integer> feeds List<Number> through extends plus super");
    List<? extends Number> produced = ints;
    Number read = produced.get(0);
    ok(read.intValue() == 1, "an extends list reads as the bound");
    List<? super Integer> consumed = new ArrayList<Number>();
    consumed.add(9);
    ok(consumed.contains(9), "a super list accepts the subtype for writing");
    Object opaque = consumed.get(0);
    ok(opaque.equals(9), "a super list only reads back as Object");
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27");
  }
```

listing 50: extends reads at the bound, super writes below it, measured (source: java/samples/src/Ch06/Variance.java, lines 40-54)

The rules are strict because they are static: an extends wildcard accepts no writes except `null`, a super wildcard reads back only `Object`. Chapter 12 shows these shapes all over the collections api, `Collections.copy` has exactly this signature.

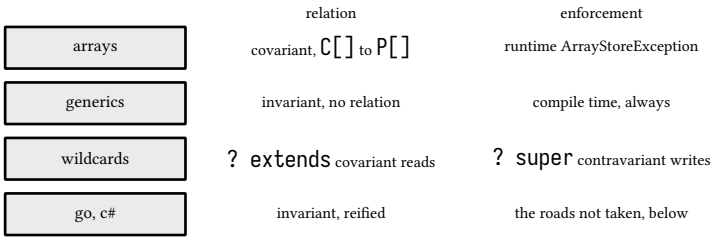


figure 11: three container relations and what enforces each

Go's generics (book 4, ch 4 (go 1.27, generics and generic methods)) took the other road in 1.18: invariant, no wildcards, no variance annotation at all, and types carried as dictionaries at compile time. C#'s (book 6, ch 5 (c# 15, f# 11, and .net 11, generics)) are reified, the runtime knows `List<int>` from `List<string>`, which is why that platform overloads on generic arguments and this one cannot.

6.4. bounds

A bound narrows what a parameter may stand for, and the bound's members become callable inside the declaration:

```
// one bound: the parameter promises Number's interface
static double sum(List<? extends Number> xs) {
  double total = 0;
  for (Number n : xs) {
    total += n.doubleValue();
  }
  return total;
}
```

listing 51: one bound, Number's interface unlocked (source: java/samples/src/Ch06/Bounded.java, lines 18-25)

Bounds stack, class first then interfaces, and a bound may mention the parameter itself, the recursive or f-bounded shape every `Comparable` hierarchy uses:

```
record Money(int cents) implements Comparable<Money>, Serializable {
  @Override public int compareTo(Money other) {
    return Integer.compare(cents, other.cents);
  }
}

// two bounds: comparable and serializable, class first, interfaces after
static <T extends Comparable<T> & Serializable> T older(T a, T b) {
  return a.compareTo(b) ≥ 0 ? a : b;
}
```

listing 52: a record meeting two bounds, and the method that demands both (source: java/samples/src/Ch06/Bounded.java, lines 27-36)

6.5. heap pollution and `@SafeVarargs`

A varargs parameter is an array, arrays are covariant, and generics erase, so `List<String>...` is really an untyped `List[]` slot anyone can write through. A variable whose static type lies about its contents is heap pollution, and the failure lands far from the cause:

```
@SafeVarargs
@SuppressWarnings("varargs")
static void pollute(List<String>... lists) {
  Object[] a = lists;
  a[0] = List.of(42);
  try {
    String s = lists[0].get(0);
    ok(false, "the polluted slot should not read back as String: " + s);
  } catch (ClassCastException e) {
    ok(true, "the polluted slot throws ClassCastException on read");
  }
}
```

listing 53: the polluted slot throwing on read, measured (source: java/samples/src/Ch06/Pollute.java, lines 29-40)

The annotation `@SafeVarargs` is the developer's promise that the method only reads the array or copies it out safely, and it silences the warning at the declaration and every call site. It arrived in java 7 for static and final methods only, java 9 extended it to private methods:

```
@SafeVarargs
private static <T> List<T> combined(List<? extends T>... parts) {
  java.util.ArrayList<T> out = new java.util.ArrayList<>();
  for (List<? extends T> part : parts) {
    out.addAll(part);
  }
  return List.copyOf(out);
}
```

listing 54: a genuinely safe varargs method, annotated, private since 9 (source: java/samples/src/Ch06/Pollute.java, lines 18-26)

This sample keeps one deliberately unsafe method to run the failure, and the `@SuppressWarnings("varargs")` scoped to it is the honest marker of that choice. The general rule stands: do not build generic arrays, and when an api hands you one, read it or copy it, never publish it.

PITFALL: raw types are still legal, still a trap

> `List list = new ArrayList()` compiles today with a warning, and every element that comes out is `Object`. Raw types exist only for migration compatibility, this book's samples run under `-Xlint:all -Werror` where any raw or unchecked use fails the build, which is the discipline production code should keep.

6.6. generics, records, and patterns

A record can be generic, and the generated members are written once over the parameters. Chapter 9 owns pattern matching, one interaction belongs here because it is erasure at work: a pattern may not spell a type argument the runtime cannot check. Measured on the pinned build, `Object o` against `Box<Integer>` fails with `Object cannot be safely cast to Box<Integer>`. The component patterns carry the check instead, through inference:

```
    // inference inside record patterns: components carry the check
    Object o = new Box<Integer>(42);
    String described = switch (o) {
      case Box(Integer i) -> "integer component " + i;
      case Box(var v) -> "anything else " + v;
      default -> "not a box";
    };
    ok(described.equals("integer component 42"),
        "the component pattern infers and checks the type argument");
    ok(o instanceof Box(var v) && v.equals(42),
        "var components defer the check to the binding's own type");
```

listing 55: the class-level argument refused, components inferred and checked (source: java/samples/src/Ch06/GenericRecords.java, lines 32-42)

The same-type selector, where the expression is already known to be `Box<Integer>`, is the one spelling that survives, because nothing is unchecked about it:

```
    Box<Integer> typed = new Box<>(42);
    if (typed instanceof Box<Integer> same) {
      ok(same.value() == 42, "a same-type selector is the one legal spelled form");
    }
```

listing 56: the one legal spelled form (source: java/samples/src/Ch06/GenericRecords.java, lines 43-46)

```
// will not compile, measured on build 27+35-2325:
Object o = List.of();
if (o instanceof List<String> l) { }
// error: Object cannot be safely cast to List<String>
```

fragment, illustrative, not executed

sources: openjdk.org/jeps/513 and the java 5 through 9 release attributions discussed here follow the verified spine in docs/audit/java.md, checked 2026-10-04. `@SafeVarargs` applicability read from the java 27 api page, docs.oracle.com, accessed 2026-10-04. Java in a Nutshell 8th edition, chapter 4, generics through nested types, pages 162 to 198. Every behavioral claim in this chapter is asserted by `java/samples/src/Ch06`, 26 checks green under `pwsh -NoProfile -File tools/run-java-samples.ps1 -Chapter Ch06` on the pinned build 27+35-2325, measured 2026-10-04.

7. enums, annotations, and nested types

Three specialized type forms round out the reference type story: enums since java 5, a class with a fixed set of self-typed instances, annotations since java 5, an interface specialized for metadata, and nested types since java 1.1, types declared inside other types. None of the three is syntax sugar, each compiles to a real class file, and each carries rules this chapter measures.

7.1. an enum is a class

`enum` declares a class with a fixed number of instances, created at class initialization and never again. The constants are static final fields of the enum's own type, the constructor runs once per constant, and the class extends `java.lang.Enum`, which is why an enum cannot extend anything else:

```
public enum Planet {
  MERCURY(3.303e+23, 2.4397e6),
  VENUS(4.869e+24, 6.0518e6),
  EARTH(5.976e+24, 6.37814e6),
  MARS(6.421e+23, 3.3972e6),
  JUPITER(1.9e+27, 7.1492e7),
  SATURN(5.688e+26, 6.0268e7),
  URANUS(8.686e+25, 2.5559e7),
  NEPTUNE(1.024e+26, 2.4746e7);

  private static final double G = 6.67300e-11;
  private final double mass;
  private final double radius;

  Planet(double mass, double radius) {
    this.mass = mass;
    this.radius = radius;
  }

  double surfaceGravity() {
    return G * mass / (radius * radius);
  }

  enum Op {
    PLUS { int apply(int a, int b) { return a + b; } },
    TIMES { int apply(int a, int b) { return a * b; } };

    abstract int apply(int a, int b);
  }
```

listing 57: constants with arguments, fields, a constructor, methods, and a nested enum with constant bodies (source: java/samples/src/Ch07/Planet.java, lines 6-34)

Everything after the constant list is ordinary class body. The nested `Op` shows the further step, an abstract method in the enum forces every constant to carry its own body, the state machine pattern in one declaration:

```
enum Op {
  PLUS { int apply(int a, int b) { return a + b; } },
  TIMES { int apply(int a, int b) { return a * b; } };

  abstract int apply(int a, int b);
}
```

listing 58: constant-specific bodies implementing one abstract method (source: java/samples/src/Ch07/Planet.java, lines 29-34)

The family machinery comes from `java.lang.Enum`. `values()` returns a fresh array every call, callers may sort it without vandalizing the type. `ordinal()` is declaration position and `compareTo` is exactly that order, `valueOf` parses a name back to the same instance, and `name` and `toString` start identical. The constants are singletons, which is the singleton story: no reflective construction, no serialization duplicate, one instance each:

```
public static void main(String[] args) throws Exception {
  ok(Config.values().length == 1, "one constant, one instance, forever");
  ok(Config.INSTANCE == Config.valueOf("INSTANCE") && Config.INSTANCE.limit() == 512,
      "valueOf hands back the same object with its state");
  var ctor = Config.class.getDeclaredConstructors()[0];
  ctor.setAccessible(true);
  try {
    ctor.newInstance("SECOND", 1);
    ok(false, "reflective enum construction refused");
  } catch (IllegalArgumentException e) {
    ok(e.getMessage().equals("Cannot reflectively create enum objects"),
        "reflective enum construction refused: " + e.getMessage());
  }
```

listing 59: the enum singleton, and the measured refusal to construct a second (source: java/samples/src/Ch07/Singleton.java, lines 24-36)

The reflection refusal is a jvm rule, `Constructor.newInstance` checks for enum classes specifically, so the classic singleton hazards of double construction and reflective bypass are structurally closed.

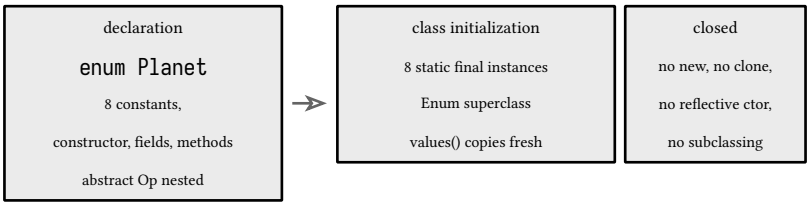


figure 12: one enum declaration, what the runtime holds

The collections answer enums with two purpose-built families. `EnumSet` is a bit vector, one long covers universes up to 64 constants, `RegularEnumSet` in the measurement, with a `JumboEnumSet` beyond. `EnumMap` is an array indexed by ordinal, which is why its iteration follows declaration order while a `HashMap` follows hashes. Both demand the enum's `Class` object at construction because the universe must be known:

```
    EnumSet<Planet> inner = EnumSet.range(MERCURY, MARS);
    ok(inner.size() == 4 && inner.contains(EARTH)
        && EnumSet.allOf(Planet.class).size() == 8,
        "EnumSet range and allOf over the declaration");
    ok(inner.getClass().getName().equals("java.util.RegularEnumSet"),
        "the small-enum implementation is the bit vector RegularEnumSet");
    EnumMap<Planet, String> moons = new EnumMap<>(Planet.class);
    moons.put(EARTH, "luna");
    moons.put(MARS, "phobos, deimos");
    ok(moons.get(EARTH).equals("luna")
        && moons.keySet().iterator().next() == EARTH
        && moons.getClass().getName().equals("java.util.EnumMap"),
        "EnumMap keyed by declaration order");
```

listing 60: EnumSet range over the declaration, EnumMap in declaration order (source: java/samples/src/Ch07/Planet.java, lines 60-72)

7.2. annotations

An annotation type is an interface with restrictions: no generics, zero-argument methods only, restricted return types, and `default` values allowed. It compiles to an interface extending `java.lang.annotation.Annotation`. The design intent was metadata without semantics, compile time hints like `@Override`, and the platform still ships those, but container frameworks put so much behavior behind runtime annotations that the strict reading has eroded:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Repeatable(Routes.class)
@interface Route {
  String path();

  int order() default 0;
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Routes {
  Route[] value();
}
```

listing 61: a repeatable annotation and its container, both RUNTIME (source: java/samples/src/Ch07/Annotations.java, lines 23-36)

Two meta-annotations do the policy work. `@Target` says where the annotation may appear, its `ElementType` enum grew `TYPE_PARAMETER` and `TYPE_USE` in java 8 so annotations could sit inside type positions. `@Retention` says how far it survives: `SOURCE` is discarded by `javac`, `CLASS` lands in the class file unseen by the jvm, `RUNTIME` is readable through reflection. The sample reads the retention of platform annotations off their own class objects, and `@Override` measures as `SOURCE`:

```
    // retention is measurable on the annotation types themselves
    Retention onOverride = Override.class.getAnnotation(Retention.class);
    Retention onFi = FunctionalInterface.class.getAnnotation(Retention.class);
    ok(onOverride != null && onOverride.value() == RetentionPolicy.SOURCE,
        "Override is SOURCE retention: discarded by javac itself");
    ok(onFi != null && onFi.value() == RetentionPolicy.RUNTIME,
        "FunctionalInterface is RUNTIME retention");
```

listing 62: retention read off the annotation types themselves (source: java/samples/src/Ch07/Annotations.java, lines 52-58)

The original java 5 platform set is `@Override`, `@Deprecated`, and `@SuppressWarnings`. Java 7 added `@SafeVarargs`, java 8 added `@FunctionalInterface` and `@Repeatable`. Repeatability works exactly as the sample shows, the class file stores the container type and reflection expands it back, so `getAnnotationsByType` flattens while `getAnnotations` shows the wrapper:

```
    Route[] routes = Endpoints.class.getAnnotationsByType(Route.class);
    ok(routes.length == 2 && routes[1].path().equals("/ready")
        && routes[0].order() == 2 && routes[1].order() == 1,
        "repeatable annotations flatten back into their order");
    Annotation[] direct = Endpoints.class.getAnnotations();
    ok(direct.length == 1 && direct[0] instanceof Routes,
        "the class file stores the container, reflection expands it");
    ok(Annotations.class.getDeclaredAnnotation(Routes.class) == null,
        "unannotated classes read as null, no empty array needed");
```

listing 63: two spellings of the same two annotations (source: java/samples/src/Ch07/Annotations.java, lines 43-51)

The service spine in chapters 19 through 31 leans on annotations lightly, the http kernel is hand-routed, but the testing chapter uses a `@Repeatable` tag the same way this sample does.

7.3. nested types

Types nest inside types in four shapes, and the vocabulary matters because “inner class” colloquially covers three of them. The declaration picks the shape: `static` member type, nonstatic member class, local class, anonymous class:

```
// static member type: no enclosing instance, sees statics only
static class Fixed {
  static int scale = 3;
}

// nonstatic member class: every instance carries an outer instance
class Tally {
  int bump() {
    counter += 1; // reads the outer private field through the link
    return Nesting.this.counter; // qualified this names the outer object
  }
}

// nested records and interfaces are implicitly static since 16
record Pair(int a, int b) {}

int localDemo() {
  int factor = 5; // effectively final, capturable below
  class Local {
    int total() {
      return factor + counter; // local class sees locals and outer state
    }
  }
  record LocalRec(int v) {} // local records since java 16
  return new Local().total() + new LocalRec(1).v();
}
```

listing 64: static nested, inner with qualified this, nested record, local class and record (source: java/samples/src/Ch07/Nesting.java, lines 20-45)

A static member type is a type in a namespace, no outer instance, the shape of `Map.Entry`. A nonstatic member class, the true inner class, carries a reference to an enclosing instance, which is how `Tally` touches the outer private field, and `Outer.this` is the unambiguous name for that link. Local classes live in a block and see the effectively final locals of it, and local records arrived with java 16 when records made the constant-field restriction obsolete. Records and interfaces nested in a class are implicitly static.

Until java 11 nesting was a compiler fiction: separate class files needed synthetic package-private bridge methods so `Tally` could read `counter` at all. Nest-based access control (JEP 181) taught the jvm the family directly, `NestHost` and `NestMembers` attributes, private access across nestmates, and reflection exposes the relation:

```
    // nestmates, java 11: the jvm now knows these files are one family
    ok(Nesting.class.getNestHost() == Nesting.class
        && Nesting.class.isNestmateOf(Tally.class)
        && !Nesting.class.isNestmateOf(String.class),
        "getNestHost and isNestmateOf expose the nest, java 11");
```

listing 65: the nest, java 11, visible through Class (source: java/samples/src/Ch07/Nesting.java, lines 71-75)

The anonymous class is a local class with no name, defined and instantiated in one `new` expression. It still earns its keep where a one-off subclass or an interface plus state is needed, but for the single-method case lambdas took over, and the takeover is about identity semantics, not syntax. The sample measures the split from both sides:

```
void thisIdentity(java.util.function.BiConsumer<Object, Object> sink) {
  Runnable lambda = () -> sink.accept(this, Nesting.this);
  Runnable anon = new Runnable() {
    @Override public void run() {
      sink.accept(this, Nesting.this); // this is the anonymous instance
    }
  };
  lambda.run();
  anon.run();
}
```

listing 66: the same two captures, lambda and anonymous class (source: java/samples/src/Ch07/Nesting.java, lines 51-60)

```
    boolean lambdaThis = selves.get(0) == outer && outers.get(0) == outer;
    boolean anonThis = selves.get(1) != outer && outers.get(1) == outer;
    ok(lambdaThis, "inside a lambda, this is the enclosing instance");
    ok(anonThis, "inside an anonymous class, this is the new instance");
    ok(!selves.get(0).getClass().isAnonymousClass()
        && selves.get(1).getClass().isAnonymousClass()
        && selves.get(1).getClass().getSimpleName().isEmpty(),
        "reflection names the anonymous class, the lambda is no class at all");
```

listing 67: this means different objects, measured (source: java/samples/src/Ch07/Nesting.java, lines 83-90)

Inside the lambda, `this` is the enclosing instance, the lambda has no object of its own. Inside the anonymous class, `this` is the fresh instance and the outer one needs qualification. Reflection tells the same story, the anonymous class has an empty simple name and `isAnonymousClass()` is true, while the lambda's class is a runtime artifact with no source-level identity at all. Chapter 8 opens with how lambdas actually compile.

NOTE: when to reach for which nest

Static nested type when the pairing is namespace only. Inner class when each helper instance genuinely belongs to one outer instance, iterators are the canonical case. Local class when a block needs a named helper with state or multiple instances. Anonymous class only when you need fields or several methods on a throwaway type. Lambda or method reference for everything single-method, chapter 8.

sources: Java in a Nutshell 8th edition, chapter 4, enums, annotations, lambda expressions, and nested types, pages 175 to 198, and its appendix on the post-17 arc. `@Repeatable` semantics read from docs.oracle.com java 27 api, accessed 2026-10-04, nestmate facts from openjdk.org/jeps/181, checked 2026-10-04. Every behavioral claim is asserted by `java/samples/src/Ch07`, 26 checks green under `pwsh -NoProfile -File tools/run-java-samples.ps1 -Chapter Ch07` on the pinned build 27+35-2325, measured 2026-10-04.

8. functional java

Java 8 (JSR 335) made functions values: lambdas, method references, and a function package of target types. The design kept java nominal, there is no function type in the language, only interfaces with one abstract method, and the compile story is unlike any other feature's: lambdas do not become anonymous classes, they become `invokedynamic` calls into `LambdaMetafactory`, which materializes a lightweight class at first use. Everything in this chapter measures that story on the pinned build.

8.1. the function package

The target types live in `java.util.function`, and the census is a fact, not a guess: 43 top-level interfaces on jdk 27, counted live from the jrt filesystem, every one carrying `@FunctionalInterface` and exactly one abstract method:

```
static int censusTopLevel() throws Exception {
  var fs = FileSystems.getFileSystem(java.net.URI.create("jrt:/"));
  int total = 0;
  try (var stream = Files.newDirectoryStream(
      fs.getPath("/modules/java.base/java/util/function"), "*.class")) {
    for (var p : stream) {
      if (!p.getFileName().toString().contains("$")) {
        total++;
      }
    }
  }
  return total;
}
```

listing 68: the census, walked straight out of the runtime image (source: java/samples/src/Ch08/Inventory.java, lines 25-37)

Four shapes cover most code: `Function<T, R>` transforms, `Predicate<T>` tests, `Supplier<T>` produces, `Consumer<T>` consumes. `BiFunction` takes two, `UnaryOperator` and `BinaryOperator` stay in one type, and the primitive specializations, `IntFunction`, `ToLongFunction`, `IntPredicate` and friends, exist because type parameters cannot hold primitives, chapter 6. The shapes compose:

```
  Function<Integer, Integer> doubled = n -> n * 2;
  Function<Integer, Integer> plus1 = n -> n + 1;
  ok(doubled.andThen(plus1).apply(5) == 11 && doubled.compose(plus1).apply(5) == 12,
      "andThen runs left to right, compose right to left");
  Predicate<String> shortWord = w -> w.length() < 5;
  ok(shortWord.negate().test("alpha") && shortWord.or(w -> w.startsWith("a")).test("aqua"),
      "predicates compose with negate, and, or");
  ok(Function.identity().apply("same").equals("same")
```

listing 69: andThen, compose, negate, and the identities (source: java/samples/src/Ch08/Inventory.java, lines 59-66)

`andThen` reads left to right, `compose` right to left, and `Function.identity` is the unit of composition. A functional interface is still an interface: default methods may pile up, exactly one method may stay abstract:

```
static int censusSam() {
  int sam = 0;
  for (Class<?> c : new Class<?>[] {Function.class, BiFunction.class,
        Predicate.class, Supplier.class, UnaryOperator.class}) {
    long abstracts = java.util.Arrays.stream(c.getMethods())
        .filter(m -> Modifier.isAbstract(m.getModifiers()))
        .filter(m -> m.getDeclaringClass() != Object.class)
        .count();
    if (abstracts == 1) {
      sam++;
    }
  }
  return sam;
}
```

listing 70: the sam rule checked by reflection over the core shapes (source: java/samples/src/Ch08/Inventory.java, lines 39-52)

`@FunctionalInterface` is optional documentation with teeth, the compiler rejects a second abstract method on an annotated type, and chapter 7 measured it as `RUNTIME` retention.

8.2. the lambda compile story

A lambda is a function literal with no name of its own, `(p, q) -> body`, converted to whatever functional interface the context expects. The conversion is target typing, the compiler reads the assignment or parameter position to know what to build. What it builds is the part worth knowing, and the sample reads it back through reflection:

```
public static void main(String[] args) {
  Runnable lambda = () -> System.out.print("");
  Runnable anon = new Runnable() {
    @Override public void run() {}
  };
  String lambdaClass = lambda.getClass().getName();
  String anonClass = anon.getClass().getName();
  ok(lambdaClass.contains("$$Lambda") && lambdaClass.startsWith("Lambdas"),
      "the lambda compiles to a runtime class, " + lambdaClass);
  ok(anonClass.equals("Lambdas$1") && anon.getClass().isAnonymousClass()
      && !lambda.getClass().isAnonymousClass(),
      "the anonymous class is Lambdas$1, a class file with a name slot");
  ok(lambda.getClass().isSynthetic() && !anon.getClass().isSynthetic(),
      "the lambda class is synthetic, generated at first use");
```

listing 71: one lambda and one anonymous class, named by the runtime (source: java/samples/src/Ch08/Lambdas.java, lines 29-42)

The anonymous class is `Lambdas$1`, a real class file with a name slot, emitted by `javac` per source occurrence. The lambda's class is `Lambdas$$Lambda/0x...`, synthetic, generated by the metafactory at first use, and its name carries a runtime identity because there is no source-level one. C#'s lambdas compile to delegates, a first-class runtime function type, book 6, ch 7 (c# 15, f# 11, and .net 11, delegates, lambdas, events) walks that design. Java chose dynamic linkage instead, the same machinery the string concat and later record bootstrap reuse, and it is why a lambda costs nothing until called and why `this` inside one means the enclosing object, chapter 7 measured that split.

8.3. method references

A method reference is a lambda with the body borrowed from an existing method, `String::length` for `s -> s.length()`. Four kinds, by what is fixed:

```
    // unbound: the receiver arrives as the first lambda parameter
    Function<String, Integer> length = String::length;
    ok(length.apply("lambda") == 6, "unbound reference: instance method, receiver as parameter");

    // bound: the receiver is captured inside the reference
    String prefix = "re-";
    Function<String, String> rebound = prefix::concat;
    ok(rebound.apply("run").equals("re-run"), "bound reference: receiver captured at creation");

    // static: no receiver at all
    Function<String, String> loud = MethodRefs::shout;
    ok(loud.apply("hey").equals("HEY"), "static reference: class method, arguments only");

    // constructor: the new expression as a function
    Supplier<ArrayList<String>> fresh = ArrayList::new;
    IntFunction<String[]> cells = String[]::new;
    ok(fresh.get().isEmpty() && cells.apply(4).length == 4,
        "constructor references: no-arg and array forms");
```

listing 72: unbound, bound, static, constructor, plus the array constructor (source: java/samples/src/Ch08/MethodRefs.java, lines 26-43)

The unbound kind has the one rule that bites: the receiver must arrive as the first parameter of the functional shape. Measured on the pinned build, `Thread::threadId` against `Supplier<Long>` fails compilation with `unexpected instance method threadId() found in unbound lookup`, because a supplier has no parameter to be the receiver. The same reference against `ToLongFunction<Thread>` compiles cleanly:

```
    // the lookup rule: an unbound reference needs the receiver type as
    // the first parameter of the functional shape. Thread::threadId
    // against Supplier<Long> fails: measured 2026-10-04, the error is
    // "unexpected instance method threadId() found in unbound lookup".
    // against ToLongFunction<Thread> it compiles, the receiver slot exists
    java.util.function.ToLongFunction<Thread> tid = Thread::threadId;
    ok(tid.applyAsLong(Thread.currentThread()) > 0,
        "unbound Thread::threadId fits a shape whose first parameter is Thread");
```

listing 73: the shape supplies the receiver slot, and the reference fits (source: java/samples/src/Ch08/MethodRefs.java, lines 50-57)

```
// measured 2026-10-04 on build 27+35-2325, refuses to compile:
Supplier<Long> tid = Thread::threadId;
// error: incompatible types: invalid method reference
//   unexpected instance method threadId() found in unbound lookup
```

fragment, illustrative, not executed

8.4. capture and the effectively final rule

A lambda body may read local variables of the enclosing scope, but only ones it can treat as constants: captured locals must be final or effectively final, assigned once, because the capture is a copy of the value, and a mutable copy would diverge from the variable silently. The compile error on a reassigned capture is the language refusing that divergence. The sample builds closures that outlive their scope, the classic loop:

```
List<IntSupplier> snapshots = new ArrayList<>();
for (int i = 0; i < 3; i++) {
  snapshots.add(CLOSURE.snap(i));
}
int sum = 0;
for (IntSupplier s : snapshots) {
  sum += s.getAsInt();
}
ok(snapshots.get(0).getAsInt() == 0 && snapshots.get(2).getAsInt() == 2 && sum == 3,
    "three closures hold three copies of one loop variable, outliving the loop");
```

listing 74: three closures over three values of one loop variable (source: java/samples/src/Ch08/Lambdas.java, lines 44-53)

Each snapshot holds its own copy, and the copies stay correct after the loop is gone. The copy is of the variable's value, so for a reference that means the reference is frozen while the object it points at remains mutable:

```
int before = 41;
IntSupplier read = () -> before; // copies the value of before
ok(read.getAsInt() == 41, "capture reads the value at lambda creation");
int[] cell = {10};
IntSupplier through = () -> cell[0]; // copies the reference, not the array
cell[0] = 20; // mutating the object is legal, only the variable is frozen
ok(through.getAsInt() == 20,
    "capture copies the reference: object contents stay visible");
```

listing 75: the array reference frozen, its contents live (source: java/samples/src/Ch08/Lambdas.java, lines 55-62)

That asymmetry is the daily shape of java closures: state you can close over is state you can only reach through, never reassign. A mutable counter needs a holder, an array as here or an `AtomicInteger` in real code, chapter 10 owns the thread story.

8.5. Optional as a return contract

`Optional<T>` is a container that may be empty, designed for one narrow job: return types, so an absent result is a value instead of a null test. The core discipline is to never call `get` on a maybe empty, and the api gives better tools for every case:

```
    defaultCalls++;
    return "default";
  }

  public static void main(String[] args) {
    ok(find(7).isPresent() && find(7).get().equals("item-7"),
        "of wraps a present value, get reads it back");
    ok(find(0).isEmpty(), "empty is a real value, not null, since java 11");
    ok(find(-1).orElse("fallback").equals("fallback"),
        "orElse supplies the absent branch");
    ok(find(5).map(s -> s.length()).orElse(0) == 6,
        "map transforms the inside, staying optional");
    ok(find(0).map(s -> s.length()).orElse(-1) == -1,
        "map on empty stays empty");
    defaultCalls = 0;
    String eager = find(5).orElse(expensiveDefault());
    int afterEager = defaultCalls;
    String lazy = find(5).orElseGet(() -> expensiveDefault());
    int afterLazy = defaultCalls;
    ok(afterEager == 1 && afterLazy == 1,
        "orElse ran the default anyway, orElseGet did not, measured");
    try {
      String s = find(0).get();
```

listing 76: of, empty, map, orElse, orElseGet, and the two throws (source: java/samples/src/Ch08/Optionals.java, lines 24-46)

The laziness difference is measurable: `orElse` computes its argument eagerly, present or not, `orElseGet` only on absence. In the run the eager default fired once with a present value, the lazy one not at all. `of(null)` throws immediately, `ofNullable` guards the nullable boundary, and `get` on empty throws `NoSuchElementException`, the runtime's answer to skipping the contract:

```
      ok(false, "get on empty throws: " + s);
    } catch (java.util.NoSuchElementException e) {
      ok(true, "get on empty throws NoSuchElementException");
    }
    try {
      Optional.of(null);
      ok(false, "of(null) throws");
    } catch (NullPointerException e) {
      ok(true, "of(null) throws NPE, ofNullable does not");
    }
    ok(Optional.ofNullable(null).isEmpty()
        && Optional.ofNullable("x").isPresent(),
        "ofNullable is the boundary guard for nullable inputs");
    try {
      find(0).orElseThrow(IllegalStateException::new);
      ok(false, "orElseThrow with a supplier customizes the failure");
    } catch (IllegalStateException e) {
      ok(true, "orElseThrow with a supplier customizes the failure");
```

listing 77: the failure paths, measured rather than asserted from memory (source: java/samples/src/Ch08/Optionals.java, lines 47-64)

NOTE: optional is a return type

Fields, parameters, and collection elements should not be optional. An `Optional` field holds a reference to another object for no information gain, and `Optional.get` in a chain is just a null test with more steps. Return it, consume it immediately, never store it.

8.6. the stream pipeline

Streams compose the function shapes over data. A pipeline is a source, zero or more lazy intermediate operations, and one terminal operation that pulls everything through:

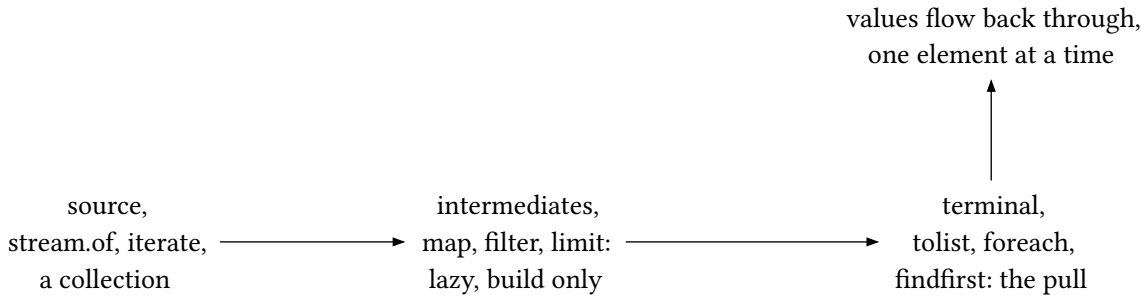


figure 13: one pipeline, one pull

The laziness is not a claim, the sample traces it. Building the pipeline runs nothing. The terminal call drives every stage, and each element traverses the whole pipeline before the next one starts, which is why a `filter` cannot be talked about before or after ordering the way sql clauses can:

```
    Stream<String> built = Stream.of("a", "b", "c")
        .map(s -> seen("map1:" + s, s.toUpperCase()))
        .filter(s -> {
          trace.add("filter:" + s);
          return !s.equals("B");
        })
        .map(s -> seen("map2:" + s, s + "!"));
    ok(trace.isEmpty(), "building the pipeline runs nothing");
    List<String> out = built.toList();
    ok(out.equals(List.of("A!", "C!")),
        "the terminal op drives the whole pipeline");
    ok(trace.equals(List.of("map1:a", "filter:A", "map2:A",
          "map1:b", "filter:B", "map1:c", "filter:C", "map2:C")),
        "elements flow one at a time, each through all stages");
```

listing 78: the empty trace before the terminal, the per-element interleave after (source: java/samples/src/Ch08/Lazy.java, lines 27-40)

Short-circuiting terminals stop the pull early, `findFirst` leaves the third element unmapped, and `limit` terminates an infinite source, the sample builds the first three even numbers from `Stream.iterate`:

```
    trace.clear();
    String first = Stream.of("x", "y", "z")
        .map(s -> seen("m:" + s, s))
        .filter(s -> s.equals("y"))
        .findFirst()
        .orElseThrow();
    ok(first.equals("y") && trace.equals(List.of("m:x", "m:y")),
        "findFirst short-circuits, z never mapped");

    List<Integer> evens = Stream.iterate(1, i -> i + 1)
        .filter(n -> n % 2 == 0)
        .limit(3)
        .toList();
    ok(evens.equals(List.of(2, 4, 6)),
        "limit over an infinite source stops the pull");
```

listing 79: findFirst stops the pull, limit caps an infinite source (source: java/samples/src/Ch08/Lazy.java, lines 42-56)

Chapter 12 owns streams in full, the collectors, the parallel variants, and the java 24 gatherers. What belongs here is the composition model: a stream is a description of work that never runs until asked, and an unused pipeline is garbage, not computation.

sources: Java in a Nutshell 8th edition, chapter 4 lambda expressions and chapter 8 streams, pages 180 to 187 and 306 to 317. JEP and release spine from docs/audit/java.md, checked 2026-10-04. Every behavioral claim is asserted by `java/samples/src/Ch08`, 34 checks green under `pwsh -NoProfile -File tools/run-java-samples.ps1 -Chapter Ch08` on the pinned build 27+35-2325, measured 2026-10-04.

9. pattern matching

Pattern matching is the longest arc project amber has run: instanceof patterns finalized in java 16, record patterns and switch patterns in 21, unnamed variables and patterns in 22, and primitive patterns still in preview, the fifth round in java 27. The destination is algebraic data types java style: sealed interfaces as the sum, records as the product, and a switch that checks its own coverage against the permits list. Chapter 2 walked the ladder, this chapter works the machinery.

9.1. instanceof patterns

The java 8 idiom was three steps, test, cast, bind, and the cast could fail if the test and the cast ever drifted apart. The pattern does all three in one expression, and the compiler scopes the binding to exactly the places where the test succeeded:

```
static String describe(Object o) {
  // test and bind in one step, s is scoped to the true branch
  if (o instanceof String s) {
    return "string of length " + s.length();
  }
  return "something else";
}
```

listing 80: test and bind in one step, s scoped to the branch (source: java/samples/src/Ch09/InstanceofPatterns.java, lines 15-21)

Flow scoping is the interesting part. The binding is live where the pattern is guaranteed true, dead where it is false, which lets it join boolean chains and survive negation without any cast anywhere:

```
static int guardedLen(Object o) {
  // the binding stays in scope for the condition that follows it
  return o instanceof String s && !s.isEmpty() ? s.length() : -1;
}

static int negationScoped(Object o) {
  if (!(o instanceof Integer n)) {
    return 0;
  }
  return n; // in scope after the failed negation, no cast anywhere
}
```

listing 81: the binding in a condition chain, and after a failed negation (source: java/samples/src/Ch09/InstanceofPatterns.java, lines 23-33)

One rule never moved: no pattern matches `null`, not even a type pattern on `Object`, so a pattern test doubles as a null test. The sample runs `null` through all three helpers to pin it.

9.2. record patterns

A record pattern deconstructs a record into its components, matching when the value is an instance of the record type and each component pattern matches. Nesting goes as deep as the data:

```
static int manhattan(Object o) {
  if (o instanceof Line(Point(var x1, var y1), Point(var x2, var y2))) {
    return Math.abs(x1 - x2) + Math.abs(y1 - y2);
  }
  return -1;
}
```

listing 82: two levels of deconstruction in one test (source: java/samples/src/Ch09/RecordPatterns.java, lines 22-27)

Component patterns may use `var`, inferring each component's declared type, which keeps the spelling short without giving up the check. In switch labels the same shapes appear after `case`:

```
Object tagged = new Tagged(1, "alpha");
String read = switch (tagged) {
  case Tagged(Integer id, String label) -> id + ":" + label;
  default -> "other";
};
ok(read.equals("1:alpha"), "record patterns in switch, full spelling");
String terse = switch (tagged) {
  case Tagged(var id, var label) -> "var " + id + label;
  default -> "other";
};
ok(terse.equals("var 1alpha"), "var spelling infers each component");
```

listing 83: full spelling and var spelling of the same two cases (source: java/samples/src/Ch09/RecordPatterns.java, lines 37-47)

One sharp edge is worth measuring because it looks like a spec bug until you read it as an optimization. A component pattern that is unconditional for the component's declared type, `String label` against a `String` component, has its runtime check elided, so it matches even a null component. A narrowing component pattern keeps the check and rejects the null:

```
Object holed = new Tagged(2, null);
String holedRead = switch (holed) {
  case Tagged(Integer id, String label) -> "label is " + label;
  default -> "rejected";
};
ok(holedRead.equals("label is null"),
    "an unconditional component pattern matches a null component, measured");
boolean nullIsInteger = null instanceof Integer i;
ok(!nullIsInteger,
    "while null instanceof Integer stays false, the edge is component-local");
```

listing 84: the unconditional component matched null, the bare type test does not (source: java/samples/src/Ch09/RecordPatterns.java, lines 51-60)

The rule is component-local and symmetric with chapter 6's erasure story: the compiler only skips checks it can prove, and `null` is assignable to every reference type.

9.3. switch patterns and exhaustiveness

Sealed types since 17 (JEP 409) say who may implement them, and switch patterns since 21 (JEP 441) read that list. Together the compiler can prove coverage, and a switch over a sealed type needs no `default`:

```
sealed interface Shape permits Circle, Rect {}

record Circle(double radius) implements Shape {}

record Rect(double w, double h) implements Shape {}

// exhaustive over the permits list, no default needed
static String kind(Shape s) {
  return switch (s) {
    case Circle c -> "circle";
    case Rect r -> "rect";
  };
}
```

listing 85: sealed interface, records, one switch, no default (source: java/samples/src/Ch09/SwitchPatterns.java, lines 15-27)

The proof is the permits list, and the compiler walks it as a decision procedure:



figure 14: exhaustiveness checking over a sealed tree

The error is precise. Add a `Triangle` to the permits list and the sample's two-case switch stops compiling with `the switch expression does not cover all possible input values`, and the missing-patterns note names it `Triangle _`, the unnamed pattern from java 22 already living in the compiler's own diagnostics. Exhaustiveness has teeth only over sealed types, an open hierarchy still needs a `default`, because anyone can add a subtype anywhere.

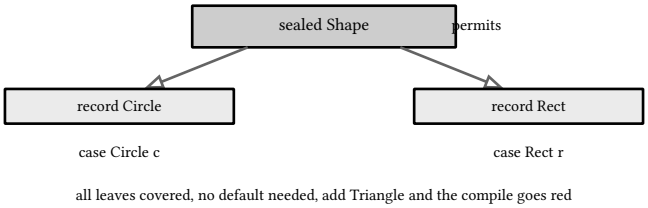


figure 15: the tree the switch is checked against

9.4. guards, null, dominance

A `when` clause guards a pattern with a boolean the compiler treats as part of the label. A guarded case does not count toward exhaustiveness by itself, it can fail, so it must pair with an unguarded case for the same type:

```
static double area(Shape s) {
  return switch (s) {
    case Circle c when c.radius() > 1 -> Math.PI * c.radius() * c.radius();
    case Circle c -> c.radius() * c.radius(); // small circles, squared
    case Rect(var w, var h) -> w * h; // deconstruction inside a case
  };
}
```

listing 86: a guarded and an unguarded case for one type, deconstruction in a third (source: java/samples/src/Ch09/SwitchPatterns.java, lines 29-35)

Null handling changed shape in 21. A null selector used to throw before any label ran, it still does when no `case null` exists, but `case null` makes absence an ordinary label. The sample measures both sides:

```
static String nullSafe(Shape s) {
  return switch (s) {
    case null -> "absent";
    case Circle _ -> "some circle"; // unnamed pattern, java 22
    case Rect _ -> "some rect";
  };
}
```

listing 87: case null first, unnamed patterns after (source: java/samples/src/Ch09/SwitchPatterns.java, lines 37-43)

Dominance rules keep the order honest: a label that covers everything an earlier label covers is an error, `case Object o` before `case Circle c` fails with `this case label is dominated by a preceding case label`, measured. A guarded case may precede its unguarded twin, that ordering is the idiom, and the sample relies on it:

```
  String shaped = switch (new Rect(2, 2)) {
    case Rect r when r.w() == r.h() -> "square";
    case Rect r -> "rect";
  };
  ok(shaped.equals("square"), "a guarded case may precede its unguarded twin");
```

listing 88: guarded first, unguarded fallback, measured order (source: java/samples/src/Ch09/SwitchPatterns.java, lines 77-81)

9.5. unnamed variables and patterns

Java 22 (JEP 456, after a 21 preview) finished the underscore's journey. _ was a legal identifier in java 1, drew a warning in 8, became an error in 9 (JEP 213), and now means unnamed: bind nothing, match anything. In a type pattern it discards the binding while keeping the type test, and as a variable it marks a value the reader should not track:

```
  // unnamed variables beyond patterns: loop, catch, assignment
  int seen = 0;
  for (var _ : java.util.List.of(1, 2, 3)) {
    seen++;
  }
  try {
    throw new IllegalStateException("ignored");
  } catch (IllegalStateException _) {
    seen += 10;
  }
  var _ = "result discarded";
  ok(seen == 13, "unnamed variables in for, catch, and assignment");
```

listing 89: unnamed variables in for, catch, and assignment (source: java/samples/src/Ch09/SwitchPatterns.java, lines 62-73)

```
// java 22, one underscore, four uses
case Circle _ -> "some circle";   // unnamed pattern
for (var _ : list) count++;       // unnamed variable
try { read(); } catch (IOException _) { log(); }  // unnamed parameter
var _ = sideEffectOnly();         // discarded result
```

fragment, illustrative, not executed

The rule that makes it safe: an unnamed variable may never be read, so it cannot be confused with a forgotten binding. Longer names keep their underscores, `_count` and `MAX_AGE` are untouched, and `_` as a digit separator in literals is a different token from 7 on.

9.6. what is still preview

Primitive types in patterns, instanceof and switch, remain preview in java 27, the fifth round (JEP 532), unchanged from the java 26 revision. The feature lets `int i` and `double d` patterns match primitives with widening conversions, `case int i when i > 0`, and it is the piece that eventually lets records hold primitives without boxing in pattern position. This book's samples do not use it, the runner compiles without `--enable-preview` by design.

NOTE: why the arc matters

Records gave java cheap products, sealed types gave it closed sums, and pattern matching gave the two a consumer. The result reads like the ml family: one declaration for the data, one switch for every case, and a compiler that notices when the two drift apart. Chapter 5 and chapter 6 built the halves, this chapter is the join.

sources: openjdk.org/jeps/305, /jeps/394, /jeps/409, /jeps/440, /jeps/441, /jeps/456, /jeps/532, and /jeps/213, statuses, preview chains, and release attributions checked 2026-10-04. The null-component rule read against JEP 440 and verified by measurement. Every behavioral claim is asserted by `java/samples/src/Ch09`, 19 checks green under `pwsh -NoProfile -File tools/run-java-samples.ps1 -Chapter Ch09` on the pinned build 27+35-2325, measured 2026-10-04.

10. concurrency: threads to virtual threads

Java shipped threads in 1.0, before any of its peers made them central. The platform rode that head start for 25 years on one model: every `Thread` is an operating system thread, 1:1, with a fixed megabyte-scale stack. Java 21 broke the mapping. Virtual threads are managed by the jvm, cost a fraction of a platform thread, and turn thread-per-request back into the sane default. This chapter walks the whole ladder, from the raw thread api through monitors and the memory model to the virtual era and the scoped-value replacement for `ThreadLocal`. book 12, ch 6 (design patterns, concurrency, and distributed systems, concurrency fundamentals and the memory model) owns the deep treatment of races, memory models, and detectors, this chapter covers the java surfaces.

10.1. the platform thread model

`Thread t = new Thread(runnable); t.start();` delegates straight to the os: the thread gets its own stack, its own program counter, and the os scheduler decides when it runs. Every java program is already multithreaded before `main` starts, the vm runs its own threads for gc and jit work alongside the main thread. Threads share one heap and see each other's objects by default, which is exactly why the rest of this chapter exists.

`Thread.State` wraps the os view into six values: new, runnable, blocked, waiting, timed waiting, terminated. Blocked means waiting for a monitor, the state a sample proves below. The methods that matter: `start()` schedules the thread, `join()` waits it out, `interrupt()` asks it to stop, `isDaemon()` marks threads that never keep the jvm alive. `stop()`, `suspend()`, and `resume()` are the deprecated failures of the original api, `stop()` can kill a thread mid-mutation and leave objects in illegal states, they must never appear in code.

10.2. synchronized and the monitor story

Every object carries a monitor. `synchronized` on a method or block acquires it, and the acquire is exclusive: a second thread blocks until the holder releases. The point is never the code inside the block, it is the state the code can temporarily make inconsistent. Acquiring the monitor does not fence the object off, an unsynchronized method can still read a half-updated field while the monitor is held. Synchronization is cooperative, all the mutating and reading paths must take the same monitor, one missing keyword breaks the whole contract.

```
static class Counter {
  private long i = 0;

  public synchronized void increment() {
    i = i + 1;
  }

  // reentrancy on display: increment() takes the same monitor this
  // method already holds, a non-reentrant lock would deadlock here
  public synchronized void incrementTwice() {
    increment();
    increment();
  }

  public synchronized long get() {
    return i;
  }
}
```

listing 90: a counter guarded by its own monitor, and a reentrancy proof (source: java/samples/src/Ch10/Monitors.java, lines 15-33)

Two threads run 100,000 increments each through `increment()` and the total is exactly 200,000, every time. Without `synchronized` the read-modify-write of `i = i + 1` interleaves and loses updates, the lost update anomaly.

`incrementTwice()` shows reentrancy: java locks are reentrant, a thread holding a monitor re-enters its synchronized regions freely, a non-reentrant lock would deadlock right there.

`wait()` and `notifyAll()` complete the monitor story. They live on `Object` and must run inside a synchronized region: `wait()` releases the monitor and parks the thread, `notifyAll()` wakes everyone parked on that monitor, and the woken thread re-acquires before continuing.

```
static class Slot {
  private String item = null;

  public synchronized void put(String value) throws InterruptedException {
    while (item ≠ null) {
      wait();
    }
    item = value;
    notifyAll();
  }

  public synchronized String take() throws InterruptedException {
    while (item == null) {
      wait();
    }
    String value = item;
    item = null;
    notifyAll();
    return value;
  }
}
```

listing 91: a one-slot buffer, the wait and notify handshake (source: java/samples/src/Ch10/Monitors.java, lines 36-56)

The guarded `while` loop around `wait()` is not decoration, a wakeup does not guarantee the condition holds again. The sample drives 1,000 productions and 1,000 consumptions through one slot and both sides finish exactly. Real code reaches for `ArrayBlockingQueue` instead, but every developer should be able to read this shape, it is what the queue does underneath.

10.3. volatile and happens-before

Monitors fix both exclusion and visibility. `volatile` buys only the second half. The java memory model lets each thread cache field values in registers and per-core caches, so a plain `boolean running` flag may never be seen as false by the thread spinning on it. A `volatile` write pushes the value to main memory and a volatile read pulls it back, and more precisely the write and the read form a happens-before edge: everything the writer did before the volatile write is visible to the reader after the volatile read. Monitor release and acquire, thread start and join, all create the same edges.

```
private volatile boolean shutdown = false;

public void shutdown() {
  shutdown = true;
}

public void run() {
  while (!shutdown) {
    // ... process another task
```

```
    }
  }
```

fragment, illustrative, not executed

The run-until-shutdown pattern, the textbook volatile use. The keyword carries no exclusion at all: a volatile counter still loses updates under `i = i + 1`, because the read-modify-write is three steps and volatile only orders the individual reads and writes. `AtomicLong` or a monitor is the fix. The full happens-before algebra and its counterexamples are book 12, ch 6 (design patterns, concurrency, and distributed systems, concurrency fundamentals and the memory model)'s subject, java's rules are one instance of that general story.

10.4. executors and CompletableFuture

Raw threads have no result and no pool. `java.util.concurrent`, in the jdk since 5, added the runtime-managed layer: an `ExecutorService` owns the threads, code submits work. A fixed pool of platform threads is the classic shape, and since 19 an executor is `AutoCloseable`, try-with-resources waits for submitted work to finish.

`CompletableFuture`, since 8, layers composition on top: stages that map, combine, and recover, with exceptions flowing through the pipeline instead of breaking it.

```
    var left = java.util.concurrent.CompletableFuture.supplyAsync(() -> 20, exec);
    var right = java.util.concurrent.CompletableFuture.supplyAsync(() -> 22, exec);
    int sum = left.thenCombine(right, Integer::sum).join();
    ok(sum == 42, "thenCombine joins two async results");
    String text = java.util.concurrent.CompletableFuture.supplyAsync(() -> "a", exec)
      .thenApply(s -> s + "b")
      .thenApply(s -> s + "c")
      .join();
    ok("abc".equals(text), "thenApply chains map over the eventual value");
    // exceptional pipelines recover without the caller seeing the throw
    int recovered = java.util.concurrent.CompletableFuture
      .<Integer>failedFuture(new IllegalStateException("boom"))
      .exceptionally(e -> -1)
      .join();
    ok(recovered == -1, "exceptionally recovers a failed stage");
    // allOf fans in: every future must complete before the join returns
    var parts = new java.util.ArrayList<java.util.concurrent.CompletableFuture<Integer>>();
    for (int i = 0; i < 4; i++) {
      int idx = i;
      parts.add(java.util.concurrent.CompletableFuture.supplyAsync(() -> idx, exec));
    }
    java.util.concurrent.CompletableFuture
      .allOf(parts.toArray(new java.util.concurrent.CompletableFuture<?>[0])).join();
    int total = 0;
    for (var p : parts) {
      total += p.join();
    }
    ok(total == 6, "allOf waits for every stage, results read separately");
  }
}
```

listing 92: pipelines over a virtual-thread-per-task executor (source: java/samples/src/Ch10/Async.java, lines 24-53)

`supplyAsync` starts a stage on the given executor, `thenApply` maps the eventual value, `thenCombine` joins two stages, `exceptionally` recovers a failed one, and `allOf` fans in. The executor passed here is the 19 addition

`newVirtualThreadPerTaskExecutor`, every task gets its own virtual thread and the pool never needs sizing, which is the bridge to the centerpiece.

10.5. ThreadLocal and its costs

`ThreadLocal`, since 1.2, gives each thread a private slot: the per-thread `SimpleDateFormat`, the per-transaction context. The cost profile was fine when a program ran tens of threads. It breaks on tens of thousands. Each thread carries a mutable map of bindings, the values live as long as the thread, and pooled platform threads glue one task's leftovers onto the next task's run. On virtual threads the jep 444 guidance is blunt: thread locals only after real thought, never to pool costly objects, since virtual threads are plentiful and short-lived, not pooled. The jdk itself stripped thread-local uses out of `java.base` to shrink per-thread footprint, and `-Djdk.traceVirtualThreadLocals` flags the first write to a thread local in a virtual thread. The replacement is scoped values, the last section of this chapter.

10.6. virtual threads

Previewed in 19 and 20 (jeps 425, 436), final in 21 (jep 444). A virtual thread is a `java.lang.Thread` whose stack lives on the heap in resizable stack chunks and whose body runs on a carrier platform thread from a fork join pool, by default one pool sized to the core count. Blocking unmounts the virtual thread, frees the carrier, and remounts it when the operation completes. The api is deliberately the same: `Thread.ofVirtual()`, or the executor above, and everything written against `Thread` keeps working.

```
  public static void main(String[] args) throws Exception {
    int n = 10_000;
    var done = new java.util.concurrent.atomic.AtomicInteger();
    var threads = new Thread[n];
    long start = System.nanoTime();
    for (int i = 0; i < n; i++) {
      threads[i] = Thread.ofVirtual().unstarted(done::incrementAndGet);
    }
    for (Thread t : threads) {
      t.start();
    }
    for (Thread t : threads) {
      t.join();
    }
    long elapsedMs = (System.nanoTime() - start) / 1_000_000;
    ok(done.get() == n, "10,000 virtual threads all ran to completion");
    ok(threads[0].isVirtual(), "a virtual thread reports isVirtual");
    // thread ids are unique and positive, threadId() since 19
    var ids = new java.util.HashSet<Long>();
    for (int i = 0; i < 100; i++) {
      ids.add(threads[i].threadId());
    }
    ok(ids.size() == 100 && threads[0].threadId() > 0, "virtual thread ids are unique and positive");
    // states: a joined thread is terminated, a fresh one is new
    ok(threads[0].getState() == Thread.State.TERMINATED, "a joined virtual thread is terminated");
    ok(Thread.ofVirtual().unstarted(() -> {}).getState() == Thread.State.NEW, "an unstarted virtual thread
is new");
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27: "
      + n + " virtual threads spawned, started, and joined in " + elapsedMs + " ms"
```

listing 93: ten thousand virtual threads, spawned, started, joined (source: java/samples/src/Ch10/Virtual.java, lines 14-41)

Measured 2026-10-04 on the pinned build, 20 reported cores: the whole spawn, start, and join of 10,000 virtual threads took 38 ms. Platform threads at a 1 mb default stack cannot stack 10,000 deep on a normal machine, virtual

threads at a few hundred bytes to start can, and order-of-magnitude more. `currentThread()` inside the task is the virtual thread itself, the carrier's identity is unavailable to it.

Identity is the one place the old instincts misfire. Virtual threads are cheap and plentiful, so caching per-thread resources keyed on identity is an anti-pattern, and the jep discourages identity-sensitive operations on them. `getId()` was deprecated in 19 because subclasses could override it, the final `threadId()` (since 19) is the replacement, and the sample confirms ids stay unique and positive.

10.6.1. pinning, before and after

The original jep 444 caveat: a virtual thread that blocks inside a `synchronized` method or block could not unmount, it pinned its carrier. One monitor held across a slow io call froze a whole carrier, and a fleet of them starved the pool. The standard mitigation was to swap `synchronized` for `java.util.concurrent.locks.ReentrantLock`, which never pinned. Jep 491 in 24 removed the limitation: monitors are now owned by the virtual thread, not the carrier, blocking on a monitor or in `Object.wait()` unmounts like any other blocking operation, and the `jdk.tracePinnedThreads` diagnostic was retired with the problem it tracked. What still pins on 27: native frames on the stack, blocking during class loading or in a class initializer, and jni or foreign-function calls that block inside native code.

```
  public static void main(String[] args) throws Exception {
    int n = 1_000;
    long sleepMs = 40;
    int cores = Runtime.getRuntime().availableProcessors();
    long syncMs = timedSynchronized(n, sleepMs);
    long lockMs = timedLock(n, sleepMs);
    // pinned carriers would serialize: ceil(n / cores) * sleepMs at minimum.
    // measured 2026-10-04 with 20 reported cores that floor is 2000 ms, so
    // a 1200 ms bound fails only if the carriers got pinned.
    long pinnedFloorMs = ((n + cores - 1) / cores) * sleepMs;
    ok(syncMs < pinnedFloorMs, "sleeping inside synchronized does not pin carriers (" + syncMs
      + " ms < " + pinnedFloorMs + " ms serialization floor)");
    ok(lockMs < pinnedFloorMs, "sleeping under ReentrantLock does not pin carriers (" + lockMs + " ms)");
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27: " + n
      + " virtual threads x " + sleepMs + " ms sleep inside synchronized: " + syncMs
      + " ms, inside ReentrantLock: " + lockMs + " ms, " + cores + " reported cores");
  }
}
```

listing 94: the pinning probe, measured against the serialization floor (source: java/samples/src/Ch10/Pinning.java, lines 61-79)

The math the check rides on: if sleeping inside `synchronized` still pinned, 1,000 sleepers of 40 ms each would serialize onto 20 carriers for a 2,000 ms floor. Measured 2026-10-04: 64 ms synchronized, 50 ms under `ReentrantLock`. Both sit far under the floor, monitors no longer pin, and the pre-24 `ReentrantLock` workaround is back to being just a lock with more features, try-lock and timed waits.

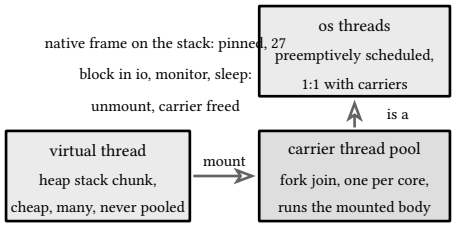


figure 16: a virtual thread mounts on a carrier, blocks, unmounts, remounts

10.7. structured concurrency, the shape of the future

Virtual threads make thread-per-task cheap, and the missing half is lifecycle: a parent that forks subtasks, waits for them as a unit, and cancels the rest on first failure. Structured concurrency is that api, still previewing, in its seventh preview in 27 (jep 533, incubated in 19, first previewed in 21).

```
try (var scope = StructuredTaskScope.open(Joiner.<String>anySuccessfulOrThrow())) {
  scope.fork(() -> fetchLeft());
  scope.fork(() -> fetchRight());
  // throws if both subtasks fail, cancels the slower one otherwise
  String firstDone = scope.join();
}
```

fragment, illustrative, not executed

The 27 preview makes `StructuredTaskScope` an interface with `open` factories and `Joiner` policies, adds a third type parameter for the exception `join()` can throw, and replaces `onTimeout()` with `timeout()`. It compiles only with `--enable-preview`, so it lives here as a snippet while the final sections use only shipped api. The pairing with scoped values is the design center: a scope's children are exactly the threads that inherit its bindings.

10.8. scoped values

Final in 25 (jep 506, previewed from 21). A `ScopedValue` is bound for the dynamic extent of a block, visible to everything the block calls, and gone when it exits. There is no `set` and no `remove`, so no leaked binding can outlive its scope, the failure mode `ThreadLocal` normalized.

```
public static void main(String[] args) throws Exception {
  ok(!USER.isBound(), "a scoped value is unbound outside any scope");
  ScopedValue.where(USER, "ada").run(() -> {
    try {
      ok("ada".equals(USER.get()), "the scope body reads the binding");
      // sharing is structured only: an unstructured executor child started
      // inside the scope does not inherit the binding, probed on 27
      try (var exec = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
        boolean bound = exec.submit(() -> USER.isBound()).get();
        ok(!bound, "an unstructured executor child does not inherit the binding");
      }
      // rebinding nests: inner scope wins, outer value restored after
      ScopedValue.where(USER, "grace").run(() -> {
        ok("grace".equals(USER.get()), "an inner scope rebinds");
      });
      ok("ada".equals(USER.get()), "leaving the inner scope restores the outer binding");
    } catch (Exception e) {
      throw new RuntimeException(e);
    }
  });
  ok(!USER.isBound(), "the binding is gone when the scope exits");
  ok("anon".equals(USER.orElse("anon")), "orElse supplies a default when unbound");
  boolean threw = false;
```

listing 95: bind, rebind, and the structured sharing rule (source: java/samples/src/Ch10/Scoped.java, lines 18-40)

Rebinding nests: the inner `where` wins and the outer value is restored when it exits. Sharing across threads is structured only, the javadoc's rule: bindings are captured by a `StructuredTaskScope` and inherited by its forked children, and the probed check above shows the inverse, an unstructured executor child sees no binding at all.

That is the deliberate design, scoped values ride structured concurrency, which is why the two features grew up together.

The guidance in one line: new code on virtual threads uses scoped values for context, leaves `ThreadLocal` to interop, never pools threads, and treats thread identity as none of its business.

sources: jep 444 (openjdk.org/jeps/444, thread locals, identity, carrier identity unavailable), jep 491 (openjdk.org/jeps/491, synchronized no longer pins, remaining native-frame pins, jdk.tracePinnedThreads removed), jep 506 scoped values final 25, jep 533 seventh preview 27 with the interface plus Joiner shape (the canonical open example read from the pinned jdk's own src.zip java.base/java/util/concurrent/StructuredTaskScope.java), jep 425 and 436 preview lineage, all accessed 2026-10-04. Thread.getId deprecated since 19 and threadId() final since 19 verified against the pinned build's src.zip java.base/java/lang/Thread.java, accessed 2026-10-04. The 8-to-17 arcs (thread lifecycle, monitors, wait and notify, volatile, the executor story) ground on Java in a Nutshell 8th edition chapter 6, pages 249 to 263. Measurements (10,000 joins in 38 ms, synchronized 64 ms against a 2,000 ms pinning floor, reentrant lock 50 ms, 20 cores, executor children do not inherit scoped bindings) produced by `java/samples/src/Ch10` under `pwsh tools/run-java-samples.ps1 -Chapter Ch10`, dated 2026-10-04 on tools/jdk27/build/jdk-27.

11. memory and runtime

Java removed the free call in 1995 and handed memory to a garbage collector. This chapter is what that trade buys: the heap's layout, the collectors that manage it, the object header shrink that landed as a default in 27, the flight recorder that watches all of it live, and the jit that makes the interpreter's bytecode fast. It ends with the measurement discipline, because every number in this chapter is either measured on this machine and dated, or documented, and the two never blur.

11.1. the heap and generations

Objects live on one shared heap, reached by tracing from the thread stacks and the other gc roots. The collector's foundational bet is the weak generational hypothesis: most objects die young. HotSpot splits the heap accordingly. New objects allocate in eden, each thread bumping a pointer through its own thread-local allocation buffer, which is nearly free. A young collection evacuates the few survivors into survivor spaces, and after surviving several cycles an object is promoted to the old generation. Evacuation never touches the dead majority, so young collections cost roughly the number of live objects, not the number allocated. When the collector must move objects that running threads can see, it stops the world at safepoints, points where every thread sits at a known state.

G1, the default everywhere in 27, reshapes that heap into regions, measured 4 MiB each on this machine. A region belongs to young, old, or free, and the collector picks the regions with the most reclaimable space first, which is the name. It compacts incrementally by evacuating whole regions, and it works toward a pause goal, `-XX:MaxGCPauseMillis=200` by default.

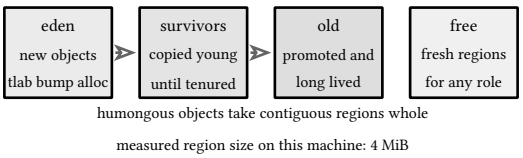


figure 17: the G1 heap: regions classified by role, evacuation moves survivors between them

11.2. the collectors

G1 became the default for server configurations in 9 (jep 248), and 27 finished the job: jep 523 makes G1 the default in every environment, the jvm no longer falls back to Serial on machines it deems constrained, a single cpu or under 1792 mb of memory. The sample reads the choice off the running vm, `UseG1GC` true with origin `ERGONOMIC`, the vm picked it, no flag was passed.

The rest of the shelf: Serial and Parallel remain available when passed explicitly, Parallel for throughput at the price of long full pauses. CMS was deprecated in 9 and removed in 14 (jep 363). Shenandoah, the Red Hat collector, evacuates concurrently with the application and hunts sub-millisecond pauses on large heaps, its generational mode arrived experimental in 24 (jep 404), became a product feature in 25 (jep 521) with `-XX:ShenandoahGCMode=generational`, and the flip of generational to the Shenandoah default is targeted for 28 (jep 535). ZGC went generational in 21 (jep 439) with the same young/old split under a concurrent collector.

11.3. compact object headers

Every object carries a header: a mark word with identity hash, lock state, and gc bits, plus a class pointer. The classic layout is 96 bits on 64-bit HotSpot with compressed class pointers, 12 bytes per object before a single field is stored. Jep 450 previewed a 64-bit header in 24, jep 519 made it a product feature behind `-XX:+UseCompactObjectHeaders` in 25, and jep 534 flipped the default in 27. The savings are per-object, so heaps full of small objects shrink proportionally.

```
    var rt = Runtime.getRuntime();
    var mem = java.lang.management.ManagementFactory.getMemoryMXBean();
    // collector and layout flags: the vm's own answer, not the docs'
    var diag = java.lang.management.ManagementFactory
      .getPlatformMXBean(com.sun.management.HotSpotDiagnosticMXBean.class);
    var g1 = diag.getVMOption("UseG1GC");
    var coh = diag.getVMOption("UseCompactObjectHeaders");
    var oops = diag.getVMOption("UseCompressedOops");
    ok(Boolean.parseBoolean(g1.getValue()), "G1 is the selected collector (origin " + g1.getOrigin() +
")");
    ok(Boolean.parseBoolean(coh.getValue()) && coh.getOrigin().toString().equals("DEFAULT"),
      "compact object headers are on as a default (origin " + coh.getOrigin() + ")");
    ok(Boolean.parseBoolean(oops.getValue()), "compressed oops are on (origin " + oops.getOrigin() + ")");
    // plain-object footprint: 2,000,000 Objects, retained, then counted.
```

listing 96: the running vm names its collector and header layout (source: java/samples/src/Ch11/Heap.java, lines 21-33)

Measured 2026-10-04 on the pinned build: `UseCompactObjectHeaders` reads true with origin `DEFAULT`, no flag was set, the 27 default at work. The footprint follows:

```
    // the identity hash read at the end defeats escape analysis, without it
    // the jit dead-code-eliminates the entire allocation loop
    final int n = 2_000_000;
    long keepAlive = 0;
    long per = -1;
    for (int round = 0; round < 3; round++) {
      System.gc();
      System.gc();
      Thread.sleep(50);
      long before = used();
      Object[] keep = new Object[n];
      for (int i = 0; i < n; i++) {
        keep[i] = new Object();
      }
      System.gc();
      System.gc();
      Thread.sleep(50);
      long delta = used() - before;
      // the array itself: 16 byte header + 4 byte compressed reference per slot
      per = (delta - (16L + 4L * n)) / n;
      keepAlive += System.identityHashCode(keep[round]);
      System.out.println("round " + round + ": heap delta " + delta + " bytes, "
        + per + " bytes per plain object");
    }
    ok(per == 8, "a plain object measures 8 bytes under the default layout");
```

listing 97: two million plain objects, measured to the byte (source: java/samples/src/Ch11/Heap.java, lines 34-58)

Three rounds agree: a plain object measures 8 bytes under the default layout, the 64-bit header and nothing else. The identity hash read after each measurement is load-bearing, without a later use the jit dead-code-eliminates the whole allocation loop and the deltas read as noise, a lesson this sample carries on purpose. Compressed oops (measured on, origin `ERGONOMIC`) keep references at 4 bytes and the sample's array arithmetic assumes them.

11.4. finalization is dead

`finalize()` ran cleanup when the collector got around to it, which is nowhere near a guarantee: no timing, no ordering, possibly never. Jep 421 terminally deprecated it in 18, and the mechanism is on the removal path.

Resources with a lexical scope go through try-with-resources, whose `close()` is guaranteed on the way out. Resources outliving any one scope register with a `Cleaner` (since 9), accepting that the clean runs at gc's discretion. Nothing new should ever override `finalize`.

11.5. java flight recorder

JFR is the jvm's built-in, low-overhead event recorder, open sourced with 11. It samples allocations, thread dumps, gc cycles, monitor contention, and method profiles into chunked recordings, cheap enough to leave running in production as a ring buffer so the data around an incident survives the incident. The usual loop runs it from outside: `jcmd <pid> JFR.start name=rec settings=profile duration=120s filename=rec.jfr`, then `jfr print` or the JDK Mission Control gui over the file. The same api works from inside:

```
public static void main(String[] args) throws Exception {
  var path = java.nio.file.Path.of(
    System.getProperty("java.io.tmpdir"), "ch11-jfr-sample.jfr");
  int allocationEvents;
  int total;
  try (var r = new jdk.jfr.Recording()) {
    ok(r.getState() == jdk.jfr.RecordingState.NEW, "a fresh recording is in the new state");
    r.enable("jdk.ObjectAllocationSample").withThreshold(java.time.Duration.ZERO);
    r.enable("jdk.ThreadSleep").withThreshold(java.time.Duration.ZERO);
    r.setDestination(path);
    r.start();
    ok(r.getState() == jdk.jfr.RecordingState.RUNNING, "a started recording is running");
    // allocation churn for the sampler to see
    for (int i = 0; i < 200_000; i++) {
      String s = new StringBuilder("x").append(i).toString();
      if (s.isEmpty()) {
        throw new IllegalStateException();
      }
    }
    Thread.sleep(500);
    r.stop();
    // with a destination set, stop writes the dump and closes the recording
    ok(r.getState() == jdk.jfr.RecordingState.CLOSED, "stop writes the destination and closes");
    allocationEvents = 0;
    total = 0;
    try (var rf = new jdk.jfr.consumer.RecordingFile(path)) {
      while (rf.hasMoreEvents()) {
        var e = rf.readEvent();
        total++;
        if (e.getEventType().getName().equals("jdk.ObjectAllocationSample")) {
          allocationEvents++;
        }
      }
```

listing 98: a recording started, fed allocation churn, and read back in-process (source: java/samples/src/Ch11/Jfr.java, lines 15-47)

Measured 2026-10-04: 153 events, 152 of them `ObjectAllocationSample`, in a 137,269-byte file. With a destination set, `stop()` writes the dump and the recording reads `CLOSED`, the state machine the checks walk. `RecordingFile` streams the parsed events, the same records Mission Control renders.

11.6. the jit at a walk level

HotSpot starts every method in the interpreter, tier 0, counting. Hot methods climb the tiers: 1 through 3 compile quickly on C1 with increasing profiling, tier 4 hands the profile to C2 for the fully optimized build. Tiered

compilation has been the default since 8, and it is why java warms up: the first seconds run interpreted and lightly compiled, peak speed arrives after the profiles do. Compiled code lives in the code cache, `ReservedCodeCacheSize` measured at roughly 240 MiB on this machine's default flags, and deoptimization drops a method back toward the interpreter whenever a speculative assumption, an unstaken branch, an inlined monomorphic call, proves wrong. This is deliberately a walk: the full compiler story belongs to the vm literature, and nothing in day-to-day java needs it beyond knowing why the p99 moves in the first five minutes.

11.7. the measurement discipline

Every claim above splits into measured or documented, and the split is stated. The measured side comes from three tools.

First, `jcmd`, the swiss knife: it talks to a running jvm by pid and its command set is grouped by subsystem.

```
public static void main(String[] args) throws Exception {
  long pid = ProcessHandle.current().pid();
  String uptime = jcmd(pid, "VM.uptime");
  ok(uptime.contains(" s"), "jcmd VM.uptime answers: " + uptime.split("\\R")[1].trim());
  String heap = jcmd(pid, "GC.heap_info");
  ok(heap.contains("garbage-first heap"), "jcmd GC.heap_info reports the G1 heap");
  ok(heap.contains("region size"), "G1 reports its region granularity");
  String flags = jcmd(pid, "VM.flags");
  ok(flags.contains("+UseG1GC"), "jcmd VM.flags shows +UseG1GC");
  // the jit and the class loader answer through their mx beans
  var comp = java.lang.management.ManagementFactory.getCompilationMXBean();
  var classes = java.lang.management.ManagementFactory.getClassLoadingMXBean();
  ok(comp.getTotalCompilationTime() ≥ 0, "jit time is reported (" + comp.getName() + ")");
  ok(classes.getLoadedClassCount() > 0, "loaded classes are counted ("
    + classes.getLoadedClassCount() + ")");
  System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27, pid " + pid + ":");
  for (String line : heap.split("\\R")) {
    if (line.contains("garbage-first heap") || line.contains("region size")) {
      System.out.println("  " + line.trim());
```

listing 99: jcmd against the sample's own process (source: java/samples/src/Ch11/Tools.java, lines 28-46)

Measured 2026-10-04, the same run: `GC.heap_info` reported `garbage-first heap total reserved 8339456K, committed 12288K, used 2512K` with `region size 4M`, and `VM.flags` showed `+UseG1GC`. Second, `jstat`, the sampled time series, `jstat -gc <pid> 1000` prints one line per second of generation occupancy and collection counts, the shape of a leak or a tuning change over time. Third, the mx beans, which answer the same questions from inside the process and are what application metrics should export: `MemoryMXBean`, `CompilationMXBean`, `ClassLoadingMXBean`, and the diagnostic bean the heap sample used to read flag values and origins.

VERIFY: measured or documented, never both

A number in this book is either measured on the pinned build and dated, like the 8-byte object and the 4 MiB regions, or it comes from a cited jep or the javadoc, like the 200 ms default pause goal. Anything a workload actually depends on gets re-measured on the deployment machine before it drives a decision, because heap sizing, region granularity, and ergonomics all move with hardware and flags.

sources: jep 523 (openjdk.org/jeps/523, G1 default in all environments, the 1792 mb and single cpu Serial fallback, JEP 522 throughput work), jep 534 compact headers default 27 with the 96 to 64 bit reduction and the 450 (24 preview) and 519 (25 opt-in) lineage, jep 248 G1 server default 9, jep 363 CMS removed 14, jep 404 generational Shenandoah experimental 24, jep 521 product 25, jep 535 generational default targeted 28, jep 439 generational ZGC 21, jep 421 finalization deprecation 18, all accessed 2026-10-04. Tiered compilation default since 8 verified

against Oracle's compilation optimization documentation, docs.oracle.com/en/java/javase/11/jrockit-hotspot/compilation-optimization.html, accessed 2026-10-04. The 8-to-17 arcs (mark and sweep, evacuation, survivor spaces, the HotSpot heap, G1 region model, parallel concurrent serial distinctions, finalization's failure modes, jcmd and jstat usage, JFR workflow) ground on Java in a Nutshell 8th edition chapter 6 pages 237 to 248 and chapter 13 pages 407 to 429. Measurements (collector ergonomics and header origins via HotSpotDiagnosticMXBean, 8 bytes per plain object over 3 rounds, 153 JFR events with 152 allocation samples in 137,269 bytes, G1 region size 4M, reserved heap 8339456K, code cache 240 MiB, loaded class count) produced by `java/samples/src/Ch11` under `pwsh tools/run-java-samples.ps1 -Chapter Ch11` and the VM.flags probe, dated 2026-10-04 on tools/jdk27/build/jdk-27.

12. collections and streams

Almost every java program moves values through `List`, `Set`, `Map`, and the stream pipeline that replaced half the loops in the language at 8. This chapter walks the framework top to bottom: the interface hierarchy, the implementations and their complexity contracts, the 21 unification of first and last access, the factory methods since 9, streams at full depth including the 24 gatherer extension point, and the swing from imperative loops to pipelines, taught as a refactor arc rather than a religion.

12.1. the hierarchy

`Collection` is the grouping contract, `Map` sits beside it, not above it, because a map of pairs is not a collection of elements. Under `Collection`: `List` is ordered with duplicates and index access, `Set` rejects duplicates, `Queue` hands out the head. `SortedSet` and `SortedMap` add ordering, `Iterator` and `Iterable` power the foreach loop over all of it. The contracts are old, 1.2 for the framework, 5 for generics, and stable since.

The implementations are the performance story, and the complexity contracts decide which one a workload wants:

type	representation	the contract that picks it
ArrayList	dynamic array	get and set O(1), add amortized O(1), middle inserts O(n)
LinkedList	doubly linked	ends O(1), indexed access O(n), also a Deque
ArrayDeque	circular array	stack and queue ends O(1), the Stack replacement
HashSet	hash table	add, remove, contains O(1), no order
TreeSet	red-black tree	sorted iteration, O(log n) ops, range views
HashMap	hash table	put, get, remove O(1), one null key
TreeMap	red-black tree	sorted keys, O(log n), head, tail, sub views

figure 18: the implementation shelf, representation against the operations it wins

`LinkedHashMap` keeps a list threaded through the hash table for insertion or access order, the access-order mode builds an lru cache in one line. `Vector` and `Stack` are the synchronized 1.0 fossils, `ArrayDeque` replaces the second. Nothing in `java.util` is threadsafe, the concurrent package carries that load: `ConcurrentHashMap` for lock-striped maps, `CopyOnWriteArrayList` for read-mostly lists, and the blocking queues chapter 10 met as the executor handoff.

The `equals` and `hashCode` contract governs membership in every hash structure: equal objects must hash equal, or a `HashSet` loses elements it is holding. Records get both for free from their components, classes write them by hand at their peril.

12.2. sequenced collections

Before 21 the ends were a dialect: `list.get(0)` and `list.get(list.size() - 1)`, `set.first()` on sorted sets only, `map.keySet().iterator()` to find a map's first key. Jep 431 (final in 21) made one interface, `SequencedCollection`, that `List`, `Deque`, and `LinkedHashSet` implement, with `SequencedSet` and `SequencedMap` beside it. One vocabulary: `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast`, `reversed`, and on maps `putFirst`, `putLast`, `firstEntry`, `lastEntry`, and the two polls.

```
  public static void main(String[] args) {
    // List has always been ordered, now it speaks first and last
    var list = new java.util.ArrayList<>(java.util.List.of("a", "b", "c"));
    ok("a".equals(list.getFirst()) && "c".equals(list.getLast()),
      "a list answers getFirst and getLast");
    list.addFirst("z");
    list.addLast("z2");
    ok("z".equals(list.getFirst()) && "z2".equals(list.getLast()),
      "addFirst and addLast push at the ends");
    ok("z2".equals(list.reversed().getFirst()), "reversed is a view, not a copy");
    ok(list.size() == 5, "the original list still holds all 5");
    ok("z".equals(list.removeFirst()) && "z2".equals(list.removeLast()),
      "removeFirst and removeLast return what they drop");
```

listing 100: a list speaks the unified ends vocabulary (source: java/samples/src/Ch12/Sequenced.java, lines 15-27)

`reversed()` returns a view, not a copy, mutations through it write through to the original, and the view costs one object. The sample then walks the same vocabulary across `LinkedHashSet` and `LinkedHashMap`:

```
    // SequencedSet: LinkedHashSet keeps insertion order both ways
    var words = new java.util.LinkedHashSet<>(java.util.List.of("one", "two", "three"));
    ok("one".equals(words.getFirst()) && "three".equals(words.getLast()),
      "a LinkedHashSet answers first and last in insertion order");
    var backwards = words.reversed();
    ok("three".equals(backwards.getFirst()) && "one".equals(backwards.getLast()),
      "a reversed set iterates last to first");
    // SequencedMap: LinkedHashMap gained the ends
    var ages = new java.util.LinkedHashMap<Integer, String>();
    ages.put(1, "one");
    ages.put(2, "two");
    ages.put(3, "three");
    ok("one".equals(ages.firstEntry().getValue()) && "three".equals(ages.lastEntry().getValue()),
      "a map answers firstEntry and lastEntry");
    ages.putFirst(0, "zero");
    ages.putLast(4, "four");
    ok(ages.firstEntry().getValue().equals("zero") && ages.lastEntry().getValue().equals("four"),
      "putFirst and putLast write at the ends");
    var reversed = ages.reversed();
    ok(reversed.firstEntry().getValue().equals("four") && reversed.lastEntry().getValue().equals("zero"),
      "a reversed map flips the iteration order");
    ok("zero".equals(ages.pollFirstEntry().getValue()) && "four".equals(ages.pollLastEntry().getValue()),
      "pollFirstEntry and pollLastEntry pop the ends");
    ok(ages.size() == 3, "the polled map keeps its middle");
  }
}
```

listing 101: sets and maps answer the same first and last questions (source: java/samples/src/Ch12/Sequenced.java, lines 28-53)

Twelve checks cover the three families. The interface shows up in api signatures too: a parameter declared `SequencedCollection` states it cares about encounter order, and `HashSet` cannot even be passed, it does not implement the interface.

12.3. factories since 9

`List.of("a", "b")`, `Set.of(1, 2, 3)`, `Map.of("one", 1)` and `Map.entry(k, v)` for the varargs form, `Map.ofEntries(entry(...),`
`entry(...))`. All return unmodifiable collections, fixed-arity overloads up to ten arguments and a varargs fallback,

`Set.of` throws on duplicate elements at creation rather than silently dropping them. Since 10 they carry the immutable collections' fight against interface bloat: `List.copyOf`, `Set.copyOf`, `Map.copyOf` take any source and return the unmodifiable shape, reusing the argument itself when it already qualifies. The empty forms, `List.of()` and friends, replace `Collections.emptyList`.

12.4. the stream api

Streams arrived at 8 as the collection pipeline: a source, zero or more lazy intermediate stages, one eager terminal. The first fact worth internalizing is the laziness itself:

```
// laziness: nothing runs until a terminal operation pulls
var seen = new java.util.concurrent.atomic.AtomicInteger();
var pipeline = java.util.List.of(1, 2, 3, 4, 5).stream()
  .peek(n -> seen.incrementAndGet())
  .filter(n -> n % 2 == 1)
  .map(n -> n * 10);
ok(seen.get() == 0, "a pipeline with no terminal op does zero work");
var odds = pipeline.toList();
ok(seen.get() == 5 && odds.equals(java.util.List.of(10, 30, 50)),
  "the terminal op pulls the whole pipeline");
// short-circuiting terminals stop early
var seen2 = new java.util.concurrent.atomic.AtomicInteger();
java.util.List.of(1, 2, 3, 4, 5).stream()
  .peek(n -> seen2.incrementAndGet())
  .filter(n -> n > 2)
  .findFirst();
ok(seen2.get() == 3, "findFirst stops the pipeline at the first hit");
```

listing 102: no terminal operation, no work, and short circuits cut the run (source: java/samples/src/Ch12/Streams.java, lines 16-32)

`peek` counts element visits, and the counter reads zero until the terminal runs. `findFirst` stops the pull after the first passing element, three visits for this data, which is the semantics infinite streams ride: `IntStream.iterate` produces forever, only a short-circuiting terminal makes it finite. Streams are single use, a consumed stream throws `IllegalStateException` on a second terminal, and they are not collections, nothing is stored, elements flow through.

Primitive streams exist because generics cannot specialize: `IntStream`, `LongStream`, `DoubleStream` avoid the `Integer` boxing on every element and carry the math terminals, `sum`, `average`, `min`, `max`, `summaryStatistics`, plus `range` and `rangeClosed` as sources. `mapToInt`, `mapToObj`, and `boxed` cross between the worlds.

```
// primitive streams: no boxing, math out of the box
var stats = java.util.stream.IntStream.rangeClosed(1, 100).summaryStatistics();
ok(stats.getSum() == 5050 && stats.getCount() == 100
  && stats.getMin() == 1 && stats.getMax() == 100,
  "IntStream summarizes 1 through 100");
ok(java.util.stream.IntStream.range(0, 5).map(n -> n * n).sum() == 30,
  "primitive streams map and sum without boxing");
// collectors: grouping, counting, joining
var words = java.util.List.of("pear", "fig", "apple", "date", "plum", "kiwi");
var byLength = words.stream()
  .collect(java.util.stream.Collectors.groupingBy(String::length, java.util.TreeMap::new,
    java.util.stream.Collectors.counting()));
ok(byLength.get(3) == 1 && byLength.get(4) == 4 && byLength.get(5) == 1,
  "groupingBy counts by length: " + byLength);
String joined = words.stream().sorted().collect(java.util.stream.Collectors.joining(", "));
ok("apple, date, fig, kiwi, pear, plum".equals(joined),
  "joining makes one string, sorted");
var lengths = words.stream()
  .collect(java.util.stream.Collectors.toMap(w -> w, String::length));
ok(lengths.get("pear") == 4 && lengths.size() == 6, "toMap builds a lookup");
// streams over maps: entries flow like anything else
int totalLen = lengths.entrySet().stream()
  .mapToInt(java.util.Map.Entry::getValue)
  .sum();
ok(totalLen == 24, "a map streams its entries into a primitive sum");
```

listing 103: primitive streams and the standard collectors (source: java/samples/src/Ch12/Streams.java, lines 33-57)

The collectors are the terminal vocabulary: `groupingBy` with a map factory and a downstream collector, `counting`, `joining`, `toMap`, `partitioningBy`, `mapping`, and `collectingAndThen` to post-process. `toList()` (16) is the unmodifiable shortcut for the most common collection of all, the sample proves the immutability by failing to add to it.

12.5. stream gatherers

The fixed stages could not express windowing, running state, or custom cardinality, and the pre-24 answer was always “write a collector or a spliterator”, both wrong shapes for a mid-pipeline operation. Jep 485 (final in 24) added `Stream.gather`, taking a `Gatherer`: an initializer for per-pipeline state, an integrator that sees state, element, and a downstream push, and optional finisher and combiner for parallel merges. The jdk ships five in `java.util.stream.Gatherers`.

```
// windowFixed: non-overlapping batches, the sql floor of pipelines
var batches = nums.stream()
  .gather(java.util.stream.Gatherers.windowFixed(3))
  .toList();
ok(batches.size() == 3 && batches.get(0).equals(java.util.List.of(1, 2, 3))
  && batches.get(2).equals(java.util.List.of(7)),
  "windowFixed(3) makes 3 batches, the last short: " + batches);
// windowSliding: overlapping runs, rolling windows
var windows = nums.stream()
  .gather(java.util.stream.Gatherers.windowSliding(3))
  .toList();
ok(windows.size() == 5 && windows.get(0).equals(java.util.List.of(1, 2, 3))
  && windows.get(4).equals(java.util.List.of(5, 6, 7)),
  "windowSliding(3) rolls 5 windows");
```

listing 104: the built-in windows, fixed and sliding (source: java/samples/src/Ch12/Gatherers.java, lines 17-30)

`windowFixed` batches, `windowSliding` rolls, `fold` scans a running value, `mapConcurrent` maps with bounded concurrency on virtual threads. Writing one is the point of the api:

```
    // a custom gatherer: running maximum, one state cell, push one output
    // per input
    java.util.stream.Gatherer<java.lang.Integer, int[], java.lang.Integer> runningMax =
      java.util.stream.Gatherer.ofSequential(
        () -> new int[] {java.lang.Integer.MIN_VALUE},
        (max, element, downstream) -> {
          max[0] = Math.max(max[0], element);
          downstream.push(max[0]);
          return true;
        });
    var maxes = java.util.List.of(3, 1, 4, 1, 5, 9, 2, 6).stream()
      .gather(runningMax)
      .toList();
    ok(maxes.equals(java.util.List.of(3, 3, 4, 4, 5, 9, 9, 9)),
      "a custom running-max gatherer carries state across elements: " + maxes);
    // a custom gatherer that changes cardinality: one output per run of
    // equal neighbors, the dedupe-consecutive shape
    java.util.stream.Gatherer<java.lang.Integer, java.lang.Integer[], java.lang.Integer> collapseRuns =
      java.util.stream.Gatherer.ofSequential(
        () -> new java.lang.Integer[] {null},
        (seen, element, downstream) -> {
          if (!element.equals(seen[0])) {
            downstream.push(element);
            seen[0] = element;
          }
          return true;
        });
    var collapsed = java.util.List.of(7, 7, 7, 8, 8, 9, 7, 7).stream()
      .gather(collapseRuns)
      .toList();
    ok(collapsed.equals(java.util.List.of(7, 8, 9, 7)),
      "a gatherer can emit fewer elements than it consumes: " + collapsed);
```

listing 105: two custom gatherers, state carried, cardinality changed (source: java/samples/src/Ch12/Gatherers.java, lines 31-62)

The running maximum keeps its state in a mutable cell the initializer supplies, one cell per pipeline evaluation, so parallel splits stay safe. The lesson the sample teaches by construction: the state must be mutable, the first draft used an `Integer` and the maximum silently reset to the input every element, the integrator can read an immutable state but never writes it back. The collapse-runs gatherer shows the cardinality freedom, fewer outputs than inputs, which no intermediate stage before 24 could do. Gatherers are lazy like every stage, the last check builds a gathered pipeline and proves zero work happens until the terminal.

12.6. the swing from imperative

The arc every codebase rides, in one shape. First the loop:

```
int total = 0;
for (String w : words) {
  if (w.length() > 3) {
    total += w.length();
  }
}
```

fragment, illustrative, not executed

Then the pipeline:

```
int total = words.stream()
  .filter(w -> w.length() > 3)
  .mapToInt(String::length)
  .sum();
```

fragment, illustrative, not executed

The pipeline names its stages, composes, parallelizes with one `.parallel()`, and drops the loop scaffolding. The discipline that keeps it readable: stages stay pure, side effects live in `peek` for debugging and `forEach` for terminal effect only, the whole pipeline fits a screen, and the terminal is where the answer leaves. When the shape needs mid-pipeline state or windowing, a gatherer extends the vocabulary instead of falling back to the loop. Loops remain correct code: small, index-coupled, or short-circuiting-on-two-conditions logic is often clearer as a loop, and the best java developers carry both and choose per problem, the service spine in chapter 19 leans on pipelines throughout because request handling is a pipeline.

sources: JEP 431 sequenced collections (openjdk.org/jeps/431) and JEP 485 stream gatherers (openjdk.org/jeps/485), accessed 2026-10-04. API since tags verified against the pinned build's own source: SequencedCollection 21, Gatherer and Gatherers 24, read from tools/jdk27/build/jdk-27/lib/src.zip. The 8-to-17 arcs (the interface hierarchy, implementation tables with complexity contracts, optional operations and UnsupportedOperationException, iteration and ConcurrentModificationException, queue semantics, Collections and Arrays utilities, the stream api's filter map reduce core and laziness) ground on Java in a Nutshell 8th edition chapter 8, pages 283 to 317. Behavior verified by `java/samples/src/Ch12` under `pwsh tools/run-java-samples.ps1 -Chapter Ch12`: 28 checks, including the mutable-state gatherer lesson (an Integer state silently no-ops, an int array state works) probed on the pinned jdk 27, dated 2026-10-04.

13. text, numbers, and time

Programs spend their days on strings, numbers, and timestamps, and java's story on all three is a long climb out of 90s decisions: a `String` api that grew modern methods across 11, 12, and 15, text blocks that made multiline literals writable, exact decimal and integer arithmetic when binary floats will not do, a `java.time` package that replaced `java.util.Date` wholesale at 8, and utf-8 as the platform charset since 18. This chapter walks each one end to end with the pitfalls next to the fixes.

13.1. String as the modern api

Strings are immutable and live in the string pool when literal, the `+` operator compiles to an `invokedynamic` concatenation strategy since 9, so hot loops no longer need hand-written `StringBuilder` churn, though building incrementally still uses one directly. The batch that modernized daily work landed in 11: `isBlank`, `lines`, `strip`, `stripLeading`, `stripTrailing`, and `repeat`.

```
public static void main(String[] args) {
  // the 11 batch: isBlank, lines, strip, repeat
  ok("   ".isBlank() && !"\t\n".isEmpty() && "\t\n".isBlank(),
    "isBlank tests whitespace, isEmpty only length");
  ok(!"  ".trim().isEmpty() && "  ".strip().isEmpty(),
    "strip is unicode-aware, trim stops at ascii space");
  var lines = "alpha\nbeta\r\ngamma".lines().toList();
  ok(lines.size() == 3 && lines.get(2).equals("gamma"),
    "lines splits on all three terminators: " + lines);
  ok("ab".repeat(4).equals("abababab"), "repeat builds copies");
```

listing 106: the 11 batch, and strip against trim on a U+2000 space (source: java/samples/src/Ch13/Strings.java, lines 15-24)

`strip` against `trim` is the sleeper difference: `trim` cuts characters up to U+0020, an ascii-era definition, `strip` uses unicode whitespace, and the check keeps a U+2000 space alive through `trim` to prove it. `lines` splits on all three line terminators and returns a `Stream`, so it chains directly into the chapter 12 vocabulary. `indent` (12) pads or strips a per-line margin, and `stripIndent` (15) applies the text-block algorithm to any string, which the next section needs.

```
  // 12 and 15: indent and stripIndent, formatted
  ok("a\nb".indent(2).equals("  a\n  b\n"), "indent(2) pads every line and adds the final newline");
  ok("  a\n  b".stripIndent().equals("a\nb"), "stripIndent removes the common prefix");
  ok("hi %s".formatted("there").equals("hi there"), "formatted is format on the instance (15)");
  // toString of valueOf vs null
  ok("null".equals(String.valueOf((Object) null)), "valueOf(null) prints null, no throw");
  // utf-8 by default since 18 (JEP 400), measured on this machine
  ok("UTF-8".equals(java.nio.charset.Charset.defaultCharset().name()),
    "the default charset is UTF-8 (18)");
  ok("UTF-8".equals(System.getProperty("file.encoding")),
    "file.encoding reads UTF-8 too");
}
```

listing 107: indent, stripIndent, formatted, and the 18 charset default (source: java/samples/src/Ch13/Strings.java, lines 25-36)

`formatted` (15) puts `String.format` on the instance, the fluent reading order the printf-style call always wanted.

13.1.1. utf-8 by default

Before 18 the default charset followed the operating system, so a `FileReader` read the same bytes differently on a Windows machine than on Linux, and files moved between them corrupted quietly. Jep 400 made utf-8 the default charset in 18 everywhere, `file.encoding` reads utf-8 on a stock jvm, and code needing a different one says

so explicitly with `new InputStreamReader(in, charset)`. The sample probes both properties on the running machine. Programs that always passed an explicit charset noticed nothing, which was the point.

13.2. text blocks

Multiline strings fought escaping for 20 years. Text blocks, previewed in 13 and 14, final in 15 (jep 378), fixed it: open with """ and a newline, close with """, and the content keeps its shape.

```
public static void main(String[] args) {
  // the common indent of the content is stripped, the content keeps
  // any depth beyond the minimum
  String tb = """
      alpha
        beta
      """;
  ok(tb.equals("alpha\n  beta" + "\n"),
    "the minimal indent goes, deeper lines keep theirs: " + repr(tb));
  // closing delimiter left of the content: its column caps the strip
  String shifted = """
      alpha
        beta
    """;
  ok(shifted.equals("  alpha\n    beta" + "\n"),
    "the closing delimiter's column caps the strip: " + repr(shifted));
```

listing 108: the indentation algorithm, minimum strip with a closing-delimiter cap (source: java/samples/src/Ch13/TextBlocks.java, lines 19-34)

The algorithm, precisely: the compiler finds the common whitespace prefix across the content lines, and the closing delimiter's own line counts when it sits left of the content, capping how much is stripped. Move the closing """ left and the content shifts right with it, slide it past the content's left edge and nothing strips. `String.stripIndent` implements the same minimum-indent rule for runtime strings, and `translateEscapes` completes the pair. The final newline before the closing delimiter is part of the content, the checks assert it, and every line's trailing whitespace is removed unless escaped.

Two escapes exist because of that trailing-whitespace rule and long lines: \ at end of line joins two source lines into one logical line, and \s pins a real space that would otherwise be stripped, both since 14.

```
    // \ escapes the newline: two source lines, one logical line
    String joined = """
        one \
        two
        """;
    ok(joined.equals("one two" + "\n"), "a trailing backslash joins lines (14)");
    // \s is an explicit space that survives trailing-whitespace removal
    String spaced = """
        keep\s
        """;
    ok(spaced.equals("keep \n"), "the \\s escape pins a trailing space");
    ok("keep \n".stripTrailing().equals("keep"),
      "without it, stripTrailing shows what the compiler would eat");
    // quotes ride free inside
    String quoted = """
        she said "hi"
        """;
    ok(quoted.equals("she said \"hi\"" + "\n"), "double quotes need no escape inside a block");
    // blocks are plain strings at runtime
    ok(("alpha\n  beta" + "\n").equals(tb) && tb instanceof String,
      "a text block is just a String");
```

listing 109: the two escapes, free quotes, and plain-String identity (source: java/samples/src/Ch13/TextBlocks.java, lines 38-58)

At runtime a text block is a `String`, nothing else, the last check compares it against the ordinary literal. Their natural habitat is embedded languages: json, sql, html ride without double escaping, and chapter 19's http kernel builds its test fixtures from blocks.

13.3. BigInteger and BigDecimal

Binary floating point cannot hold 0.1, and money is counted in tenths. `java.math` carries the exact pair: `BigInteger` for unbounded integers, `BigDecimal` for arbitrary-precision decimal. The one rule that prevents the classic bug: construct from strings or `valueOf`, never from a double.

```
// BigDecimal: exact decimal arithmetic, constructed from strings
var price = new java.math.BigDecimal("19.99");
var qty = new java.math.BigDecimal("3");
ok(price.multiply(qty).compareTo(new java.math.BigDecimal("59.97")) == 0,
  "BigDecimal multiplies exactly: 19.99 x 3 = 59.97");
// the classic pitfall: the double constructor inherits binary error
var fromDouble = new java.math.BigDecimal(0.1);
var fromString = new java.math.BigDecimal("0.1");
ok(!fromDouble.equals(fromString) && fromString.equals(java.math.BigDecimal.valueOf(0.1)),
  "new BigDecimal(0.1) is not 0.1, valueOf and strings are: " + fromDouble);
// non-terminating division throws unless told where to stop
boolean threw = false;
try {
  java.math.BigDecimal.ONE.divide(new java.math.BigDecimal("3"));
} catch (ArithmeticException e) {
  threw = true;
}
ok(threw, "1/3 with no scale throws ArithmeticException");
var rounded = java.math.BigDecimal.ONE
  .divide(new java.math.BigDecimal("3"), 4, java.math.RoundingMode.HALF_UP);
ok(rounded.compareTo(new java.math.BigDecimal("0.3333")) == 0,
  "with a scale and rounding mode it rounds: " + rounded);
// equals is scale-sensitive, compareTo is not
ok(!new java.math.BigDecimal("1.0").equals(new java.math.BigDecimal("1.00"))
  && new java.math.BigDecimal("1.0").compareTo(new java.math.BigDecimal("1.00")) == 0,
  "equals sees scale, compareTo sees value");
```

listing 110: exact arithmetic, the double constructor trap, and the two equality semantics (source: java/samples/src/Ch13/Numbers.java, lines 16-41)

`new BigDecimal(0.1)` prints `0.1000000000000000055511151231257827021181583404541015625`, the exact binary double the literal names, while the string and `valueOf` forms hold 0.1. Division without a target scale throws `ArithmeticException` on non-terminating results, 1/3 has no decimal end, so real code passes a scale and a `RoundingMode`. And the equality trap: `equals` compares scale, `1.0` is not `1.00`, `compareTo` compares values, so sorted maps of decimals key on `compareTo` or normalize first.

```
    // BigInteger: unbounded integers, measured
    var big = java.math.BigInteger.TWO.pow(10_000);
    ok(big.bitLength() == 10_001 && big.toString().length() == 3_011,
      "2^10000 has " + big.toString().length() + " digits, bitLength " + big.bitLength());
    ok(java.math.BigInteger.TWO.pow(64)
      .subtract(java.math.BigInteger.TWO.pow(64).subtract(java.math.BigInteger.ONE))
      .equals(java.math.BigInteger.ONE),
      "arithmetic that would overflow long stays exact");
    // compact number formatting since 12, locale aware
    var us = java.util.Locale.US;
    String shortB = java.text.NumberFormat
      .getCompactNumberInstance(us, java.text.NumberFormat.Style.SHORT)
      .format(1_000_000_000L);
    String longB = java.text.NumberFormat
      .getCompactNumberInstance(us, java.text.NumberFormat.Style.LONG)
      .format(1_000_000_000L);
    ok("1B".equals(shortB) && "1 billion".equals(longB),
      "compact formatting: short \"" + shortB + "\", long \"" + longB + "\"");
    String currency = java.text.NumberFormat.getCurrencyInstance(us).format(1_234_567.891);
    ok("$1,234,567.89".equals(currency), "currency format groups and rounds: " + currency);
  }
}
```

listing 111: BigInteger measured, compact formatting since 12, locale currency (source: java/samples/src/Ch13/Numbers.java, lines 42-63)

Measured 2026-10-04 on the pinned build: 2^10000 has 3,011 decimal digits at a bitLength of 10,001, arithmetic that would overflow `long` stays exact. Compact number formatting arrived with 12 (`getCompactNumberInstance`), 1,000,000,000 formats as `1B` short and `1 billion` long, locale aware, and the currency instance groups and rounds, `$1,234,567.89`. For the floating-point side, 17 made every operation strictly IEEE 754 (jep 306), the `strictfp` keyword became a no-op, and the pitfalls that remain are binary representation itself, the reason this section exists.

13.4. java.time

The 8 overhaul (jsr 310) replaced `Date` and `Calendar` with a package built on one distinction: a point on the timeline is an `Instant`, a machine reading with nanoseconds since the epoch. Calendar-shaped types, `LocalDate`, `LocalTime`, `LocalDateTime`, carry fields without a zone, `ZonedDateTime` adds one, and `OffsetDateTime` carries just the utc offset. Everything is immutable and the api is fluent: `plus`, `minus`, `with`, `truncatedTo`.

```
    // Instant: a point on the utc timeline, nanosecond resolution
    var epoch = java.time.Instant.ofEpochSecond(0);
    ok(epoch.equals(java.time.Instant.EPOCH), "the epoch instant");
    var withNanos = java.time.Instant.ofEpochSecond(5, 250_000_000);
    ok(withNanos.getNano() == 250_000_000, "instants carry nanoseconds");
    // Duration: exact seconds-and-nanos, Period: calendar units
    var d = java.time.Duration.between(
      java.time.LocalTime.of(9, 0), java.time.LocalTime.of(10, 30));
    ok(d.toMinutes() == 90, "Duration measures machine time: 90 minutes");
    var p = java.time.Period.between(
      java.time.LocalDate.of(2026, 1, 31), java.time.LocalDate.of(2026, 3, 1));
    ok(p.getMonths() == 1 && p.getDays() == 1,
```

listing 112: Instant, and Duration's machine time against Period's calendar walk (source: java/samples/src/Ch13/Time.java, lines 16-27)

`Duration` counts seconds and nanoseconds, exact machine time. `Period` counts years, months, days on the calendar, and the check shows the difference on real dates: January 31 to March 1, 2026 is one month and one day through the calendar, 29 days on a timeline. Mixing them up breaks billing logic.

```
    // zones and the dst gap: 2026-03-08 02:30 does not exist in New York
    var gap = java.time.LocalDateTime.of(2026, 3, 8, 2, 30)
      .atZone(java.time.ZoneId.of("America/New_York"));
    ok(gap.getHour() == 3, "a nonexistent local time shifts forward to " + gap.getHour() + ":" +
gap.getMinute());
    // and the fall-back overlap resolves to the earlier offset
    var overlap = java.time.LocalDateTime.of(2026, 11, 1, 1, 30)
      .atZone(java.time.ZoneId.of("America/New_York"));
    ok(overlap.getOffset().getTotalSeconds() == -4 * 3600,
      "an ambiguous time takes the earlier offset: " + overlap.getOffset());
    // formatting and parsing are two calls on one formatter
    var iso = java.time.format.DateTimeFormatter.ISO_LOCAL_DATE;
    ok("2026-10-04".equals(java.time.LocalDate.of(2026, 10, 4).format(iso))
      && java.time.LocalDate.parse("2026-10-04", iso).getDayOfWeek() == java.time.DayOfWeek.SUNDAY,
      "formatters format and parse");
    var pretty = java.time.format.DateTimeFormatter.ofPattern("MMMM d, yyyy");
    ok("October 4, 2026".equals(java.time.LocalDate.of(2026, 10, 4).format(pretty)),
```

listing 113: dst edges resolved, formatting and parsing both ways (source: java/samples/src/Ch13/Time.java, lines 29-44)

Time zones are rules, not offsets, and the rules change: the jvm carries its own tzdata copy, so zone-rule updates ride jdk updates. The two hard edges behave deterministically: a local time inside the spring-forward gap shifts forward, 02:30 on March 8, 2026 in New York becomes 03:30, and an ambiguous fall-back time takes the earlier offset. Formatters parse what they print, `DateTimeFormatter` patterns and predefined ISO constants, and `MonthDay` and `YearMonth` cover the partial shapes.

```
    // the injected clock: same code, deterministic time
    var fixed = java.time.Clock.fixed(
      java.time.Instant.parse("2026-10-04T12:00:00Z"), java.time.ZoneOffset.UTC);
    ok(java.time.LocalDate.now(fixed).equals(java.time.LocalDate.of(2026, 10, 4)),
      "a fixed clock makes now() deterministic, the test seam");
    var shifted = java.time.Clock.offset(fixed, java.time.Duration.ofHours(48));
    ok(java.time.LocalDate.now(shifted).equals(java.time.LocalDate.of(2026, 10, 6)),
      "Clock.offset travels without sleeping");
    // legacy conversion: Date.toInstant bridges old code in
    var legacy = new java.util.Date(0).toInstant();
    ok(legacy.equals(java.time.Instant.EPOCH), "Date.toInstant bridges the old api");
  }
}
```

listing 114: the injected clock, deterministic time in tests (source: java/samples/src/Ch13/Time.java, lines 46-58)

`Clock` is the seam that makes time-dependent code testable: pass a `Clock` in, production gets `Clock.systemUTC()`, a test gets `Clock.fixed` for one instant and `Clock.offset` to travel, and `now(clock)` becomes a pure function of its argument. The service spine's token expiry checks in chapter 22 ride exactly this. Legacy `Date` converts one way, `toInstant`, and stays at the boundary.

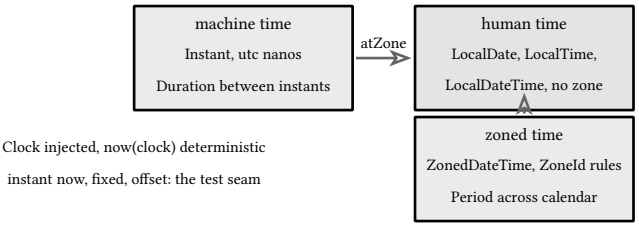


figure 19: the type lattice: machine time left, calendar right, zones bridged

sources: JEP 378 text blocks final 15 (openjdk.org/jeps/378, the re-indentation algorithm, stripIndent equivalence, line-join and s escapes delivered by the 14 preview JEP 368), JEP 400 utf-8 default 18, JEP 306 strict floating point 17, all accessed 2026-10-04. API since tags verified against the pinned build's own source in tools/jdk27/build/jdk-27/lib/src.zip: String isBlank, lines, repeat, strip 11, indent 12, stripIndent and formatted 15, read 2026-10-04. Compact number formatting since 12 confirmed from the same source's NumberFormat (getCompactNumberInstance since 12) and the openjdk JDK 12 project page, which lists it as a non-JEP enhancement, accessed 2026-10-04. The 8-to-17 arcs (string literals and immutability, the concatenation story and its 9 invokedynamic change, toString and valueOf, regex at a mention, two's complement, IEEE 754 and the Goldberg reading, the java.time walkthrough with Instant, Duration, TemporalQuery, adjusters, zone data, and legacy conversion) ground on Java in a Nutshell 8th edition chapter 9, pages 319 to 341. Measurements and behaviors verified by `java/samples/src/Ch13` under `pwsh tools/run-java-samples.ps1 -Chapter Ch13`, dated 2026-10-04 on tools/jdk27/build/jdk-27: 38 checks including the U+2000 strip/trim split, the text-block indentation outcomes with the closing-delimiter cap and final newline, 2^10000 at 3,011 digits, compact 1B and 1 billion, currency $1,234,567.89, the dst gap and overlap resolutions, and the fixed and offset clock determinism.

14. io and networking

Java's input output stack has three ages stacked on each other: the 1996 byte and character streams, the 1.4 channels and buffers built for bulk movement, and the java 7 `java.nio.file` api, `Path` and `Files`, that replaced the old `File` class as the everyday surface. Networking has the same shape, sockets from the beginning and the `java.net.http` client standard since 11, HTTP/2 native and HTTP/3 since 26. This chapter walks the stack bottom up and ends with the websocket client measured against a hand rolled peer, which is where the capstone's chat server starts. Chapter 2 walked the ladder, here is the io rung at work.

14.1. streams and try-with-resources

The oldest layer is a pair of abstract classes, `InputStream` and `OutputStream`, sequential bytes, plus `Reader` and `Writer` over them for characters. `System.in` and `System.out` are these. Real code layers them, a buffer over a reader over a file, and the language feature that makes the layering safe is try-with-resources, since java 7. Any type implementing `AutoCloseable` can sit in the resource clause, several at once, and the compiler closes them in reverse order, suppressing secondary exceptions. Since java 9 the resource can be an effectively final variable declared before the try, so a stream opened in one method can be scoped in another:

```
try (var writer = Files.newBufferedWriter(note, StandardCharsets.UTF_8)) {
  writer.write("alpha\n");
  writer.write("beta\n");
}
var reader = Files.newBufferedReader(note, StandardCharsets.UTF_8);
try (reader) {
  ok("alpha".equals(reader.readLine()), "the buffered reader reads the first line");
}
```

listing 115: the same try closes a fresh declaration and a preexisting effectively final reader (source: java/samples/src/Ch14/IoFiles.java, lines 31-38)

The close order matters when data flows between resources: the writer must flush before the reader underneath it closes. Manual close calls after every operation were the classic leak, and the platform's own pre-7 resource code was full of it, which is why the rule is simple: if it is `AutoCloseable`, it goes in the try.

14.2. path and files

The pre-7 `java.io.File` class could tell you it exists but not read itself, modeled attributes inconsistently, and walked trees badly. It survives for interop. Java 7 replaced it with `Path`, an interface to a location that may not exist, and `Files`, a wall of static operations over paths. `Path.of` is a static interface method, legal since java 8 on interfaces, and the whole family is expressive enough that file plumbing in modern java is a few lines:

```
// readString and writeString arrived in java 11, utf-8 by default
Files.writeString(dir.resolve("note.txt"), "héllo wörld");
ok("héllo wörld".equals(Files.readString(dir.resolve("note.txt"))),
    "writeString/readString round trip (since 11)");
ok(Files.size(dir.resolve("note.txt")) == 13L, "two of those chars encode as 2 bytes, size 13");
Files.writeString(dir.resolve("note2.txt"), "héllo wörld!");
ok(Files.mismatch(dir.resolve("note.txt"), dir.resolve("note2.txt")) == 13L,
    "mismatch reports the first differing byte (since 11)");
```

listing 116: readString and writeString since 11, mismatch finding the first differing byte (source: java/samples/src/Ch14/IoFiles.java, lines 40-47)

`readString`, `writeString`, and `mismatch` arrived in java 11, with utf-8 as the default charset, itself a decision java 18 made global. Before them the same job needed a `BufferedReader` loop or `readAllBytes` plus a decode. Directory work rides the same types: `Files.walk` streams a whole tree, `Files.find` filters it by path and attributes, and `newDirectoryStream` globs one level.

The watch service, also java 7, turns the directory itself into an event source. A registered directory reports create, modify, and delete events for its direct children, buffered until taken:

```
      var names = new HashSet<String>();
      long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(10);
      while (kinds.size() < 3 && System.nanoTime() < deadline) {
        var key = watcher.poll(1, TimeUnit.SECONDS);
        if (key == null) {
          continue;
        }
        for (WatchEvent<?> event : key.pollEvents()) {
          kinds.add(event.kind().name());
          names.add(event.context().toString());
        }
```

listing 117: three writes on a temp dir surface as watch events (source: java/samples/src/Ch14/WatchDir.java, lines 30-40)

Two facts the sample prints rather than asserts: modify events can coalesce, two quick writes may report once, and the event context is the child name, so resolving it against the watched directory is the caller's job.

14.3. channels and buffers

The nio answer to bulk data is the channel and the buffer. A `ByteBuffer` is a typed window over memory with three integers: capacity, fixed at allocation, position, the read or write cursor, and limit, the first byte that does not belong. Writing fills toward capacity, `flip` swaps position and limit for draining, `compact` moves the tail to the front for refilling, `clear` resets both for reuse. The pattern is always fill, flip, drain:

```
    // file channel plus byte buffer, the nio way of moving bytes
    try (var ch = FileChannel.open(dir.resolve("data.bin"), StandardOpenOption.CREATE,
        StandardOpenOption.WRITE, StandardOpenOption.READ)) {
      var buf = ByteBuffer.allocate(8);
      buf.putInt(0x0A0B0C0D);
      buf.putInt(1234);
      buf.flip();
      ok(buf.position() == 0 && buf.limit() == 8, "flip rewinds position and pins the write mark as
limit");
      ch.write(buf);
      ok(ch.position() == 8, "the channel position advanced to 8");
      ch.position(0);
      var head = ByteBuffer.allocate(4);
      while (head.hasRemaining()) {
        ch.read(head);
      }
      head.flip();
      ok(head.getInt() == 0x0A0B0C0D, "the channel read restores the first int in file order");
    }
    ok(ByteBuffer.allocateDirect(64).isDirect(), "allocateDirect hands back a direct buffer");
    var be = ByteBuffer.allocate(4).order(ByteOrder.BIG_ENDIAN);
```

listing 118: a file channel round trip through one buffer, plus direct allocation and byte order (source: java/samples/src/Ch14/IoFiles.java, lines 57-76)

`allocate` is heap memory the garbage collector owns, `allocateDirect` is native memory the channel can move to and from the operating system without a copy, worth it for large or hot buffers only. Endianness is a buffer property, defaulting to big endian, `LITTLE_ENDIAN` for x86 wire formats. A `FileChannel` is the channel over a file, and its `map` hands back a `MappedByteBuffer`, the file mapped into the address space, the closest java gets to memory mapped io.

14.4. tcp sockets

The socket pair, `Socket` and `ServerSocket`, is 1996 api that java 13 quietly reimplemented over the nio infrastructure without changing a signature. A server socket accepts, the accepted socket is a duplex byte stream, and both sides read and write through the stream hierarchy from the first section:

```
  static void blockingSockets() throws Exception {
    var loop = InetAddress.getLoopbackAddress();
    try (var server = new ServerSocket(0, 1, loop);
        var client = new Socket(loop, server.getLocalPort());
        var peer = server.accept()) {
      client.getOutputStream().write("ping\n".getBytes(StandardCharsets.UTF_8));
      client.getOutputStream().flush();
      var peerIn = new BufferedReader(new InputStreamReader(peer.getInputStream(),
StandardCharsets.UTF_8));
      ok("ping".equals(peerIn.readLine()), "the server socket read the client line");
      peer.getOutputStream().write("pong\n".getBytes(StandardCharsets.UTF_8));
      peer.getOutputStream().flush();
      var clientIn = new BufferedReader(new InputStreamReader(client.getInputStream(),
StandardCharsets.UTF_8));
      ok("pong".equals(clientIn.readLine()), "the client socket read the reply");
    }
  }
```

listing 119: connect, accept, and one line each way (source: java/samples/src/Ch14/Net.java, lines 42-57)

The channel equivalents, `SocketChannel` and `ServerSocketChannel`, move the same bytes through buffers instead of streams. They are blocking by default and switch to nonblocking with `configureBlocking` for the selector world, where one thread multiplexes many channels, the scalability answer before virtual threads existed. Chapter 10 covered why virtual threads make blocking sockets the plain choice again, and the service spine in chapter 19 uses blocking handlers on virtual threads throughout:

```
  static void channelSockets() throws Exception {
    try (var ssc = ServerSocketChannel.open()) {
      ssc.bind(new InetSocketAddress(InetAddress.getLoopbackAddress(), 0));
      try (var client = SocketChannel.open(ssc.getLocalAddress());
          var peer = ssc.accept()) {
        var hello = ByteBuffer.wrap("channel ping".getBytes(StandardCharsets.UTF_8));
        while (hello.hasRemaining()) {
          client.write(hello);
        }
        var inbound = ByteBuffer.allocate(64);
        while (inbound.position() == 0) {
          peer.read(inbound);
        }
        inbound.flip();
        var echo = ByteBuffer.allocate(inbound.remaining());
```

listing 120: the same echo over channels and buffers (source: java/samples/src/Ch14/Net.java, lines 59-73)

14.5. the http client

Before 11, http in the jdk was `HttpURLConnection`, protocol version 1.0 habits under a 1996 era api. Java 11 standardized `java.net.http` (JEP 321, incubated in 9) with http/2 native, a builder for requests, and one call for the blocking case:

```
        } catch (Exception e) {
          // the server socket closes when the client is done, that ends the loop
        }
      });
      responder.start();
      // standard since 11: the client prefers http/2 and falls back
      var http = HttpClient.newHttpClient();
      var uri = URI.create("http://127.0.0.1:" + port + "/hello");
      var req = HttpRequest.newBuilder(uri).build();
      var resp = http.send(req, HttpResponse.BodyHandlers.ofString());
      ok(resp.statusCode() == 200, "the blocking send read the status line");
      ok("hello over tcp".equals(resp.body()), "the body handler decoded the payload");
      ok(resp.version() == HttpClient.Version.HTTP_1_1,
          "an http/2 client falls back to 1.1 against a 1.1-only peer");
      ok(resp.headers().firstValue("Content-Type").orElse("").startsWith("text/plain"),
```

listing 121: the blocking send against the hand rolled responder, version fallback included (source: java/samples/src/Ch14/Net.java, lines 113-127)

The client prefers http/2 and negotiates down, the sample measures the fallback to 1.1 against a 1.1-only peer. `sendAsync` returns a `CompletableFuture`, the same pipeline type chapter 10 composed, and `BodyHandlers.ofString`, `ofFile`, `ofInputStream` decide where the body lands without touching the call site. Java 26 adds HTTP/3 (JEP 517), QUIC over UDP with TLS 1.3 inside: opt in with `Version.HTTP_3` on the client or per request, the client races an HTTP/3 attempt against HTTP/2 unless discovery mode pins it, and there is no server-side implementation, the client is the whole feature.

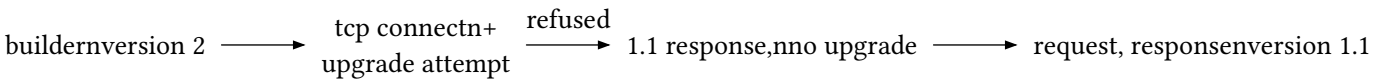


figure 20: an http/2 client against a 1.1 peer: the upgrade attempt fails, the same connection answers

For a zero-config server peer the jdk ships one since java 18 (JEP 408): `java -m jdk.httpserver`, the jwebserver tool, a static file server over the same `com.sun.net.httpserver` package the service spine builds on. The sample starts it as a subprocess with `--port 0`, reads the bound port off its startup line, and fetches a file through the http client.

14.6. websockets, client and peer

`java.net.http.WebSocket`, standard since 11 with the http client, is a client only, and the capstone needs the server half hand rolled, so this chapter builds the peer first. The protocol, RFC 6455, is an http upgrade followed by small binary frames. The client opens with `GET` plus `Sec-WebSocket-Key`, a 16 byte nonce, the server answers `101 Switching Protocols` with `Sec-WebSocket-Accept`, the base64 of sha1 over the nonce and a fixed guid, and from then on the connection carries frames: an opcode byte with a fin bit, a length, up to 7 bits inline or 16 or 64 extended, a 4 byte mask on every client frame, and the payload xored with that mask:

```
  static void serve(ServerSocket server, List<Frame> frames, AtomicInteger closeCode,
      boolean[] sawVersion13, boolean[] sawWsKey) throws IOException,
java.security.NoSuchAlgorithmException {
    try (Socket sock = server.accept()) {
      sock.setTcpNoDelay(true);
      var in = new BufferedInputStream(sock.getInputStream());
      var out = sock.getOutputStream();

      var head = new StringBuilder();
      while (!head.toString().endsWith("\r\n\r\n")) {
        head.append((char) in.read());
      }
      String key = null;
      for (String line : head.toString().split("\r\n")) {
        var lower = line.toLowerCase(Locale.ROOT);
        if (lower.startsWith("sec-websocket-key:")) {
          key = line.substring(line.indexOf(':') + 1).trim();
          sawWsKey[0] = true;
        }
        if (lower.startsWith("sec-websocket-version: 13")) {
          sawVersion13[0] = true;
        }
      }
      var accept = Base64.getEncoder().encodeToString(
          MessageDigest.getInstance("SHA-1").digest((key + GUID).getBytes(StandardCharsets.UTF_8)));
      out.write(("HTTP/1.1 101 Switching Protocols\r\n"
          + "Upgrade: websocket\r\nConnection: Upgrade\r\n"
          + "Sec-WebSocket-Accept: " + accept + "\r\n\r\n").getBytes(StandardCharsets.UTF_8));
      out.flush();
```

listing 122: the upgrade handshake: parse the key, answer the accept, then frames (source: java/samples/src/Ch14/WebSocketEcho.java, lines 116-144)

```
      while (true) {
        int b0 = in.read();
        int b1 = in.read();
        boolean fin = (b0 & 0x80) != 0;
        int opcode = b0 & 0x0f;
        boolean masked = (b1 & 0x80) != 0;
        long len = b1 & 0x7f;
        if (len == 126) {
          len = ((in.read() & 0xffL) << 8) | (in.read() & 0xffL);
        } else if (len == 127) {
          len = 0;
          for (int i = 0; i < 8; i++) {
            len = (len << 8) | (in.read() & 0xffL);
          }
        }
        var mask = new int[4];
        if (masked) {
          for (int i = 0; i < 4; i++) {
            mask[i] = in.read() & 0xff;
          }
        }
        var payload = new byte[(int) len];
        for (int i = 0; i < len; i++) {
          payload[i] = (byte) (in.read() ^ mask[i % 4]);
        }
        frames.add(new Frame(fin, opcode, len, masked));

        if (opcode == 8) {
```

listing 123: the frame loop: unmask, record, echo, mirror the close code (source: java/samples/src/Ch14/WebSocketEcho.java, lines 146-173)

The measured facts on this build, 2026-10-04, pinned jdk 27: a text message of 16384 characters is one final frame, 16385 is two, a 16384 byte frame plus a 1 byte continuation, so `sendText` fragments above 16384. Every client frame arrives masked, the one rule that distinguishes the two directions on the wire. The close handshake echoes: the client sends close with code 1000, the peer mirrors the code, and the client's `onClose` sees its own 1000 come back.

The listener is a pull model, the one surprise in the api. The default `onOpen` grants one invocation, and every override that consumes a message must call `request(1)` or delivery stops mid message, silently:

```
      var received = new StringBuilder();
      var closed = new int[] {-1};
      var listener = new WebSocket.Listener() {
        @Override
        public CompletionStage<?> onText(WebSocket ws, CharSequence data, boolean last) {
          received.append(data);
          // the listener is a pull model: without request(1) the next
          // invocation never happens, measured on this build
          ws.request(1);
          return null;
        }

        @Override
        public CompletionStage<?> onClose(WebSocket ws, int statusCode, String reason) {
          closed[0] = statusCode;
```

listing 124: onText appends and renews the permit, onClose records the echoed code (source: java/samples/src/Ch14/WebSocketEcho.java, lines 57-71)

VERIFY: measured, not documented

The 16384 boundary, the masking, and the close echo are behavior of this build, printed by `WebSocketEcho` with its date, not promises of the specification. RFC 6455 only requires that fragments exist and that clients mask. Re-measure before relying on a number.

The capstone in chapter 32 grows this peer into the line clone's server: one accept loop, one frame parser, the same close handshake.

sources: Java in a Nutshell 8th edition, chapter 10 (File Handling and I/O) and chapter 13 (Platform Tools) for the 8-through-17 stream, channel, path, socket, and urlconnection grounding, rewritten here. openjdk.org/jeps/321 (http client standard, 11), /jeps/213 (effectively final try-with-resources, 9), /jeps/517 (HTTP/3, 26), /jeps/408 (simple web server, 18), docs.oracle.com javadoc for `java.nio.file.Files` at java 11 (readString, writeString, mismatch), RFC 6455 (tools.ietf.org/html/rfc6455) for the websocket handshake, frame, and close rules, all accessed 2026-10-04. Every number in this chapter is measured live on `tools/jdk27/build/jdk-27` by the Ch14 samples under `pwsh tools/run-java-samples.ps1 -Chapter Ch14`: 38 checks, the fragmentation boundary, the masked frames, the echoed close code, and the jwebserver fetch, dated 2026-10-04.

15. modules and jlink

Before java 9 the runtime was one monolithic `rt.jar` plus a flat classpath. Every class was reachable from every other, the only encapsulation was the package private keyword, and half the ecosystem reached into `sun.misc.Unsafe` and friends, which is why the jdk could not change its own internals for a decade. Java 9 shipped the java platform module system, split the jdk itself into modules, and gave the language a descriptor that states what a code unit requires, what it offers, and what it keeps private. This chapter builds one module live, breaks its boundary on purpose, analyzes it with jdeps, links it into a custom runtime with jlink, and ends with the java 25 import that shrinks module consumption to one line.

15.1. the classpath world and the unnamed module

The classpath still exists and this book's own samples ride it. All classpath code lands in the unnamed module, which reads every module that exports an api and can be read by nobody in return. That is the migration bridge: no module descriptor, no benefits, full compatibility:

```
  static void unnamedModule() {
    var own = Modules.class.getModule();
    ok(!own.isNamed(), "classpath code runs in the unnamed module");
    ok(own.getName() == null, "the unnamed module reports a null name");
    var base = Object.class.getModule();
    ok(own.canRead(base), "the unnamed module reads every module that exports an api");
    ok(base.getDescriptor().requires().isEmpty(), "java.base requires nothing, it is the root of the
graph");
    var boot = ModuleLayer.boot();
    ok(boot.findModule("java.net.http").isPresent(), "java.net.http resolves for a classpath launch");
    ok(boot.findModule("jdk.httpserver").isPresent(), "jdk.httpserver is a default root for classpath
code");
    ok(boot.findModule("jdk.crypto.cryptoki").isPresent(), "jdk.crypto.cryptoki rides along as an exporting
system module");
    ok(boot.modules().size() == 61, "the boot layer resolves 61 modules for this classpath launch");
    System.out.println("java.base exports "
        + base.getDescriptor().exports().size() + " packages, boot layer 61 modules");
  }

  // the greet module: one exported api package, one hidden impl package,
```

listing 125: the sample asserting its own unnamed module facts (source: java/samples/src/Ch15/Modules.java, lines 59-75)

The measured numbers on the pinned jdk 27: a classpath launch resolves 61 modules into the boot layer, `java --list-modules` observes 66, `java.base` requires nothing and exports 116 packages, and the exporters among the `jdk.*` set, `jdk.httpserver`, `jdk.crypto.cryptoki`, `jdk.crypto.ec`, are default roots, which is why chapter 14's jwebserver probe needed no `--add-modules` flag.

15.2. the descriptor

A module is a `module-info.java` at the source root. The ten words it is built from, `module`, `requires`, `exports`, `opens`, `to`, `uses`, `provides`, `with`, `open`, `transitive`, are restricted keywords, tokens only inside the descriptor, so no variable named `opens` anywhere breaks. The chapter's module states a dependency on `java.net.http`, exports one api package, and binds a service to a class in a package it does not export:

```
    Files.writeString(src.resolve("module-info.java"), """
        module greet {
          requires java.net.http;
          exports greet.api;
          uses greet.api.Greeter;
          provides greet.api.Greeter with greet.internal.SecretGreeter;
        }
        """);
```

listing 126: the greet descriptor: requires, exports, uses, provides (source: java/samples/src/Ch15/Modules.java, lines 81-88)

Compilation turns it into `module-info.class`, bytecode metadata, not a manifest convention. Running it is a module path launch, `java -p out -m greet/greet.api.Main`, and the resolution happens before main: the launcher reads the descriptor, pulls `java.net.http` and `java.base` into the graph, and refuses to start if a require is missing.

The boundary is real at compile time. A classpath class that says `new greet.internal.SecretGreeter()` fails with `package greet.internal is not visible`, because module greet does not export that package, and the sample captures that exact diagnostic before reopening it with the escape hatch, `--add-exports greet/greet.internal=ALL-UNNAMED`, which compiles and runs the same source. `--add-opens` is the reflective sibling, it allows `setAccessible` into the package where `--add-exports` only allows static references. Both are migration bandages, never shipped configuration.

`exports` controls compile time and deep reflection is `opens`. An `open module` opens every package, the honest first step when migrating a reflection-heavy framework, and `opens some.package to specific.module` narrows it back down. Chapter 16 returns to this from the reflection side.

15.3. services

The services mechanism solves the last gap: the point of an api package is to hide implementations, but `new` needs the implementation class reachable. `provides` declares the binding inside the provider's own module, `uses` declares consumption, and `ServiceLoader` does the lookup:

```
    Files.writeString(src.resolve("greet/api/Main.java"), """
        package greet.api;

        import java.net.http.HttpClient;
        import java.util.ServiceLoader;

        public class Main {
          public static void main(String[] args) {
            var greeter = ServiceLoader.load(Greeter.class).findFirst()
                .orElseThrow(() -> new IllegalStateException("no provider found"));
            System.out.println("greeting=" + greeter.greet()
                + " http=" + HttpClient.Version.HTTP_1_1.name());
            System.out.println("module=" + Main.class.getModule().getName()
                + " named=" + Main.class.getModule().isNamed());
          }
        }
        """);
    var out = work.resolve("out/greet");
```

listing 127: the main class loads the service and proves the required module is live (source: java/samples/src/Ch15/Modules.java, lines 108-125)

The provider sits in `greet.internal`, unexported, unreachable by import, and fully usable through the interface. Before modules the same trick needed `META-INF/services` files on the classpath, which still works, unnamed module code reads those, but the descriptor form carries the contract where the compiler and the linker can see it.

15.4. jdeps and jlink

Two jdk tools consume the descriptor. `jdeps` is static analysis: it reads bytecode, reports package and module dependencies, and its `-m greet` run against the built module prints the `java.net.http` edge and both packages, the exported and the hidden. It is the tool for the migration question, what does this jar actually need.

`jlink` assembles modules into a custom runtime image, a directory with its own `bin/java`, that runs with no installed jdk:

```
  static void jlink(Path jlinkBin, Path jdkHome, Path greet, Path work) throws Exception {
    var image = work.resolve("image");
    var modulePath = greet + File.pathSeparator + jdkHome.resolve("jmods");
    var run = run(jlinkBin, List.of("--module-path", modulePath,
        "--add-modules", "greet", "--launcher", "greet=greet/greet.api.Main",
        "--output", image.toString()));
    ok(run.exit() == 0, "jlink links a custom runtime image for the module");
    var bin = image.resolve("bin");
    var exes = 0;
    long imageBytes = 0;
    try (var stream = Files.walk(image)) {
      for (var p : stream.toList()) {
        imageBytes += Files.size(p);
        if (p.getParent().equals(bin) && p.getFileName().toString().endsWith(".exe")) {
          exes++;
        }
      }
    }
    ok(Files.exists(bin.resolve(isWindows() ? "java.exe" : "java")), "the image carries its own java
launcher");
    ok(Files.exists(bin.resolve(isWindows() ? "greet.bat" : "greet")), "the image carries the greet
launcher script");
    ok(exes == 3, "the image bin holds 3 executables: java, keytool, and the launcher");
    var runImage = run(bin.resolve(isWindows() ? "java.exe" : "java"), List.of("-m", "greet/
greet.api.Main"));
    ok(runImage.exit() == 0 && runImage.out().contains("module=greet named=true"),
        "the image runs the module without any installed jdk");
    long jdkBytes = 0;
```

listing 128: linking the image, counting it, and running it (source: java/samples/src/Ch15/Modules.java, lines 192-216)

Measured on this machine, 2026-10-04: the greet image is 51.5 mb against the full 369.2 mb jdk, and its `bin` holds 3 executables, `java`, `keytool`, and the `greet` launcher script the `--launcher greet=greet/greet.api.Main` option writes. The module path is the compiled module plus the jdk's own `jmods` directory, because jlink links platform modules too. The image is the deployment story for cli tools and containers, and it only exists because modules made the dependency graph explicit enough to trace.

15.5. module import declarations

Java 25 finalized module import declarations (JEP 511, previewed in 23 and 24): `import module java.base;` imports every public top level type of every package the module exports, so the 54 package imports a `java.base`-heavy file needs collapse to one line:

```
    Files.writeString(src.resolve("ModImp.java"), """
        import module java.base;

        public class ModImp {
          public static void main(String[] args) throws Exception {
            var lines = new BufferedReader(new StringReader("a\\nb")).lines().toList();
            System.out.println("sizes=" + List.of(1, 2, 3).size() + lines);
          }
        }
```

listing 129: one import line replaces the per-package imports, final since 25 (source: java/samples/src/Ch15/Modules.java, lines 230-238)

The sample compiles it live with the in-process compiler. Two rules keep it predictable: single type imports shadow module imports, so an ambiguous simple name like `Date`, `java.util` versus `java.sql`, is resolved by writing the specific import you mean, and `import module java.se` fails in classpath code because `java.se` is not a default root for the unnamed module. Classpath code can import named modules, never the unnamed one back.

NOTE: modules at this book's scale

The service spine in chapters 19 through 31 runs on the classpath like these samples do, one `jdk.httpserver` application with no descriptor, because a single-module application gains nothing from describing itself. Modules pay off the moment a codebase splits into parts that must not see each other, or ships as a jlink image.

sources: Java in a Nutshell 8th edition, chapters 12 (Java Platform Modules) and 13 (Platform Tools: jdeps, jlink, jar) for the 8-through-17 grounding, rewritten here, openjdk.org/jeps/511 (module import declarations final in 25, preview history 23 and 24) and openjdk.org/projects/jdk/9 (the jep 261 module system, jshell, and jlink landing) accessed 2026-10-04. Behavior verified live on `tools/jdk27/build/jdk-27` by the Ch15 sample under `pwsh tools/run-java-samples.ps1 -Chapter Ch15`: 28 checks covering the unnamed module facts, the compile, run, encapsulation refusal with its exact diagnostic, the add-exports reopening, jdeps output, the 51.5 mb image running headless, and the module import compile, all dated 2026-10-04.

16. reflection and method handles

Reflection is the runtime looking at types it did not know at compile time: what methods a class has, what annotations sit on them, how to build an instance from a name. Every framework that wires objects together, every test runner, every serializer, is reflection underneath. Java grew two answers to it, the 1996 `java.lang.reflect` api and the java 7 `java.lang.invoke` api, method handles, built for `invokedynamic` and now the machinery under every lambda. This chapter walks both, then the hardening arc, records that refuse mutation since 16 and the final means final warning since 26.

16.1. class objects

A `Class` object is a live type inside the running process, and there are three doors into one, a literal, `getClass`, and a lookup by name:

```
static void classObjects() throws Exception {
  Class<?> byLiteral = Square.class;
  Class<?> byInstance = new Square(2).getClass();
  Class<?> byName = Class.forName("Reflect$Square");
  ok(byLiteral == byInstance && byLiteral == byName,
      "literals, getClass, and forName agree on one class object per type");
  ok(int.class == Integer.TYPE, "int.class and Integer.TYPE are the same object");
  ok(Integer.class != int.class, "the wrapper class differs from the primitive class");
  ok(Square.class.isRecord(), "records identify themselves since 16");
  var comps = Square.class.getRecordComponents();
  ok(comps.length == 1 && comps[0].getName().equals("side") && comps[0].getType() == double.class,
      "getRecordComponents lists the accessor signature since 16");
  ok(Square.class.getPermittedSubclasses() == null,
      "a record without permits has no sealed hierarchy, isRecord and permits are separate facts");
}
```

listing 130: one class object per type, wrappers distinct from primitives, records visible (source: java/samples/src/Ch16/Reflect.java, lines 59-74)

Primitives participate: `int.class` and `Integer.TYPE` are the same object, `Integer.class` is a different one, and the awkward `Object[]` based api shows its age right there. The class object carries the metadata the compiler used, constructors, fields, method signatures, annotations, and the reflective api is a walk over that metadata.

16.2. invoking and constructing

The workhorse is `Method`. `getMethod` returns public methods, `getDeclaredMethod` reaches the private ones after `setAccessible`, and `invoke` calls with an `Object[]` for arguments, boxing the primitives both ways:

```
static void invoking() throws Exception {
  var area = Square.class.getMethod("area");
  var box = (Double) area.invoke(new Square(3));
  ok(box == 9.0, "Method.invoke boxes the primitive return into Object");
  var ctor = Square.class.getDeclaredConstructor(double.class);
  var made = ctor.newInstance(4);
  ok(made.side() == 4.0, "Constructor.newInstance builds without the new keyword");
}
```

listing 131: invoke boxes the return, the constructor builds the record (source: java/samples/src/Ch16/Reflect.java, lines 75-83)

Constructors are methods without names, found by parameter types, `Constructor.newInstance` builds without `new`. The old shortcut `Class.newInstance` is deprecated since java 9, and the annotation on it is itself readable reflectively, which is the next subject.

16.3. annotations at runtime

An annotation only exists at runtime if its retention policy says so, `RetentionPolicy.RUNTIME`. Class file retention, the default, keeps it for compilers and tools but reflection cannot see it. The runtime-retained ones come back as proxy objects generated by the same machinery as the last section of this chapter:

```
static void annotations() throws Exception {
  var area = Square.class.getMethod("area");
  ok(area.isAnnotationPresent(Audit.class), "runtime retention keeps a custom annotation live");
  ok(area.getAnnotation(Audit.class).value().equals("area"), "annotation values read reflectively");
  var newInstance = Class.class.getMethod("newInstance");
  ok(newInstance.isAnnotationPresent(Deprecated.class),
      "the jdk's own deprecations, Class.newInstance since 9, are readable the same way");
  ok(newInstance.getAnnotation(Deprecated.class).since().equals("9"),
      "since and forRemoval are annotation data, this one deprecates without removal");
}
```

listing 132: a custom runtime annotation, and the jdk's own deprecation metadata (source: java/samples/src/Ch16/Reflect.java, lines 84-94)

16.4. records and the reflective contract

Records, final since java 16, are transparent data carriers and the reflection api knows it: `isRecord` and `getRecordComponents` expose the component list, name and type, that generated the accessor, `equals`, `hashCode`, and `toString`. The contract cuts the other way too, the components are truly final. `setAccessible(true)` on a record field succeeds, and the write still throws `IllegalAccessException`, measured in the sample, because the language promised the fields never change after construction. That promise is what makes records safe as map keys and value objects.

16.5. final means final

Records were the first hardening step. Java 26 (JEP 500) extends the same discipline to every final field, reflectively writable since 1.2 for serialization and mocking frameworks, and now on a removal path. On this build the mutation still works, but it warns:

```
static void finalFieldWarning() throws Exception {
  var dir = Files.createTempDirectory("ch16-final");
  Files.writeString(dir.resolve("Mutant.java"), """
      public class Mutant {
        static class Holder { final int f; Holder(int v) { f = v; } }
        public static void main(String[] args) throws Exception {
          var h = new Holder(1);
          var field = Holder.class.getDeclaredField("f");
          field.setAccessible(true);
          field.setInt(h, 42);
          System.out.println("value=" + field.getInt(h));
        }
      }
      """);
  var javaBin = ProcessHandle.current().info().command().orElseThrow();
  var proc = new ProcessBuilder(javaBin, dir.resolve("Mutant.java").toString())
      .redirectErrorStream(true).start();
  var out = new String(proc.getInputStream().readAllBytes(), StandardCharsets.UTF_8);
  proc.waitFor();
  ok(out.contains("value=42"), "on 27 the mutation still succeeds, warn is the default");
  ok(out.contains("WARNING: Final field f in class Mutant$Holder has been mutated reflectively"),
      "the warning names the field, the class, and the caller");
  ok(out.contains("--enable-final-field-mutation"),
      "the warning names the flag that opts a module back in");
  ok(out.contains("blocked in a future release"),
      "the warning states the schedule, deny comes later");
}
```

listing 133: the subprocess probe compiling and running the mutation, capturing the warning (source: java/samples/src/Ch16/Reflect.java, lines 113-139)

The measured warning names the field, the class, and the caller's module, points at `--enable-final-field-mutation=ALL-UNNAMED` as the opt out, and states the schedule, a future release denies by default. `--illegal-final-field-mutation=deny` front-runs that today, `Field.set` and `MethodHandles.Lookup.unreflectSetter` then throw `IllegalAccessException`, and a JFR event records offenders. The warning cannot be silenced with `--add-opens`, opening a package grants access, not mutation rights.

PITFALL: constant folding hides the mutation

A `final int f = 1` initialized with a constant expression is a constant variable, and javac inlines reads of it at the use site. Reflectively mutated or not, the direct expression still prints 1, because the class reading it never looked at the field. Half the folklore that reflective final mutation works is this illusion.

16.6. dynamic proxies

`Proxy.newProxyInstance` builds a class at runtime that implements an interface and routes every call, including `toString`, `equals`, and `hashCode`, to one `InvocationHandler`:

```
static void proxies() {
  InvocationHandler handler = (proxy, method, margs) -> switch (method.getName()) {
    case "area" -> 42.0;
    case "name" -> "proxy";
    case "toString" -> "proxied";
    case "hashCode" -> 7;
    case "equals" -> proxy == margs[0];
    default -> null;
  };
  var proxy = (Shape) Proxy.newProxyInstance(Reflect.class.getClassLoader(),
      new Class<?>[] {Shape.class}, handler);
  ok(proxy.area() == 42.0 && "proxy".equals(proxy.name()),
      "a dynamic proxy implements any interface at runtime");
  ok(proxy.toString().equals("proxied"), "object methods route through the handler too");
}
```

listing 134: one handler answers an interface with a switch over method names (source: java/samples/src/Ch16/Reflect.java, lines 141-155)

Proxies are the smallest mock framework and the core of chapter 18's test doubles discussion. They only proxy interfaces, the handler is the entire implementation, and since java 16 that handler is usually one lambda with a switch over the method name.

16.7. method handles

Method handles are the second reflection, designed not for frameworks looking at unknown types but for call sites that change their target at runtime, and `invokedynamic`, the bytecode instruction behind every lambda since java 8. The type of a handle is a `MethodType`, return type first, argument types after, no method name and no receiver, because any method of that shape can ride it:

```
  public static void main(String[] args) throws Throwable {
    var mtString = MethodType.methodType(String.class);
    var mtParse = MethodType.methodType(int.class, String.class);
    ok(mtString.toMethodDescriptorString().equals("()Ljava/lang/String;"),
        "a method type is the descriptor without the name");
    ok(MethodType.methodType(int.class, String.class).equals(mtParse),
        "method types with the same shape are one interned value");

    var lookup = MethodHandles.lookup();
    var tostring = lookup.findVirtual(String.class, "toString", mtString);
    ok("hi".equals(tostring.invoke("hi")), "findVirtual binds the receiver at invocation");
    var parse = lookup.findStatic(Integer.class, "parseInt", mtParse);
    ok((int) parse.invoke("42") == 42, "findStatic takes a static by exact type");

    var generic = parse.asType(MethodType.methodType(Object.class, Object.class));
    ok(generic.invoke("41").equals(41), "asType widens to the object signature when composition needs it");

    var bound = tostring.bindTo("bound");
    ok("bound".equals(bound.invoke()), "bindTo pins the receiver into a new handle");

    IntUnaryOperator reference = Handles::inc;
    ok(reference.applyAsInt(1) == 2, "a method reference is an invokedynamic site, not reflection");

    timings();
  }

  // micro comparison: direct, lambda, method handle, generic handle, reflection.
  // numbers are machine specific, printed not asserted, only correctness asserts
```

listing 135: method types, lookup, findVirtual and findStatic, asType and bindTo (source: java/samples/src/Ch16/Handles.java, lines 26-53)

The big difference from `java.lang.reflect` is access control. A `Lookup` object is created at a specific call site, `MethodHandles.lookup()`, and it can only find what that site could have written `someMethod` for. There is no `setAccessible` equivalent and no way to subvert it, which is why the same jdk can offer handles to untrusted frameworks. Composition is the other strength, `asType` adapts signatures, `bindTo` pins a receiver, `filterArguments` and `collectArguments` reshape pipelines, operations that on `java.lang.reflect` would be manual array shuffling.

The cost story, measured on this machine, best of five rounds of a million calls, 2026-10-04:

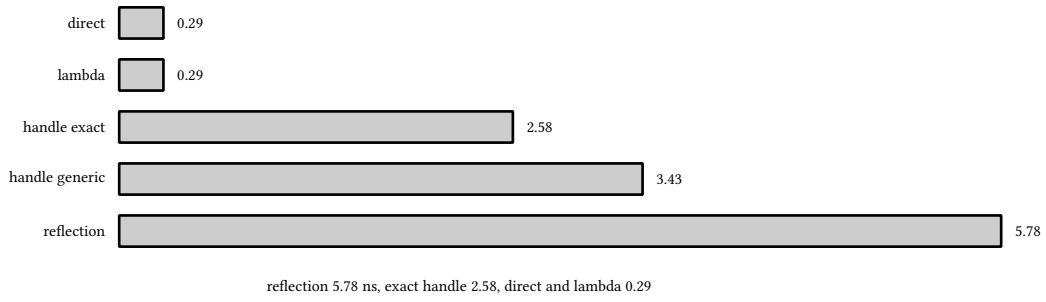


figure 21: call mechanisms by measured cost, nanoseconds per call, lower is better

VERIFY: micro numbers, read with care

These are best-of-five nanoseconds on one machine with the jit warm, printed by the `Handles` sample each run. The honest claims are the shape, direct and lambda are free after inlining, an exact method handle is single digit nanoseconds, the object-typed adaptation pays for boxing, and reflection costs about double a generic handle. The table on another cpu, or another jdk, shifts.

The equality assertion next to the measurement is the part that matters for correctness, all five paths compute the same sum. When the mechanism, not the speed, is the reason to reach for handles, varhandles, the field flavored sibling with memory ordering modes, are chapter 11 territory.

sources: Java in a Nutshell 8th edition, chapter 11 (Classloading, Reflection, and Method Handles) for the class object, method, constructor, proxy, method type, and lookup grounding, rewritten here, openjdk.org/jeps/500 (prepare to make final mean final, 26) and /jeps/395 (records final, 16) accessed 2026-10-04, plus the java 27 javadoc for `Deprecated` annotation attributes. Behavior verified live on `tools/jdk27/build/jdk-27` by the Ch16 samples under `pwsh tools/run-java-samples.ps1 -Chapter Ch16`: 28 checks, Reflect 20 and Handles 8, covering the class object identities, the boxing round trip, the runtime annotation reads including `Class.newInstance` since 9 forRemoval false, the record field refusal after setAccessible, the final field mutation warning captured from a subprocess with its opt out flag text, the proxy handler answering object methods, the method type descriptor, and the five way timing equality, all dated 2026-10-04.

17. security and cryptography

Java ships its cryptography as an architecture, not a library. Algorithms are requested by name, `MessageDigest.getInstance("SHA-256")`, and a chain of providers answers, the jdk's own first and hardware or third party implementations behind registration. This build resolves 13 providers, from `SUN` through `SunJCE` and `SunJSSE` to the Windows `SunMSCAPI` and the `SunPKCS11` bridge, measured by the sample. The payoff is that no line of this chapter names an implementation class, and the platform can swap what answers.

17.1. digests and macs

A digest is a one way fingerprint, a mac is a keyed one. Both are `getInstance` by algorithm, both verify against known answer vectors, published input and output pairs, which is how the sample proves it is calling the real algorithm and not something shaped like it:

```
  static void digests() throws Exception {
    var first = MessageDigest.getInstance("SHA-256").digest("abc".getBytes(StandardCharsets.UTF_8));
    var second = MessageDigest.getInstance("SHA-256").digest("abc".getBytes(StandardCharsets.UTF_8));
    ok(Arrays.equals(first, second), "digests are deterministic across instances");
    ok(first.length == 32, "sha-256 emits 32 bytes");
    ok(HexFormat.of().formatHex(first)
        .equals("ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"),
        "the sha-256 known answer vector for abc");
  }

  static void macs() throws Exception {
    var mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(HexFormat.of().parseHex("0b".repeat(20)), "HmacSHA256"));
    var tag = mac.doFinal("Hi There".getBytes(StandardCharsets.UTF_8));
    ok(HexFormat.of().formatHex(tag)
        .equals("b0344c61d8db38535ca8afceaf0bf12b881dc200c9833da726e9376c2e32cff7"),
        "the hmac-sha256 rfc 4231 test case 1 vector");
  }
```

listing 136: the sha-256 vector for abc and the hmac rfc 4231 test case 1 (source: java/samples/src/Ch17/Crypto.java, lines 40-57)

Digests detect corruption, macs detect tampering, and neither hides anything, both outputs are public. Encryption comes later in the chapter.

17.2. secure random

`SecureRandom` is the randomness source for every key and nonce in this chapter. Its default algorithm on this build is `DRBG`, the nist deterministic random bit generator, seeded from the operating system. The determinism check runs the other way, two draws must differ, and any code that seeds it with a constant or a timestamp is the vulnerability report of some other book.

17.3. key derivation

Passwords are not keys, they are low entropy strings, and turning them into key material needs a function that is deliberately expensive. PBKDF2, `SecretKeyFactory` with `PBKDF2WithHmacSHA256`, takes the password, a random salt, an iteration count, and an output width:

```
  static void pbkdf2() throws Exception {
    var random = new SecureRandom();
    var salt = new byte[16];
    random.nextBytes(salt);
    var factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
    var spec = new PBEKeySpec("correct horse".toCharArray(), salt, 100_000, 256);
    long t0 = System.nanoTime();
    var key = factory.generateSecret(spec).getEncoded();
    long nanos = System.nanoTime() - t0;
    var same = factory.generateSecret(
        new PBEKeySpec("correct horse".toCharArray(), salt, 100_000, 256)).getEncoded();
    var otherSalt = new byte[16];
    random.nextBytes(otherSalt);
    var other = factory.generateSecret(
        new PBEKeySpec("correct horse".toCharArray(), otherSalt, 100_000, 256)).getEncoded();
    ok(key.length == 32, "256 requested bits arrive as 32 bytes");
    ok(Arrays.equals(key, same), "same password and salt derive the same key");
    ok(!Arrays.equals(key, other), "a different salt derives a different key");
    ok(nanos > 10_000_000L, "100,000 iterations cost real time, " + nanos / 1_000_000 + " ms measured");
  }

  // the kdf api, final since 25 (jep 510), hkdf via SunJCE
```

listing 137: salted, iterated derivation with the same-salt determinism checks (source: java/samples/src/Ch17/Crypto.java, lines 69-90)

The salt kills precomputed rainbow tables, the iteration count makes each guess cost the attacker the same tens of milliseconds it costs you once, measured in the sample, and the width feeds an aes key directly. Chapter 22's authn pipeline derives its stored verifier exactly this way.

For high entropy input, raw key material rather than passwords, java 25 finalized the key derivation function api (JEP 510) with HKDF as the bundled function. On this build SunJCE answers with `HKDF-SHA256`, `HKDF-SHA384`, and `HKDF-SHA512`:

```
  static void hkdf() throws Exception {
    var kdf = javax.crypto.KDF.getInstance("HKDF-SHA256");
    var ikm = new byte[32];
    var salt = new byte[32];
    var info = "handshake".getBytes(StandardCharsets.UTF_8);
    var spec = javax.crypto.spec.HKDFParameterSpec.ofExtract()
        .addIKM(ikm).addSalt(salt).thenExpand(info, 32);
    var key = kdf.deriveKey("AES", spec);
    var again = kdf.deriveKey("AES", spec);
    ok(key.getAlgorithm().equals("AES") && key.getEncoded().length == 32,
        "hkdf derives a ready aes key, final since 25");
    ok(Arrays.equals(key.getEncoded(), again.getEncoded()), "hkdf is deterministic for the same inputs");
    var expanded = kdf.deriveData(javax.crypto.spec.HKDFParameterSpec.expandOnly(key, info, 16));
    ok(expanded.length == 16, "expandOnly derives raw bytes from a pseudorandom key");
  }
```

listing 138: extract then expand through the builder, deriveKey and deriveData (source: java/samples/src/Ch17/Crypto.java, lines 91-106)

The shape is extract, squeeze the input into a pseudorandom key, then expand, stretch that key into as much output as needed with an info label for domain separation. It is the derivation style of modern protocols, TLS 1.3 included.

17.4. aes-gcm

Authenticated encryption in one primitive: `AES/GCM/NoPadding` takes a key, a unique nonce, and produces ciphertext with a 16 byte authentication tag appended, tampering fails the tag instead of decrypting to garbage:

```
static void gcm() throws Exception {
  var random = new SecureRandom();
  var keyBytes = new byte[32];
  random.nextBytes(keyBytes);
  var key = new SecretKeySpec(keyBytes, "AES");
  var iv = new byte[12];
  random.nextBytes(iv);
  var plaintext = "attack at dawn".getBytes(StandardCharsets.UTF_8);

  var cipher = Cipher.getInstance("AES/GCM/NoPadding");
  cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
  var sealed = cipher.doFinal(plaintext);
  ok(sealed.length == plaintext.length + 16, "gcm appends its 16 byte auth tag to the ciphertext");

  cipher.init(Cipher.DECRYPT_MODE, key, new GCMParameterSpec(128, iv));
  ok(Arrays.equals(cipher.doFinal(sealed), plaintext), "the round trip restores the plaintext");

  // the classic gcm sin is nonce reuse, and the platform refuses it
  try {
    cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
    cipher.doFinal(plaintext);
    ok(false, "the platform refuses gcm iv reuse");
  } catch (java.security.InvalidAlgorithmParameterException e) {
    ok(true, "sunjce refuses gcm iv reuse in encrypt mode: " + e.getMessage());
  }

  var tampered = sealed.clone();
```

listing 139: the round trip, the appended tag, the tamper failure, and the refused nonce reuse (source: java/samples/src/Ch17/Crypto.java, lines 107-133)

The measured facts: the ciphertext is exactly 16 bytes longer than the plaintext, a single flipped bit raises `AEADBadTagException`, and re-initializing encryption with the same key and nonce throws `InvalidAlgorithmParameterException: Cannot reuse iv for GCM encryption`, SunJCE tracks nonces per key and refuses the classic sin at the api level. GCM with a repeated nonce is a catastrophe, the authentication key leaks, and the platform simply will not do it.

17.5. tls end to end

The transport layer rides the same provider architecture through `SunJSSE`. The sample builds the whole stack live: `keytool` mints a self signed certificate into a pkcs12 keystore with the loopback address as a subject alternative name, `KeyManagerFactory` hands the server side to an `SSLContext`, and the client trusts exactly that one certificate, the pinning pattern, instead of a store of public authorities:

```
    TrustManager pin = new X509TrustManager() {
      @Override
      public void checkClientTrusted(X509Certificate[] chain, String authType) {
      }

      @Override
      public void checkServerTrusted(X509Certificate[] chain, String authType) {
        if (!chain[0].equals(serverCert)) {
          throw new SecurityException("unexpected server certificate");
        }
      }

      @Override
      public X509Certificate[] getAcceptedIssuers() {
        return new X509Certificate[] {serverCert};
      }
    };
    var serverContext = SSLContext.getInstance("TLS");
    serverContext.init(kmf.getKeyManagers(), null, new SecureRandom());
    var clientContext = SSLContext.getInstance("TLS");
    clientContext.init(null, new TrustManager[] {pin}, new SecureRandom());
```

listing 140: a trust manager that accepts one certificate, and the context wiring (source: java/samples/src/Ch17/Tls.java, lines 72-93)

```
      try (var raw = (SSLSocket) clientContext.getSocketFactory()
          .createSocket(loop, port)) {
        raw.getOutputStream().write("GET / HTTP/1.1\r\nHost: 127.0.0.1\r\nConnection: close\r\n\r\n"
            .getBytes(StandardCharsets.UTF_8));
        raw.getOutputStream().flush();
        var body = new String(raw.getInputStream().readAllBytes(), StandardCharsets.UTF_8);
        ok(body.endsWith(BODY), "the raw ssl socket reads the response body");
        ok(raw.getSession().getProtocol().equals("TLSv1.3"),
            "the negotiated protocol is tls 1.3, " + raw.getSession().getCipherSuite());
        System.out.println("cipher suite " + raw.getSession().getCipherSuite());
      }

      // client 2: the standard http client over the same pinned context
      var http = HttpClient.newBuilder().sslContext(clientContext).build();
```

listing 141: a raw ssl socket client, protocol and suite read off the session (source: java/samples/src/Ch17/Tls.java, lines 123-136)

Three clients then ride the same listener: the raw `SSLSocket` negotiating TLSv1.3 with the `TLS_AES_256_GCM_SHA384` suite, measured, the `java.net.http` client from chapter 14 over the same pinned context, and the 1996 era `HttpsURLConnection`, which still works and still has no reason to be chosen.

Java 27 turns on post-quantum key exchange by default (JEP 527). The hybrid group `X25519MLKEM768`, elliptic curve X25519 multiplied with the ML-KEM lattice scheme from FIPS 203, is the first entry in the default named group list, measured on a fresh client socket, followed by the classic groups, `x25519`, `secp256r1`, the ffdhe family. A tls 1.3 client offers key shares for the hybrid and for `x25519` together, so a peer that lacks the hybrid falls back without a failed handshake. Opting out means `jdk.tls.namedGroups` or `SSLParameters.setNamedGroups`, pinning the traditional list.

Two adjacent 27 facts, verified live in the sample: PEM encodings of cryptographic objects (JEP 538) are in their third preview, the `java.security.PEMEncoder` and `PEMDecoder` types compile only under `--enable-preview --release 27` and refuse without it with `PEMEncoder is a preview API`, and the preview run prints real RFC 7468 output, `-----BEGIN PUBLIC KEY-----`. And the key derivation api note above is the settled sibling, final since 25.

VERIFY: protocol facts are build facts

The first-named group, the negotiated suite, the refusal text for iv reuse, these are this build's behavior, printed with dates by the samples. The specification fixes less than the output suggests, a different provider order or a disabled algorithm list shifts them.

17.6. the security manager's exit

The sandbox api had a long goodbye: deprecated for removal in java 17 (JEP 411), permanently disabled in java 24 (JEP 486). On 27 the measured behavior is blunt, `System.getSecurityManager` always returns null and `System.setSecurityManager` throws `UnsupportedOperationException`, and the deliberate probe of the second is the one place in this book's samples that carries a `@SuppressWarnings("removal")`, because `-Xlint:all -Werror` flags any touch of removal marked api and the touch is the point:

```
@SuppressWarnings("removal")
static void securityManager() {
  ok(System.getSecurityManager() == null, "getSecurityManager returns null, always, since 24");
  try {
    System.setSecurityManager(null);
    ok(false, "setSecurityManager throws");
  } catch (UnsupportedOperationException e) {
    ok(true, "setSecurityManager throws UnsupportedOperationException since 24, jep 486");
  }
```

listing 142: the null and the throw, the whole remaining surface (source: java/samples/src/Ch17/Crypto.java, lines 145-153)

There is no replacement sandbox inside the jdk, the position is that isolation belongs to the operating system, containers, hypervisors, and process boundaries. What remains of the security package is the cryptography above and the policy free `AccessController` remnants scheduled to follow the manager out.

sources: openjdk.org/jeps/510 (key derivation function api final in 25, HKDF), /jeps/527 (post-quantum hybrid key exchange default in 27, X25519MLKEM768 ordering and opt outs), /jeps/538 (pem encodings third preview in 27), /jeps/411 (deprecate security manager, 17) and /jeps/486 (permanently disable it, 24), all accessed 2026-10-04, RFC 4231 (identifying hmac test vectors, rfc-editor.org), RFC 7468 (pem text encoding), docs.oracle.com javadoc for `javax.crypto.KDF`, `HKDFParameterSpec`, and `Cipher` on java 27. Behavior verified live on `tools/jdk27/build/jdk-27` by the Ch17 samples under `pwsh tools/run-java-samples.ps1 -Chapter Ch17`: 30 checks, Crypto 19 and Tls 11, covering both known answer vectors, the drbg default, the pbkdf2 cost in milliseconds, hkdf determinism, the gcm tag length, tamper failure and nonce reuse refusal, the full tls 1.3 exchange over a keytool minted certificate with three clients, the X25519MLKEM768 default group order, the pem preview refusal and acceptance, and the security manager null and throw, all dated 2026-10-04.

18. testing and idioms

This book tests java without a build tool, without a framework for most of it, and with one vendored jar where a framework is the right answer. The reason is the corpus contract: every chapter's claims are checked by code the reader can run with the pinned jdk and nothing else, the same way the c and python books run their samples. This chapter states the contract, builds the pieces a framework would otherwise hand you, and closes the manual half of the book with the idiom summary, what java 27 code looks like when none of the java 8 habits survive.

18.1. the ok contract

Every runnable sample embeds the same helper. `ok(boolean, String)` counts, prints `ok N name` per passing check, and on the first failure prints `FAIL name (check N)` to stderr and exits 1. The runner, `tools/run-java-samples.ps1`, compiles a chapter directory under `-Xlint:all -Werror`, runs every main bearing class, and goes red on a nonzero exit or a class that printed no ok lines. A green run prints its check count and a dated measurement line, which is where every number in this book's java prose comes from:

```
public static void main(String[] args) throws Exception {
  ok(true, "ok counts a passing check, a failing one prints FAIL and exits 1");

  // asserting failures is half of testing
  var divided = assertThrows(ArithmeticException.class, () -> divide(1, 0));
  ok("/ by zero".equals(divided.getMessage()), "assertThrows hands back the exception for inspection");
  var empty = assertThrows(java.util.NoSuchElementException.class, () -> List.of().getLast());
  ok(empty != null, "the sequenced collection getLast on empty throws, tested not assumed");

  // parameterized tests are tables: the one place values repeat by design
  record Row(String input, int expected) {
  }
  var rows = List.of(
      new Row("", 0),
      new Row("one", 1),
```

listing 143: the contract in use: count, assert the failure paths, run the table (source: java/samples/src/Ch18/OkContract.java, lines 43-57)

The shape is deliberate end to end testing. A sample drives real files, real sockets, real subprocesses, the way chapters 14 through 17 did, because mocks over the jdk would test the mock. The contract is also the honest budget: an assertion is a boolean and a name, anything fancier needs a reason.

18.2. asserts and tables by hand

The two framework conveniences worth building are exception assertion and parameterization. `assertThrows` returns the caught exception so the message can be checked, the failure mode a test cares about:

```
static <T extends Throwable> T assertThrows(Class<T> type, Runnable action) {
  try {
    action.run();
  } catch (Throwable thrown) {
    if (type.isInstance(thrown)) {
      return type.cast(thrown);
    }
    throw new AssertionError("expected " + type.getSimpleName() + " but " + thrown, thrown);
  }
  throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
}
```

listing 144: the junit shape without the framework, ten lines (source: java/samples/src/Ch18/OkContract.java, lines 21-32)

Parameterized tests are tables. A record per row, a list of rows, a loop of `ok` calls with the row in the name, and a failing row is legible in the output without a report file. The corpus makes one exception for repeated values, inline test tables, and the rule claws back anything that escapes the table:

```
static int wordCount(String text) {
  var words = text.strip().split("\\s+");
  return text.strip().isEmpty() ? 0 : words.length;
}

static int divide(int a, int b) {
  return a / b;
}
```

listing 145: the function under test and its table shape (source: java/samples/src/Ch18/OkContract.java, lines 34-41)

18.3. interleavings and contention

Java has no `-race` equivalent. The go book leans on a race detector, book 4, ch 6 (go 1.27, concurrency), this book cannot, so correctness of concurrent code rests on two disciplines. First, deterministic interleavings: choreograph the threads with latches and barriers until the order under test is the only possible order:

```
  static void interleavings() throws Exception {
    var box = new int[] {0};
    var bothRead = new CyclicBarrier(2);
    var done = new CountDownLatch(2);
    for (int t = 0; t < 2; t++) {
      Thread.ofPlatform().start(() -> {
        int value = box[0];
        try {
          bothRead.await();
        } catch (Exception e) {
          throw new RuntimeException(e);
        }
        box[0] = value + 1;
        done.countDown();
      });
    }
    done.await();
    ok(box[0] == 1,
        "two read modify writes over one barrier lose an update, 2 increments land as 1");

    var atomic = new AtomicLong();
    var finished = new CountDownLatch(2);
    for (int t = 0; t < 2; t++) {
      Thread.ofPlatform().start(() -> {
        atomic.incrementAndGet();
        finished.countDown();
      });
    }
    finished.await();
    ok(atomic.get() == 2, "the atomic fix counts both increments");
  }
```

listing 146: a barrier forces the lost update window, then the atomic fix (source: java/samples/src/Ch18/OkContract.java, lines 71-102)

The `CyclicBarrier` demo is the honest version of a race: both threads read, the barrier proves both reads happened before either write, and two increments land as one. It is a data race by design, isolated in two lines, with the joins providing the happens-before edge that makes the final read sound. Second, contention loops, the jcstress style from the openjdk's own concurrency torture tool: exact answers under load, asserted, with the timing printed:

```
  static void contention() throws Exception {
    final int threads = 4;
    final int each = 250_000;
    final long expected = (long) threads * each;

    long t0 = System.nanoTime();
    var atomicTotal = new AtomicLong();
    var atomicDone = new CountDownLatch(threads);
    for (int t = 0; t < threads; t++) {
      Thread.ofPlatform().start(() -> {
        for (int i = 0; i < each; i++) {
          atomicTotal.incrementAndGet();
        }
        atomicDone.countDown();
      });
    }
    atomicDone.await();
    long atomicNanos = System.nanoTime() - t0;
    ok(atomicTotal.get() == expected,
        "4 threads x 250,000 atomic increments land exactly 1,000,000");

    t0 = System.nanoTime();
    var monitor = new Object();
    var box = new long[] {0};
    var syncDone = new CountDownLatch(threads);
    for (int t = 0; t < threads; t++) {
      Thread.ofPlatform().start(() -> {
        for (int i = 0; i < each; i++) {
          synchronized (monitor) {
            box[0]++;
```

listing 147: 4 threads x 250,000 increments, atomic and monitor, exactness asserted (source: java/samples/src/Ch18/OkContract.java, lines 104-133)

Measured on this machine, 2026-10-04: the atomic loop ran 24.5 ms and the monitor loop 141.4 ms for the same million increments, both exact. The number is machine specific, the exactness is not. The service spine's chapter 31 suite builds its race story on these two disciplines plus golden vectors, fixed input and output pairs that pin the whole http pipeline end to end.

18.4. the junit lane

For class shaped tests the book vendors one jar, `junit-platform-console-standalone-6.1.3.jar` under `tools/junit/`, the fat jar from maven central dated 2026-08-07. The 6.x console is subcommand based, `execute`, `discover`, `engines`, and the lane this chapter's sample drives is:

```
static final Path JAR = Path.of("tools/junit/junit-platform-console-standalone-6.1.3.jar");

public static void main(String[] args) throws Exception {
  ok(JAR.getFileName().toString().equals("junit-platform-console-standalone-6.1.3.jar"),
      "the lane pins junit 6.1.3, the newest console standalone on central");
  if (!Files.exists(JAR)) {
    // wave 2 owns vendoring the jar; this branch prints, it never fails
    System.out.println("junit vendoring lands in wave 2");
    System.out.println("when the jar is present this lane compiles OkTests and runs:");
    System.out.println("java -jar " + JAR + " execute --select-class OkTests"
        + " --fail-if-no-tests --disable-ansi-colors --disable-banner -cp classes");
    System.out.println("exit codes: 0 pass, 1 failure, 2 no tests under --fail-if-no-tests");
    System.out.println("measured 2026-10-04 on tools/jdk27/build/jdk-27");
    return;
  }
  runLane();
}
```

listing 148: the pinned jar path, and the honest branch while wave 2 owns vendoring (source: java/samples/src/Ch18/JUnitLane.java, lines 21-37)

Exit codes carry the verdict: 0 for green, 1 when any test or container failed, 2 when nothing was discovered under `--fail-if-no-tests`, 3 for invalid input. Test classes follow the `*Tests` naming convention, junit 5 style, so discovery finds `OkTests` and skips helpers. Inside, the api is the jupiter one: `assertEquals`, `assertThrows` from `org.junit.jupiter.api`, and `@ParameterizedTest` with `@CsvSource` from the bundled jupiter-params, the framework version of the table loop above.

NOTE: vendoring status, stated

The jar lands in wave 2 of this book's build, so the sample's jar-free branch is what runs today: it prints the pinned coordinates and the exact command line it will run, and it never fakes a green line. When the jar appears, the same class compiles the `OkTests` source, runs `execute`, asserts the 5 successful count, asserts zero module warnings on 27, and asserts the exit 2 no-tests path. The `assertThrows` and csv assertions in this chapter's hand-built lane already run green today.

18.5. java 27 idioms, the wrap-up

Eighteen chapters in, the manual's working style is fixed. The java 8 habits and their replacements, in the order this book met them:

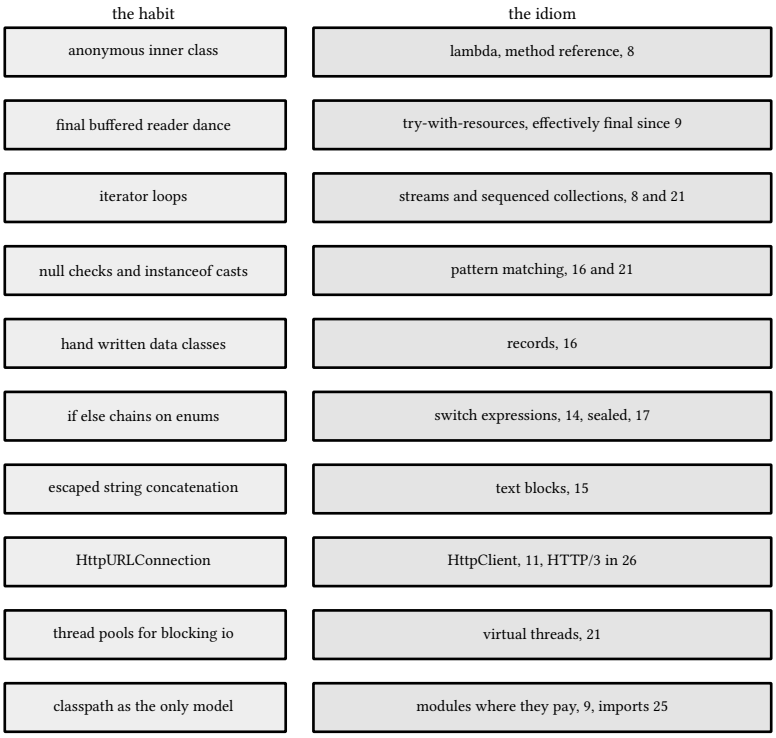


figure 22: java 8 habit on the left, java 27 idiom on the right

The two-line version of the whole manual: name what a thing is with a record or a sealed hierarchy, move data with streams or channels, block freely on a virtual thread, and let the compiler's warnings be errors. The service spine that starts in chapter 19 holds this line for thirteen chapters, and the suite in chapter 31 tests it with the contract from this one.

```
class Greeter {
  private final String name;
  Greeter(String name) { this.name = name; }
  String greet() { return "hello " + name; }
  // plus equals, hashCode, toString, getters
}
// versus, since 16:
record Greeter(String name) {
  String greet() { return "hello " + name; }
}
```

fragment, illustrative, not executed

sources: repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone (the 6.1.3 version listed, dated 2026-08-07) and docs.junit.org/6.1.3 user guide, console launcher page (the execute subcommand, the option set, the 0, 1, 2, 3 exit codes), both accessed 2026-10-04, github.com/openjdk/jcstress for the concurrency torture tool the contention style borrows, and book 4 of this corpus, chapter 6, for the go race detector contrast. Behavior verified live on `tools/jdk27/build/jdk-27` by the Ch18 samples under `pwsh tools/run-java-samples.ps1 -Chapter Ch18`: 13 checks, OkContract 12 and JUnitLane 1, covering the assertThrows contract, the parameter table, the barrier forced lost update, the atomic and monitor exactness with timings, the latch choreography order, and the pinned junit coordinates with the honest vendoring branch, all dated 2026-10-04.

19. http kernel

The service part of this book builds one api across thirteen chapters, and everything in it sits on the package this chapter builds: `javabook.httpx` under `java/api/src`. The jdk's own server, `jdk.httpserver`, gives us a listener, an exchange object, and not much else, so the kernel owns the rest: the route table with methods and path parameters, the constant error envelope every non-2xx response answers with, the json the jdk never shipped, the decode ladder every request body climbs, and the skeleton that later chapters wire their families into. The go book built the same layer on its standard mux, book 4, ch 18 (go 1.27, http kernel), and the contrasts with that chapter are the point of this one: where go's library hands you a router and a codec, java hands you a socket and a choice.

19.1. the server and the exchange

`com.sun.net.httpserver.HttpServer` lives in the `jdk.httpserver` module, which is a jdk module outside `java.se`, so the first line of the service is a module declaration that requires it by name:

```
// The service module for the book's api part. jdk.httpserver is a jdk
// module outside java.se, so it must be required by name; everything
// else the kernel needs lives in java.base.
module javabook {
  // transitive: javabook.httpx's public api speaks HttpExchange and
  // Filter, so every client of this module reads jdk.httpserver too.
  requires transitive jdk.httpserver;
  // the one custom jfr event, javabook.obs.RequestEvent; transitive
  // because the event type's superclass is public api from jdk.jfr
  requires transitive jdk.jfr;
  // the load client's one dependency, java.net.http.HttpClient
  requires java.net.http;

  exports javabook;
  exports javabook.httpx;
  exports javabook.middleware;
  exports javabook.conc;
  exports javabook.cache;
  exports javabook.limit;
  exports javabook.obs;
  exports javabook.load;
  exports javabook.user;
  exports javabook.authn;
  exports javabook.authz;
  exports javabook.store;
}
```

listing 149: the service module: jdk.httpserver by name, transitive because the kernel's api speaks its types, exports grow one line per family (source: java/api/src/module-info.java, lines 1-26)

The server itself is three calls. `HttpServer.create` binds the listening socket at construction, an `InetSocketAddress` of port 0 asks the kernel for an ephemeral port, which is what every test in this part boots on. `setExecutor` must run before `start`, and the executor is the whole concurrency story: `Executors.newVirtualThreadPerTaskExecutor()` means one virtual thread per request, the chapter 10 material doing real work, so a handler may block freely and a thousand slow clients cost a thousand cheap threads. `start` returns immediately, it does not block the caller the way go's `ListenAndServe` does, which is why `Main` ends by joining its own thread forever:

```
  public App(Config config) throws IOException {
    this.config = config;
    this.server = HttpServer.create(new InetSocketAddress(config.port()), 0);
  }

  /** The bound port, meaningful with port 0 before or after start. */
  public int port() {
    return server.getAddress().getPort();
```

listing 150: create binds at construction, port 0 is the ephemeral port the tests ride (source: java/api/src/javabook/httpx/App.java, lines 54-61)

The request side of the server is one object, the `HttpExchange`. It is both the request and the response at once: request method, uri, and headers on the way in, `sendResponseHeaders` and one response `OutputStream` on the way out, an attributes map the filters use for per-request state, and a `close` that ends the exchange. Nothing buffers, nothing is parsed beyond the request line and headers, and the response length contract is exact: a positive `long` is a fixed content length the handler must write to the byte, zero means chunked and the handler closes the stream to terminate, and since java 26 the constants `RSPBODY_EMPTY` and `RSPBODY_CHUNKED` name the two special values instead of leaving bare `-1` and `0` in the code.

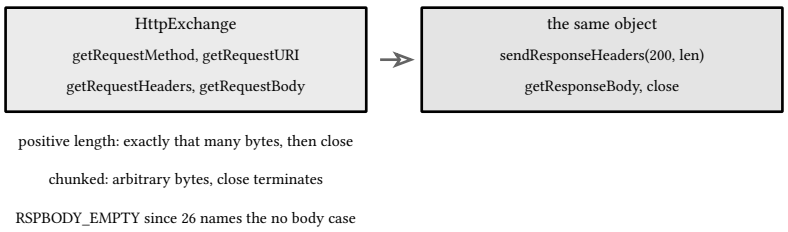


figure 23: one object is the whole transaction: request in, response out, close at the end

19.2. the route table

The server's own dispatch is prefix matching: the context whose path is the longest matching prefix of the request path wins, matched literally, and a context of `/api/users` will also see `/api/users/anything`. That rule buys nothing for an api with methods and path parameters, so the kernel registers exactly one context, the root, and owns the table itself. A pattern is segment-exact with `{name}` wildcards that match one segment, a route is a method plus a pattern plus a handler, and dispatch walks the list:

```
  private void dispatchRoutes(HttpExchange ex) throws ApiError, IOException {
    // the raw path keeps %2F inside one segment, so an encoded slash
    // cannot collapse two segments into one and reroute the request
    String path = ex.getRequestURI().getRawPath();
    Route match = null;
    boolean pathKnown = false;
    for (Route r : routes) {
      if (!r.pathMatches(path)) {
        continue;
      }
      pathKnown = true;
      if (r.method().equals(ex.getRequestMethod())) {
        match = r;
        break;
      }
    }
    if (match == null) {
      if (pathKnown) {
        Set<String> allow = new TreeSet<>();
        for (Route r : routes) {
          if (r.pathMatches(path)) {
            allow.add(r.method());
          }
        }
        ex.getResponseHeaders().set("Allow", String.join(",", allow));
        throw ApiError.failAt(405, Code.NOT_FOUND, "method not allowed");
      }
      throw ApiError.fail(Code.NOT_FOUND, "route not found");
    }
    match.bindParams(path, ex);
    match.handler().handle(ex);
  }
```

listing 151: the whole router: segment match, wildcard bind, 405 with Allow, else the 404 envelope (source: java/api/src/javabook/httpx/App.java, lines 116-148)

Three answers fall out of the walk. A path that matches no pattern answers 404 with the envelope. A path that matches a pattern but not the method answers 405, keeps an `Allow` header built from the methods the path does serve, and rides the same `not_found` code the go book chose, because the closed code set has no method member. And a matched route binds its wildcards as exchange attributes under `path.`, decoded, so the user chapter's handlers read `Kernel.pathValue(ex, "id")` the way go handlers read `r.PathValue`.

Segment exactness is two decisions, both measured before they were kept. The table matches the raw path, not the decoded one, so a percent-encoded slash stays inside one segment: `/items/a%2Fb` matches `{id}` once and the handler reads the decoded `a/b`, where routing the decoded path would have collapsed the segments and sent the request to a different route. And an interior or trailing empty segment, a double slash or a trailing slash, matches nothing: the pattern `/healthz` is not `/healthz/`, and the strictness test pins both. The boundary is the server's own front door: a request target the server rejects before dispatch, `//healthz`, `OPTIONS *`, a bad percent escape in the query, is answered by the server's own plain error page and never reaches the kernel's envelope. Everything the server dispatches gets the contract shape.

One honest omission: the go mux serves `HEAD` for every `GET` pattern automatically, this table does not, and a `HEAD` to a `GET` route answers 405 until someone teaches it otherwise.

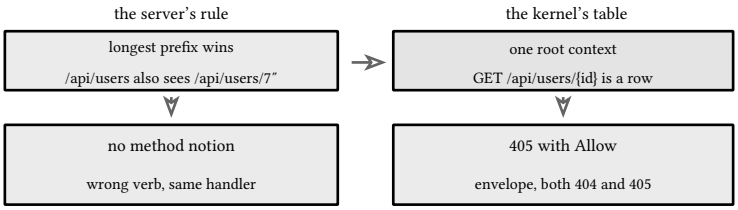


figure 24: prefix matching versus the kernel's table: methods and wildcards are ours either way

19.3. the error envelope

The contract freezes one failure shape for the whole api, and it is the go book's shape member for member: an error object carrying a code from a closed set of eleven, one human sentence, `details` on `validation_failed` only, and the request id. Every non-2xx rides it, including the 404 and 405 the router owns, the 422 the user chapter adds, and the 429 and 503 the rate and load chapters add. A constant envelope beats bare statuses because a client writes one error path instead of one per status, and the day a status needs to move the code does not.

Java's version of the taxonomy is an enum whose constants carry their own wire name and default status, which is the single table the go book needed a map for:

```
public enum Code {
  INVALID_JSON("invalid_json", 400),
  VALIDATION_FAILED("validation_failed", 422),
  PAYLOAD_TOO_LARGE("payload_too_large", 413),
  UNAUTHORIZED("unauthorized", 401),
  FORBIDDEN("forbidden", 403),
  NOT_FOUND("not_found", 404),
  CONFLICT("conflict", 409),
  PRECONDITION_FAILED("precondition_failed", 412),
  RATE_LIMITED("rate_limited", 429),
  OVERLOAD("overload", 503),
  INTERNAL("internal", 500);
```

listing 152: eleven codes, eleven distinct default statuses, and the split pair adds 415 and 405 on top (source: java/api/src/javabook/httpx/Code.java, lines 10-21)

The failure carrier is a checked exception. `Handler` declares `throws ApiError, IOException`, so a handler's signature states its failures the way a go handler's error return does, and the compiler keeps the list honest:

```
 * rendering whatever travels it.
 */
@FunctionalInterface
public interface Handler {
  void handle(HttpExchange ex) throws ApiError, IOException;
}
```

listing 153: the handler contract: write successes, throw failures (source: java/api/src/javabook/httpx/Handler.java, lines 10-15)

The adapter that turns a `Handler` into the server's `HttpHandler` has two catch lanes. An `ApiError` renders as its own envelope. An `IOException` means the client hung up mid-response and there is nobody left to answer. Anything unchecked is deliberately not caught here: that is the panic lane, and chapter 20's filter owns it, which is also why the measured behavior without that filter is a closed connection and a client holding an `IOException`, stated as a test:

```
public static HttpHandler adapt(Handler handler) {
  return ex -> {
    try {
      handler.handle(ex);
    } catch (ApiError e) {
      error(ex, e);
    } catch (IOException e) {
      System.err.println("client gone: " + e);
    }
  };
}
```

listing 154: the adapter: declared failures become the envelope, the client gone is let go (source: java/api/src/javabook/httpx/Kernel.java, lines 39-49)

The envelope writer itself is one method, and it is the only place a failure body is written. It builds the exact bytes by hand, member order fixed, `details` present only when nonempty, and the request id resolved at write time. The committed-response guard at the top uses the sentinel the class documents: `getResponseCode` answers `-1` while nothing has been sent, so any other value means the handler already answered and a late failure can only be logged and the exchange closed:

```
public static void error(HttpExchange ex, ApiError e) throws IOException {
  if (ex.getResponseCode() != RESPONSE_UNSENT) {
    System.err.println("late failure after " + ex.getResponseCode() + ": " + e);
    ex.close();
    return;
  }
  String body = envelope(e, requestId(ex));
  byte[] bytes = body.getBytes(StandardCharsets.UTF_8);
  ex.getResponseHeaders().set("Content-Type", "application/json");
  ex.sendResponseHeaders(e.status(), bytes.length);
  try (OutputStream out = ex.getResponseBody()) {
    out.write(bytes);
  }
}
```

listing 155: the one failure writer, guarded by the unsent sentinel (source: java/api/src/javabook/httpx/Kernel.java, lines 69-82)

NOTE: the -1 sentinel, measured

The class documentation of `HttpExchange.getResponseCode` promises `-1` if the response code is not available yet. The first version of this kernel tested `!= 0` and every failure path answered nothing, which the kernel tests caught on their first run: of 21 tests, 13 held `EOF` where an envelope belonged. The sentinel is now a named constant and the tests pin it end to end.

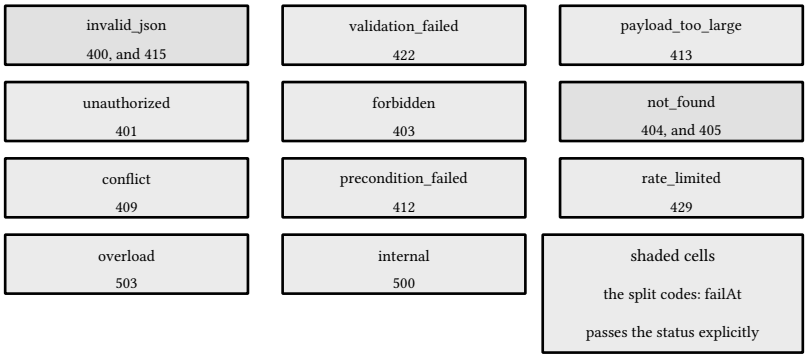


figure 25: eleven codes, eleven default statuses, thirteen on the wire

19.4. json by hand

The jdk has no json. There is a `JSON-P` api in jakarta and a `JSON-B` binding in jakarta, and the flight recorder ships a json writer for `jfr print --json`, but a grep across the pinned build's own sources finds no json parser anywhere in the jdk, and nothing in `java.se` a service can call. The corpus bans external dependencies, so the kernel is the json, and being the json is smaller than it sounds: one writer, one reader, and a value model that is just the maps and lists the language already has.

The value model is `null`, `Boolean`, `String`, `Long`, `Double`, `List`, and `LinkedHashMap` in insertion order. The map type is the load-bearing choice: a `LinkedHashMap` serializes its members in the order they were put, so wire bytes are deterministic without a reflection layer or an annotation, and the health route's body is the same string on every machine that runs it. Numbers split by shape, integers without a fraction or exponent parse as `Long`, everything else as `Double`, because the resources this api moves carry ids and counts, not decimal money.

The writer is a switch over the value's runtime type, with strings escaped per the grammar and every other type appended. The whole thing:

```
  static void writeValue(StringBuilder sb, Object value) {
    switch (value) {
      case null -> sb.append("null");
      case String s -> string(sb, s);
      case Boolean b -> sb.append(b.booleanValue() ? "true" : "false");
      case Double d -> {
        if (d.isInfinite() || d.isNaN()) {
          throw new IllegalArgumentException("cannot write a non-finite number");
        }
        sb.append(d.doubleValue());
      }
      case Float f -> {
        if (f.isInfinite() || f.isNaN()) {
          throw new IllegalArgumentException("cannot write a non-finite number");
        }
        sb.append(f.floatValue());
      }
      case Number n -> sb.append(n);
      case Map<?, ?> m -> {
        sb.append('{');
        boolean first = true;
        for (Map.Entry<?, ?> e : m.entrySet()) {
          if (!first) {
            sb.append(',');
          }
          first = false;
          string(sb, String.valueOf(e.getKey()));
          sb.append(':');
          writeValue(sb, e.getValue());
        }
        sb.append('}');
      }
      case List<?> l -> {
        sb.append('[');
        boolean first = true;
        for (Object item : l) {
          if (!first) {
            sb.append(',');
          }
          first = false;
          writeValue(sb, item);
        }
        sb.append(']');
      }
      default -> throw new IllegalArgumentException(
          "not a json value: " + value.getClass().getName());
    }
  }
```

listing 156: the writer: insertion order is the byte order, strings escaped, non-finite numbers and other types rejected (source: java/api/src/javabook/httpx/Json.java, lines 31-79)

The reader is a recursive descent parser over the source string, one position, strict RFC 8259: whitespace only where allowed, no leading zeros, no trailing content, control characters rejected inside strings, every escape including `\uXXXX` honored with surrogate pairs validated, non-finite numbers rejected so the reader never accepts a `1e400` the writer would have to refuse, and a nesting cap so a hostile body cannot recurse the stack. Objects build the same `LinkedHashMap` the writer consumes, which is what makes round trips a property the tests can assert:

```
public static Object parse(String src) {
  Parser p = new Parser(src);
  p.skipWhitespace();
  Object value = p.value(0);
  p.skipWhitespace();
  if (p.pos != src.length()) {
    throw p.fail("trailing content");
  }
  return value;
```

listing 157: parse: one document, nothing after it, offsets in every failure (source: java/api/src/javabook/httpx/Json.java, lines 108-116)

VERIFY: the ternary that unboxed our numbers

The number reader first returned `integral ? Long.valueOf(text) : Double.valueOf(text)`, and every number came back a `Double`, `42` as `42.0`. That is correct java: a conditional with two numeric operand types applies binary numeric promotion, unboxes both sides to `double`, and boxes the result. The fix is branches, which the source now carries with a comment, and the typing test pins `42` as a `Long` forever.

19.5. the decode ladder

Reading a request body is stricter than writing one, because request bodies are input. `Kernel.bindJson` is the one decode path, and it enforces the same four rungs the go kernel enforced, in the same order. The media type must be `application/json` before the `;`, anything else is 415. The body must fit the 1 MiB cap, checked while reading, so the 413 fires without parsing a byte past the cap. The json must parse, 400. And every member must be one the route accepts, because a typed struct in go got that from `RejectUnknownMembers` and a map gets it from an explicit allowed set the caller passes:

```
public static Map<String, Object> bindJson(HttpExchange ex, Set<String> allowed)
    throws ApiError, IOException {
  String ct = ex.getRequestHeaders().getFirst("Content-Type");
  String base = ct == null ? "" : ct.split(";", 2)[0].trim();
  if (!base.equalsIgnoreCase("application/json")) {
    throw ApiError.failAt(415, Code.INVALID_JSON, "content type must be application/json");
  }
  Object v;
  try {
    v = Json.parse(new String(readCapped(ex.getRequestBody()), StandardCharsets.UTF_8));
  } catch (IllegalArgumentException e) {
    throw ApiError.fail(Code.INVALID_JSON, "malformed json body");
  }
  if (!(v instanceof Map)) {
    throw ApiError.fail(Code.INVALID_JSON, "body must be a json object");
  }
  @SuppressWarnings("unchecked")
  Map<String, Object> m = (Map<String, Object>) v;
  for (String key : m.keySet()) {
    if (!allowed.contains(key)) {
      throw ApiError.fail(Code.INVALID_JSON, "unknown member: " + key);
    }
  }
  return m;
}
```

listing 158: the ladder: 415, 413, 400, 400, then the member map is the caller's (source: java/api/src/javabook/httpx/Kernel.java, lines 135-160)

The capped read is its own helper, and the cap check runs per chunk with the total accumulated, so the failure throws before the buffer ever holds the oversized body. There is no `MaxBytesReader` equivalent in the jdk, there is just an `InputStream` and arithmetic:

```
static byte[] readCapped(InputStream in) throws ApiError, IOException {
  ByteArrayOutputStream buf = new ByteArrayOutputStream();
  byte[] chunk = new byte[8192];
  int n;
  long total = 0;
  while ((n = in.read(chunk)) != -1) {
    total += n;
    if (total > MAX_BODY_BYTES) {
      throw ApiError.fail(Code.PAYLOAD_TOO_LARGE, "request body exceeds 1 MiB");
    }
    buf.write(chunk, 0, n);
  }
  return buf.toByteArray();
}
```

listing 159: no library helper: read, count, refuse past the cap (source: java/api/src/javabook/httpx/Kernel.java, lines 162-175)

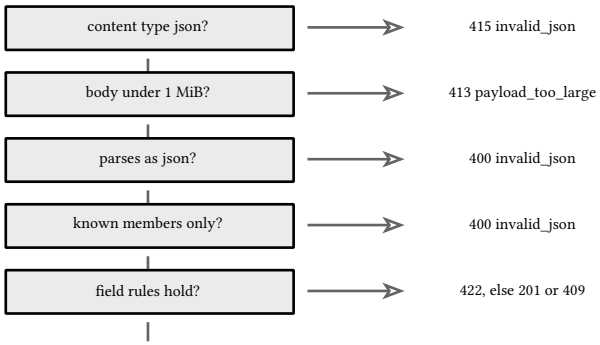


figure 26: the decode ladder: each rung rejects a class of bad input before the next runs

19.6. the skeleton and its lifecycle

`App` is the service skeleton the whole part wires into. A `Config` record carries the port, the build version, and a readiness probe, `Config.fromEnv` reads `JBAPI_PORT` and `JBAPI_VERSION` so one module runs anywhere, and port 0 is the ephemeral port every test boots on. The kernel's two routes are the deployment contract: `GET /healthz` answers 200 with the status and the version, so a deployment can prove what it is running, and `GET /readyz` answers 200 while the probe says ready and the envelope's `overload` code at 503 the moment it does not, the hook the load chapter's drain wiring flips:

```
public void registerKernelRoutes() {
  route("GET", "/healthz", ex -> {
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("status", "ok");
    body.put("version", config.version());
    Kernel.writeJson(ex, 200, body);
  });
  route("GET", "/readyz", ex -> {
    if (config.ready() == null || config.ready().getAsBoolean()) {
      Kernel.writeJson(ex, 200, Map.of("status", "ready"));
    } else {
      throw ApiError.fail(Code.OVERLOAD, "not ready");
    }
  });
}
```

listing 160: the two deployment routes, the pattern every later family copies (source: java/api/src/javabook/httpx/App.java, lines 75-90)

`start` wires the one root context with the filter stack attached, installs the virtual thread executor, and starts serving. `stop` is the graceful drain, and its semantics come straight from the class documentation: the listening socket closes immediately, in-flight exchanges get up to the grace argument in seconds to finish, then open connections close, and a stopped server cannot be reused:

```
public int start() throws IOException {
  server.createContext("/", dispatch).getFilters().addAll(filters);
  server.setExecutor(executor);
  server.start();
  return port();
}

/**
 * Graceful stop: the listener closes now, in-flight exchanges get up
 * to graceSeconds to finish, then open connections close. A stopped
 * server cannot be reused.
 */
public void stop(int graceSeconds) {
  server.stop(graceSeconds);
  executor.shutdown();
}

@Override
public void close() {
  stop(0);
}
```

listing 161: start wires context, filters, executor. stop drains then shuts the pool down (source: java/api/src/javabook/httpx/App.java, lines 92-113)

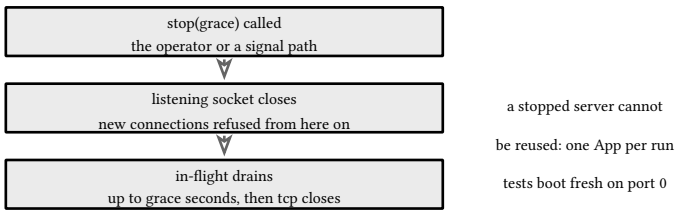


figure 27: the stop order: listener first, drain second, connections last

19.7. testing through a real socket

Go's kernel chapter tested handlers through `httptest` recorders with no socket anywhere. The jdk has no recorder, and this kernel's honesty lives one layer down anyway: the route table, the exchange lifecycle, and the drain all belong to the server, so the tests boot the real thing. Every test constructs an `App` on port 0, starts it, and drives it with the standard `HttpClient`, which means the bytes on the wire are the bytes asserted:

```
  private static App boot() throws IOException {
    App app = new App(new App.Config(0, "test", null));
    app.registerKernelRoutes();
    app.start();
    return app;
  }

  private static HttpResponse<String> get(App app, String path) throws Exception {
    return send(app, HttpRequest.newBuilder().uri(uri(app, path)).GET().build());
  }

  private static URI uri(App app, String path) {
    return URI.create("http://localhost:" + app.port() + path);
  }

  private static HttpResponse<String> send(App app, HttpRequest request) throws Exception {
    return CLIENT.send(request, HttpResponse.BodyHandlers.ofString());
  }
```

listing 162: the harness: boot on an ephemeral port, one shared client, small send helpers (source: java/api/test/javabook/httpx/HttpServerKernelTests.java, lines 24-41)

The assertions are exact where the contract freezes bytes, the health body for instance, and structural everywhere else, decoding the envelope back through the kernel's own reader to check codes, details arrays, and that `request_id` always parses as a uuid v4. The drain test is the one that proves the lifecycle story: a slow handler holds a request open, `stop(2)` runs while it sleeps, the in-flight response still arrives complete, and the next connection is refused:

```
void stopDrainsInFlightAndRefusesNewConnections() throws Exception {
  try (App app = new App(new App.Config(0, "t", null))) {
    app.route("GET", "/slow", ex -> {
      try {
        Thread.sleep(300);
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
      Kernel.writeJson(ex, 200, Map.of("done", true));
    });
    app.start();
    var response = new java.util.concurrent.atomic.AtomicReference<HttpResponse<String>>();
    Thread client = Thread.ofVirtual().start(() -> {
      try {
        response.set(get(app, "/slow"));
      } catch (Exception e) {
        throw new RuntimeException(e);
      }
    });
    Thread.sleep(100); // let the request arrive before stopping
    app.stop(2);
    client.join();
    assertEquals(200, response.get().statusCode());
    Exception refused = assertThrows(Exception.class, () -> get(app, "/healthz"));
    assertInstanceOf(IOException.class, refused);
```

listing 163: the drain proof: in-flight completes, new arrivals refused (source: java/api/test/javabook/httpx/HttpServerKernelTests.java, lines 248-272)

The lane is the one chapter 18 vendored. `javac -Xlint:all -Werror` compiles `src` and `test` separately, the test compile carrying `build/classes` and the junit jar on its classpath, and the console launcher's `execute --scan-classpath` runs every `*Tests` class against both class directories. On this machine, dated 2026-10-04, the classes this chapter owns carry 24 green tests between them, 14 over the socket and 10 on the json reader, and every claim in this chapter that can be executed is one of them or one of their assertions.

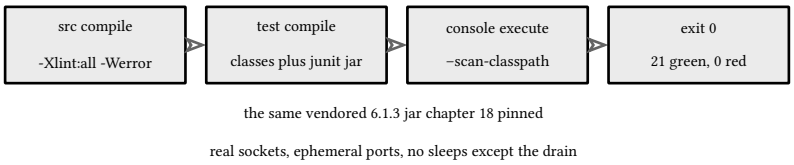


figure 28: the test lane: two compiles, one console run, exit 0 green

sources: the jdk.httpserver api read from the pinned build's own sources (`tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325): the longest-matching-prefix and path-prefix matching notes plus the `sun.net.httpserver.pathMatcher` property from `HttpServer.java`, the `stop(int)` drain contract and the executor-before-start rule from the same file, `RSPBODY_EMPTY` and `RSPBODY_CHUNKED` marked since 26 and the `-1` sentinel documented in `HttpExchange.java`, and `Filter.Chain` from `Filter.java`. RFC 8259 for the json grammar. The go contrasts against the standard mux chapter of book 4 of this corpus. Accessed and verified live 2026-10-04 by the `javabook.httpx` and json tests over the kernel and json classes of the module at `books/java/api`, 24 tests green under the vendored junit 6.1.3 lane, covering the exact health body, the 503 flip, the 404 and 405 envelopes with Allow, the uuid request ids, the 415 413 400 400 ladder, path parameters, the strict raw-path routing with encoded slashes and rejected double slashes, virtual thread handlers, 64 concurrent requests, the drain and refusal after stop, and the json round trips, escapes, finiteness, surrogates, and rejections.

20. middleware

Cross-cutting behavior does not belong inside handlers, and the jdk's server already agrees: `com.sun.net.httpserver.Filter` is a chain type built into the same module as the kernel. This chapter composes the service's filter stack in `javabook.middleware`, four layers over the chapter 19 kernel: request ids born once and quoted everywhere, panic recovery into the contract envelope, the access log that reads the status off the exchange after the fact, timing measured on the unwind, and the wiring order that makes the stack correct rather than merely present. The go book built the same stack as handler-wrapping functions, book 4, ch 19 (go 1.27, middleware), and where its shapes differ from java's, the differences are the chapter.

20.1. filters are the chain

A filter is an abstract class with two methods. `doFilter` receives the exchange and a `Chain`, and the chain is the entire middleware contract: call `chain.doFilter(ex)` to run everything inside you, decline to call it and you have answered the request yourself. The chain object is a `ListIterator` over the context's filter list plus the handler at the end, so the list order is the runtime order, the first filter in the list is the outermost layer, it sees the request first and the response last, and a filter's code after its `chain.doFilter` call runs on the unwind. One filter is one class here rather than one function value as in go, and one abstract method beyond `doFilter`, a `description` string the javadoc asks for. Nothing in the server ever calls it, a grep across the pinned build's sources finds no caller, so it is documentation in code and the four layers here state their jobs in it.

The second built-in is the request-scoped state go put in a context. `HttpExchange` carries an attributes map, `setAttribute` and `getAttribute`, documented for exactly this use, and it is how the layers talk without touching headers: the router's path parameters already live there under `path.`, this chapter's id lives under `request.id`, and the timing layer publishes its measurement the same way for the log outside it.

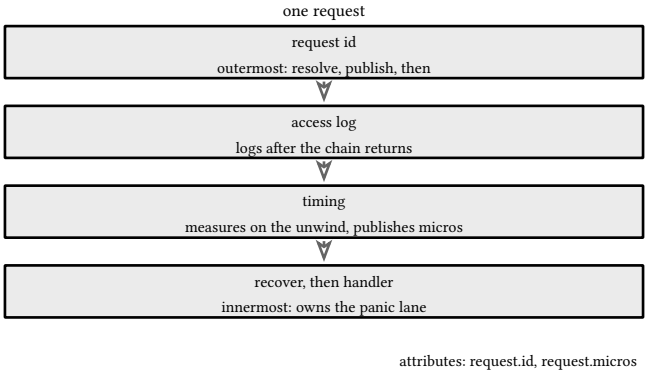


figure 29: the chain is a list iterator: list order is runtime order, code after the call runs on the unwind

20.2. request id

Every request needs one id that every artifact agrees on: the access log line, the error envelope, the panic log. The layer resolves it exactly once, at the edge, using the kernel's order from chapter 19: an id already resolved, else the client's `X-Request-ID` when it parses as a uuid, else a fresh `UUID.randomUUID`, which is version 4. The jdk's other generator, `nameUUIDFromBytes`, builds version 3 names, so time-ordered v7 ids like the go book's would have to be hand-rolled. Parsing is not politeness, it is contract: the envelope promises `request_id` is a uuid, and a client-supplied free-text header echoed into logs is log injection. Garbage is replaced, never echoed.

The layer then does two small things with the value: it sets the response header so a client correlates its own logs with the server's, and it installs the id as an exchange attribute so everything downstream reads one value without touching headers again:

```
  public void doFilter(HttpExchange ex, Chain chain) throws IOException {
    String id = Kernel.requestId(ex);
    ex.setAttribute(Kernel.REQUEST_ID, id);
    ex.getResponseHeaders().set(Kernel.HEADER_REQUEST_ID, id);
    chain.doFilter(ex);
  }

  @Override
  public String description() {
    return "resolves one request id per exchange";
  }
```

listing 164: resolve once at the edge, echo it, publish it as the attribute (source: java/api/src/javabook/middleware/RequestId.java, lines 19-29)

Three births, two tests: a client uuid echoes verbatim, garbage is replaced by a v4, and an absent header generates one. The uuid version nibble is asserted, so the day someone swaps in a hand-rolled v7 the tests ask whether that was intended.

20.3. recover, the panic lane

Java has two failure lanes and chapter 19 split them. The declared lane is `ApiError` plus the `IOException` of a departed client, both caught by the kernel adapter. The undeclared lane is everything else, any `RuntimeException` from anywhere inside a handler, and a bare exchange has no way to promise a client anything when one fires. Measured on the pinned build: an unhandled `IllegalStateException` in a handler closes the connection, and the client's `HttpClient` throws `IOException` with `HTTP/1.1 header parser received no bytes`. The recover filter exists so that never happens to a real caller:

```
  public void doFilter(HttpExchange ex, Chain chain) throws IOException {
    try {
      chain.doFilter(ex);
    } catch (Throwable t) {
      var sw = new StringWriter();
      t.printStackTrace(new PrintWriter(sw));
      log.log(System.Logger.Level.ERROR,
          "unhandled throw, request " + Kernel.requestId(ex) + "\n" + sw);
      try {
        Kernel.error(ex, ApiError.fail(Code.INTERNAL, "internal error"));
      } catch (IOException e) {
        // the client left while the envelope was being written
        System.err.println("client gone: " + e);
      }
    }
  }
```

listing 165: catch Throwable, log stack and id, answer the internal envelope (source: java/api/src/javabook/middleware/Recover.java, lines 30-46)

Three rules live in those lines. The log line carries the stack and the request id, because a panic without a stack is a mystery and one without an id is an orphan. The envelope says `internal error` and nothing else, so the `IllegalStateException`'s message never reaches the client, the same information discipline the go layer kept. And the envelope write itself can fail, because a throw often means a half-written response and a client that already gave up, so the inner catch acknowledges the client-gone case instead of throwing from inside a catch. A response the handler already committed is left alone, the kernel's writer checks the `-1` sentinel and only logs, and the test for that pins a handler that writes a 201 and then throws: the client keeps its 201, the log records the late throw.

There is no rethrown sentinel here. Go's middleware rethrows `http.ErrAbortHandler` because the server itself speaks it, and java's server has no equivalent signal, so the catch is total, `Throwable` and all. The tradeoff is honest: an `Error` in the stack is caught, answered, and logged, on the theory that a service that tells the client `internal error` while it dies is better than one that answers nothing. That is a policy this book owns, not a library default.

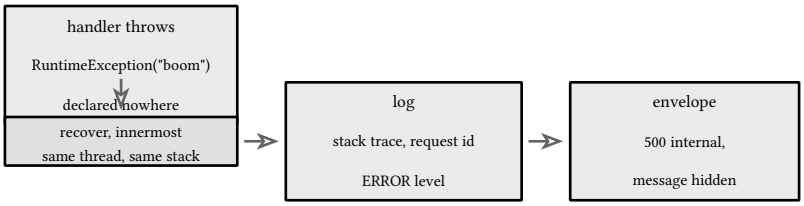


figure 30: panic unwind: one path to the log, one to the envelope

20.4. the access log, no wrapper needed

Go's access log had to wrap the `ResponseWriter` in a status-capturing type, because a go writer does not tell you what went out, and the wrapper then broke every optional interface it did not forward. The jdk exchange does tell you: `getResponseCode` answers the status after the chain returns, so the java access log is five lines with no wrapper and no forwarding problem, and the first-status-wins trap of the go chapter cannot exist here because the exchange is one object remembering its own commit. Body byte counts are not exposed, so the line carries what is: method, path, status, request id, and the duration when a timing layer ran inside this one:

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  chain.doFilter(ex);
  Object micros = ex.getAttribute(Timing.MICROS);
  String line = ex.getRequestMethod() + " " + Logs.oneLine(ex.getRequestURI().getPath())
      + " " + ex.getResponseCode() + " " + Kernel.requestId(ex);
  if (micros instanceof Long us) {
    line += " " + us + "us";
  }
  log.log(System.Logger.Level.INFO, line);
}
```

listing 166: after the chain: the status is on the exchange, the duration is an attribute, the path is sanitized (source: java/api/src/javabook/middleware/AccessLog.java, lines 25-35)

The path in that line is client input that arrives percent-decoded, so a request to `/ok/x%0D%0Ainjected` would otherwise carry a real crlf into the log and forge a second line. The id lane refuses this by parsing, a garbage header never reaches the log, and the path lane refuses it by escaping: `Logs.oneLine` turns the three line-breaking characters into their two-character spelled forms before anything is concatenated, and the test that pins it requests exactly such a path and asserts the captured line carries the spelled `\r` and `\n` rather than the bytes they name.

20.5. timing on the unwind

The timing layer is `System.nanoTime` around the chain and a published attribute. What it cannot do is put the duration in a response header, and the reason is the jdk's own commit discipline: `sendResponseHeaders` sends the headers the moment it runs, which is before the handler's work is done, so by the time a duration exists the header block is gone. A duration header would have to be a trailer, the exchange api has no trailers, and the honest answer is that timing travels outward through the stack, as an attribute and a log line, where the access log outside it picks the number up for its own line:

```
@Override
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  long start = System.nanoTime();
  chain.doFilter(ex);
  long micros = (System.nanoTime() - start) / 1_000;
  ex.setAttribute(MICROS, micros);
  log.log(System.Logger.Level.INFO,
      "dur " + micros + "us " + ex.getRequestMethod() + " "
          + Logs.oneLine(ex.getRequestURI().getPath()));
}
```

listing 167: nanoTime around the chain, micros published as the attribute and logged (source: java/api/src/javabook/middleware/Timing.java, lines 26-35)

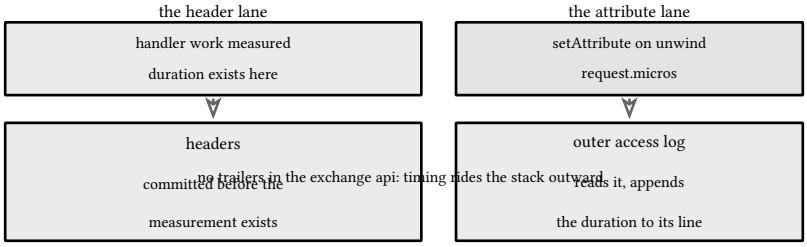


figure 31: two ways a duration can travel: the header lane is closed at commit, the attribute lane is open

20.6. wiring order and why

The order of the four layers is contract, and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log line, the panic log, the envelope. Access log second, outside recover, so a recovered 500 is logged as the 500 the client saw. This is not aesthetic, it is mechanical: an access log's line runs after its `chain.doFilter` returns normally, and a throw unwinds past that line entirely, so a recover placed outside the log means panics never get logged at all. Timing third, inside the log, so the log can carry the number. Recover innermost, next to the handler, where every other layer's post-chain code still runs on its answer. The wiring is four `use` calls in list order, and the program's own `Main` wires the same four in the same order before any route registers:

```
private static Wired stack() throws IOException {
  CapturingLogger log = new CapturingLogger();
  App app = new App(new App.Config(0, "t", null));
  app.use(new RequestId());
  app.use(new AccessLog(log));
  app.use(new Timing(log));
  app.use(new Recover(log));
  app.route("GET", "/ok", ex -> Kernel.writeJson(ex, 200, Map.of("ok", true)));
  app.route("GET", "/boom", ex -> {
    throw new IllegalStateException("boom");
  });
  app.route("GET", "/late", ex -> {
    Kernel.writeJson(ex, 201, Map.of("late", true));
    throw new IllegalStateException("after the commit");
  });
  app.start();
  return new Wired(app, log);
}
```

listing 168: the contract stack: id, log, timing, recover, then the routes (source: java/api/test/javabook/middleware/MiddlewareTests.java, lines 61-78)

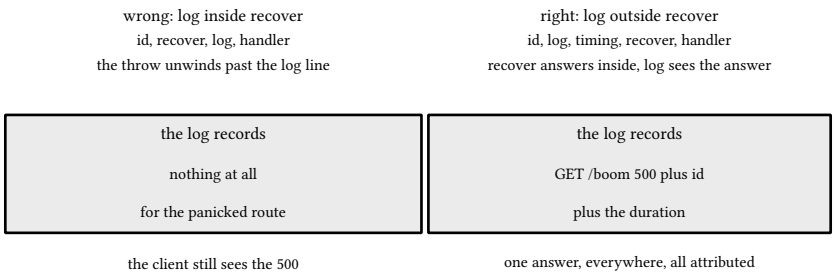


figure 32: wrong order versus right: where the log sits decides what it sees

20.7. testing the stack

The tests boot the real stack on an ephemeral port and capture the logs through the seam the jdk already ships: `System.Logger`, the logging facade of `java.base` since java 9, which the logging layers take as a constructor argument and the tests implement with a capturing class. No global logger is configured, nothing is parsed back out of a console, and the assertion reads the same lines production would hand a real backend.

One race had to be owned rather than hidden. The body bytes flush to the client before the filter stack finishes unwinding, so a log line written after `chain.doFilter` can land moments after the client already has its response. A test that asserts the line immediately after `send` passes in isolation and fails under a full-suite run, which is exactly how it was found. The honest harness polls for its line with a deadline, and the polling helper carries the comment explaining why:

```
/**
 * Post-chain log lines race the client's receipt of the response:
 * the body is flushed before the filter stack finishes unwinding,
 * so a log assertion polls for its line instead of assuming it.
 */
private static boolean awaitLine(CapturingLogger log, String needle) throws InterruptedException {
  long deadline = System.nanoTime() + 2_000_000_000L;
  while (System.nanoTime() < deadline) {
    if (log.lines.stream().anyMatch(l -> l.contains(needle))) {
      return true;
    }
    Thread.sleep(5);
  }
  return log.lines.stream().anyMatch(l -> l.contains(needle));
}
```

listing 169: the polling helper: log lines race the client’s receipt, so assertions wait (source: java/api/test/javabook/middleware/MiddlewareTests.java, lines 94-110)

The marquee test is the wiring’s own: the counterexample app wires recover outside the access log on purpose, panics a route, and asserts the access log stayed silent, the precise failure mode the order contract exists to prevent:

```
  void recoverOutsideTheAccessLogLosesThe500() throws Exception {
    // the counterexample the wiring contract exists to prevent: with
    // recover outside, the throw unwinds past the log line entirely
    CapturingLogger log = new CapturingLogger();
    try (App app = new App(new App.Config(0, "t", null))) {
      app.use(new RequestId());
      app.use(new Recover(log));
      app.use(new AccessLog(log));
      app.route("GET", "/boom", ex -> {
        throw new IllegalStateException("boom");
      });
      app.start();
      HttpResponse<String> r = get(app, "/boom", null);
      assertEquals(500, r.statusCode());
      assertTrue(log.lines.stream().noneMatch(l -> l.contains("GET /boom")));
    }
  }
}
```

listing 170: the counterexample: wrong order, the 500 reaches the client and no line records it (source: java/api/test/javabook/middleware/MiddlewareTests.java, lines 226-243)

sources: `Filter`, `Filter.Chain`, and the exchange attributes read from the pinned build's own sources (`tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325), the chain-as-iterator contract and the must-answer-if-you-stop-the-chain rule from `Filter.java`. `System.Logger` from `java.lang` of the same build. The go contrasts, the statusWriter wrapper, `ErrAbortHandler`, and the timeout layer's goroutine rule, against book 4, chapter 19. Accessed and verified live 2026-10-04 by the `javabook.middleware` tests under the vendored junit 6.1.3 lane, 9 tests, run green three times consecutively as a stability check: the three id births, the envelope answer with the throw logged and the detail hidden, the committed-201 case left alone, the status-carrying access lines, the duration attribute and its pickup, the crlf-forcing path escaped in the log, the outside-in filter order, and the wrong-order counterexample.

21. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, a creation time, and a version. This chapter builds the whole resource layer on the chapter 19 kernel in `javabook.user`: the representation and its explicit wire writer, registration with validation from first principles, the store as a port with an in-memory implementation, the password hashing seam that carries chapter 17's cryptography into the service, keyset pagination with byte-exact cursors, the etag contract on read, and the handler tests that pin all of it over a real socket with a fixture the code owns. The go book built the same layer, book 4, ch 20 (go 1.27, the user resource), and this chapter keeps its contracts while spending its java differences, explicit writers instead of struct tags, injected functions instead of interface fields, where they earn their keep.

21.1. the representation

A record is the whole stored type: six components, one line of syntax, equality and accessors free. The wire representation is a method, not a second type: `toJson` builds a `LinkedHashMap` in fixed member order, and that explicitness is what keeps the password hash off the wire. Go needed a dash tag on `PasswordHash` so its reflection serializer would skip it, and the tag was the one line where a missing character leaks a hash. Here the writer writes what it lists, nothing else exists, and the test that pins the 201 body asserts the absence directly:

```
public record User(
    String id,
    String email,
    String displayName,
    Instant createdAt,
    int version,
    String passwordHash) {

  /** The wire bytes in fixed member order; the etag is the sha256 of exactly these. */
  public Map<String, Object> toJson() {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("id", id);
    m.put("email", email);
    m.put("display_name", displayName);
    m.put("created_at", createdAt.toString());
    m.put("version", version);
    return m;
  }
}
```

listing 171: the record and its wire writer: the members on the wire are the members toJson puts (source: java/api/src/javabook/user/User.java, lines 15-33)

The `created_at` member is `Instant.toString`, the iso-8601 form with a `Z` suffix, deterministic for a fixed instant, which is what makes the etag and the test bytes stable. The version member is the quiet third actor: it starts at 1, bumps once per successful write when updates arrive in chapter 25's scope, and becomes the compare-and-swap substrate there.

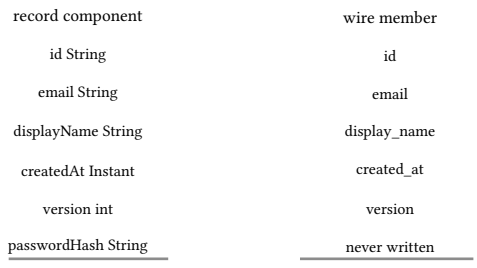


figure 33: the record components against the wire members, one component never written

21.2. create, validation, and the 422 versus 409 boundary

The create handler is the decode ladder from chapter 19 with a resource on top of it, and the order is the contract: the kernel's `bindJson` answers the 415, the 413, the parse 400, and the unknown member 400, then the typed member read answers a 400 for a non-string value, and only then does validation see clean strings:

```
private static Handler create(Deps d) {
  return ex -> {
    Map<String, Object> body = Kernel.bindJson(ex, Set.of("email", "password", "display_name"));
    String email = member(body, "email");
    String password = member(body, "password");
    String displayName = member(body, "display_name");
    List<String> details = Validation.validateRegister(email, password, displayName);
    if (!details.isEmpty()) {
      throw ApiError.fail(Code.VALIDATION_FAILED, "validation failed",
          details.toArray(String[]::new));
    }
    User u = new User(d.ids().get(), Validation.normalizeEmail(email), displayName,
        d.clock().instant(), 1, d.hasher().hash(password));
    try {
      d.store().create(u);
    } catch (ConflictException e) {
      throw ApiError.fail(Code.CONFLICT, "email already registered");
    }
    ex.getResponseHeaders().set("Location", "/api/users/" + u.id());
    ex.getResponseHeaders().set("ETag", etag(u));
    Kernel.writeJson(ex, 201, u.toJson());
  };
}
```

listing 172: strict bind, typed members, every rule checked, then the store (source: java/api/src/javabook/user/Users.java, lines 42-64)

The rules come from the contract, not from a library, because the jdk has no bean validation and the numbers should be greppable anyway: passwords 12 to 128 characters, display names 1 to 64, one email shape. The email rule is a seven-token regex, one `@`, nonempty local part, dotted domain, no whitespace, capped at 254 characters, the approximation that accepts the addresses people hold. Every rule reports into one details array rather than failing on the first, so one round trip tells the client everything wrong:

```java
/** Checks a registration payload. Empty means valid. */
public static List<String> validateRegister(String email, String password, String displayName) {
  List<String> details = new ArrayList<>();
  if (email.isEmpty()) {
    details.add("email is required");
  } else if (!validEmail(email)) {
    details.add("email is not a valid address");
  }
  if (password.isEmpty()) {
    details.add("password is required");
  } else if (password.length() < MIN_PASSWORD) {
    details.add("password must be at least 12 characters");
  } else if (password.length() > MAX_PASSWORD) {
    details.add("password must be at most 128 characters");
  }
  if (displayName.isEmpty()) {
    details.add("display_name is required");
  } else if (displayName.codePointCount(0, displayName.length()) > MAX_DISPLAY_NAME) {
    details.add("display_name must be at most 64 characters");
  }
  return details;
}
```

listing 173: one detail per violated rule, in field order, nothing short-circuits (source: java/api/src/javabook/user/Validation.java, lines 33-54)

The 422 and the 409 are different animals and the boundary is worth stating. A rule violation is about the request alone, checkable without touching state, so it is 422 validation. The email conflict is about the store's current contents, so it is 409 conflict, and it surfaces only after validation passed and the store answered its exception. The store speaks outcomes, the handler maps them to statuses, and no layer does the other's job. Normalization closes the last gap: emails are trimmed and lower-cased before the store sees them, so `Ada@Example.ORG` cannot dodge a conflict with `ada@example.org`, and the test registers both spellings and asserts the 409.

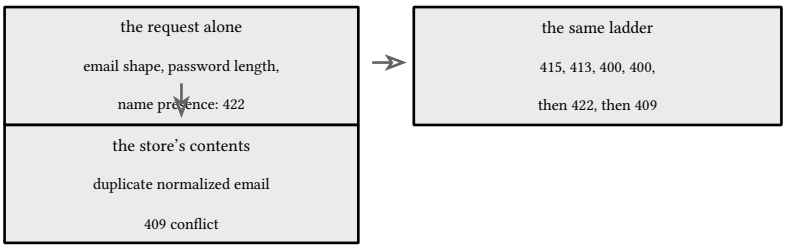


figure 34: same request shape, two truths: rules about the request, conflict about the store

21.3. the store seam

The persistence port is three methods, create, get, list, frozen now because chapter 24 implements the same interface over a real engine and two chapters build against one shape. The in-memory implementation is deliberately boring: one monitor, an id map, an email map, and an order list kept sorted by the keyset key with a binary insertion, so `list` walks one sorted slice and reads live records through the map:

```
  private static final Comparator<User> KEY =
      Comparator.comparingLong((User u) -> u.createdAt().toEpochMilli())
          .thenComparing(User::id);

  private final Map<String, User> byId = new LinkedHashMap<>();
  private final Map<String, String> byEmail = new HashMap<>();
  private final List<String> order = new ArrayList<>();

  @Override
  public synchronized void create(User u) throws ConflictException {
    if (byEmail.containsKey(u.email())) {
      throw new ConflictException();
    }
    byId.put(u.id(), u);
    byEmail.put(u.email(), u.id());
    int at = Collections.binarySearch(order, u.id(),
        (a, b) -> KEY.compare(byId.get(a), byId.get(b)));
    order.add(at < 0 ? -at - 1 : at, u.id());
  }
```

listing 174: one monitor, two maps, a sorted order slice with binary insertion (source: java/api/src/javabook/user/MemStore.java, lines 23-41)

Every method is `synchronized`, the whole locking story for a page this size. The chapter 10 concurrency material would replace the monitor with a `ReentrantReadWriteLock` if reads dominated, and the store chapter will replace the entire class with an engine, and neither change touches a handler, which is the point of the seam.

21.4. the password seam

The handler never hashes. It calls a `Hasher`, a one-method interface whose production implementation is the pbkdf2 chapter 17 measured: 100,000 iterations of hmac-sha256 over a fresh 16 byte salt, the salt, the count, and the digest all encoded into the stored string so the authn chapter can verify against a self-describing value:

```
  static Hasher pbkdf2() {
    return password -> {
      byte[] salt = new byte[16];
      new SecureRandom().nextBytes(salt);
      PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, 100_000, 256);
      try {
        SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        byte[] hash = factory.generateSecret(spec).getEncoded();
        Base64.Encoder b64 = Base64.getEncoder();
        return "pbkdf2-sha256$100000$" + b64.encodeToString(salt) + "$"
            + b64.encodeToString(hash);
      } catch (NoSuchAlgorithmException e) {
        throw new IllegalStateException("pbkdf2 missing from the jdk", e);
      } catch (java.security.spec.InvalidKeySpecException e) {
        throw new IllegalStateException("pbkdf2 rejected its own spec", e);
      } finally {
        spec.clearPassword();
      }
    };
  }
}
```

listing 175: the production hasher: chapter 17's pbkdf2, self-describing output (source: java/api/src/javabook/user/Hasher.java, lines 23-43)

Two choices worth their ink. The hash lands in the record at registration because storing plaintext is not an option and computing the hash lazily would mean a write path later. And the test fixture injects a fake `p -> "fake:" + p`, because 100,000 iterations per registration is the cost production pays on purpose and no fixture wants to pay it 25 times a page test: measured on this machine, a full register round trip through the booted program with the real hasher took 186 ms on one run and 34 to 145 ms across a re-measured set, all of it the hash, while the whole test class with the fake runs its 11 tests in well under a second.

21.5. keyset pagination

Offset pagination asks for row 86041 onward, and it fails twice: the store walks and discards every skipped row, and offsets are positions in a moving file, so one registration between requests makes a row appear twice or vanish. Keyset pagination remembers where it stopped instead of counting from the front. The order is the pair, creation millisecond then id, and the cursor is that pair as json, base64url without padding, opaque to clients and decodable in a test:

```
public static String encode(CursorKey k) {
  StringBuilder json = new StringBuilder();
  json.append("{\"t\":").append(k.t()).append(",\"i\":");
  Json.string(json, k.id());
  json.append('}');
  return Base64.getUrlEncoder().withoutPadding()
      .encodeToString(json.toString().getBytes(StandardCharsets.UTF_8));
}
```

listing 176: the cursor: two members, json, base64url unpadded (source: java/api/src/javabook/user/Cursor.java, lines 45-53)

The decode path is strict in the same direction as everything else in the kernel: a cursor is either one this service issued or garbage, never a missing page, so every failure down the base64, the json parse, and the member checks answers the same 422, and a forged cursor carrying members this service never writes, an extra `junk` key for instance, is refused the same way. The limit rejects anything outside 1..100 with a default of 20, and the list handler asks the store for `limit + 1` rows so the presence of one extra row is the whole has-more computation, no counts, no second query:

```
private static Handler list(Deps d) {
  return ex -> {
    int limit = Cursor.parseLimit(queryParam(ex, "limit"));
    CursorKey after = CursorKey.START;
    String raw = queryParam(ex, "cursor");
    if (raw != null && !raw.isEmpty()) {
      after = Cursor.decode(raw);
    }
    List<User> page = d.store().list(after, limit + 1);
    boolean more = page.size() > limit;
    if (more) {
      page = page.subList(0, limit);
    }
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("items", page.stream().map(User::toJson).toList());
    body.put("next_cursor",
        more ? Cursor.encode(CursorKey.of(page.get(page.size() - 1))) : null);
    Kernel.writeJson(ex, 200, body);
  };
}
```

listing 177: limit plus one read, one extra row means has-more, the cursor names the last row kept (source: java/api/src/javabook/user/Users.java, lines 170-189)

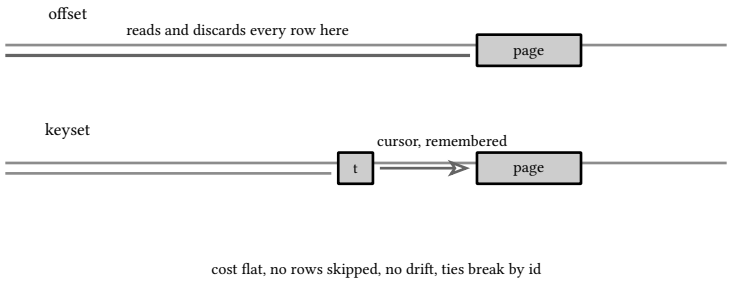


figure 35: offset walks the whole prefix, keyset jumps to the remembered position

21.6. get and the etag

The etag is not stored, it is computed: sha256 over the exact wire bytes of the representation, hex, quoted. Because the etag is a function of the body the two can never disagree, and because the writer is deterministic the same record yields the same etag forever. The get handler is four moves: resolve the id off the path parameter, answer 404 through the one failure path, run the `If-None-Match` comparison and answer 304 with the header and no body when it hits, else 200 with the etag and the body. The comparison is the rfc's, not plain string equality: a star matches any current representation, a comma list matches when the etag appears in it, and both comparisons are weak, so a `W/` prefix still counts:

```
private static Handler get(Deps d) {
  return ex -> {
    String id = Kernel.pathValue(ex, "id");
    User u = d.store().get(id)
        .orElseThrow(() -> ApiError.fail(Code.NOT_FOUND, "user not found"));
    String etag = etag(u);
    if (noneMatch(etag, headerJoined(ex, "If-None-Match"))) {
      ex.getResponseHeaders().set("ETag", etag);
      ex.sendResponseHeaders(304, HttpExchange.RSPBODY_EMPTY);
      return;
    }
    ex.getResponseHeaders().set("ETag", etag);
    Kernel.writeJson(ex, 200, u.toJson());
  };
}
```

listing 178: 404 or 304 or 200: the conditional read in fifteen lines (source: java/api/src/javabook/user/Users.java, lines 66-80)

```
/**
 * RFC 9110's If-None-Match: a star matches any current
 * representation, a list matches when the etag appears in it, and
 * both comparisons are weak, so a W/ prefix still counts.
 */
static boolean noneMatch(String etag, String header) {
  if (header == null) {
    return false;
  }
  for (String candidate : header.split(",")) {
    String t = candidate.trim();
    if (t.equals("*")) {
      return true;
    }
    if (t.startsWith("W/")) {
      t = t.substring(2);
    }
    if (t.equals(etag)) {
      return true;
    }
  }
  return false;
}
```

listing 179: noneMatch: star, list, and weak forms, the rfc 9110 rules (source: java/api/src/javabook/user/Users.java, lines 146-168)

The 304 uses the `RSPBODY_EMPTY` constant from chapter 19, the no-body length the exchange api names, and the same etag string rides both answers so a client caching on it cannot be misled. Chapter 25's update scope will add `If-Match` against the same value, which is when the version member starts earning its keep.

21.7. handler tests over http

The fixture owns everything time-dependent, and in java that is three lambdas and a clock subclass: a `StepClock` advancing one second per `instant` call, so five registrations hold five distinct creation times without sleeping, an id source counting `u1` through `u25`, and the fake hasher. The boot helper reads like the production wiring in `Main`, which is the review trick: the wiring table and the fixture differ only in the injected functions:

```
/** Boots the resource with a fixed fixture: step clock, counted ids, fake hash. */
private static App boot() throws IOException {
  AtomicLong seq = new AtomicLong();
  App app = new App(new App.Config(0, "t", null));
  Users.register(app, new Users.Deps(
      new MemStore(), new StepClock(),
      () -> "u" + seq.incrementAndGet(), p -> "fake:" + p));
  app.start();
  return app;
}
```

listing 180: the fixture: step clock, counted ids, fake hash, real routes (source: java/api/test/javabook/user/UserResourceTests.java, lines 50-59)

```
    app.registerKernelRoutes();
    Users.register(app, new Users.Deps(
        users, clock,
        () -> UUID.randomUUID().toString(), Hasher.pbkdf2()));
    Login.register(app, new Login.Deps(users, clock, secret, Passwords::verify));
    Admin.register(app, new Admin.Deps(users, grants));
```

listing 181: the wiring table: kernel routes, then the user family, one register call per family as later chapters add theirs (source: java/api/src/javabook/Main.java, lines 96-101)

Exactness is the point. The page-walk test seeds five users and asserts the first page's whole body, byte for byte, including the cursor, which for the fixture's second user is `eyJ0IjoxNzY3MjI1NjAxMDAwLCJpIjoidTIifQ`: the base64url of `{"t":1767225601000,"i":"u2"}`, the 2026-01-01 creation second the step clock handed out. The rest of the slice covers the ladder, every violation at once, the normalized conflict, the 404, the 304 on a matching etag, the star, list, and weak forms beside it, the forged cursor shape refused, the limit rejections, and the default 20 with its 5-row tail, 11 tests, run green three times consecutively. When the cursor changes the code is wrong, and when the code changes an output byte the test fails: drift has nowhere to hide.

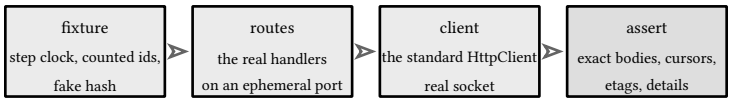


figure 36: a golden test: the fixture drives the bytes, the socket carries them, the assertions read them back

sources: RFC 9110, HTTP Semantics, sections 8.8.3 and 13.1.1 on etags and conditional requests, at rfc-editor.org. The pbkdf2 construction, `PBKDF2WithHmacSHA256`, 100,000 iterations, 16 byte salt, is the one chapter 17 verified against rfc 8018 and measured on this build, and the register round trips through the booted `javabook.Main` with the production hasher measured 186 ms once and 34 to 145 ms across a re-measured set on 2026-10-04. The keyset versus offset argument follows the same use-the-index-luke reasoning the go book cited. Verified live 2026-10-04 by the `javabook.user` tests under the vendored junit 6.1.3 lane, 11 tests green three consecutive runs, 44 across the module, covering the 201 body, Location, and 64-hex etag, the whole decode ladder including the unknown member and non-string messages, the three-violation 422 details array, the normalized 409, the 404, the 304 and etag equality plus the star, list, and weak forms, the byte-exact three-page walk with cursor `eyJ0IjoxNzY3MjI1NjAxMDAwLCJpIjoidTIifQ`, the forged cursor shape, the limit and cursor rejections, and the default-20 tail page.

22. authn

Chapter 21 stored a password hash and never used it. This chapter is the use: `javabook.authn` builds the whole answer to who is calling, a json web token assembled from `javax.crypto.Mac` rather than a library, the login endpoint that proves a password against chapter 21's pbkdf2 form and issues a fifteen minute token, the filter that extracts a bearer token and installs the actor everything else reads, and the 401 that closes every route the wiring does not declare open. The go book built the same layer with argon2id, sessions, refresh rotation, and cookies, book 4, ch 21 (go 1.27, authn), and the smaller scope here is deliberate: the token and the proof carry every idea the larger machine adds machinery around, and one chapter owes them justice before the authz chapter decides what the caller may do.

22.1. the token, three segments and one primitive

A json web token is a header, a claims object, and a signature over the first two, all base64url without padding. That is the entire format, and the jdk's crypto floor covers it: `Mac` with `HmacSHA256` is the signature, `Base64.getUrlEncoder` is the alphabet, and the kernel's json writer is the serializer. The one representation choice worth naming is determinism: the header and claims serialize through `LinkedHashMap` in fixed member order, so the same secret and the same facts always produce the same bytes, which is what lets a test pin a token exactly:

```
/** The token payload. Declaration order is the wire order, alphabetical. */
public record Claims(long exp, long iat, String iss, String sub) {}

private Jwt() {}

/** Signs claims into the three-segment token. */
public static String sign(byte[] secret, Claims c) {
  Map<String, Object> header = new LinkedHashMap<>();
  header.put("alg", ALG);
  header.put("typ", "JWT");
  Map<String, Object> claims = new LinkedHashMap<>();
  claims.put("exp", c.exp());
  claims.put("iat", c.iat());
  claims.put("iss", c.iss());
  claims.put("sub", c.sub());
  Base64.Encoder b64 = Base64.getUrlEncoder().withoutPadding();
  String hs = b64.encodeToString(Json.write(header).getBytes(StandardCharsets.UTF_8));
  String ps = b64.encodeToString(Json.write(claims).getBytes(StandardCharsets.UTF_8));
  String sig = b64.encodeToString(mac(secret, hs + "." + ps));
  return hs + "." + ps + "." + sig;
}
```

listing 182: the claims and sign: json through the kernel writer, hmac over the first two segments, base64url unpadded (source: java/api/src/javabook/authn/Jwt.java, lines 58-78)

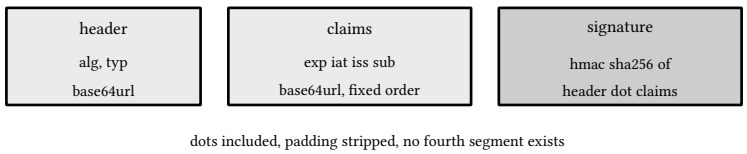


figure 37: three segments, the signature covers exactly the first two

The claims are four, and the two that matter as policy are `exp`, answered against the clock at verification, and `iss`, the service's own name. There is no roles claim, on the same reasoning the go book kept: authorization reads the store, so a token carries identity and nothing that can go stale. And there is no session id, which is a real divergence from the go chapter and worth its own sentence: this book keeps no server-side session, so a token

cannot be revoked before its fifteen minutes run out, and revocation is expiry. At the scale a book ships that is the cheap honest trade, and the store chapter's engine would be the place a session row would live if the trade ever needed revisiting.

22.2. verification, order and constant time

Verification is a fixed order and the order is the security: shape, algorithm pin, signature, then claims. The algorithm check runs before any payload byte is trusted, so the classic `alg: none` forgery and a token correctly signed under `HS384` both die at the header, and the signature comparison is `MessageDigest.isEqual`, whose contract in this build's own sources states that the calculation time depends only on the length of the first argument:

```
public static Claims verify(String token, byte[] secret, Instant now) {
  String[] parts = token.split("\\.", -1);
  if (parts.length != 3
      || parts[0].isEmpty()
      || parts[1].isEmpty()
      || token.indexOf('=') >= 0) {
    throw new Rejected(Reason.MALFORMED);
  }
  Object h = parse(decode(parts[0]));
  if (!(h instanceof Map<?, ?>)) {
    throw new Rejected(Reason.MALFORMED);
  }
  if (!ALG.equals(((Map<?, ?>) h).get("alg")) || !"JWT".equals(((Map<?, ?>) h).get("typ"))) {
    // alg is pinned before any payload byte is trusted: the alg none
    // forgery and every cross-algorithm confusion die here
    throw new Rejected(Reason.ALG_MISMATCH);
  }
  byte[] given = decode(parts[2]);
  byte[] want = mac(secret, parts[0] + "." + parts[1]);
  if (!MessageDigest.isEqual(want, given)) {
    throw new Rejected(Reason.BAD_SIGNATURE);
  }
  Object p = parse(decode(parts[1]));
  if (!(p instanceof Map<?, ?> cm)
      || !(cm.get("exp") instanceof Long exp)
      || !(cm.get("iat") instanceof Long iat)
      || !(cm.get("iss") instanceof String iss)
      || !(cm.get("sub") instanceof String sub)) {
    throw new Rejected(Reason.MALFORMED);
  }
  if (!ISSUER.equals(iss)) {
    throw new Rejected(Reason.WRONG_ISSUER);
  }
  if (now.getEpochSecond() >= exp) {
    throw new Rejected(Reason.EXPIRED);
  }
  return new Claims(exp, iat, iss, sub);
}
```

listing 183: verify: shape, alg pin, constant-time signature, then the claim rules (source: java/api/src/javabook/authn/Jwt.java, lines 87-124)

VERIFY: the constant-time property, stated not measured

`MessageDigest.isEqual`'s documentation in the pinned build carries the `@implNote` that when the second digest is nonempty, calculation time depends only on the first argument's length, never on either digest's contents,

> and the implementation is a loop that xors every byte unconditionally. A timing test would measure the jvm's jit, not the property, so the tests pin tamper rejection and the chapter pins the contract: equality decided in time independent of where the bytes differ.

Every refusal is one of five reasons, malformed, algorithm mismatch, bad signature, wrong issuer, expired, and the caller that matters, the filter, maps all five onto one 401, because a forged token and an expired one are the same non-event to a caller. Cost is part of the story too, measured on this machine: one hmac over a 150 byte span answers in 0.3 microseconds, while one pbkdf2 verification of chapter 21's stored form answers in 28 to 34 milliseconds across four runs, five orders of magnitude apart, which is the whole argument for tokens as the per-request credential and passwords as the once-per-login one.

22.3. login, prove then issue

The login endpoint is chapter 19's decode ladder wearing one job: bind the two members, prove the password, mint the token. The proof runs against the stored form through an injected verifier so a test fixture never pays 100,000 iterations, and the production verifier is `Passwords.verify`, which parses the self-describing string chapter 21 wrote, algorithm label, iteration count, salt, digest, re-derives with exactly those parameters, and compares digests through the same `isEqual`:

```
public static boolean verify(String stored, String password) {
  String[] parts = stored.split("\\$", -1);
  if (parts.length != 4 || !"pbkdf2-sha256".equals(parts[0])) {
    return false;
  }
  final int iterations;
  final byte[] salt;
  final byte[] want;
  try {
    iterations = Integer.parseInt(parts[1]);
    salt = Base64.getDecoder().decode(parts[2]);
    want = Base64.getDecoder().decode(parts[3]);
  } catch (IllegalArgumentException e) {
    return false;
  }
  if (iterations < 1 || salt.length == 0 || want.length == 0) {
    return false;
  }
  PBEKeySpec spec =
      new PBEKeySpec(password.toCharArray(), salt, iterations, want.length * 8);
  try {
    SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
    byte[] got = factory.generateSecret(spec).getEncoded();
    return MessageDigest.isEqual(got, want);
  } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
    throw new IllegalStateException("pbkdf2 missing from the jdk", e);
```

listing 184: verify replays the stored parameters, never today's defaults, and never throws on a foreign string (source: java/api/src/javabook/authn/Passwords.java, lines 28-53)

The endpoint's one security argument is timing. A login that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch, and the fix is one line in the unknown-email branch: run the same pbkdf2 against a dummy hash, discard the answer, return the identical envelope. Both branches cost one derivation and answer the same bytes:

```
    var user = d.store().getByEmail(Validation.normalizeEmail(email));
    if (user.isEmpty()) {
      // unknown email: pay the same pbkdf2 the wrong password pays,
      // against the dummy hash, then answer the identical bytes
      d.verifier().test(Passwords.dummyHash(), password);
      throw ApiError.fail(Code.UNAUTHORIZED, "invalid credentials");
    }
    if (!d.verifier().test(user.get().passwordHash(), password)) {
      throw ApiError.fail(Code.UNAUTHORIZED, "invalid credentials");
    }
    long now = d.clock().instant().getEpochSecond();
    Jwt.Claims claims =
```

listing 185: both failure branches answer the same 401 after the same work (source: java/api/src/javabook/authn/Login.java, lines 60-71)

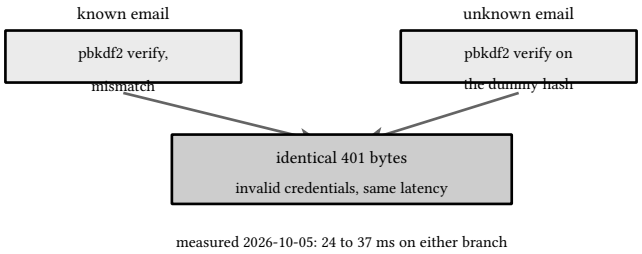


figure 38: the timing-equalized login: unknown email pays what a wrong password pays

The success path is two facts, prove then issue, and the issue is one call: `Jwt.sign` over the clock's second, fifteen minutes ahead, the service's issuer, and the user's id. The response body is `access_token` and nothing else, no user representation, because the caller just proved which user it is and can ask chapter 21's routes with the token it holds.

22.4. the filter, and the routing question it must ask first

The identity layer is a `Filter`, like chapter 20's stack, and chapter 20 established the constraint that shapes it: filters run before the kernel dispatches, so the path parameters a handler reads do not exist yet and the filter cannot ask them. What the filter needs instead is the route pattern the request will match, open or not, and that is a question only the kernel's table can answer. The chapter adds one query method to `App`, `resolvePattern`, which walks the same table the dispatcher walks and answers the `METHOD pattern` string or nothing:

```
  public Optional<String> resolvePattern(String method, String rawPath) {
    for (Route r : routes) {
      if (r.method().equals(method) && r.pathMatches(rawPath)) {
        return Optional.of(r.method() + " " + r.pattern());
      }
    }
    return Optional.empty();
  }

  /** Every registered route as "METHOD pattern", the authz audit's inventory. */
  public List<String> routes() {
    List<String> out = new ArrayList<>();
    for (Route r : routes) {
      out.add(r.method() + " " + r.pattern());
    }
    return out;
  }
```

listing 186: the pattern query: filters ask the table instead of guessing at paths (source: java/api/src/javabook/httpx/App.java, lines 156-172)

The filter itself is then short. Extract the bearer, verify it, install the actor as an exchange attribute, and for a pattern that is not open and an actor that did not resolve, answer the 401 envelope with the rfc 9110 challenge header and stop the chain:

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  Actor actor = bearer(ex);
  String pattern =
      app.resolvePattern(ex.getRequestMethod(), ex.getRequestURI().getRawPath())
          .orElse(null);
  if (actor == null && pattern != null && !open.test(pattern)) {
    // one message for the missing and the invalid token alike: the
    // envelope never teaches a forger which layer refused
    ex.getResponseHeaders().set("WWW-Authenticate", "Bearer");
    Kernel.error(ex, ApiError.fail(Code.UNAUTHORIZED, "authentication required"));
    return;
  }
  if (actor != null) {
    ex.setAttribute(ATTRIBUTE, actor);
  }
  chain.doFilter(ex);
}

/**
 * Extracts and verifies the bearer token, null when there is no
 * actor. The scheme word is matched case-insensitively, the rfc
 * 9110 rule for auth-scheme, while the token itself is exact.
 */
private Actor bearer(HttpExchange ex) {
  String header = ex.getRequestHeaders().getFirst("Authorization");
  if (header == null || header.length() < 7
      || !header.substring(0, 7).equalsIgnoreCase("Bearer ")) {
    return null;
  }
  String token = header.substring(7);
  if (token.isEmpty()) {
    return null;
  }
  try {
    Jwt.Claims c = Jwt.verify(token, secret, clock.instant());
    return new Actor(c.sub(), roles.apply(c.sub()));
  } catch (Jwt.Rejected e) {
    return null;
  }
}
```

listing 187: install the actor, close the closed routes, one message for missing and invalid alike (source: java/api/src/javabook/authn/Authn.java, lines 59-98)

Three decisions in those lines. A bad token on an open route is not a failure: it means no actor, exactly like a request that carried nothing, so health checks and registrations do not break because a stale token rode along. A request that matches no pattern at all falls through untouched, because 404 and 405 are the kernel's answers, not authentication verdicts. And the 401 carries `WWW-Authenticate: Bearer`, the header rfc 9110 section 11.6.1 requires a 401 to carry, which most hand-rolled servers forget and every standards-strict client checks. The scheme word itself is matched case-insensitively, `bearer` and `Bearer` both parse, because rfc 9110 section 11.1 makes the auth-scheme case-insensitive, while the token that follows is exact bytes and the test pins that the same token upper-cased answers no actor.

The actor is the whole handoff to the rest of the book: a record of the user id and the roles a function reads from the store at request time. The roles function arrives in the next chapter's wiring as the grants table, and the token never carries them, so a demoted admin loses the role on the next request, not at the next login.

22.5. the wiring

The program's `Main` grows the family the way chapter 21's did, one filter line and one register call, plus the secret. The secret comes from `JBAPI_SECRET` when the environment carries one, else a fresh random 32 bytes at boot, and the ephemeral lane is stated in the code's own comment: every token dies with the process that signed it, correct for a dev boot and a lockout for a production one. The open set names the routes that take anonymous traffic, the two probes, registration, and login itself, and everything else in the table now demands a bearer token:

```
    app.use(new RequestId());
    app.use(new AccessLog(log));
    app.use(new Timing(log));
    // the load-shedding pair: the cheap per-key 429 decision runs
    // before the wait-for-permit one, so a flood is answered without
    // ever spending admission queue budget on it
    app.use(new RateLimit(intEnv(RATE_BURST_ENV, 30),
        intEnv(RATE_REFILL_MS_ENV, 1000) * 1_000_000L, 10_000, 60_000_000_000L,
        System::nanoTime, RateLimit::byRemoteAddress));
    app.use(new Admit(
        Math.max(64, Runtime.getRuntime().availableProcessors() * 8), 500_000_000L));
    app.use(new Authn(app, secret, clock, grants::of,
        pattern -> table.get(pattern) == Policy.OPEN));
    app.use(new Guard(app, table));
    app.use(new Recover(log));
    app.registerKernelRoutes();
    Users.register(app, new Users.Deps(
        users, clock,
        () -> UUID.randomUUID().toString(), Hasher.pbkdf2()));
    Login.register(app, new Login.Deps(users, clock, secret, Passwords::verify));
```

listing 188: the wiring: the filter stack in its order, login reading the same store registration writes (source: java/api/src/javabook/Main.java, lines 81-100)

One store instance serves both families, stated in a comment because it is the one wiring mistake this chapter can make silently: a login route reading a second `MemStore` would answer unknown email for every registered user and nothing else would look wrong. The filter sits inside timing, so its 401s are timed and logged like any answer, and outside recover, which states a boundary worth knowing: the recover filter protects the chain it wraps, not the stack that wraps it, so a defect that throws inside the authn layer itself escapes the envelope lane and closes the connection, fail closed, the same boundary chapter 20 drew around its own layers.

22.6. the tests

The vector is the discipline. A fixed secret, a fixed clock at 2026-01-01T00:00:00Z, and the token is bytes: the tests pin `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NjcyMjY1MDAs...` through `xHf6iLieoQ5F_avvH6L0zHTlMJn_KDbL4-kBGZ3J4FQ`, signature included, and any drift in the writer, the encoder, or the hmac fails a string comparison. Around it: tampering each of the three segments, the wrong secret, expiry to the second on both sides of the boundary, the `alg: none` forgery and a foreign algorithm correctly signed, malformed shapes from the empty token to a padded segment to a non-object header, and a foreign issuer under a valid signature. The socket side boots the chapter's whole wiring on an ephemeral port: the deterministic token the fixture's step clock mints, the two failure branches asserting byte-identical envelopes minus the request id and the dummy hash's presence in the counting verifier's log, presence validation, the ladder, the 401 with its challenge header for anonymous, garbage, and expired bearers, the open routes staying open with a garbage bearer aboard, the case-insensitive scheme word beside the exact-bytes token, the 404 that authn never touches, and the real-hasher round trip that prints its cost. Sixteen tests this chapter, green three consecutive runs in a module now numbering 92.

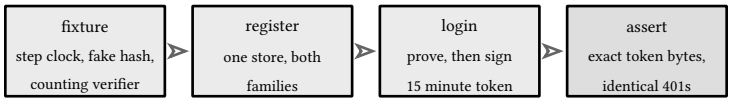


figure 39: the golden flow: fixed inputs in, frozen bytes out, over a real socket

sources: RFC 7519 for the json web token claims and the exp semantics, RFC 9110 sections 11.1, 11.6.1, and 15.5.1 for the case-insensitive auth-scheme, the bearer challenge header, and the 401, rfc-editor.org, accessed 2026-10-05. `MessageDigest.isEqual`'s time-independence `@implNote` read from the pinned build's own sources, `tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325, as was `Mac`'s contract. The pbkdf2 construction is chapter 17′s, verified against rfc 8018 there. The go contrasts, argon2id, sessions, refresh rotation, cookies, against book 4, chapter 21. Verified live 2026-10-05 by the `javabook.authn` tests under the vendored junit 6.1.3 lane, 16 tests green three consecutive runs in a module of 92: the frozen token vector, tamper and wrong-key rejection, second-precise expiry, the alg pin, malformed shapes, the foreign issuer, the deterministic login token, identical failure envelopes with the dummy hash burned, the 401 with WWW-Authenticate for missing, garbage, and expired bearers, open routes staying open, the case-insensitive scheme word with its exact-bytes token, the login ladder, and the real-hasher round trip measuring 43 to 61 ms to register, 24 to 37 ms to log in, and 24 to 31 ms to refuse a wrong password across the suite's runs on this machine, beside 0.3 microseconds for one hmac-sha256 over a 150 byte span.

23. authz and rbac

Chapter 22 answered who is calling. This chapter answers what that caller may do, in `javabook.authz`: roles as stored data with a grant and revoke surface, the policy table that maps every route pattern to one plain function, the object-level check whose absence the industry names IDOR, the 401 versus 403 discipline that keeps the two answers from swapping places, and the deny-by-default guard that makes an unaudited route unreachable instead of quietly open. The go book built the same layer over its mux's own pattern matching, book 4, ch 22 (go 1.27, authz and rbac), and the java difference runs through the whole chapter: java's filters run before the kernel dispatches, so the table asks the kernel's own route resolution instead of reading path values the framework would have filled.

23.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication runs first and installs an actor or does not, chapter 22's filter. Authorization reads that actor against the route and, on the object routes, against the record. A caller that never proved anything has no authorization to deny, only an authentication demand, so 401 means try again with credentials and 403 means do not bother, this identity will never pass. The wiring makes the sequence physical, the authn filter sits outside the guard, so by the time a policy runs the actor question is settled and a policy never parses a token, checks a signature, or hashes a password:

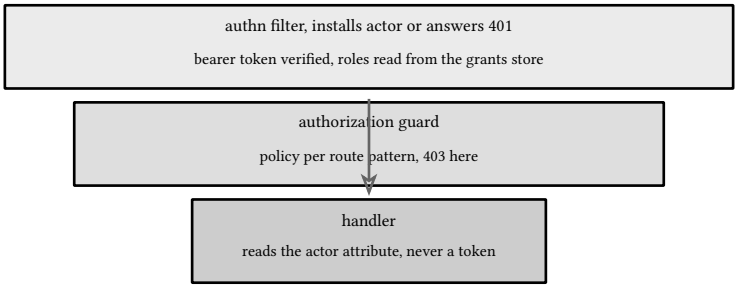


figure 40: two layers in wiring order: authn outside, guard inside, handler last

23.2. policies as functions

A policy is one function from exchange, actor, and pattern to a verdict: return to allow, throw the contract's `ApiError` to deny. The whole surface composes from four of them, open, any actor, a required role, an owner-or-admin check, and each is short enough to read in one breath:

```
Policy OPEN = (ex, actor, pattern) -> {};

/** Demands a resolved actor and nothing more. */
static Policy authenticated() {
  return (ex, actor, pattern) -> {
    if (actor == null) {
      throw ApiError.fail(Code.UNAUTHORIZED, "authentication required");
    }
  };
}

/**
 * Demands one role. No actor is an authentication failure, a
 * present actor without the role is an authorization failure, and
 * the two codes never mix.
 */
static Policy role(String role) {
  return (ex, actor, pattern) -> {
    if (actor == null) {
      throw ApiError.fail(Code.UNAUTHORIZED, "authentication required");
    }
    if (!actor.roles().contains(role)) {
      throw ApiError.fail(Code.FORBIDDEN, "insufficient role");
    }
  };
}

/**
 * The object policy: the actor owns the path's id or holds admin.
```

listing 189: open, authenticated, and role: the 401 versus 403 discipline lives in the last two lines of role (source: java/api/src/javabook/authz/Policy.java, lines 29-57)

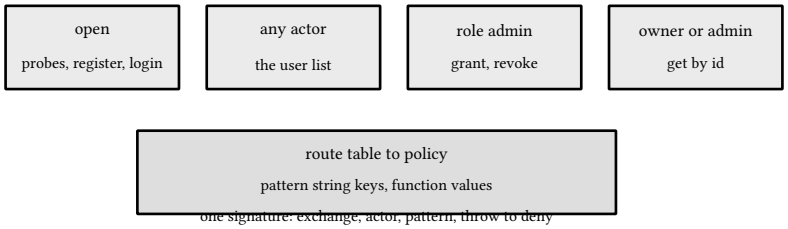


figure 41: four policies, one signature, the table maps routes onto them

Plain functions beat a rules dsl or annotation mining at this size because the compiler type-checks the table, the audit test can walk it, and a new rule is a function beside its siblings. The role vocabulary itself is two constants and deliberately flat, admin and user with no hierarchy: an `admin implies user` rule would be one line, and the day the implicit expansion hides a grant from the person auditing the table, that line costs more than it ever bought.

23.3. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. `GET /api/users/` followed by someone else's id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check needs the record, whether it exists and who owns it, and in go that meant the handler after the fetch. Java's guard has another option worth taking: the policy extracts the wildcard itself, walking the raw path and the pattern in lockstep, so the object check can live in the table and still answer all three questions correctly:

```
static Policy ownerOrAdmin(UserStore users) {
  return (ex, actor, pattern) -> {
    if (actor == null) {
      throw ApiError.fail(Code.UNAUTHORIZED, "authentication required");
    }
    String id = pathParam(ex.getRequestURI().getRawPath(), pattern, "id");
    if (users.get(id).isEmpty()) {
      throw ApiError.fail(Code.NOT_FOUND, "user not found");
    }
    if (!actor.userId().equals(id) && !actor.roles().contains(Roles.ADMIN)) {
      throw ApiError.fail(Code.FORBIDDEN, "not the owner");
    }
  };
}

/**
 * The production table: every pattern the service registers mapped
 * to its policy. The guard denies any registered pattern missing
 * from here, the audit test walks both directions, and the authn
 * filter's open set is read off this same table so there is one
 * source of truth for what takes anonymous traffic. Later chapters
 * append rows, none ever edit existing ones.
```

listing 190: the object policy: extract the wildcard, 404 the unknown id, 403 the stranger, allow owner or admin (source: java/api/src/javabook/authz/Policy.java, lines 64-85)

The store read inside the policy is the price of the 404-first order, and it is stated rather than hidden: the handler fetches again, one row read twice on the object route, in exchange for keeping the unknown id answering the same 404 a nonexistent route would. What the order does not hide is existence from an authenticated caller: a stranger probing ids still splits 404 from 403, and which ids are real is exactly what that split reveals. The go chapter made the same trade and named it, the alternative is answering 404 for both, and the choice here is debuggability over disclosure at book scale, stated where the code states it rather than paid silently.

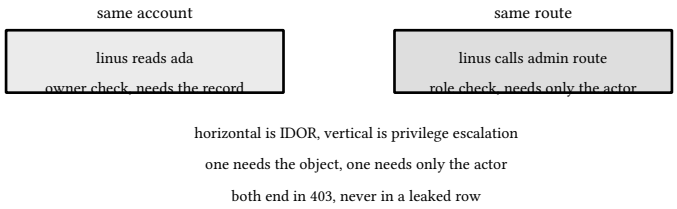


figure 42: horizontal versus vertical: two axes of privilege, two different checks

23.4. roles as stored data

Roles are rows, not magic. The port is three verbs over user id to role set, the in-memory implementation is one monitor, and the invariant that matters lives where the only verb that can break it runs: a revoke that would leave the whole table without one admin throws, so the demotion is refused with nothing written:

```
public synchronized void revoke(String userId, String role) throws LastAdminException {
  TreeSet<String> rows = table.get(userId);
  if (rows == null || !rows.contains(role)) {
    return;
  }
  if (Roles.ADMIN.equals(role) && countHolders(Roles.ADMIN) ≤ 1) {
    throw new LastAdminException();
  }
  rows.remove(role);
}

private TreeSet<String> rows(String userId) {
  return table.computeIfAbsent(userId, k → new TreeSet<>());
}

private int countHolders(String role) {
  int n = 0;
  for (TreeSet<String> roles : table.values()) {
    if (roles.contains(role)) {
      n++;
    }
  }
  return n;
}
```

listing 191: the never-zero-admins invariant, checked before the row changes (source: java/api/src/javabook/authz/MemGrants.java, lines 44-67)

The admin routes are the grant surface, add one role, remove one role, answering the resulting set, and they are themselves governed by the admin policy row, so the only way in is to already hold the role. The first admin cannot come from this surface, and the bootstrap rule is a decorator at the wiring seam instead: a create that lands while the grants table has never granted anything mints the admin. The question the decorator asks is the empty grants table, not the empty user store, and the difference is fail-safety: a crash between one create and its grant leaves the table still empty, so the next registration bootstraps instead of an admin surface locked out forever, which the fail-safe test pins with a grants source whose first grant throws. Chapter 21's handlers and this chapter's table read the same wrapped store, and neither knows the rule ran:

```
public void create(User u) throws ConflictException {
  inner.create(u);
  if (grants.isEmpty()) {
    grants.grant(u.id(), Roles.ADMIN);
  }
}
```

listing 192: the bootstrap rule as a decorator: the empty grants table mints the admin (source: java/api/src/javabook/authz/FirstAdmin.java, lines 33-39)

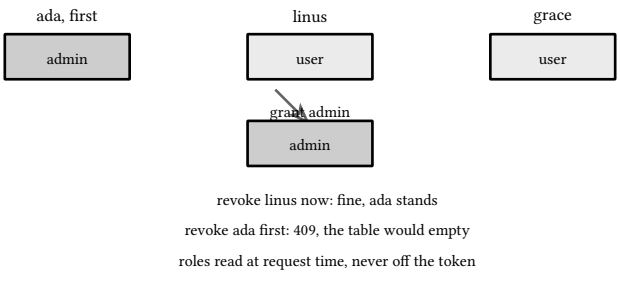


figure 43: the grant table: one column per user, the admin row must never empty

The decorator is also where the actor's role source meets the token: chapter 22's filter reads `grants.of(user)` at request time, so a demoted admin loses the role on the very next request, not at the next login, and the matrix test's last cell pins exactly that, the same bearer token crossing an admin route successfully before the grant and again after it with no reissue anywhere.

23.5. the table and deny by default

One map names every registered pattern and its policy, and the guard wraps the whole route set in it. A resolved pattern missing from the table answers 403 before a handler byte runs, so a new route that forgets its policy is loudly unreachable instead of quietly open. Requests that match no pattern at all fall through untouched, because 404 and 405 are the kernel's answers, not authorization verdicts:

```
static Map<String, Policy> productionTable(UserStore users) {
  Map<String, Policy> table = new LinkedHashMap<>();
  table.put("GET /healthz", OPEN);
  table.put("GET /readyz", OPEN);
  table.put("POST /api/users", OPEN);
  table.put("POST /api/auth/login", OPEN);
  table.put("GET /api/users", authenticated());
  table.put("GET /api/users/{id}", ownerOrAdmin(users));
  table.put("PATCH /api/users/{id}", ownerOrAdmin(users));
  table.put("POST /api/admin/users/{id}/roles", role(Roles.ADMIN));
  table.put("DELETE /api/admin/users/{id}/roles/{role}", role(Roles.ADMIN));
  return Collections.unmodifiableMap(table);
}
```

listing 193: the production table: nine rows today, later chapters append, none ever edit (source: java/api/src/javabook/authz/Policy.java, lines 87-99)

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  String pattern =
      app.resolvePattern(ex.getRequestMethod(), ex.getRequestURI().getRawPath())
          .orElse(null);
  if (pattern == null) {
    chain.doFilter(ex);
    return;
  }
  Policy policy = table.get(pattern);
  if (policy == null) {
    Kernel.error(ex, ApiError.fail(Code.FORBIDDEN, "no policy for route"));
    return;
  }
  try {
    policy.check(ex, Authn.actor(ex), pattern);
  } catch (ApiError e) {
    Kernel.error(ex, e);
    return;
  }
  chain.doFilter(ex);
}

@Override
```

listing 194: the guard: no row, no response, routing answers pass through (source: java/api/src/javabook/authz/Guard.java, lines 35-57)

The open set chapter 22 wired as a hand-written predicate is now read off this same table, `table.get(pattern) == Policy.OPEN`, so there is one source of truth for what takes anonymous traffic and the authn filter and the guard cannot drift apart. An anonymous call to an unaudited route meets the authn layer's 401 first and an authenticated one meets the guard's 403, and either way the route is unreachable, which is the property deny by default buys.

The audit closes the loop from the other side and both directions walk it: every table row must resolve, through the kernel's own `resolvePattern`, to a registered route with exactly that pattern, and every registered route, enumerated by the `App.routes()` inventory chapter 22 added, must carry a row. A route cannot exist unaudited, a row cannot point at nothing, and the counts face each other in the test, nine against nine today.

23.6. the wiring

The program's `Main` grows the family once more. The grants table and the store are constructed before the filters because the policy table reads the store and the authn filter reads the grants, and one store instance wrapped in the bootstrap decorator serves registration, login, the policy table, and the admin routes alike:

```
    // one grants table and one store serve every family, and the
    // policy table reads the same store the handlers do
    FileGrants grants = new FileGrants(engine);
    UserStore users = new FirstAdmin(new FileStore(engine), grants);
    Map<String, Policy> table = Policy.productionTable(users);
    app.use(new RequestId());
    app.use(new AccessLog(log));
    app.use(new Timing(log));
    // the load-shedding pair: the cheap per-key 429 decision runs
    // before the wait-for-permit one, so a flood is answered without
    // ever spending admission queue budget on it
    app.use(new RateLimit(intEnv(RATE_BURST_ENV, 30),
        intEnv(RATE_REFILL_MS_ENV, 1000) * 1_000_000L, 10_000, 60_000_000_000L,
        System::nanoTime, RateLimit::byRemoteAddress));
    app.use(new Admit(
        Math.max(64, Runtime.getRuntime().availableProcessors() * 8), 500_000_000L));
    app.use(new Authn(app, secret, clock, grants::of,
        pattern -> table.get(pattern) == Policy.OPEN));
    app.use(new Guard(app, table));
```

listing 195: the guarded wiring: table before filters, one store under everything (source: java/api/src/javabook/Main.java, lines 76-94)

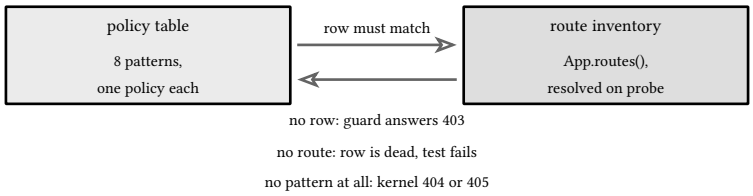


figure 44: the audit: table and route inventory face each other, both directions checked

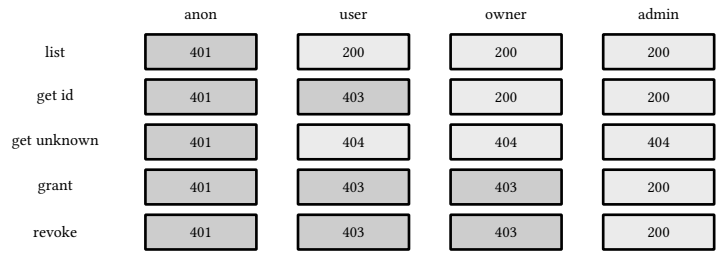
One honest omission against the go chapter: its user package projected the wire representation per reader, the public projection for everyone but admins, field-level authorization inside the record. This book keeps chapter 21's one representation and guards routes, so an authenticated caller on the list route sees emails. The reason is the same discipline the whole part runs on, the pinned test bytes: a second projection would fork the writer and the etag with it, and the trade is stated here rather than paid silently.

23.7. the rbac matrix

The grid is the proof: every route times every actor kind, anonymous, regular user, the stranger on someone else's object, the admin, one expected status per cell, asserted in one table-driven test through the full guarded stack over a real socket, the same path production traffic takes. Each row is a cell with a name that fails loudly:

```
    // route, actor kind, expected status, one cell per row
    String[][] rows = {
        {"GET /api/users", "anon", "401"},
        {"GET /api/users", "user", "200"},
        {"GET /api/users/u1", "anon", "401"},
        {"GET /api/users/u1", "stranger", "403"},
        {"GET /api/users/u1", "owner", "200"},
        {"GET /api/users/u1", "admin", "200"},
        {"GET /api/users/u9", "owner", "404"},
        {"POST /api/admin/users/u2/roles", "anon", "401"},
        {"POST /api/admin/users/u2/roles", "user", "403"},
        {"DELETE /api/admin/users/u1/roles/user", "user", "403"},
        // the grant lands: the same token, the store's truth moved
        {"POST /api/admin/users/u2/roles", "admin", "200"},
        {"POST /api/admin/users/u1/roles", "promoted", "200"},
    };
    for (String[] row : rows) {
      String bearer = switch (row[1]) {
        case "anon" -> null;
        case "user", "stranger", "promoted" -> user;
        case "owner", "admin" -> admin;
        default -> throw new IllegalArgumentException(row[1]);
      };
      HttpResponse<String> r =
          send(app, methodOf(row[0]), pathOf(row[0]), bodyOf(row[0]), bearer);
      assertEquals(Integer.parseInt(row[2]), r.statusCode(), row[0] + " as " + row[1]);
    }
```

listing 196: the matrix rows: route times actor, one status each, the promoted cell last (source: java/api/test/javabook/authz/AuthzTests.java, lines 125-151)



shaded: the refused cells. ownership opens nothing administrative

figure 45: the matrix: routes one way, actors the other, statuses in the cells

Around the matrix: the code split pinned on its own, the 401 envelope with its challenge header beside the 403 with the rule's own words and the 404 the unknown id answers first, the last-admin chain, demote the only admin for the 409, grant a second for the 200, demote the first again for the 200, and the 409 returns to whoever now stands alone, the grant surface validating unknown users and unrecognized roles, the deny-by-default counterexample wired with one row deleted and proving the 403 by name, the fail-safe bootstrap with a throwing grants source, and the audit walking both directions. Eight tests this chapter, green three consecutive runs in a module now numbering 92.

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.com, for the IDOR class and the deny-by-default guidance, and RFC 9110 sections 15.5.1

to 15.5.4 for the 401, 403, and 404 semantics, accessed 2026-10-05. The go contrasts, the handler-side owner check, the field-level projection, and the mux-driven audit, against book 4, chapter 22. Verified live 2026-10-05 by the `javabook.authz` tests under the vendored junit 6.1.3 lane, 8 tests green three consecutive runs in a module of 92, covering the twelve matrix cells, the code split with messages, the last-admin chain, the grant surface's 404 and 422 lanes, the deny-by-default counterexample, the fail-safe bootstrap, the two direction audit at nine rows against nine routes, and the flat role set with the bootstrap grant.

24. store

The service has run so far on maps, which tests love and restarts hate. The obvious answer is an embedded database, and the jdk does not have one: `java.sql` is a driver surface with no driver for any local engine behind it, and the last bundled database, java db, the rebranded derby, left the distribution after java 8. The corpus lane is stdlib exclusive anyway, so this chapter owns the machinery outright in `javabook.store`: a write-ahead log of crc32c framed records, replay that recovers state and tolerates a torn tail, transaction markers that make groups atomic, snapshot compaction with an atomic rename, and the `UserStore` port from chapter 21 implemented over it all behind the same interface, no handler any the wiser. The go book built the same engine on the same reasoning when its dependency ruling retired the sqlite driver, book 4, ch 23 (go 1.27, store), and that chapter's depth is the bar this one aims at, with java's own shapes: no `defer`, so close is the rollback, no `crc32` table to build, the Castagnoli polynomial ships in `java.util.zip` since 9, and no sorted slice to maintain by hand, a `TreeMap` is the sorted key view.

24.1. the frame

One log record is one frame: an eight byte header, payload length then its crc32c, wrapped around a payload of version byte, op byte, and the op's fields, every length little endian so a payload can carry a full request body. The op set is four: put, tombstone, and the two transaction markers, and the version byte is the codec's contract with the future, a frame this build cannot parse stops the walk rather than being guessed at:

```
static byte[] payload(Record r) {
  ByteArrayOutputStream p = new ByteArrayOutputStream();
  p.write(VERSION);
  p.write(r.op().tag);
  switch (r.op()) {
    case PUT -> {
      writeU32(p, r.key().length);
      p.writeBytes(r.key());
      writeU32(p, r.value().length);
      p.writeBytes(r.value());
    }
    case TOMBSTONE -> {
      writeU32(p, r.key().length);
      p.writeBytes(r.key());
    }
    case TX_BEGIN, TX_COMMIT -> writeU64(p, r.seq());
  }
  return p.toByteArray();
}

/**
 * Walks frames in b until the bytes run out or a frame is torn,
 * appending each fully decoded record to out and answering the
```

listing 197: the payload writer: version, op, and the op's fields, lengths u32 little endian (source: java/api/src/javabook/store/Wal.java, lines 91-113)

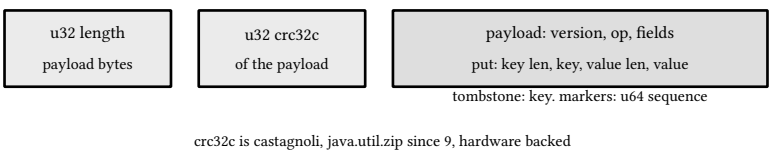


figure 46: one frame: the checksum covers exactly the payload, the payload carries its own version

Decoding walks frames until the bytes run out or a frame is torn: a remaining tail shorter than the header, a length that overruns, a checksum that lies, or a payload this build cannot parse, and the walk stops there without an error, exactly what recovery wants. A torn tail is the crash artifact the design accepts, not a corruption event:

```
public static int decode(byte[] b, List<Record> out) {
  int off = 0;
  while (off + HEADER_LEN ≤ b.length) {
    int n = readU32(b, off);
    int sum = readU32(b, off + 4);
    // the bound is written subtraction-side first: an overflowing
    // length field must read as an overrun, never wrap negative
    if (n < 2 || n > b.length - off - HEADER_LEN) {
      return off;
    }
    byte[] payload = Arrays.copyOfRange(b, off + HEADER_LEN, off + HEADER_LEN + n);
    CRC32C crc = new CRC32C();
    crc.update(payload, 0, payload.length);
    if ((int) crc.getValue() ≠ sum) {
      return off;
    }
    Record r = decodePayload(payload);
    if (r == null) {
      return off;
    }
    out.add(r);
    off += HEADER_LEN + n;
  }
  return off;
}
```

listing 198: the walk: every refusal is the same stop, the offset names where (source: java/api/src/javabook/store/Wal.java, lines 116-141)

The appender is the write side, one channel opened for append with groups written whole, one buffer per group in as few write calls as the channel takes, because a channel write may write partially and the loop is the honesty. `sync` is `force`, the durability point a commit means when the caller wants it promised.

24.2. replay, the recovery story

Open reads the log and rebuilds memory from it, and the rule that makes groups atomic is the marker pair: ops apply only between a begin and its matching commit, an orphan group whose commit never arrived is discarded whole, and the sequence resumes past the last begin seen so a reopened engine never reuses a committed number:

```
public static Engine open(Path walPath) throws IOException {
  Path parent = walPath.getParent();
  if (parent != null) {
    Files.createDirectories(parent);
  }
  Engine e = new Engine(walPath);
  boolean snapLoaded = e.loadSnapshot();
  byte[] log = Files.exists(walPath) ? Files.readAllBytes(walPath) : new byte[0];
  List<Wal.Record> recs = new ArrayList<>();
  int good = Wal.decode(log, recs);
  for (Wal.Record r : recs) {
    if (r.op() == Wal.Op.TX_BEGIN && r.seq() > e.seq) {
      e.seq = r.seq();
    }
  }
  replay(e.data, recs);
  if (good < log.length) {
    // a torn tail sits behind the decode stop: truncate it before the
    // appender opens, or every later frame lands past dead bytes no
    // future decode can cross and the engine serves commits it cannot
    // recover
    try (FileChannel ch = FileChannel.open(walPath, StandardOpenOption.WRITE)) {
      ch.truncate(good);
    }
  }
  e.wal = Wal.Appender.open(walPath);
  if (snapLoaded) {
    Files.deleteIfExists(e.walPath.resolveSibling(walPath.getFileName() + ".old"));
  }
  return e;
}
```

listing 199: open: snapshot first when one stands, then the committed groups replayed over it (source: java/api/src/javabook/store/Engine.java, lines 60-90)

```java
static void replay(TreeMap<String, byte[]> state, List<Wal.Record> recs) {
  List<Wal.Record> group = null;
  long groupSeq = 0;
  for (Wal.Record r : recs) {
    switch (r.op()) {
      case TX_BEGIN -> {
        group = new ArrayList<>();
        groupSeq = r.seq();
      }
      case TX_COMMIT -> {
        if (group != null && groupSeq == r.seq()) {
          for (Wal.Record op : group) {
            apply(state, op);
          }
        }
        group = null;
      }
      default -> {
        if (group != null) {
          group.add(r);
        }
      }
    }
  }
}
```

listing 200: replay: no marker pair, no application, the whole group dies together (source: java/api/src/javabook/store/Engine.java, lines 377-401)

The invariant is named in the test that proves it, exhaustively rather than by fuzz: for any truncation of a framed log, replaying the decodable prefix answers exactly the state of the transactions committed inside that prefix. The test builds a three transaction log and asserts it twice over. At every frame boundary the recovered state is compared against a hand-computed expectation, eleven boundaries from the empty log to the full one, written out in the test so the oracle is not the implementation agreeing with itself. And between the boundaries, at every byte length from zero to the whole file, the reopened engine is compared against a decode of the same prefix, so a torn tail changes nothing anywhere else. A crash mid-frame, mid-group, mid-header, all one geometry, and geometry is not timing. Open also creates the log's parent directories, one line, because a boot into a fresh directory should not die on a missing folder:

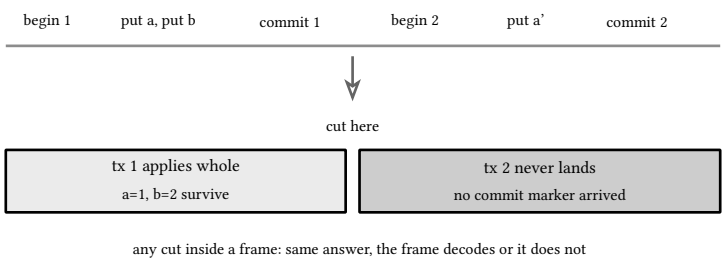


figure 47: a log recovers to the last committed group, whatever the cut

24.3. the transaction, append then mutate

One shape covers every write: take the transaction, run, commit. The write side holds the engine's full lock from begin to finish, so exactly one writer holds a group at a time and every read inside it is current without a second mechanism. Commit is append then mutate: the group's frames, the begin marker, the ops, the commit marker, go out in one write, and only then does memory take the group, values copied so nothing aliases the caller's array:

```
    public void commit() throws IOException {
      if (finished) {
        throw new IllegalStateException("transaction already finished");
      }
      finished = true;
      List<Wal.Record> frames = new ArrayList<>(ops.size() + 2);
      frames.add(Wal.Record.marker(Wal.Op.TX_BEGIN, txSeq));
      frames.addAll(ops);
      frames.add(Wal.Record.marker(Wal.Op.TX_COMMIT, txSeq));
      try {
        wal.append(frames);
      } catch (IOException e) {
        // the append tore mid-log: memory stays untouched, and the
        // engine is poisoned so no later commit can bury the tear
        // under frames recovery would never reach
        broken = true;
        lock.writeLock().unlock();
        throw e;
      }
      for (Wal.Record r : ops) {
        switch (r.op()) {
          case PUT -> data.put(new String(r.key(), StandardCharsets.UTF_8),
              r.value().clone());
          case TOMBSTONE -> data.remove(new String(r.key(), StandardCharsets.UTF_8));
          default -> throw new IllegalStateException("marker staged as an op");
        }
      }
      seq = txSeq;
      lock.writeLock().unlock();
    }
```

listing 201: the commit order: the group's frames append in one write, then memory mutates (source: java/api/src/javabook/store/Engine.java, lines 196-225)

The order is the guarantee. The log is always at least as current as memory, so a crash between the append and the mutation recovers to the committed state, and the inverse order would leave memory holding a state the log cannot rebuild, the exact failure recovery exists to prevent. The write reaches the operating system but is not forced, the line sqlite's synchronous NORMAL draws: durable across application crashes, relaxed about power loss, with sync as the operator's stronger call.

Java has no `defer`, and the discard lane is the idiom that fills the gap: the transaction is `AutoCloseable`, and close without a commit is the rollback. Every caller is a try-with-resources block, so the normal exit, the early throw, the store's own conflict exception, and a panic unwinding through all land in the same close, the group discarded with nothing to undo because nothing was appended until commit:

And the append can fail. A disk that throws mid-group leaves a torn frame in the file and the channel positioned after it, and any commit that landed later would bury that tear under frames recovery stops before ever reaching, memory silently ahead of the readable log. The engine refuses that outcome by construction: a failed append poisons it, the lock is released, memory stays untouched, and every later `begin` answers `IllegalStateException` until the process restarts and recovers to the last whole group. Fail closed, stated plainly, because the alternative is the one inconsistency the design cannot repair.

```
public void create(User u) throws ConflictException {
  try (Engine.WriteTx tx = engine.begin()) {
    if (tx.get(EMAIL + u.email()) != null) {
      throw new ConflictException();
    }
    tx.put(REC + u.id(), stored(u));
    tx.put(EMAIL + u.email(), u.id().getBytes(StandardCharsets.UTF_8));
    tx.put(orderKey(u), u.id().getBytes(StandardCharsets.UTF_8));
    tx.commit();
  } catch (IOException e) {
    throw new UncheckedIOException("store write failed", e);
  }
}
```

listing 202: the caller's shape: one tx, three ops, commit inside try-with-resources (source: java/api/src/javabook/store/FileStore.java, lines 47-59)

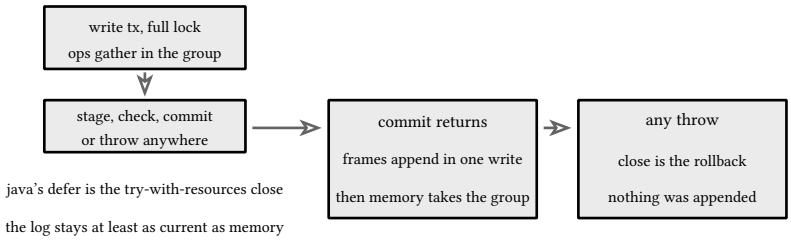


figure 48: a transaction's exits, every one of them clean

24.4. the read view

A read takes one clone of the key space under the read lock and keeps it for its whole life: no dirty reads, because memory only mutates after a group is logged, no non-repeatable reads within one transaction, because the view never moves. That is snapshot isolation, built here rather than configured, and the demo is deterministic single threaded code, the statement order is the schedule, no thread, no barrier, no sleep: pin a view, commit past it, read again, still the old bytes, a fresh view sees the new ones. Writers serialize on the full lock, one at a time, and the concurrency test races eight virtual threads through ten groups each and finds eighty whole groups and 240 frames, every one accounted for. The clone is the stated price, a tree map copied per read, and at book scale, a few hundred entries, that is cheap enough to keep the engine plain.

24.5. compaction

The log only grows, and compaction is the answer: write the current state to a snapshot file, then rotate the log. The snapshot is a tiny format, magic, codec version, the last committed sequence number, and a crc32c of the body, and the body is the entries in sorted key order, so the bytes are deterministic. The sequence rides the header because the open sequence's no-reuse promise must survive compaction too: a post-compaction reopen resumes numbering where the old log stopped instead of restarting it, and the test pins the number walking across a compact and two reopens. The write lands on a tmp file, is forced, and is moved over atomically, `Files.move` with `ATOMIC_MOVE`, replace-existing on this build's windows and posix alike, and only then does the log rotate, rename to `.old`, create fresh, remove `.old`. Every interruption of that order is recoverable by the open sequence, because the snapshot already subsumes everything the old log held:

```
  public void compact() throws IOException {
    lock.writeLock().lock();
    try {
      byte[] body = encodeBody();
      java.io.ByteArrayOutputStream out = new java.io.ByteArrayOutputStream();
      out.writeBytes(MAGIC);
      Wal.writeU32(out, CODEC);
      // the sequence rides the header so a post-compaction reopen
      // resumes numbering instead of restarting it
      Wal.writeU64(out, seq);
      CRC32C crc = new CRC32C();
      crc.update(body, 0, body.length);
      Wal.writeU32(out, (int) crc.getValue());
      out.writeBytes(body);

      Path tmp = snapPath.resolveSibling(snapPath.getFileName() + ".tmp");
      try (FileChannel ch = FileChannel.open(tmp, StandardOpenOption.CREATE,
          StandardOpenOption.WRITE, StandardOpenOption.TRUNCATE_EXISTING)) {
        ByteBuffer b = ByteBuffer.wrap(out.toByteArray());
        while (b.hasRemaining()) {
          ch.write(b);
        }
        // the force is the point: a renamed but unwritten snapshot is
        // the one corruption the open sequence cannot survive
        ch.force(true);
      }
      try {
        Files.move(tmp, snapPath, StandardCopyOption.ATOMIC_MOVE,
            StandardCopyOption.REPLACE_EXISTING);
      } catch (java.nio.file.AtomicMoveNotSupportedException e) {
        Files.move(tmp, snapPath, StandardCopyOption.REPLACE_EXISTING);
      }

      wal.close();
      Files.move(walPath, walPath.resolveSibling(walPath.getFileName() + ".old"),
          StandardCopyOption.REPLACE_EXISTING);
      wal = Wal.Appender.open(walPath);
      Files.deleteIfExists(walPath.resolveSibling(walPath.getFileName() + ".old"));
    } finally {
      lock.writeLock().unlock();
    }
  }
```

listing 203: compact: force, atomic move, then the rotation whose recovery argument relies on the order (source: java/api/src/javabook/store/Engine.java, lines 257-298)

NOTE: the one loss the design accepts

A snapshot that arrives torn despite the force is not trusted: the open sequence falls back to log-only recovery, and the pre-compaction state is gone, because the rotation already removed the log that held it. The torn-snapshot test documents exactly this, the old key missing, the post-compaction writes back, and the alternative, trusting a checksum that failed, is not an alternative at all.

Measured by the suite on this machine, dated 2026-10-05: sixty superseded puts grow the log to 3770 bytes, compaction leaves a fresh log of 0 bytes and a 43 byte snapshot holding the one surviving value, and a thousand users, three thousand records and 300,010 log bytes, reopen by replay in 10.1 to 11.9 ms, while the compacted

form, a 234,036 byte snapshot, reloads in 7.1 to 9.0 ms. Compaction is a call the operator or the wiring makes, not a background thread, and the honesty is the point: nothing in the engine wakes up on its own.

24.6. the user store over the engine

The adapter implements chapter 21's port, `create`, `get`, `getByEmail`, `list`, and the key scheme is the documentation the keys enforce. The record lives under `u:rec:`, the email uniqueness claim under `u:email:`, and the keyset order index under `u:ord:`, where the creation millisecond rides sign-flipped fixed width hex beside the id, exactly the pair the pagination cursor carries, so storage order and keyset order are one notion of position:

```
/** The order index key: sign-flipped millis keep ascending time ascending in hex. */
static String orderKey(User u) {
  return orderKey(u.createdAt().toEpochMilli(), u.id());
}

static String orderKey(long millis, String id) {
  return ORD + String.format("%016x", millis ^ Long.MIN_VALUE) + ":" + id;
}
```

listing 204: the order key: sign-flipped millis keep ascending time ascending in hex, the id tiebreak beside it (source: java/api/src/javabook/store/FileStore.java, lines 125-132)

The stored form is not the wire form, and the split is worth naming. The record serializes all six components including the password hash, because the store must verify logins after a restart, while the wire writer chapter 21 pinned lists five and never the hash: the hash is stored, never served, and the test asserts both directions, the stored bytes contain it, the wire bytes do not. The etag story closes the loop: the etag is sha256 over the wire bytes, the wire writer is deterministic, so the same record yields the same etag forever, and the restart test asserts the bytes did not move across a close and reopen:

```
public List<User> list(CursorKey after, int limit) {
  List<User> out = new ArrayList<>(Math.min(limit, 64));
  String from = orderKey(after.t(), after.id());
  SortedMap<String, byte[]> view = engine.view();
  for (String key : view.tailMap(from).keySet()) {
    if (!key.startsWith(ORD)) {
      break;
    }
    if (key.equals(from)) {
      // the cursor names its own row, the page is strictly after it
      continue;
    }
    String id = new String(view.get(key), StandardCharsets.UTF_8);
    decode(view.get(REC + id)).ifPresent(out::add);
    if (out.size() == limit) {
      break;
    }
  }
  return out;
}
```

listing 205: the list walk: one view, the range after the cursor, the id tiebreak free from the key (source: java/api/src/javabook/store/FileStore.java, lines 104-123)

The grants ride the same engine under `r:grant:`, one json array per user in sorted order, with the never-zero-admins invariant checked inside revoke against the engine's own live keys, and one detail worth its ink: the emptied set is a tombstone, not a row, so the key drops and the log gets to carry a `TOMBSTONE` frame, the op earning its keep.

24.7. the wiring and the reboot proof

`Main` swaps `MemStore` and `MemGrants` for the engine-backed pair, one engine, one wal, one snapshot, both key spaces, opened at a directory from `JBAPI_DATA` with a shutdown hook closing it, and nothing else in the program changes: the first-admin decorator wraps the file store now, the policy table reads it, the handlers cannot tell. The end to end test is the chapter's proof: boot the whole api over a temp directory, register, login, list, then close everything, open a fresh engine and a fresh app over the same directory, and the second login verifies against the persisted hash, the second list answers the persisted user, and the token minted before the restart still carries the persisted admin role, because roles are store truth, chapter 23′s rule, and the store now survives:

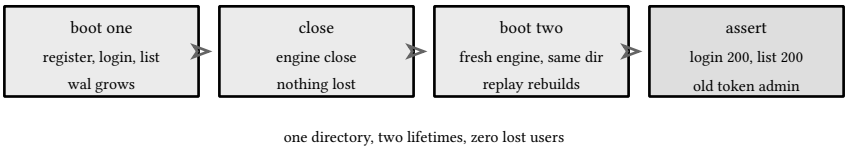


figure 49: the reboot proof: two processes, one directory, everything comes back

The test story is the chapter's spine: the codec round trip and its refusals, corruption, a future codec version under an honest checksum, a lying length and a near-INT_MAX one that must not wrap into a two gigabyte allocation, every truncation at the codec level and every truncation again through a real reopen with the hand-computed boundary oracle, the orphan group, the rolled-back group, the poisoned engine after a failed append with its lock released, the sequence surviving compaction, the pinned view, concurrent writers, compaction shrinking with the state intact and the post-compaction writes replaying over the snapshot, the torn snapshot falling back to the log, the parent directory created on open, the store's round trips, the surviving email claim, the stable etag, the pagination walk with the tie, the grants surviving with their invariant and their tombstone, and the reboot. Twenty-four tests this chapter, green three consecutive runs in a module now numbering 92, and the whole engine is four files, 904 lines including javadoc, small enough to be read whole, which is the point of owning it.

sources: the jdk's own sources for `java.util.zip.CRC32C` (since 9, the castagnoli polynomial), `FileChannel.force`, `Files.move` and `ATOMIC_MOVE`, and `ByteBuffer`'s byte orders, read from `tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325. The java db removal from the jdk stated against oracle's own migration guide, "removed tools and components" at docs.oracle.com/en/java/javase/26/migrate/, which says java db was bundled with jdk 7 and 8 and is no longer included, accessed 2026-10-05. The wal and synchronous NORMAL analogy follows the go book's store chapter, book 4, chapter 23, and the sqlite documentation it cited. Verified live 2026-10-05 by the `javabook.store` tests under the vendored junit 6.1.3 lane, 24 tests green three consecutive runs in a module of 92, with the measurements above printed by the runs: compaction 3770 to 0 bytes of wal beside a 43 byte snapshot, a thousand users replaying 300,010 bytes in 10.1 to 11.9 ms, the 234,036 byte compacted snapshot reloading in 7.1 to 9.0 ms.

25. concurrency control

Chapter 10 built the thread ladder and measured the thread itself. This chapter owns the question underneath a running service: what limits anything. The kernel's executor is `newVirtualThreadPerTaskExecutor`, one virtual thread per request, no queue, no cap, no rejection, and that is correct for the thread but silent about every other resource. A flood of requests is a flood of threads, each holding heap, an open exchange, and a turn at whatever the handler touches, and unlimited concurrency is just unbounded memory with extra steps. The service answer is three layers built over this chapter and the two after it: admission control in `javabook.conc`, the bounded state tables of `javabook.cache`, and the fairness budget of `javabook.limit`. The go book walked the same territory from the lost update up, book 4, ch 24 (go 1.27, concurrency control), and its singleflight and admission material lands here in java's shapes, with the lost update itself closing the chapter as the promise chapter 21 made and this one keeps.

25.1. thread-per-request removes a cap that was doing work

A platform-thread pool of 200 was a crude load shedder by accident: request 201 waited in the pool's queue and timed out there. Virtual threads deleted the pool, deleted the queue, and deleted the rejection handler with it, so the service must rebuild all three on purpose. The jep 444 wording is the design center and worth quoting exactly: virtual threads "are cheap and plentiful, and thus should never be pooled", and code that needs to cap concurrent access to a limited resource should use "constructs specifically designed for that purpose, such as semaphores". That sentence is this chapter's whole architecture: not a pool, a semaphore at the door.

What the flood actually costs is not thread creation, and the next section measures that honestly, it is the resources the threads hold while parked or running: heap for the stack chunks, file descriptors and buffers for each open exchange, and turns at the carriers while a handler computes. Backpressure is the name for refusing work before those costs pile up, and its two knobs are how much may run at once and how long the rest may wait.

25.2. one hundred thousand threads, measured

```
int virtualCount = 100_000;
var virtualDone = new java.util.concurrent.atomic.AtomicInteger();
var virtualThreads = new Thread[virtualCount];
long virtualStart = System.nanoTime();
for (int i = 0; i < virtualCount; i++) {
  virtualThreads[i] = Thread.ofVirtual().unstarted(virtualDone::incrementAndGet);
}
for (Thread t : virtualThreads) {
  t.start();
}
for (Thread t : virtualThreads) {
  t.join();
}
long virtualNanos = System.nanoTime() - virtualStart;
```

listing 206: 100,000 virtual threads spawned, started, and joined, timed (source: java/samples/src/Ch25/Scale.java, lines 19-32)

Measured 2026-10-05 on this machine, 20 reported cores, the pinned build 27+35-2325: 100,000 virtual threads spawned, started, and joined in 111 ms, 1.12 microseconds per thread, and all 100,000 were simultaneously alive at a latch, each parked on the heap, the 20 carriers free for other work. The platform comparison in the same sample creates, starts, and joins 2,000 platform threads one at a time, 85.11 microseconds per thread, a ratio of 76 platform-thread costs per virtual-thread cost, before touching the 1 mb default stack reservation that makes stacking 100,000 of them a memory planning question rather than a rounding error. The lesson is not that virtual threads are fast, it is where the bottleneck moved: creation is noise, so anything that limits a virtual-thread service limits the resources the threads hold, never the threads themselves.

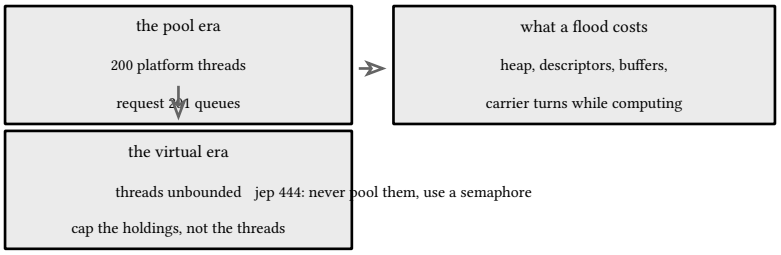


figure 50: the cap moved: pools capped threads, the service must cap what threads hold

25.3. executor sizing, the question that came back

Virtual threads answered sizing for blocking work by deleting it, and cpu-bound work puts the question right back. A carrier runs one mounted virtual thread at a time, a thread that computes for a second holds its carrier for a second, and enough of them starve every blocked-and-unmountable thread behind them. So the hybrid is the standing rule: virtual threads where the work waits, a fixed platform pool sized to the core count where the work computes, and the two meet when a request's virtual thread submits its compute to the sized pool. The pinned build's own `ThreadPoolExecutor` is that pool, and its constructor is where bounded queues and rejection policies live:

```
    var pool = new java.util.concurrent.ThreadPoolExecutor(cores, cores, 0L,
        java.util.concurrent.TimeUnit.MILLISECONDS,
        new java.util.concurrent.ArrayBlockingQueue<>(4), policy);
    for (int i = 0; i < 100; i++) {
      try {
        pool.submit(() -> {
          if (Thread.currentThread() == caller) {
            callerRan.incrementAndGet();
          }
          sleep(25);
          completed.incrementAndGet();
          return null;
        });
      } catch (java.util.concurrent.RejectedExecutionException e) {
        refused.incrementAndGet();
      }
```

listing 207: one measured run: core count threads, a 4-slot queue, a rejection handler (source: java/samples/src/Ch25/Bounded.java, lines 67-82)

The four shipped handlers are four sentences. `AbortPolicy` throws `RejectedExecutionException`, loud refusal. `CallerRunsPolicy` runs the overflow on the submitting thread, backpressure that slows the producer to the consumer's pace. `DiscardPolicy` drops the task silently, and `DiscardOldestPolicy` drops the queued head to make room for the new. Measured 2026-10-05, 20 cores, 100 tasks of 25 ms submitted flat out: abort completed 24 and refused 76, caller-runs completed all 100 with the main thread running 12 of them itself, and the virtual-thread executor ran the same 100 sleeps in 38 ms with no sizing knobs at all, because sleeps block and blocking unmounts. For request work the silent discards are disqualified on their face, a dropped request is an unexplained timeout to its client, so the choice is refuse loudly or push back, and the admission layer below is the refuse-loudly answer at the http door.

25.4. admission control

The semaphore the jep names, as a filter. At most `permits` requests are inside the stack at once, an arriving request beyond the cap waits up to `maxWaitNanos`, and one that waited out its budget is refused with the 503 overload envelope and a `Retry-After`:

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  boolean admitted;
  try {
    admitted = gate.tryAcquire(maxWaitNanos, TimeUnit.NANOSECONDS);
  } catch (InterruptedException e) {
    // shutdown or cancellation: restore the flag, refuse this one,
    // the permit state is untouched
    Thread.currentThread().interrupt();
    admitted = false;
  }
  if (!admitted) {
    rejected.increment();
    long seconds = Math.max(1L, (maxWaitNanos + 999_999_999L) / 1_000_000_000L);
    ex.getResponseHeaders().set("Retry-After", Long.toString(seconds));
    Kernel.error(ex, ApiError.fail(Code.OVERLOAD, "server at capacity, retry later"));
    return;
  }
  try {
    chain.doFilter(ex);
  } finally {
    gate.release();
  }
}
```

listing 208: tryAcquire with a budget, refuse with the overload envelope, release in the finally (source: java/api/src/javabook/conc/Admit.java, lines 61-83)

Three decisions carry the weight. The wait is the bounded queue: `tryAcquire` with a timeout parks the request's own virtual thread, a parked virtual thread costs its few hundred bytes, so the queue is as long as the clients make it and exactly as patient as the budget. The refusal is the rejection policy, and it is the overload code chapter 19 reserved for exactly this, mapped through the kernel's one envelope writer. And the release lives in a `finally`, because the one leak that matters in an admission layer is the permit that never comes back: the tests pin it against a handler that returns, a handler that throws past the adapted route, and a flood of 50 requests against 4 permits where every request answers, 200 or 503, none lost. The interrupt lane answers overload too, restores the flag, and leaves the semaphore untouched.

The answer is written, never thrown, the same fact chapter 22's authn coded against: the kernel's `ApiError` adapter wraps the route handler at the bottom of the chain, so a filter exception would climb past it unanswered and drop the connection. The leak test is the one worth reading whole, because an admission layer that leaks permits turns every error into a capacity cut:

```
@Test
void thePermitComesBackWhenTheHandlerThrows() throws Exception {
  try (Wired w = slowApp(1, 50 * MS, 200)) {
    // the adapted route handler lets the unchecked throw climb: the
    // exchange dies, and the release in the finally still runs
    assertThrows(IOException.class, () -> get(w.app(), "/boom"));
    assertEquals(200, get(w.app(), "/slow").statusCode(),
        "a thrown request cannot leak its permit");
    assertEquals(0, w.admit().rejected());
  }
}
```

listing 209: the throw climbs past the adapted handler, the exchange dies, and the finally still releases (source: java/api/test/javabook/conc/AdmitTests.java, lines 122-132)

25.5. where the pair sits in the stack

The rate layer of chapter 27 lands next to this one, and the wiring order between them is contract. The cheap decision runs first: a rate check is a map lookup and an arithmetic take, so a flooding client collects 429s without ever being granted a queue wait, and admission's budget is spent only on traffic that already passed the fairness gate. Both sit inside the access log and timing, so a shed request is logged as the 503 or 429 the client saw, the chapter 20 order rule applied to two more layers:

```
    // the load-shedding pair: the cheap per-key 429 decision runs
    // before the wait-for-permit one, so a flood is answered without
    // ever spending admission queue budget on it
    app.use(new RateLimit(intEnv(RATE_BURST_ENV, 30),
        intEnv(RATE_REFILL_MS_ENV, 1000) * 1_000_000L, 10_000, 60_000_000_000L,
        System::nanoTime, RateLimit::byRemoteAddress));
    app.use(new Admit(
        Math.max(64, Runtime.getRuntime().availableProcessors() * 8), 500_000_000L));
```

listing 210: the load-shedding pair in the stack, rate outside admission, both inside the log (source: java/api/src/javabook/Main.java, lines 84-91)

The permit count in the wiring is `max(64, 8 per core)`, a blocking handler holds its permit across io so the number wants headroom over the core count, and the wait budget is 500 ms, about half a round trip of client patience. Both are constructor arguments, not constants burned into the class, because the right numbers are a deployment fact.

25.6. pinning, one paragraph

Chapter 10 measured it and this chapter only inherits the conclusion: `synchronized` no longer pins, jep 491 since 24, the 1,000 sleeper probe ran 64 ms against a 2,000 ms pinning floor, so the monitors in this book's store and limiter are safe under virtual threads. What still pins on 27 is native frames on the stack, blocking inside class initialization, and jni or foreign calls that block in native code, which is why the store chapter's file channel work and this chapter's semaphore wait are the shapes to keep: both unmount.

25.7. structured task scope, waiting to land

Handler-level fan-out, one request racing two backend calls and cancelling the loser, is the shape the jdk has been previewing since 21 and still previews in 27, jep 533, the seventh preview:

```
try (var scope = StructuredTaskScope.open()) {
  var left = scope.fork(() -> fetchLeft());
  var right = scope.fork(() -> fetchRight());
  scope.join();
  // one scope, one lifetime: both subtasks are done or cancelled here
}
```

fragment, illustrative, not executed

It compiles only under `--enable-preview`, so it lives here as a snippet and never rides the samples runner, the wave's standing rule. The pairing to watch is scoped values, chapter 10′s other final feature: a scope's forked children are exactly the threads that inherit its bindings, which is why the two apis grew up together and why the fan-out above is also the context-propagation story a service needs.

25.8. compare-and-swap, the promise chapter 21 made

Chapter 21 parked the user's version member as the substrate for optimistic concurrency, and this is the mechanism and the route that spend it. The lost update is the oldest race in the book: two readers fetch version 3, both write back, and one overwrite silently wins. The lock-free answer is compare-and-swap, `AtomicReference`'s one instruction: set the value to the candidate only if it is still the expected one, and let the loser re-read and retry.

```
var box = new java.util.concurrent.atomic.AtomicReference<Long>(3L);

// thread A and thread B both hold the version 3 they read
boolean aWon = box.compareAndSet(3L, 4L);  // true
boolean bWon = box.compareAndSet(3L, 5L);  // false: the world moved

// bWon is false, so B re-reads, merges onto 4, and tries again
// losing is detected, never guessed at
```

fragment, illustrative, not executed

The update scope turns that instruction into a route family: `PATCH /api/users/\{id\}` carrying `If-Match`, answered by the 412 precondition code on the stale version and the version bump on the win. The stack answers in three layers, fast fail, invariant, atomicity. Fast fail is the handler: the If-Match is required, a missing header answers 412 before any decode work runs, the choice the go book made for the same route over the 428 the rfc merely suggests, and the header is checked against the current etag before the body is read:

```
  /**
   * The update scope: existence first, the same discipline the authz
   * chapter kept, then the precondition, then the decode, then the
   * compare-and-swap. The If-Match is required, and the contract
   * chose 412 over the 428 the rfc merely suggests, the choice the
   * go book made for the same route. A missing or stale precondition
   * refuses before any decode work runs.
   */
  private static Handler patch(Deps d) {
    return ex -> {
      String id = Kernel.pathValue(ex, "id");
      User current = d.store().get(id)
          .orElseThrow(() -> ApiError.fail(Code.NOT_FOUND, "user not found"));
      String etag = etag(current);
      String header = headerJoined(ex, "If-Match");
      if (header == null) {
        throw ApiError.fail(Code.PRECONDITION_FAILED, "If-Match is required for updates");
      }
      if (!ifMatch(etag, header)) {
        throw ApiError.fail(Code.PRECONDITION_FAILED, "precondition failed");
      }
      Map<String, Object> body = Kernel.bindJson(ex, Set.of("email", "display_name"));
      String email = patchMember(body, "email");
      String displayName = patchMember(body, "display_name");
      List<String> details = Validation.validateUpdate(email, displayName);
      if (!details.isEmpty()) {
        throw ApiError.fail(Code.VALIDATION_FAILED, "validation failed",
            details.toArray(String[]::new));
      }
      try {
        User updated = d.store().update(id, current.version(),
            email != null ? Validation.normalizeEmail(email) : current.email(),
            displayName != null ? displayName : current.displayName());
        ex.getResponseHeaders().set("ETag", etag(updated));
        Kernel.writeJson(ex, 200, updated.toJson());
      } catch (ConflictException e) {
        throw ApiError.fail(Code.CONFLICT, "email already registered");
      } catch (StaleVersionException e) {
        // another writer won between the read and the swap: the same
        // answer the stale etag already carries
        throw ApiError.fail(Code.PRECONDITION_FAILED, "precondition failed");
      }
    };
  }
```

listing 211: existence, then the precondition, then the decode: the stale request never burns a parse (source: java/api/src/javabook/user/Users.java, lines 82-125)

The invariant is the version member, and the comparison runs the rfc's If-Match rules, which differ from the If-None-Match two sections below it in exactly one word: the comparison is strong. A star matches any current representation, a list matches when the etag appears in it, and a weak `W/` form never matches, where the 304 lane counts it. A sender may split the list across repeated header lines, and the read combines them into one list before it compares, the comma join the rfc's field-value grammar allows. The star's truth is evaluated where the precondition is, at the read, so a writer landing between that read and the swap still meets the swap's own 412 and a retry succeeds, the precondition is a gate, not a reservation:

```
/**
 * RFC 9110's If-Match, and the one rule that differs from the
 * If-None-Match two sections below it: the comparison is strong.
 * A star matches any current representation, a list matches when
 * the etag appears in it, and a weak W/ form never matches.
 */
static boolean ifMatch(String etag, String header) {
  if (header == null) {
    return false;
  }
  for (String candidate : header.split(",")) {
    String t = candidate.trim();
    if (t.equals("*") || t.equals(etag)) {
      return true;
    }
  }
  return false;
}
```

listing 212: ifMatch: strong comparison, star and list carry, weak never matches (source: java/api/src/javabook/user/Users.java, lines 127-144)

And the atomicity is the store's, the only layer that is. The port grew one method, `update` gated on the version the caller read, and the file implementation rides the whole compare-and-swap inside one engine write transaction: the version check, the email claim swap, and the record write commit or discard together, because chapter 24's full lock across a transaction makes the check atomic with the write it gates:

```
@Override
public User update(String id, int ifVersion, String email, String displayName)
    throws ConflictException, StaleVersionException {
  // the whole compare-and-swap rides one write transaction, and the
  // engine holds its full lock across the transaction, so the
  // version check is atomic with the write it gates
  try (Engine.WriteTx tx = engine.begin()) {
    User u = decode(tx.get(REC + id))
        .orElseThrow(() -> new NoSuchElementException("no user " + id));
    if (u.version() != ifVersion) {
      throw new StaleVersionException();
    }
    if (!u.email().equals(email)) {
      byte[] claim = tx.get(EMAIL + email);
      if (claim != null && !id.equals(new String(claim, StandardCharsets.UTF_8))) {
        throw new ConflictException();
      }
      tx.delete(EMAIL + u.email());
      tx.put(EMAIL + email, id.getBytes(StandardCharsets.UTF_8));
    }
    User updated =
        new User(u.id(), email, displayName, u.createdAt(), u.version() + 1, u.passwordHash());
    tx.put(REC + id, stored(updated));
    tx.commit();
    return updated;
  } catch (IOException e) {
    throw new UncheckedIOException("store write failed", e);
  }
}
```

listing 213: the CAS in one write transaction: check, swap the claim, bump, commit, or discard whole (source: java/api/src/javabook/store/FileStore.java, lines 60-89)

The proof is the duel, two PATCHers carrying the same valid etag, released together by one barrier. Both may pass the handler's check, only the swap decides, and every run answers exactly one 200 and one 412 with version 2 standing after:

```
@Test
void theDuelCrownsExactlyOneWriter() throws Exception {
  try (App app = boot()) {
    String e1 = registerForEtag(app);
    java.util.concurrent.CountDownLatch go = new java.util.concurrent.CountDownLatch(1);
    var codes = new java.util.concurrent.ConcurrentLinkedQueue<Integer>();
    List<Thread> duel = new java.util.ArrayList<>();
    for (int i = 0; i < 2; i++) {
      final int n = i;
      duel.add(Thread.ofVirtual().start(() -> {
        try {
          go.await();
          codes.add(patch(app, "/api/users/u1", e1, "application/json",
              "{\"display_name\":\"Writer" + n + "\"}").statusCode());
        } catch (Exception e) {
          throw new RuntimeException(e);
        }
      }));
    }
    go.countDown();
    for (Thread t : duel) {
      t.join();
    }
    int ok = 0;
    for (int code : codes) {
      if (code == 200) {
        ok++;
      } else {
        assertEquals(412, code);
      }
    }
    assertEquals(1, ok, "two writers, one etag, one winner: the store's CAS decides");
    assertEquals(2L, body(get(app, "/api/users/u1", null)).get("version"));
  }
}
```

listing 214: the duel: one barrier releases both writers, the store's CAS crowns one (source: java/api/test/javabook/user/UserResourceTests.java, lines 358-392)

Around the duel sit the contract's edges, each its own test: the missing and the stale and the weak and the list-without-the-etag refused, the star and the carrying list accepted, the bump serving a fresh etag that feeds the chapter 21 conditional read, the email claim following the patch in both directions with the freed address registrable again, json null members decoding as the leave-alone form while every member null still leaves nothing to update, the patch validation lanes including the empty object and a display name ceiling counted in characters rather than utf-16 units, and the unknown id answering 404 before any precondition work. The store side pins the same swap at its own level, the stale version refused, the claim swap atomic, the bumped record surviving a reopen with its hash intact, and the guarded wiring pinning the policy row the route rides, anonymous 401, stranger 403, owner and admin 200.

The chapter's own tests are five over a real socket: three permits serving three concurrent slow requests, the fourth refused after its budget with the envelope and the header, the permit returned on completion and on a handler throw, and the 50-request flood against 4 permits losing none. Twenty-nine tests land across the three packages of this wave, 14 in the cache chapter and 10 in the rate chapter, and the update scope adds 12 more over the user and store packages, green three consecutive runs with the module lane green beside them.

sources: jep 444 at openjdk.org/jeps/444, the pooling prohibition and the semaphore guidance quoted from its text, accessed 2026-10-05. Jep 491 and the pinning measurement are chapter 10's, re-verified there 2026-10-04 against

openjdk.org/jeps/491. Jep 533, structured concurrency seventh preview in 27, verified 2026-10-05 at openjdk.org/jeps/533. Rfc 9110 section 13.1.1, the If-Match strong comparison mandate, the star rule, and the lost-update framing, read from the full text at rfc-editor.org, accessed 2026-10-05. The rejection policy semantics read from the pinned build's own `ThreadPoolExecutor` sources, `tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325, accessed 2026-10-05. `Semaphore.tryAcquire` timeout semantics and `AtomicReference.compareAndSet` from the same src.zip. Measurements (100,000 virtual threads in 111 ms at 1.12 microseconds per thread, 2,000 platform threads at 85.11 microseconds each, ratio 76.1, 100,000 simultaneously alive, abort 24 completed 76 refused, caller-runs 100 completed with 12 on the caller, the same 100 sleeps in 38 ms on the virtual executor, 20 reported cores) produced by `java/samples/src/Ch25` under `pwsh tools/run-java-samples.ps1 -Chapter Ch25`, dated 2026-10-05. Verified live 2026-10-05 by the `javabook.conc` tests under the vendored junit 6.1.3 lane, 5 tests green, 29 across this wave's three packages, and the update scope verified by the `javabook.user` and `javabook.store` tests, 47 green three consecutive runs, 133 across the module, the duel answering one 200 and one 412 every run.

26. caching

A cache is a promise with two prices. The promise is freshness within a stated window, the first price is memory, which must be bounded, and the second price is the stampede, the crowd of concurrent misses that all decide to fill the same key and hammer the backend they were supposed to protect. This chapter builds both answers in `javabook.cache`: a bounded table of expiring entries over one `ConcurrentHashMap`, and a single-flight fill so one loader runs per key no matter how many callers arrive together. The go book built the same layer from lru upward, book 4, ch 25 (go 1.27, caching), and its stampede section is the one this chapter's future map answers directly. Java's jdk has no cache of its own, `ConcurrentHashMap` is the raw material and everything above it is owned here, which is the stdlib-exclusive lane's whole trade.

26.1. ttl from first principles

An entry carries its own deadline. On read, a deadline that has passed means the entry is gone, removed by the very read that noticed it, and no thread ever sweeps the table on a timer. The boundary is exact and the tests pin it: an entry with a 100 nanos ttl answers at 99 and is dead at 100, at-or-after means gone, and a ttl of zero or less is refused outright because an entry born dead is a defect, not a policy. The clock is injected, `System::nanoTime` in production and a hand-driven counter in every test, so not one test sleeps:

```
public V get(String key) {
  Cell<V> c = table.get(key);
  if (c == null) {
    return null;
  }
  if (nanos.getAsLong() ≥ c.deadline()) {
    table.remove(key, c);
    return null;
  }
  return c.value();
}
```

listing 215: the read: live values only, the noticing read reaps, no timer anywhere (source: java/api/src/javabook/cache/TtlTable.java, lines 45-55)

26.2. bounded: the eviction decision

Memory must be capped, and past the cap the table evicts in a fixed order. First the dead: an expired entry is free space the sweep reclaims before anything live is touched. Then, when everything held is live, the entry dying soonest goes, the one whose remaining life is shortest, so the table sheds the entries it was about to lose anyway. That is not lru: tracking access order means a queue updated on every read and locks around it, and the deadline the entry already carries is a free eviction rank that deterministic tests can pin exactly. The whole write path, eviction plus insert, runs under one monitor, and reads never take it, cells are immutable records published through the map's own atomics:

```
  public void put(String key, V value, long ttlNanos) {
    if (ttlNanos ≤ 0) {
      throw new IllegalArgumentException("ttlNanos must be positive");
    }
    synchronized (writeLock) {
      if (table.get(key) == null && table.size() ≥ maxEntries) {
        makeRoomLocked();
      }
      table.put(key, new Cell<>(value, nanos.getAsLong() + ttlNanos));
    }
  }
```

listing 216: the write: room is made under the monitor, a replaced key never evicts a neighbor (source: java/api/src/javabook/cache/TtlTable.java, lines 63-73)

The edge table is the contract: a full table of three entries with deadlines 100, 50, and 200 loses the 50 on the fourth insert, a dead entry is swept in preference to any live one, replacing an existing key never evicts another, and a flood of 50 distinct keys against a cap of 3 ends with 3 held. The monotone claim the tests assert is `size() ≤ maxEntries` under any interleaving, including 8 virtual threads racing 200 writes each.

The eviction scan is linear in the table, walked only when a new key arrives at a full table, and that cost is a number the design owns rather than hides: at book scale, thousands of entries, the scan is microseconds of work under the write monitor, and the escape hatch when it stops being cheap is a shard per core or a heap keyed on the deadline, both named here so the ceiling is a decision and not a discovery. The cache chapter of the go book answered crowding with an lru list whose every read moves a node, bookkeeping this design declines upfront: reads here stay untouched by the capacity machinery entirely.

26.3. reads without locks

A read is one `get` on the map plus a deadline comparison, and it never touches the monitor, which is a claim about memory visibility that deserves its proof. Every stored cell is an immutable record, one value and one deadline, published through the `ConcurrentHashMap` put, and the map's atomics create the happens-before edge chapter 10 established: a reader that observes the cell observes a fully constructed object, never a torn one. The conditional remove, `remove(key, cell)` with the cell as witness, means two readers racing to reap the same dead entry both succeed harmlessly, the second's removal finds nothing and returns false, and a writer that replaced the cell in between is not undone, because the witness no longer matches. The whole read path is three lock-free steps, and the concurrency claims are pinned twice, 8 racing writers holding the cap exact, and 16 threads sliding one hot key 8,000 times without ever recreating it.

26.4. the sliding window, the same table serving reuse

One more read shape earns its keep in the next chapter: the sliding window, where every hit slides the deadline to now plus the window. An entry that keeps being touched never expires, an idle one dies one window after its last touch, which turns the same bounded table into an idle-key reaper. The rate limiter stores its per-key buckets here, so a key that stops asking costs nothing after its window and a key that keeps asking keeps its bucket, and the cache's production presence in the service is exactly that seat:

```
public V sliding(String key, long ttlNanos, Function<String, V> maker) {
  if (ttlNanos ≤ 0) {
    throw new IllegalArgumentException("ttlNanos must be positive");
  }
  long now = nanos.getAsLong();
  Cell<V> slid = table.computeIfPresent(key, (k, c) ->
      now < c.deadline() ? new Cell<>(c.value(), now + ttlNanos) : null);
  if (slid ≠ null) {
    return slid.value();
  }
  synchronized (writeLock) {
    Cell<V> c = table.get(key);
    long fresh = nanos.getAsLong();
    if (c ≠ null && fresh < c.deadline()) {
      // another thread created or slid the key between the failed
      // slide and the monitor: take its word
      table.put(key, new Cell<>(c.value(), fresh + ttlNanos));
      return c.value();
    }
    if (c ≠ null) {
      table.remove(key, c);
    } else if (table.size() ≥ maxEntries) {
      makeRoomLocked();
    }
    V made = maker.apply(key);
    table.put(key, new Cell<>(made, nanos.getAsLong() + ttlNanos));
    return made;
  }
}
```

listing 217: sliding: the hit slides in one per-key atomic step, only creation takes the monitor (source: java/api/src/javabook/cache/TtlTable.java, lines 88-116)

The hot path deserves its own sentence: a hit is one `computeIfPresent` that swaps the cell for one with a slid deadline, atomic per key, no table-wide monitor and no capacity scan, so the rate chapter's per-request bucket touch never queues behind an eviction. The maker runs under the write monitor on the creation path, so it must be cheap, and that rule carries weight: the expensive fill belongs to the single flight, which waits outside every lock, and the reason is the next section's whole argument.

26.5. single flight

The stampede, precisely: a popular key expires, 64 requests miss in the same instant, and each one walks to the backend because each innocently observed a miss. The fix is one future per key. The first caller installs a `CompletableFuture` in an inflight map and becomes the filler, every caller that arrives while the fill runs joins that same future, and when the fill lands the entry is stored and the future completes for everyone:

```
  public V get(String key, Function<String, V> loader) {
    V hit = table.get(key);
    if (hit != null) {
      return hit;
    }
    CompletableFuture<V> mine = new CompletableFuture<>();
    CompletableFuture<V> running = inflight.putIfAbsent(key, mine);
    if (running != null) {
      return running.join();
    }
    // a fill may have landed between the read above and the install:
    // the filler stores before it removes its future, so a completed
    // fill is always visible here or was still installed for the
    // putIfAbsent to find. Re-checking closes the double-fill window
    // and keeps the one-loader guarantee airtight rather than
    // probabilistic.
    V late = table.get(key);
    if (late != null) {
      inflight.remove(key, mine);
      mine.complete(late);
      return late;
    }
    V value;
    try {
      value = Objects.requireNonNull(loader.apply(key), "loader returned null");
    } catch (RuntimeException | Error t) {
      mine.completeExceptionally(t);
      inflight.remove(key, mine);
      throw t;
    }
    // order is contract: the entry is stored before the future is
    // removed, never the reverse
    table.put(key, value, ttlNanos);
    inflight.remove(key, mine);
    mine.complete(value);
    return value;
  }
```

listing 218: one future per key: the winner fills, riders join, failure is never stored (source: java/api/src/javabook/cache/Cache.java, lines 47-83)

Why not `computeIfAbsent` and be done: the map's own contract holds the bin lock through the mapping function, so a 150 ms fill pins every key that hashes into the same bin behind it, and the jdk's docs say so plainly, the computation "should be short and simple". The future map pays nothing while the fill runs, the filler is the only thread working, and the riders park on the join, which under virtual threads unmounts them off their carriers:

```
@Test
void singleFlightRunsOneFillForSixtyFourRacers() throws Exception {
  AtomicInteger fills = new AtomicInteger();
  CountDownLatch start = new CountDownLatch(1);
  Cache<String> c = new Cache<>(16, 1_000 * SECOND, () -> 0L);
  int racers = 64;
  String[] results = new String[racers];
  Thread[] threads = new Thread[racers];
  for (int i = 0; i < racers; i++) {
    final int slot = i;
    threads[i] = Thread.ofVirtual().unstarted(() -> {
      await(start);
      results[slot] = c.get("k", k -> {
        fills.incrementAndGet();
        sleep(150);
        return "v";
      });
    });
  }
  for (Thread th : threads) {
    th.start();
  }
  start.countDown();
  for (Thread th : threads) {
    th.join();
  }
  assertEquals(1, fills.get(), "the stampede collapses to one loader run");
  for (int i = 0; i < racers; i++) {
    assertEquals("v", results[i], "every racer rode the same fill");
  }
}
```

listing 219: 64 racers behind one latch, one fill, every racer served (source: java/api/test/javabook/cache/CacheTests.java, lines 125-156)

A failed fill completes its future exceptionally and is never cached: riders observe the failure, wrapped by the join's `CompletionException`, the inflight entry is removed, and the next get starts fresh, which the tests pin with a loader that throws once then succeeds. A null from a loader is refused loudly rather than stored as an eternal miss.

The one-loader guarantee has to be airtight, not probabilistic, and the race that breaks naive versions is worth naming because a test with a sleeping loader can never catch it. A loser reads a miss, the winner's whole fill lands, and only then does the loser install its own future and fill again, two backend calls for one stampede. The closing move is an ordering argument, not a lock: the filler stores the entry before it removes its future, so for a loser to install into an empty map the winner's removal must already have happened, which means the winner's store happened too, which means the loser's re-check after installing finds the entry and stands down. The blind attackers ran 400,000 two-racer rounds against this exact window and filled twice in none of them.

One recursion rule closes the section: a loader must not read its own key through this cache. It would join the very future it is filling and wait on itself forever, the single-flight contract turned inside out, and the javadoc says so in those words.

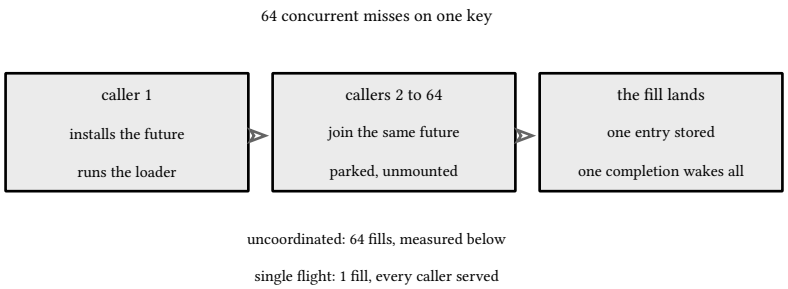


figure 51: the stampede collapsed: one fill, everyone else parked on its future

26.6. measured: the stampede and the hit rate

The counterfactual runs beside the real thing in the test class. On one side, 64 virtual threads race an uncoordinated get-or-load over a plain map, check, miss, sleep, store: measured 2026-10-05 on this machine, 64 fills, every racer paid the backend. On the other side the same 64 race the cache: 1 fill, all 64 served. The hit rate has its own deterministic workload, 1,000 gets over 10 distinct keys under a frozen clock: 10 misses, 990 hits, 99.0 percent, an exact number because the sequence and the clock are both fixed and the test asserts the miss count rather than eyeballing a ratio. Both numbers print in the junit lane's output, dated by the run.

26.7. invalidation on write

The ttl bounds staleness from above, and the write path can do better than waiting for it. Derived data whose source changed should be recomputed or dropped at the moment of the write, and the honest shape over this table is small: the writer knows the keys its write touches, so it stores the fresh value under the same key the reader will ask for, or lets the next read refill by simply not storing anything. The layer offers `put` for the first and a short ttl for the second, and the choice between them is one question, does the rebuild cost more than the write-path complexity. What the layer refuses to grow is the catalog of dependencies, key A invalidates keys B through D, because that graph is where caches earn their reputation for being clever in ways nobody can audit. A reader that can name what it wants and a writer that can name what it changed is the whole contract.

The user family needs none of this yet, which is worth stating as a fact rather than an omission: the store's reads are a tree map clone at book scale and the hash of the representation is cheaper than the bytes it names, so the cache waits for a reader expensive enough to earn it. The layer exists because that reader arrives with the observability and load chapters, and building it now against the limiter's real seat means its boundedness and its stampede answers are already tested the day it is first pointed at a handler.

26.8. the etag interplay

Chapter 21′s conditional read closes a loop with this layer. The etag is sha256 over the exact wire bytes and the writer is deterministic, so a cached representation and a recomputed one carry the same etag forever, which means a cache in front of the representation layer can never cause a 304 to lie: if the client's `If-None-Match` matches the cached bytes' hash, it matches the fresh bytes' hash too, because the same record produces the same bytes both ways. The 304 already saves the body, the cache saves the rebuild, and the two compose without coordination. That is not luck, it is the payoff of chapter 21′s decision to compute the etag rather than store it, a function cannot drift from a value it never held separately.

26.9. what not to cache

The layer takes a key and a loader, so the discipline is in what gets keyed. Roles are never cached, chapter 23′s rule that authz reads store truth at request time, and a cached role set is an access decision frozen past its revocation. Anything derived from a token's claims is not cache state, the token is already the client's own compact representation. And a personal response under a shared key is a leak in waiting, the key must name the audience

the value serves. The positive cases are exactly the expensive and the communal: derived aggregates, third-party answers, the kind of read a stampede hurts.

The chapter's tests are fourteen: the ttl boundary both sides of the deadline, the zero-ttl refusal, the three-way eviction order, the dead-before-living sweep, the replace-without-eviction, the sliding window's slide and death and rebuild, the flood bound, the 64-racer single flight at exactly one fill, the uncoordinated counterfactual at 64, the failed fill never cached, the rider seeing the wrapped failure, the exact hit rate, the 8-thread bounded stress, and the 16-thread hot-key slide that never recreates. The rate chapter adds 10 more against the same table through the limiter.

sources: `ConcurrentHashMap.computeIfAbsent`'s short-and-simple guidance and its locking behavior read from the pinned build's own sources, `tools/jdk27/build/jdk-27/lib/src.zip`, oracle jdk 27 ga build 27+35-2325, accessed 2026-10-05, and `CompletableFuture.join`'s `CompletionException` contract from the same zip. The stampede and hit-rate measurements (64 uncoordinated fills against 1 single-flight fill, 990 of 1,000 hits over 10 keys) printed by the `javabook.cache` tests under the vendored junit 6.1.3 lane, dated 2026-10-05 on tools/jdk27/build/jdk-27, 20 reported cores, 14 tests green three consecutive runs. The ttl, lru, and invalidation framing follows the go book's cache chapter, book 4, chapter 25, read for the parallel contracts.

27. rate limiting

Concurrency control answers how much at once, admission answers how long the rest may wait, and neither answers the client that asks too fast forever. Rate limiting is the fairness layer: a budget per client, spent one token per request, refilled as time passes, and refused requests answered with the one status code built for them, 429, through the envelope the kernel already owns. This chapter builds `javabook.limit`, a token bucket per key behind a filter, with the buckets stored in the last chapter's sliding table so the limiter's memory is bounded the moment it is born. The go book built its limiter on `x/time/rate` and then owned the same contract questions, book 4, ch 26 (go 1.27, rate limiting), and this chapter owns them again on nothing but `java.base`.

27.1. why limits exist

Three reasons, and they rate differently. Fairness first: one client's unbounded appetite is every other client's latency, a shared server that does not budget its callers is arbitraged by whoever asks fastest. Cost second: every request spends real money in compute and in the downstream calls it fans out to, and a scraper or a runaway retry loop can spend a day of budget in an hour. Abuse third, the deliberate flood, where the limit is a defense and the attacker's goal is exactly to be the loudest caller. The same mechanism answers all three, which is why it lives as one layer at the stack's edge rather than as per-handler counters, and why its refusals must be cheap: a limiter that spends as much refusing as the requests cost asking protects nothing.

The budget contract is client-visible, not internal policy. The 429 tells the client the shape of its budget, `Retry-After` tells it when to come back, and a well-behaved client backs off and spaces its work, which is the cheapest capacity upgrade a service ever ships. A limiter that answers nothing, or answers 500, teaches clients to retry harder, the exact opposite of the intent.

27.2. the token bucket from first principles

One bucket per key: it holds at most `burst` tokens, starts full, every request spends one, and one token accrues per `refillPeriodNanos` of elapsed time, capped at the burst. The trick worth naming is that no timer drips anything: refill is computed lazily, from the nanos elapsed since the last refill point, at the moment a take arrives, so a key nobody asks about costs exactly one small object and zero wakeups. There is no borrowing, a take with less than one token is refused and the balance stays put, and the refusal names the wait for a whole token:

```
public synchronized TokenBucket.Attempt tryTake(long now) {
  long elapsed = now - lastRefill;
  if (elapsed > 0) {
    tokens = Math.min(burst, tokens + (double) elapsed / refillPeriodNanos);
    lastRefill = now;
  }
  if (tokens ≥ 1.0 - EPSILON) {
    tokens = Math.max(0.0, tokens - 1.0);
    return new Attempt(true, 0L);
  }
  double deficitNanos = (1.0 - tokens) * refillPeriodNanos;
  long micros = Math.round(deficitNanos / 1_000.0);
  return new Attempt(false, Math.max(1L, micros * 1_000L));
}
```

listing 220: lazy refill by division, spend within an epsilon, refuse with the deficit rounded through microseconds (source: java/api/src/javabook/limit/TokenBucket.java, lines 62-75)

One arithmetic decision is pinned by the boundary tests. The accrued amount is elapsed divided by the period, not elapsed times a precomputed rate, because a whole period divides back to exactly 1.0 in ieee doubles and the contract wants exactly that: one period elapsed means exactly one token. Accrual spread across several takes sums

fractions, and a sum that is exactly one token in real arithmetic can land a last ulp low in doubles, so the spend compares within a tolerance and clamps at zero rather than refusing a token the bucket holds. The refusal's deficit is rounded through whole microseconds before the ceil to seconds, which strips the last-ulp noise a non-binary period would otherwise inflate into a whole extra second of `Retry-After`: at a three second period, two thirds of a token answers 2, not 3. The take itself is `synchronized` on the bucket, one monitor per key, so keys never contend each other and the same-key race below is a monotone spend.

The boundary table, all under a frozen clock:

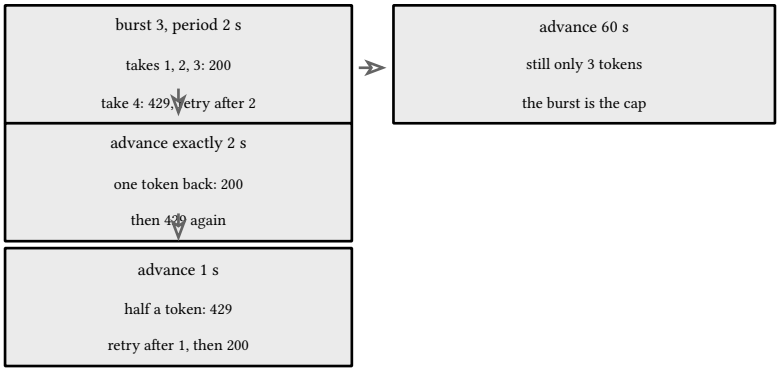


figure 52: the bucket's answers, every boundary exact under a frozen clock

At the burst, below it, above it, exactly one period later, far past it, and half a period later: six cells across four tests, no sleeps, the clock a hand-driven counter the filter and the buckets share, and a third-period test walks accrual in thirds to pin the floating-point tolerance and the uninflated header.

27.3. the 429 contract

Rfc 6585 defines the status: "the user has sent too many requests in a given amount of time". It says a response `MAY` include `Retry-After`, and it says a 429 response `MUST NOT` be stored by a cache, which chapter 26's table satisfies by construction, a 429 is an answer about the request, never a stored value. The filter writes the envelope itself through the kernel's one failure writer, with the deficit rounded up to whole seconds, the delay-seconds form rfc 9110 defines for the header:

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  String key = keyer.apply(ex);
  TokenBucket bucket = buckets.sliding(key, idleWindowNanos,
      k -> new TokenBucket(burst, refillPeriodNanos, nanos.getAsLong()));
  TokenBucket.Attempt take = bucket.tryTake(nanos.getAsLong());
  if (!take.allowed()) {
    long seconds = Math.max(1L, (take.retryAfterNanos() + 999_999_999L) / 1_000_000_000L);
    ex.getResponseHeaders().set("Retry-After", Long.toString(seconds));
    Kernel.error(ex, ApiError.fail(Code.RATE_LIMITED, "rate limit exceeded, retry later"));
    return;
  }
  chain.doFilter(ex);
}
```

listing 221: one take per request, refusal written by the filter with the header riding the envelope (source: java/api/src/javabook/limit/RateLimit.java, lines 65-78)

The header is set before the envelope is written because the exchange keeps its own response headers and the kernel writer sends what is there, and the answer is written, never thrown, the chain fact chapter 22 established: the kernel's `ApiError` adapter wraps the route handler at the bottom of the filter chain, so a filter exception would climb past it unanswered and drop the connection. The refused request costs the handler nothing, which the tests

assert with a hit counter, and the envelope carries the request id like every other failure, so a rate-limited client can correlate the refusal in its own logs.

27.4. what to key on

The keyer is an injected function, the same seam every layer in this service takes, and the choice is policy. Keying on the bearer token gives per-account budgets that die with logout. Keying on the remote address, the production default here, gives one budget per client machine that an authenticated abuser cannot dodge by dropping their token, and shares the anonymous and authenticated traffic of one address, the conservative reading. Routes can be keyed too, an expensive family given a smaller budget than a cheap one, by folding the resolved pattern into the key. The tests key on a header for the same reason production keys on the address, one app serving many independent budgets.

27.5. burst and the sustained rate

The bucket's two numbers say different things and clients feel both. The burst is the short-term allowance, a fresh or idle key may spend `burst` requests back to back, which is what makes interactive clients workable, a page load that fans into 6 requests should not meet a limiter on its first second. The refill period is the sustained rate, one token per 2 seconds in the tests, 30 per second in the wiring, and past the burst the client is throttled to exactly that pace, one refusal with a truthful `Retry-After` per step. The alternative shape, a leaky bucket that smooths output to a constant rate with no burst, protects a fragile downstream better and treats clients worse, and a windowed counter that resets each second lets 2 times the budget through at the boundary, 30 at the end of one second and 30 at the start of the next. The token bucket sits between them, its burst explicit and its sustained rate exact, and the boundary tests pin both edges: the burst is reachable in one instant and never exceeded by any refill, however long the wait. The wiring runs burst 30 with one token per second per address, thirty back to back for a fresh caller, then a steady one per second.

27.6. bounded per-key state

Every key that asks once gets a bucket, so the limiter is a memory leak unless something reaps. The buckets live in the cache chapter's `TtlTable` under its sliding window: every request slides the key's deadline, so a busy key keeps its bucket forever and an idle key's bucket dies one window after its last request, reclaimed by the next write that needs the slot. The table's cap is the hard bound, 10,000 keys in the wiring, and the constructor refuses an idle window shorter than one refill period because a window that tight would reap live buckets between requests:

```
  /**
   * The production keyer: one bucket per client address. The
   * anonymous and the authenticated traffic of one address share its
   * bucket, which is the conservative choice, an authenticated abuser
   * cannot dodge their address's budget by dropping their token.
   */
  public static String byRemoteAddress(HttpExchange ex) {
    return ex.getRemoteAddress().getAddress().getHostAddress();
  }
}
```

listing 222: the production keyer: one bucket per client address, shared across its identities (source: java/api/src/javabook/limit/RateLimit.java, lines 84-93)

The flood test walks 12 keys past a cap of 2 and asserts 2 held at the end, the same monotone claim the cache chapter's stress made, reached here through the filter's own door.

The cap has a second edge the design owns rather than hides: when every held bucket is live, the soonest-deadline eviction retires the least recently touched key, and if that key comes back it returns to a fresh bucket with a full

burst. A key flood past the cap can therefore reset a quiet client's budget, an over-admission bounded by one burst per resurrection, and the honest account is that the alternative is unbounded per-key memory. The resurrection test pins exactly this behavior, one token spent, two livelier keys evicting the victim inside its window, and the victim's next request served from a fresh full bucket. The flood's other price is the scan: only a new key at a full table pays the eviction walk, the hit path is one per-key atomic step, so hot traffic never queues behind the flood, the sharded-table escape the cache chapter named.

27.7. the admission contrast

Chapter 25′s admission layer and this one refuse requests from the same stack, and keeping their meanings straight is the contract. The 429 says the client asked too fast, it is the client's budget and the client's clock, the retry is warranted and `Retry-After` says when. The 503 says the server is at capacity right now, it is the server's state and not the client's fault, and a well-behaved client backs off longer and does not count it against itself. The two layers sit together because they protect different things, fairness and capacity, and the order between them is the wiring's sentence: rate outside admission, so the cheap arithmetic refusal answers a flood before the flood is ever granted a wait, and a client hammering a saturated server collects honest 429s, not ambiguous 503s that look like the server's failure. Go's limiter chapter closed on the same pairing, and the go book's admission preview is this book's chapter 25, landed one chapter early because java's executor story forced the question first.

27.8. limits under test

The race is the one that matters: 10 concurrent requests against a burst of 5, clock frozen so no refill can accrue no matter how the scheduler interleaves, and exactly 5 answer 200 while 5 answer 429, the take's monitor making the spend monotone under any ordering:

```
@Test
void aRaceOfTwiceTheBurstAdmitsExactlyTheBurst() throws Exception {
  try (Wired w = wired(5, 16, 60 * SECOND)) {
    int racers = 10;
    CountDownLatch start = new CountDownLatch(1);
    Queue<Integer> codes = new ConcurrentLinkedQueue<>();
    List<Thread> threads = new ArrayList<>();
    for (int i = 0; i < racers; i++) {
      threads.add(Thread.ofVirtual().start(() -> {
        try {
          start.await();
          codes.add(get(w, "a").statusCode());
        } catch (Exception e) {
          throw new RuntimeException(e);
        }
      }));
    }
    start.countDown();
    for (Thread t : threads) {
      t.join();
    }
    int allowed = 0;
    for (int code : codes) {
      if (code == 200) {
        allowed++;
      } else {
        assertEquals(429, code);
      }
    }
    assertEquals(5, allowed, "the frozen clock accrues nothing, the burst is the ceiling");
  }
}
```

listing 223: twice the burst racing at one frozen instant: exactly the burst admitted, every other answer the 429 (source: java/api/test/javabook/limit/RateLimitTests.java, lines 220-251)

That is the whole difference between a limiter tested against wall-clock sleeps, which proves nothing on a loaded machine, and one tested against arithmetic, which proves the contract. Ten tests: four carrying the six boundary cells, the third-period accrual walk, the live-key eviction and its fresh burst, the independence of two keys' budgets, the request id riding the 429 envelope, the memory bound under the key flood, and the race. The wiring seats the layer outside admission, so a flooding address collects 429s without ever being granted an admission wait, and inside the access log, so every refusal is a logged line a client can be pointed at.

sources: rfc 6585, additional http status codes, section 4, the 429 definition, the MAY on Retry-After and the MUST NOT on storing 429 responses, at rfc-editor.org, accessed 2026-10-05. The Retry-After delay-seconds form from rfc 9110, section 10.2.3, same access date. `Semaphore` and monitor semantics inherited from chapter 10′s jep 444 and jep 491 verifications. The token bucket's lazy refill and burst semantics follow the go book's rate limiting chapter, book 4, chapter 26, read for the parallel contracts, and `x/time/rate`'s behavior was not re-measured here because nothing in this chapter depends on it. Verified live 2026-10-05 by the `javabook.limit` tests under the vendored junit 6.1.3 lane, 10 tests green three consecutive runs, boundaries asserted under a frozen clock on tools/jdk27/build/jdk-27.

28. observability

The spine has logs, one line per exchange with its request id, and logs answer who did what. Nothing yet answers how much and how slow, and those are the questions a pager asks at three in the morning. This chapter builds `javabook.obs` on nothing outside the jdk: the 2 series a service actually reads, one counter per route and status and one latency histogram per route, both on `LongAdder` and rendered straight into the prometheus text format, the ops listener that serves the exposition on its own port beside the api port, the module's one custom JFR event that is free the moment no recording runs, and the percentile arithmetic the next chapter's load reports quote. The go book built the same layer with slog's redaction and a hand-rolled traceparent on top, book 4, ch 27 (go 1.27, observability), and this chapter owns the same contracts where they differ on java: the filter stack owns the tick, the exchange remembers its own status, and the runtime owns the profiling surface.

28.1. the two series

A counter only ever goes up and a histogram only ever answers which bucket, and both are registry types here rather than library imports. `javabook.requests_total` is one row per route and status code, the route label taken from the pattern table itself, and `javabook_request_duration_seconds` is one per route with the sum and count a scraper needs beside the buckets. Every observation is one method:

```
public void observe(String route, int code, long elapsedNanos) {
  counters.computeIfAbsent(new CounterKey(route, code), k -> new LongAdder()).increment();
  Hist h = histograms.computeIfAbsent(route, r -> new Hist());
  h.count.increment();
  h.nanos.add(elapsedNanos);
  for (int i = 0; i < BOUNDS_NANOS.length; i++) {
    if (elapsedNanos ≤ BOUNDS_NANOS[i]) {
      // cumulative: the observation fills this bound and every one above
      h.buckets[i].increment();
    }
  }
}

/** The observation count one route's histogram holds, the tests' total. */
public long count(String route) {
```

listing 224: one observation: the counter row, the route's histogram, and the bucket walk (source: java/api/src/javabook/obs/Metrics.java, lines 77-91)

Three decisions in that walk earn their sentences. The buckets are cumulative, an observation fills every bound at or above it, which is the prometheus shape and what makes the counts monotone in the bound. The bounds live in whole nanoseconds and compare in whole nanoseconds, so no double conversion ever decides which bucket a latency lands in, the same arithmetic discipline the token bucket chapter kept for its refill. And the per-bucket cells are eager `LongAdders`, one per bound per route, a few dozen bytes each, because a lazily created cell is a check-then-act race under a route's first concurrent observations: the adversarial round measured lost increments exactly there, a fresh route's first 8 simultaneous observations could sum 7 in a bucket against 8 in the count, and the cumulative property would stay broken forever after. Zero buckets are still elided, the render simply skips cells whose sum is zero, and monotone counts keep the zeros a prefix below the smallest observation.

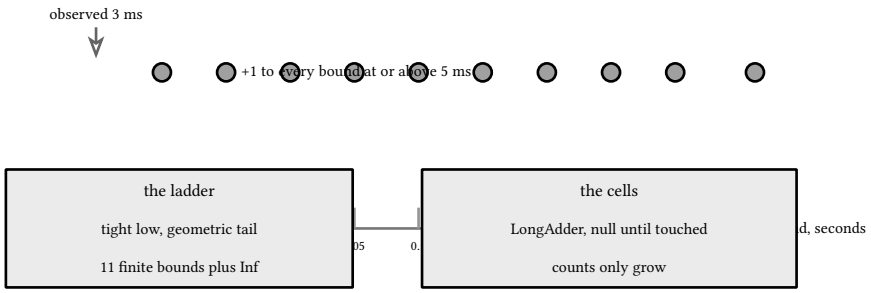


figure 53: cumulative buckets: one observation fills every bound above it

The ladder itself is a policy statement: tight at 1, 2.5, and 5 milliseconds where service latencies live, geometric through 50, 100, 250, 500 and the seconds, because a bucket ladder spends its resolution where the tail questions are asked and nowhere else.

28.2. the text exposition

The registry renders the prometheus text format directly, `#` comment lines carrying the types, series sorted by label tuple so byte-for-byte comparisons work in tests, bucket lines ascending with `+Inf` last, then the sum and the count:

```
public String render() {
  StringBuilder sb = new StringBuilder();
  sb.append("# TYPE javabook_requests_total counter\n");
  Map<CounterKey, Long> rows = new TreeMap<>((a, b) -> {
    int byRoute = a.route().compareTo(b.route());
    return byRoute != 0 ? byRoute : Integer.compare(a.code(), b.code());
  });
  counters.forEach((k, v) -> rows.put(k, v.sum()));
  for (Map.Entry<CounterKey, Long> e : rows.entrySet()) {
    sb.append("javabook_requests_total{route=\"").append(e.getKey().route())
        .append("\",code=\"").append(e.getKey().code()).append("\"} ")
        .append(e.getValue()).append('\n');
  }
  sb.append("# TYPE javabook_request_duration_seconds histogram\n");
  for (String route : histograms.keySet().stream().sorted().toList()) {
    Hist h = histograms.get(route);
    for (int i = 0; i < BOUNDS_NANOS.length; i++) {
      if (h.buckets[i].sum() > 0) {
        sb.append("javabook_request_duration_seconds_bucket{route=\"").append(route)
            .append("\",le=\"").append(seconds(BOUNDS_NANOS[i])).append("\"} ")
            .append(h.buckets[i].sum()).append('\n');
      }
    }
    sb.append("javabook_request_duration_seconds_bucket{route=\"").append(route)
        .append("\",le=\"+Inf\"} ").append(h.count.sum()).append('\n');
    sb.append("javabook_request_duration_seconds_sum{route=\"").append(route)
        .append("\"} ").append(seconds(h.nanos.sum())).append('\n');
    sb.append("javabook_request_duration_seconds_count{route=\"").append(route)
        .append("\"} ").append(h.count.sum()).append('\n');
  }
  return sb.toString();
}
```

listing 225: the whole writer: sorted rows, elided zero buckets, seconds as exact decimals (source: java/api/src/javabook/obs/Metrics.java, lines 97-128)

Two render decisions are pinned by tests. The seconds are `BigDecimal` over whole nanoseconds with trailing zeros stripped, so `903400000` nanoseconds prints `0.9034` and never a float artifact, and every histogram line carries the route as its one label because the code label already lives on the counter rows, two series that answer two different questions. The frozen vector test drives 3 observations of 3 ms, 0.4 ms, and 900 ms through a bare registry and pins the exact 17 lines, the 2 type comments and 15 series lines, including the `0.001` bucket the 0.4 ms observation fills. The elision has its own test, the divergence from the client libraries, which print every finite bucket at zero as well: a registry whose smallest observation is 3 ms renders no `0.001` or `0.0025` line at all, and a scraper reading `+Inf` and `_count` still gets the totals, so the exposition stays honest about what was observed rather than loud about what was not.

28.3. the ops port

Where the exposition serves from is a placement decision the contract forces. The metrics route is unauthenticated by convention, a scraper carries no bearer token, and chapter 23's guard denies by default, every registered route needs a policy row. So the exposition belongs on a listener the guard never wraps: `jdk.httpserver` has no second mux on one server, the answer is a second server, bound to `JBAPI_METRICS_PORT` beside the api port, serving one exact path:

```
    server.createContext("/metrics", ex -> {
      // the context matches by prefix and the path arrives decoded, so
      // the exactness check reads the raw path: /met%72ics is not the
      // contract's path whatever it decodes to
      if (!ex.getRequestURI().getRawPath().equals("/metrics")) {
        ex.sendResponseHeaders(404, HttpExchange.RSPBODY_EMPTY);
        ex.close();
        return;
      }
      byte[] body = metrics.render().getBytes(StandardCharsets.UTF_8);
      ex.getResponseHeaders().set("Content-Type", "text/plain; version=0.0.4; charset=utf-8");
      ex.sendResponseHeaders(200, body.length);
      try (OutputStream out = ex.getResponseBody()) {
        out.write(body);
      }
    });
    server.setExecutor(executor);
    server.start();
  }
```

listing 226: the ops listener: one path, the exposition's content type, prefix matches refused (source: java/api/src/javabook/obs/OpsServer.java, lines 28-46)

The context matches by prefix, `/metrics/anything` would match `/metrics`, and the handler refuses that on purpose, one exact path is the whole surface. The exactness check reads the raw path, because the decoded one would accept `/met%72ics` as the same path it spells, and the contract is bytes here, not spellings. The content type is the format's own `text/plain; version=0.0.4; charset=utf-8`, scrapers read it before the body. In production the port publishes inside the deployment only, chapter 30's compose file maps it to host 19390 beside the api's 19380, and nothing about the api listener changes: same routes, same guard, same envelope, one more process-local listener answering `docker compose`'s probe questions.

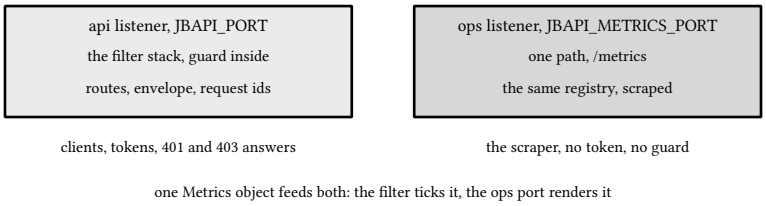


figure 54: two listeners, one process: the api port behind the guard, the ops port beside it

28.4. the filter at the edge

The layer seats outermost in the wiring, before the request id filter, so its numbers cover the whole stack and its counter ticks at the status that really went out. The route label comes from the app's own pattern resolution, the same `resolvePattern` call the authn and authz layers make, and the observation lands in a `finally`:

```
public void doFilter(HttpExchange ex, Chain chain) throws IOException {
  long start = System.nanoTime();
  String route = app == null ? UNMATCHED
      : app.resolvePattern(ex.getRequestMethod(), ex.getRequestURI().getRawPath())
          .map(p -> p.substring(p.indexOf(' ') + 1)).orElse(UNMATCHED);
  RequestEvent event = new RequestEvent();
  event.begin();
  try {
    chain.doFilter(ex);
  } finally {
    int code = ex.getResponseCode();
    observe(route, code < 100 ? 0 : code, System.nanoTime() - start);
    event.route = route;
    event.status = code < 100 ? 0 : code;
    event.commit();
  }
}
```

listing 227: the tick: pattern asked of the app, status read off the exchange, finally so nothing escapes (source: java/api/src/javabook/obs/Metrics.java, lines 136-152)

Reading the status off the exchange is the quiet gift of this server api: chapter 20's access log already relied on it, go had to wrap the writer to learn the same fact. A request the limiter refused ticks a 429 row, a path nobody routed ticks `unmatched` with a 404, and an exchange that died before any header went out reads `getResponseCode` as the documented −1 and records code 0, because a dropped connection is a fact about the service too. The pattern is stripped to its path half so the label reads `/healthz`, not `GET /healthz`, one row per route across methods that share it. The label's one blind spot is named rather than hidden: `resolvePattern` matches the method too, so a known path under the wrong method answers 405 and lands under `unmatched` beside the 404s, the code dimension separates them, and the honest reading of that row is requests the router did not serve.

28.5. the custom jfr event

Metrics answer aggregates, and the aggregate says the tail moved without saying which request moved it. The jdk's answer is JFR, the flight recorder already inside the runtime, and the module's whole commitment to it is one event class:

```
@Name("javabook.Request")
@Label("javabook request")
@Category({"javabook", "request"})
public final class RequestEvent extends Event {

  /** Jfr event classes are instantiated by the recording site, ours by the metrics filter. */
  public RequestEvent() {}

  /** The matched route pattern, the same label the metrics series carry. */
  @Label("route")
  public String route;

  /** The response status that went out, 0 when the connection died unsent. */
  @Label("status")
  public int status;
}
```

listing 228: the one custom event: name, category, 2 fields, and the duration every event carries (source: java/api/src/javabook/obs/RequestEvent.java, lines 17-32)

The filter allocates one, calls `begin` before the chain and `commit` after, and with no recording running that is the entire cost, one small allocation and 2 calls that return immediately. Nothing buffers, nothing spawns, nothing ships. A recording changes that: started from outside the process with `jcmd <pid> JFR.start` or at boot with `-XX:StartFlightRecording`, it begins consuming events, ours beside the runtime's own, and the measurement is a capture, never a promise:

```
javabook.Request {
  startTime = 06:18:53.870 (2026-10-05)
  duration = 0.853 ms
  route = "/healthz"
  status = 200
  eventThread = "" (javaThreadId = 1699, virtual)
  stackTrace = [
    javabook.obs.Metrics.doFilter(HttpExchange, Filter$Chain) line: 142
    com.sun.net.httpserver.Filter$Chain.doFilter(HttpExchange) line: 101
    sun.net.httpserver.ServerImpl$Exchange.run() line: 914
    java.util.concurrent.ThreadPerTaskExecutor$TaskRunner.run() line: 291
    java.lang.VirtualThread.run(Runnable) line: 470
  ]
}
```

listing 229: measured 2026-10-05 on this machine: jcmd started a recording on the running server, one closed loop ran through it, the dump answered (source: java/api/captures/2026-10-05-jfr.txt, lines 8-21)

1204 events for 1204 fires, 28898 bytes in all, every one naming its route, its status, its duration, and the virtual thread that served it, the stack trace arriving for free because the event was committed on that thread. Everything wider, allocation profiles, gc pauses, safepoint stalls, method sampling, the runtime's own event set records, which is why the module defines exactly one custom event and stops: the custom part is the 2 fields the business knows and the runtime does not.

NOTE: why not a flight recording always on

A recording costs throughput even at default settings, and a service that ships with one always on is spending every client's latency on diagnostics nobody is reading. The lane's stance: the event classes compile into the module, the recording starts when a human asks, chapter 29's load runs are the moments worth asking.

28.6. p99 done honestly

The percentile definition is nearest rank: sort the samples, take the value at the rank that is the ceil of p over 100 times n, no interpolation anywhere. The bucket approximation is the second half of the honesty, a scraped histogram cannot answer nearest rank, it can only say which bound the rank lands in:

```
  public static long nearestRank(long[] samples, double p) {
    if (samples.length == 0) {
      throw new IllegalArgumentException("a percentile of nothing is not a number");
    }
    long[] sorted = samples.clone();
    java.util.Arrays.sort(sorted);
    long rank = rank(p, sorted.length);
    rank = Math.max(1, Math.min(sorted.length, rank));
    return sorted[(int) (rank - 1)];
  }

  /**
   * The percentile a cumulative histogram can answer: the smallest
   * bound whose cumulative count reaches the rank, empty when the rank
   * falls past the last finite bound, because there the honest answer
   * is a bound on nothing. boundsNanos and cumulativeCounts run in
   * parallel, ascending, and total is the histogram's own count, the
   * +Inf bucket, which is the only place the samples beyond the last
```

listing 230: nearest rank: sort a copy, ceil the rank, clamp both ends (source: java/api/src/javabook/obs/Quantile.java, lines 29-46)

The rank is one arithmetic decision the adversarial round forced honest. Computing `ceil(p * n / 100.0)` in doubles can sit an ulp above the integer and ceil too far, measured concretely at p 1.1 over 3000 samples and p 4.4 over 750, both exactly integral ranks that the double product rounded up, so the rank is computed in exact decimal arithmetic on the percent value instead:

```
  private static long rank(double p, long n) {
    if (Double.isNaN(p)) {
      throw new IllegalArgumentException("a percentile of NaN is not a number");
    }
    double clamped = Math.max(0.0, Math.min(100.0, p));
    // divide to scale 0 under ceiling: an exact 33 stays 33, an exact
    // 99.9 ceils to 100, and no double product decides either
    return java.math.BigDecimal.valueOf(clamped).multiply(java.math.BigDecimal.valueOf(n))
        .divide(java.math.BigDecimal.valueOf(100), 0, java.math.RoundingMode.CEILING)
        .longValueExact();
  }
}
```

listing 231: the rank: exact decimal arithmetic, ceiling at scale zero, no double product anywhere (source: java/api/src/javabook/obs/Quantile.java, lines 72-83)

```
public static OptionalLong fromBuckets(long[] boundsNanos, long[] cumulativeCounts,
    long total, double p) {
  if (boundsNanos.length ≠ cumulativeCounts.length) {
    throw new IllegalArgumentException("bounds and counts must run in parallel");
  }
  for (int i = 1; i < boundsNanos.length; i++) {
    if (boundsNanos[i] ≤ boundsNanos[i - 1]) {
      throw new IllegalArgumentException("bounds must strictly increase");
    }
  }
  if (total ≤ 0) {
    return OptionalLong.empty();
  }
  long rank = Math.max(1, rank(p, total));
  for (int i = 0; i < boundsNanos.length; i++) {
    if (cumulativeCounts[i] ≥ rank) {
      return OptionalLong.of(boundsNanos[i]);
    }
  }
  return OptionalLong.empty();
}
```

listing 232: the bucket form: the smallest bound holding the rank, empty when only +Inf sees it (source: java/api/src/javabook/obs/Quantile.java, lines 49-70)

The boundary tests pin the contrasts that matter. p50 of 1, 1, 2, 3, 5, 8 milliseconds is 2, the third smallest, and p99 of the same 6 is 8, the largest, because a percentile past the data is a bound, not a measurement. Ninety-nine samples at 0.9 milliseconds plus one at 1.1 gives an exact p99 of 0.9 milliseconds and a bucket p99 of 1, the bound the rank lands in, and both halves of that sentence are pinned, the exact one by the vector and the bound one by the cumulative counts. And a rank that falls past the last finite bound answers empty rather than inventing the bound's value, because past the ladder only `+Inf` sees the samples and infinity is not a latency. The load chapter's reports read the exact form, its scrapes read the bound form, and the difference between the 2 numbers is the resolution the histogram traded away, stated in the report instead of hidden in a footnote.

28.7. one request fully observed

The wiring is 4 lines in the program's table. The metrics object is built after the app and used before everything else, the ops server boots after the api's `start` returns, and its close rides a shutdown hook beside the engine's:

```
Metrics metrics = new Metrics(app);
app.use(metrics);
```

listing 233: outermost: the first filter used is the outermost at request time (source: java/api/src/javabook/Main.java, lines 74-75)

```
OpsServer ops = new OpsServer(metrics, intEnv(METRICS_PORT_ENV, 9090));
Runtime.getRuntime().addShutdownHook(new Thread(ops::close));
System.out.println("javabook api listening on port " + port + ", metrics on port "
    + ops.port() + ", data at "
```

listing 234: the ops listener boots beside the api and closes with the process (source: java/api/src/javabook/Main.java, lines 105-108)

One request through the whole surface, read back: the access log carries its line with the request id, the counter row for its route and code ticks, its latency fills the histogram's tail cells, its jfr event sits in the buffer waiting for a recording, and the envelope it may have been answered with carries the same request id the log line did. The observed surface is exactly what the wiring wires, pinned by the tests: the filter test drives a real app and asserts the label and the counts, the concurrency tests assert the bucket cells agree with the count under contention and

on a fresh route's first simultaneous observations, the unsent test cuts a connection mid-exchange and asserts the code 0 row, the ops test scrapes a real port and asserts the content type and the one path, and the jfr test starts a real `Recording` in process and reads the events back out of the dump file.

The lane's boundary is stated as plainly: distributed tracing is not built here. That same go chapter parsed and rendered the W3C traceparent format from scratch, spans in context and a tree printer, and the java lane leaves the surface to it because this service has one hop and no tree to print. The moment a second hop exists, the same seam takes it, one more filter reading one header and one more field on the request's jfr event.

sources: the prometheus exposition format documentation, the content type line, the histogram bucket semantics and the optionality of HELP lines, at prometheus.io/docs/instrumenting/exposition_formats, accessed 2026-10-05. The jdk.jfr javadoc for Event, Name, Label, Category, and Recording, and the jcmd and jfr tool pages, at docs.oracle.com/en/java/javase/27, same access date, read against tools/jdk27/build/jdk-27 build 27+35-2325. Verified live 2026-10-05 by the javabook.obs tests, 17 green three consecutive runs under the vendored junit 6.1.3 lane: the frozen 17-line exposition vector, the bucket cells agreeing with the count under 8-thread contention at 8000 observations and on a fresh route's first 8 simultaneous observations, the exact-decimal rank against the vectors where a double product ceils wrong, the filter over a real app including the unmatched 404 and the cut connection's code 0, the ops port over a real socket including the refused prefix, and the recording round trip. The jfr capture is books/java/api/captures/2026-10-05-jfr.txt, dated, committed.

29. load and p99

A service whose p99 is a guess is a service shipped on faith. This chapter builds the tool that replaces the guess, `javabook.load`, and then points it at the real server: the closed loop and the open loop over 3 injected seams, the report with its honest percentiles read out of chapter 28's arithmetic, and the dated captures this machine produced, ladder, cliff, and budget, committed beside the code. The go book built the same harness around a `RoundTripper` seam, book 4, ch 28 (go 1.27, load and p99), and the java lane builds it around a virtual thread per worker and a blocking `send`, which is the same design in the other idiom: park the thread, never pool it.

29.1. the two loops

The closed loop holds concurrency constant: `workers` virtual threads, each firing one request at a time and starting the next only after the previous response lands, so offered load is whatever the server can keep up with and never exceeds the worker count. The open loop holds arrival rate constant: one pacer schedules arrival `i` at `i` periods past the start and fires it on its own virtual thread whether or not anything has come back, so when the server slows, requests pile up exactly the way they do in production, and the cliff becomes visible. Little's law, in flight equals arrival rate times service time, predicts the pile: an open loop at fixed arrival rate turns rising latency directly into a rising in-flight count:

```
  public Result closed(int workers, int warmup, int measured) throws InterruptedException {
    Sink sink = new Sink();
    List<Thread> ts = new ArrayList<>(workers);
    for (int w = 0; w < workers; w++) {
      ts.add(Thread.ofVirtual().unstarted(() -> round(sink, warmup, measured)));
    }
    for (Thread t : ts) {
      t.start();
    }
    for (Thread t : ts) {
      t.join();
    }
    return sink.result(warmup * workers);
  }
```

listing 235: the closed loop: one virtual thread per worker, joined, warmup lands nowhere in the sink (source: java/api/src/javabook/load/Harness.java, lines 73-86)

```
public Result open(long periodNanos, int arrivals, int warmup)
    throws InterruptedException {
  Sink sink = new Sink();
  warm(sink, warmup);
  Thread pacer = Thread.ofVirtual().start(() -> {
    long start = clock.getAsLong();
    for (int i = 0; i < arrivals; i++) {
      long deadline = start + i * periodNanos;
      while (clock.getAsLong() < deadline) {
        try {
          sleeper.sleep(deadline - clock.getAsLong());
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
          return;
        }
      }
      Thread.ofVirtual().start(() -> round(sink, 0, 1));
    }
  });
  pacer.join();
  // the pacer's return means every arrival was dispatched; the fires
  // it spawned still need joining, which their shared count below waits on
  sink.awaitArrivals(arrivals);
  return sink.result(warmup);
}
```

listing 236: the open loop: warmup establishes the pool, the pacer fires on schedule regardless of in flight (source: java/api/src/javabook/load/Harness.java, lines 94-118)

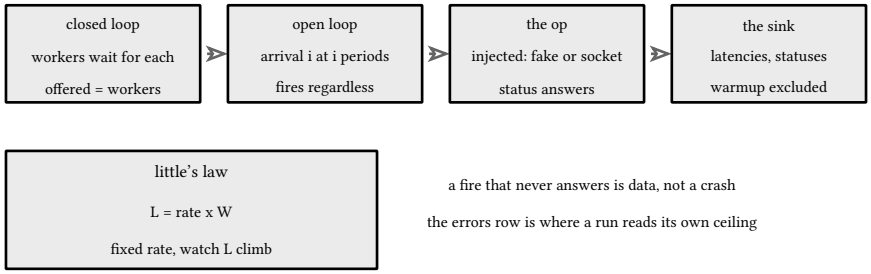


figure 55: workers, pacer, server, sink: one harness, two loops

Both loops share the 3 seams: the op, so tests drive deterministic responses with no socket, the clock, so latency is arithmetic on ticks, and the sleeper, so the open loop's pacing is a function the test steps instead of a wall it waits on. A fire that throws is not a crash, the round records it as an error and keeps offering, because a load tool that dies on the first refused connection loses exactly the data the cliff produces. Only a run where nothing ever answered exits red.

29.2. tested without sleeping

The harness tests run under a frozen clock and a fake sleeper, the same discipline every time-dependent test in this spine kept: the closed loop with 1 worker, 3 warmup, 5 measured, and an op that costs 100 ticks answers with 8 fires, 5 latencies of exactly 100, and a warmup count of 3, deterministic under any scheduler because the clock only moves when the fakes move it:

```
@Test
void closedLoopExcludesWarmupAndMeasuresOnlyTheRealRounds() throws Exception {
  Nanos clock = new Nanos();
  FakeOp op = new FakeOp(clock, 100);
  Harness.Result r = harness(clock, op).closed(1, 3, 5);
  assertEquals(8, op.fires.get(), "warmup runs first and pays the dial");
  assertEquals(5, r.measured());
  assertEquals(3, r.warmupFired());
  assertEquals(500, r.windowNanos(),
      "the window spans the measured rounds only, warmup cannot inflate the rate");
  for (long lat : r.latencies()) {
    assertEquals(100, lat, "each measured latency is the op's own cost under the frozen clock");
  }
}
```

listing 237: warmup excluded and latencies exact, all under arithmetic (source: java/api/test/javabook/load/LoadTests.java, lines 60-73)

The open loop's own test pins the property the loop exists for: with one arrival per 10 nanos and an op costing 50, all 4 arrivals fire, because a slow server never holds back an open loop's schedule. The interleaved latencies are asserted only from below, at least the op's cost, since 2 workers sharing one frozen clock can stretch a window but never shrink one.

29.3. the report

One run folds into one report: the mode line naming the loop's parameters, the elapsed window measured from the first measured start to the last measured end so warmup cannot inflate the rate, the status histogram, the protocol the responses actually rode, and the 4 percentiles by nearest rank, every column naming its unit:

```
public static String render(String modeLine, Harness.Result r, String protocol) {
  StringBuilder sb = new StringBuilder();
  row(sb, "mode", modeLine);
  if (r.windowNanos() > 0) {
    long rps = Math.round(r.measured() * 1e9 / (double) r.windowNanos());
    row(sb, "elapsed", seconds(r.windowNanos()) + " s, " + rps + " rps");
  } else {
    row(sb, "elapsed", "no measured window");
  }
  StringBuilder statuses = new StringBuilder();
  r.statuses().forEach((code, count) -> {
    if (statuses.length() > 0) {
      statuses.append(' ');
    }
    statuses.append(code).append('=').append(count);
  });
  row(sb, "status", statuses.length() == 0 ? "none" : statuses.toString());
  if (r.errors() > 0) {
    row(sb, "errors", Long.toString(r.errors()));
  }
  row(sb, "protocol", protocol == null ? "none" : protocol);
  percentile(sb, "p50", r, 50);
  percentile(sb, "p90", r, 90);
  percentile(sb, "p99", r, 99);
  percentile(sb, "p99.9", r, 99.9);
  return sb.toString();
}
```

listing 238: the whole report: units in every header, nearest rank underneath, errors only when they happened (source: java/api/src/javabook/load/Report.java, lines 19-45)

The golden test pins the bytes from a fixed vector, 6 latencies of 1, 1, 2, 3, 5, and 8 milliseconds over an 8 millisecond window: 750 rps, p50 of 2, and p90, p99, and p99.9 all clamped to 8, the largest, because a percentile past the data is a bound. The protocol column is the report's honesty lever, and the client earns it a sentence: the harness's one real op builds its `HttpClient` for HTTP/2, the kernel's `jdk.httpserver` listener is HTTP/1.1 and refuses the upgrade, and the client settles, so the report prints `HTTP_1_1` from `response.version()` rather than pretending the negotiation won:

```
  HttpClient client = HttpClient.newBuilder()
      .version(HttpClient.Version.HTTP_2)
      .connectTimeout(Duration.ofSeconds(10))
      .build();
  AtomicReference<String> protocol = new AtomicReference<>();
  Harness harness = new Harness(() -> {
    HttpResponse<Void> res = client.send(
        HttpRequest.newBuilder(URI.create(url)).GET()
            .timeout(Duration.ofSeconds(30)).build(),
        HttpResponse.BodyHandlers.discarding());
    protocol.compareAndSet(null, res.version().name());
    return res.statusCode();
  }, System::nanoTime, Load::park);
```

listing 239: one shared client built for HTTP/2, connect and request both timed, the version it actually got (source: java/api/src/javabook/load/Load.java, lines 34-46)

A blocking `send` on a virtual thread parks it, chapter 10's whole contract, so 128 workers cost 128 parked stacks and no pool tuning, and the one client shares its connections across all of them.

29.4. the ladder, measured

The capture is dated, machine-named, and committed at `books/java/api/captures/2026-10-05-load.txt`. The server is the real `Main` on this machine's loopback, its budget raised through the wiring's own `JBAPI_RATE_BURST` knob because the measurement's subject is the stack, not the limiter, and the production budget gets its own capture below:

```
mode            closed, workers 1, warmup 200, measured 5000
elapsed         1.4890235 s, 3358 rps
status          200=5000
protocol        HTTP_1_1
p50             0.243 ms
p90             0.5155 ms
p99             0.9285 ms
p99.9           1.8455 ms
mode            closed, workers 8, warmup 100, measured 5000
elapsed         1.6238383 s, 24633 rps
status          200=40000
protocol        HTTP_1_1
p50             0.2138 ms
p90             0.5841 ms
p99             1.8901 ms
p99.9           5.5165 ms
mode            closed, workers 32, warmup 100, measured 5000
elapsed         3.1180764 s, 51314 rps
status          200=160000
protocol        HTTP_1_1
p50             0.4783 ms
p90             0.9391 ms
p99             1.8773 ms
p99.9           20.5361 ms
mode            closed, workers 128, warmup 100, measured 2000
elapsed         4.8311648 s, 52989 rps
status          200=256000
protocol        HTTP_1_1
p50             2.035 ms
p90             3.1804 ms
p99             5.028 ms
p99.9           95.2404 ms
mode            open, rate 5000/s, arrivals 10000, warmup 100
elapsed         2.0001607 s, 5000 rps
status          200=10000
protocol        HTTP_1_1
p50             0.3873 ms
p90             0.6687 ms
p99             1.2939 ms
p99.9           4.0482 ms
```

listing 240: the ladder: 4 closed loops and one open loop, this machine, 2026-10-05 (source: java/api/captures/2026-10-05-load.txt, lines 10-49)

Reading it: 1 worker answers 3358 rps, latency-bound on one connection at a p50 of 0.243 ms. 8 workers reach 24633, 32 reach 51314, and 128 reach 52989, the plateau that says the server is saturated and more workers only queue. The tail tells the same story from the other end: p50 holds 0.21 to 0.48 ms through 32 workers and only then moves, 2.035 ms at 128, while p99.9 climbs 1.8, 5.5, 20.5, then 95.2 ms, queueing is what grows, and the percentile ladder is where it shows. The open loop at 5000 offered per second achieves exactly 5000 with all 10000 arrivals

served, an unremarkable number that is the point: below capacity, the open loop confirms the closed loop's story, and only above capacity do the 2 designs differ.

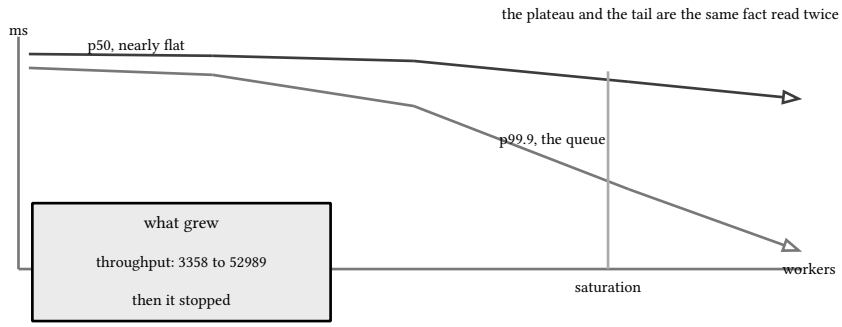


figure 56: the ladder's p50 and p99.9 against offered load, the tail is the story

29.5. the cliff

Offered far past capacity, the same loop finds a ceiling, and on one machine the first ceiling it meets may not be the server's:

```
mode            open, rate 80000/s, arrivals 40000, warmup 200
elapsed         9.3078911 s, 935 rps
status          200=8704
errors          31296
protocol        HTTP_1_1
p50             7545.5565 ms
p90             8733.1242 ms
p99             9071.1111 ms
p99.9           9177.8575 ms
```

listing 241: 80000 offered per second: the pile, the errors, and the client machine's own port ceiling (source: java/api/captures/2026-10-05-load.txt, lines 50-58)

31296 of 40000 arrivals never answered, `ConnectException` on the loopback because the client and server share one machine's ephemeral port space, and the 8704 that landed queued behind the pile, p50 at 7.5 seconds. The honest reading is stated rather than laundered: this run measured where the loopback fills, not where the service bends, and a real capacity number needs the client on its own machine. What the run does prove is the tool's own contract, the errors row counts every refused fire, the percentiles cover what landed, and the report survives its ceiling instead of dying on the first exception, which is the difference between a measurement and a crash. The exit policy is stated with the same plainness: only a run where nothing landed exits red, so this cliff run exits 0 by design, partial failure is data and the errors row is the signal, and a caller who wants a stricter gate reads the row instead of the code.

29.6. the production budget under load

The ladder raised the budget through `JBAPI_RATE_BURST`, and the wiring's defaults are unchanged from chapter 27, burst 30 with one refill per second per address. Against those defaults the same closed loop meets the layer that chapter built:

```
mode            closed, workers 8, warmup 1, measured 200
elapsed         0.2853373 s, 5607 rps
status          200=22 429=1578
protocol        HTTP_1_1
p50             1.062 ms
p90             2.6051 ms
p99             4.6858 ms
p99.9           8.6735 ms
```

listing 242: 8 workers offer 1600, the budget answers 30, the exposition counts both (source: java/api/captures/2026-10-05-budget.txt, lines 5-12)

22 of 200 measured requests per worker answered 200, 1578 answered 429, and the exposition off the ops port reads 30 and 1578, the warmup 8 included, because chapter 28′s filter seats outside the limiter and ticks the refusals at the status that really went out. The run's pooled p99, 4.7 ms across both statuses, is the limiter's design claim measured: refusing is cheap, the flood costs the handler nothing, and the budget is spent exactly where the token bucket said it would be. The report pools its percentiles over every measured sample, 200s and 429s together, and says so plainly rather than attributing the number to one status.

NOTE: the knob, not a bypass

The measurement boot raises the budget through the same environment variable the wiring reads, `JBAPI_RATE_BURST`, whose default stays 30. Nothing about the deployed service changes, and the capture names the knob and its value in its header, so the number can never be mistaken for the default's behavior.

29.7. what the run leaves behind

Three artifacts outlive the session, and they are the point of the chapter. The captures, dated and committed, so the next run on the next machine has a baseline to diff against and no one re-derives today's numbers from memory. The exposition, whose counter row and histogram cells carry the same run the report printed, one registry observed from 2 sides. And the tool itself, 6 tests green, which is what makes next quarter's number comparable to this one: a load report nobody can rebuild byte for byte is a slide, not a measurement.

sources: the java.net.http javadoc for HttpClient version negotiation, the HTTP/2 upgrade attempt over clear-text and the fallback to HTTP/1.1, and HttpResponse.version, at docs.oracle.com/en/java/javase/27, accessed 2026-10-05, verified live by the protocol column reading HTTP_1_1 against the kernel's listener on this machine. Little's law as stated across the corpus, book 4, ch 11 (go 1.27, profiling: pprof, the tracer, benchstat, p99) owns the profiling-side treatment. Verified live 2026-10-05 by the javabook.load tests, 6 green three consecutive runs under the vendored junit 6.1.3 lane on tools/jdk27/build/jdk-27, and by the 3 committed captures quoted above, measured on a 12th gen i7-12700F under windows/amd64 with client and server on one loopback.

30. ship it

The spine ends as a directory of tests unless it also ships. This chapter turns the module into an artifact and a lane: the jlink recipe that derives a custom runtime from the pinned jdk, the multi-stage image that repeats the recipe on a pinned temurin 27 base, the compose file that brings it up healthy on host ports 19380 and 19390, and the make gate that replays the wire contract over that stack. Everything leans on what the spine already built: the health route chapter 19 owns, the strong etags chapter 21 computes, the ops listener chapter 28 binds, the health probe class the load wave added. The go book shipped the same service shape one idiom over, book 4, ch 29 (go 1.27, ship it), and its chapter carries the platform envelopes, helm, terraform, ci, as taught copies, which stay there: this chapter owns the narrow band between a green module and a running container.

30.1. what a java service ships as

A jar needs a java, and "install a jre" is a deployment dependency the artifact can absorb. Chapter 15 built the machinery, jlink assembles a runtime image from a module list, and the recipe here is one script with the module list derived rather than hand-written:

```
# 2. derive the module list from the compiled module
$deps = & (Join-Path $Jdk "bin" "jdeps.exe") --module-path "build/image-classes" `
  --add-modules javabook --print-module-deps --ignore-missing-deps build/image-classes
if ($LASTEXITCODE -ne 0) { exit 1 }
$modules = "$deps,javabook"
Write-Host "jlink-image: modules $modules"

# 3. link: stripped, compressed, one launcher named after the module
$out = "build/image"
if (Test-Path $out) { Remove-Item -Recurse -Force $out }
& (Join-Path $Jdk "bin" "jlink.exe") `
  --module-path "build/image-classes;$(Join-Path $Jdk 'jmods')" `
  --add-modules $modules `
  --strip-debug --no-header-files --no-man-pages --compress zip-6 `
  --launcher javabook=javabook/javabook.Main `
  --output $out
```

listing 243: jdeps answers what the module reads, jlink links exactly that plus the module itself (source: java/api/deploy/jlink-image.ps1, lines 31-46)

The derived list is `java.base, java.net.http, jdk.httpserver, jdk.jfr, javabook`: the kernel's server module, the load client's http module, the observability event's recorder module, and the module itself, nothing more. The derivation is the whole point, a new `requires` in `module-info.java` rides the next build, while a hand-written list silently ships a runtime that fails at startup with a module not found. The recipe compiles fresh, links with stripped debug and compressed modules, names one launcher after the module, and states its own size, measured 32.8 MB of files on this machine against the 370 MB of the pinned jdk 27 home it links from. The image carries `jfr.exe` because `jdk.jfr` brings its tool, and carries no `jcmd`, which lives in `jdk.jcmd` and is not required, so a recording on the shipped thing starts from outside the container, the operator's machine, not from inside it.

30.2. the image, two stages

The Dockerfile repeats the same recipe inside a pinned base, and the runtime stage carries only the linked image:

```
# multi-stage keeps the jdk out of the shipped image: the builder
# compiles the module and links a custom runtime for it, the runtime
# stage copies that image and nothing else. the module list is derived
# with jdeps, so module-info.java stays the single source of what
# ships. base tags resolved 2026-10-05: eclipse-temurin:27_35-jdk
# (manifest-list digest sha256:2771efbbc159b89dc38b82ebe01312fd1b5f226071ff715ca9edd5411a389761),
# ubuntu:noble (sha256:534baea6a22c03a63003dbc8dbe78fe34bc0d7e595d9a9dc9834884ff530eb55),
# the temurin build 27+35 matching the repo's pinned jdk.
FROM eclipse-temurin:27_35-jdk AS build
WORKDIR /src
COPY src /src/src
RUN mkdir -p /out /srcs \
    && find src -name '*.java' > /srcs/list \
    && javac -Xlint:all -Werror -d /out @/srcs/list \
    && MODULES="$(jdeps --module-path /out --add-modules javabook \
        --print-module-deps --ignore-missing-deps /out),javabook" \
    && echo "linking modules: $MODULES" \
    && jlink --module-path "/out:$JAVA_HOME/jmods" \
        --add-modules "$MODULES" \
        --strip-debug --no-header-files --no-man-pages --compress zip-6 \
        --launcher javabook=javabook/javabook.Main \
        --output /image \
    && mkdir /data && chown 65532:65532 /data

# the runtime: ubuntu noble, the glibc the temurin builder links
# against, carrying the linked image and an empty /data the non-root
# user owns so the store volume mounts writable on first run.
FROM ubuntu:noble
ARG VERSION=dev
ENV JBAPI_VERSION=${VERSION} JBAPI_DATA=/data
COPY --from=build --chown=65532:65532 /image /image
COPY --from=build --chown=65532:65532 /data /data
USER 65532:65532
EXPOSE 8080 9090
ENTRYPOINT ["/image/bin/javabook"]
```

listing 244: the whole image: temurin builder, jlink inside it, noble runtime, non-root (source: java/api/Dockerfile, lines 1-36)

Four decisions in the file earn their sentences. The builder is `eclipse-temurin:27_35-jdk`, resolved 2026-10-05 at manifest digest `sha256:2771efbbc159b89dc38b82ebe01312fd1b5f226071ff715ca9edd5411a389761`, the temurin build 27+35 matching the repo's pinned jdk, so the image links against the same bits the lane tests. The runtime is `ubuntu:noble`, digest `sha256:534baea6a22c03a63003dbc8dbe78fe34bc0d7e595d9a9dc9834884ff530eb55`, because the temurin builder links against noble's glibc and a jlink image is binaries, not a portable format, a bookworm base would be smaller and wrong. The user is numeric, 65532, the same convention the go lane's image set, so the platform and the image agree on who runs the process without leaning on the base's user table. And `/data` is created and chowned in the builder then copied with its owner, because a named volume inherits the image content's ownership on first mount and a root-owned directory would deny the container user its store. The built image measured 188 MB on this machine, 54.7 MB compressed, the base's share included.

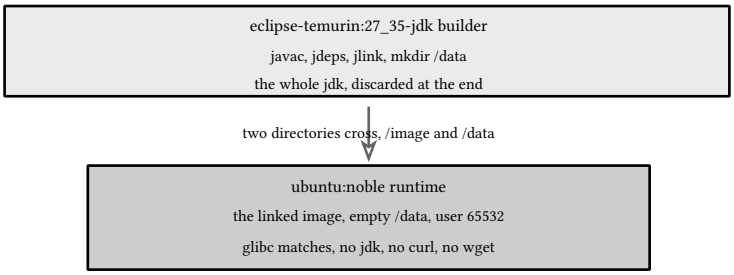


figure 57: two stages, two directories cross: the builder dies with the build, noble runs them

The build context carries `src/` alone because a `.dockerignore` trims the rest, and the rest matters here more than it did for the go lane: `build/` can hold a whole linked runtime image from the recipe above, 33 MB of context the builder would upload only to ignore, and the captures, tests, and integration suite ride none of it into the image. The go chapter chose no ignore file and stated the trade, this one chooses the file because its build directory is not merely sources.

30.3. compose, one command

The compose file is one service and its volume, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds the image, creates the container, starts it, and blocks until the healthcheck passes, so a green up means the image linked, the wal engine opened its directory, and both listeners answer. `down -v` removes the volume in the same breath, so every lane run starts from store zero:

```
    environment:
      JBAPI_SECRET: dev-only-not-a-secret
      JBAPI_DATA: /data
      JBAPI_PORT: "19380"
      # the ops listener: the exposition inside the container, on its
      # own port, published for the lane's assertions, never behind
      # the api listener's authz guard
      JBAPI_METRICS_PORT: "19390"
      # the lane raises the shared per-address budget so the replay is
      # not the limiter's subject, chapter 27's tests hold that
      # contract; the drain test below still meets the real 429 shape
      JBAPI_RATE_BURST: "100"
    ports:
      - "19380:19380"
      - "19390:19390"
    volumes:
      - data:/data
    healthcheck:
      # the image carries no curl and no wget, and the probe that
      # knows the wiring is the module's own class: the runtime's
      # java running javabook.Health against the app's own port
      test: ["CMD", "/image/bin/java", "-m", "javabook/javabook.Health"]
```

listing 245: the whole system: the env the wiring reads, both ports published, the probe the image runs (source: java/api/compose.yml, lines 22-43)

The healthcheck is the load wave's `Health` class, the runtime's own `java` running the module's probe against the app's own port. The image carries no curl and no wget, and installing either would grow the runtime for a diagnostic the module already knows how to run, which is the same trade the go lane made when it gave its binary a `-health` flag. The cadence is 5 seconds, measured against a probe that boots a jvm in about 0.9 s inside the container: a 2 second cadence would burn a third of a core just asking. The environment block carries exactly what construction demands: the secret present so authn signs with a known key, the data directory on the volume, both

ports pinned to the lane's 19380 and 19390, and the shared budget raised to 100 so the replay behind the healthcheck is not the limiter's subject, the same knob the load chapter named, its default untouched. The migration discipline of the store chapter holds unchanged, `docker compose up -d` is the only command that touches the data, because the wal engine applies its own log at startup.

30.4. the docker lane as a gate

The lane is one make target, and its shape is the infrastructure book's two-gate rule, book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) owns the compose depth and book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) the pid 1 and signal story. The plain module lane never needs a daemon, and the docker lane never runs silently, a missing daemon is a loud red with the instruction to start it, never a skip:

```
verify-javaapi-docker:
  @docker info >/dev/null 2>&1 || { echo "docker daemon not reachable." >&2; exit 1; }
  @cd books/java/api && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    <compile src and integration under the pinned jdk, then the junit console
     over --select-package javabook.it with JBAPI_URL and JBAPI_METRICS_URL set>
```

fragment, illustrative, not executed

The suite lives outside the plain junit scan path, `integration/` instead of `test/`, so `--scan-classpath` in the ordinary lane never sees it and only the docker target compiles and runs it. Its tests replay the contract over the real listener with a real clock and real uuids, the statuses, the header relations, and the byte-identical guarantees that must survive all three. The etag check is the sharpest one, recomputed over the wire bytes:

```
private static void wantStrongEtag(Result r) throws Exception {
  String etag = r.header("ETag");
  assertNotNull(etag, "no ETag header: " + r.text());
  assertTrue(etag.startsWith("\"") && etag.endsWith("\""), "the etag must be strong: " + etag);
  byte[] sum = MessageDigest.getInstance("SHA-256").digest(r.body());
  assertEquals(HexFormat.of().formatHex(sum), etag.substring(1, etag.length() - 1),
      "the etag must be the sha256 of the exact body bytes");
}
```

listing 246: the strong etag rule recomputed over live bytes, not frozen ones (source: java/api/integration/javabook/it/IntegrationTests.java, lines 114-121)

Five tests cover the scenarios the contract names. Health and routing pins both probes and the two routing envelopes, including the 405 carrying the not_found code plus an Allow header. Register and login walks the 201 with Location and the sha256 etag, the 409 the same email answers, which is this contract's replay answer where the go contract's idempotency key replayed the stored snapshot, the login pair with the two 401 shapes that must not differ enough to enumerate accounts, and the owner's read with its etag. The patch duel releases 2 concurrent patches on one If-Match and demands exactly one 200 and one 412, then a 304 off the fresh etag and a stale 412:

```
    String v1 = get.header("ETag");

    CountDownLatch start = new CountDownLatch(1);
    java.util.concurrent.ConcurrentLinkedQueue<Result> out = new
java.util.concurrent.ConcurrentLinkedQueue<>();
    List<Thread> patchers = new java.util.ArrayList<>();
    for (String name : new String[] {"a", "b"}) {
      Map<String, String> h = new LinkedHashMap<>();
      h.put("Authorization", auth.get("Authorization"));
      h.put("Content-Type", "application/json");
      h.put("If-Match", v1);
      Thread t = Thread.ofVirtual().unstarted(() -> {
        try {
          start.await();
          out.add(call("PATCH", "/api/users/" + ops[0], h,
              "{\"display_name\":\"ops renamed " + name + "\"}"));
        } catch (Exception e) {
          throw new RuntimeException(e);
        }
      });
      patchers.add(t);
      t.start();
    }
    start.countDown();
```

listing 247: the duel over the wire: one latch, 2 virtual-thread patchers, the collector concurrent (source: java/api/integration/javabook/it/IntegrationTests.java, lines 210-232)

The remaining 2 are the exposition, read off the ops port the wired service binds it to, both series present with the route label the pattern table produces, and the budget drain, ordered last because it spends the shared per-address budget down to zero on purpose, the 429's envelope and its Retry-After asserted at the status the limiter really wrote.

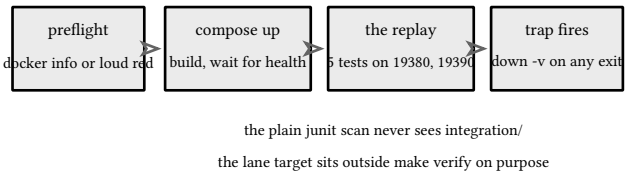


figure 58: the gate: preflight, compose, replay, teardown, no skips

30.5. the shipped checklist

The ship lane ends where it started, with the property each spine chapter contributed, every row a behavior the lane or the module gate observes and names where it is proven:

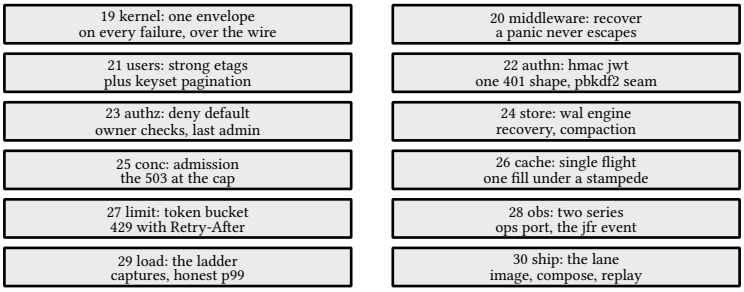


figure 59: the readiness table: twelve chapters, twelve properties, all proven

The chapter also declares what it does not teach again. Docker's storage model, the container runtime, and networking are the infrastructure book's chapters, the compose internals and the one-click philosophy are book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems), and the migration discipline the wal log rides is the store chapter's own. The cloud side stays with the go book's ship chapter, whose helm chart and terraform slice are the same service's platform envelopes taught line by line, and a java copy of them would be the same files wearing a new name. What this chapter owns is the band between a green module and a running service, and the band is small enough to read in a sitting and boring enough to trust.

sources: docker docs on multi-stage builds and `docker compose up --wait` semantics, at docs.docker.com, accessed 2026-10-05. The eclipse-temurin and ubuntu tag listings and digests read from hub.docker.com's registry api on 2026-10-05, recorded in the Dockerfile's header comment. The jlink and jdeps tool pages at docs.oracle.com/en/java/javase/27, same access date, run against tools/jdk27/build/jdk-27, build 27+35-2325. Verified live 2026-10-05: the jlink recipe run end to end on this machine (32.8 MB of files, the boot, the probe, and the exposition all checked against the linked image before the Dockerfile repeated it), and `make verify-javaapi-docker` green 3 consecutive runs, the compose stack healthy on 19380 and 19390, 5 integration tests green each run, the down -v trap observed firing on both the passing and a deliberately broken early exit.

31. the test suite

The service spine ends with the service tested, and this chapter turns the suite around and examines it as its own artifact. Every chapter from 19 on applied one technique to its own component, frozen clocks, injected seams, channel choreography, barrier duels, and what is left is the discipline the whole suite runs on: the inventory matched to the tree, the one lane gate and its legs, junit without a build tool on the stdlib-exclusive ruling, the seams that make time and chance deterministic, and the adversarial history each wave recorded, including this wave's own. The go book closed its spine the same way over six fuzz targets and a strict recording double, book 4, ch 30 (go 1.27, the test suite), and the java lane's honest inventory differs in one row: it holds no standing property engine targets yet, and says so instead of borrowing the claim.

31.1. what the suite holds

The counts are taken from the tree, not from memory, and they are the commit's counts. The module's junit suite holds 158 tests in 17 files across 12 packages, and the docker lane holds 5 more outside the plain scan path:

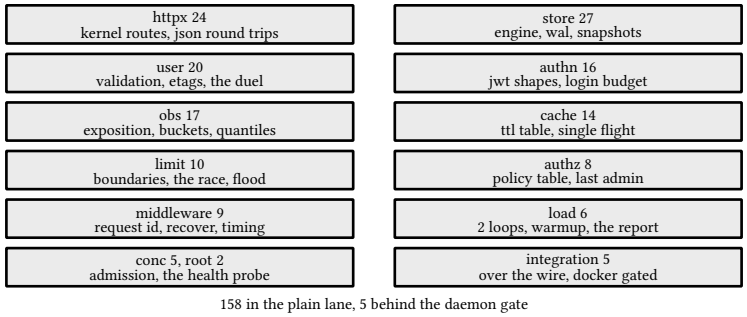


figure 60: the inventory by package, each row the tests that pin it

Beside the module suite stands the samples contract, the manual arc's own gate: 58 runnable classes under `samples/src`, one public class per file, each embedding the `ok` helper and printing its own dated checks, 493 of them across chapters 1 through 18 plus chapter 25's 12 more. The two contracts answer different questions and share one runner: a sample teaches a language fact and proves it on stdout, a junit test pins a service behavior and fails loudly in a report, and neither substitutes for the other.

31.2. the one lane gate

Everything runs through `tools/verify-java.ps1`, and its legs are the whole anatomy. The vendored junit console jar rehashes against its sha before anything runs, so a corrupted vendor is a loud red naming the re-vendor path, never a silent swap. The samples runner walks every chapter directory and fails on any class that prints no check or exits non-zero. The module lanes compile src then test under `-Xlint:all -Werror` on the pinned jdk and run the console over both class directories, the api module first, then the capstone module and its web leg as those waves landed them. The docker lane is the one leg that lives outside the gate, `make verify-javaapi-docker`, its absent daemon a loud red, its compose stack brought up healthy and torn down with volumes on every exit:

```
& $java -jar $jar execute --scan-classpath --fail-if-no-tests `
  --disable-ansi-colors --disable-banner `
  -cp "$root/build/classes;$root/build/test-classes;$jar"
```

fragment, illustrative, not executed

Three facts about that invocation earn their sentences. The scan discovers tests by classpath, so the `*Tests` naming convention is load bearing, a helper class named like a test would run and fail on its missing fixtures. The `--fail-`

`if-no-tests` flag turns an empty scan into a red, so a mistyped package in a future leg can never pass by testing nothing. And the semicolons are the windows classpath separator, the reason the script resolves the pinned jdk's own binaries and never the machine's.

31.3. junit without a build tool

The spine's standing rule is stdlib-exclusive, no gradle, no maven, no spring, and the test runtime is the same ruling carried to its conclusion: one vendored console jar, `junit-platform-console-standalone` 6.1.3, sha-gated in the vendor manifest, carrying jupiter and its params. Nothing resolves from the network, nothing generates sources, and the build is two `javac` invocations an agent can run and re-run in seconds. The cost is stated plainly: no dependency management means every third-party want becomes a vendoring decision, and the spine made exactly one, the test console, because a language needs a test runner the way a book needs a printer. The 6.x console is subcommand based, `execute` then the selectors, which the older documentation's bare flags would silently misrun.

31.4. the deterministic seams

Every boundary the suite proves is injected, never waited for. The clocks freeze, `Nanos` in the limit, load, and observability tests, `StepClock` in the store and user tests, and a refill boundary or an expiry is arithmetic on ticks under any scheduler interleaving. The ids are suppliers, so a cursor's wire bytes are reproducible. The hashers are functions, so pbkdf2′s cost never runs inside a table test. The limiter's keyer is a function, so one app serves many independent budgets keyed on a header. The load harness's op is a function, so the closed loop's latencies are exactly the fake's cost and the open loop's schedule is stepped, not slept. That is what makes the race tests meaningful: 10 concurrent requests against a frozen burst of 5 answering exactly 5 and 429 is a contract, not a timing observation.

A handful of sleeps stand in the tree, and each is a wait on a fact or the scenario itself rather than a proof of a boundary: one poll loop awaits a log line under a deadline, and the choreography sleeps hold a single-flight window open, or let one request arrive inside another's admission wait, the kernel stop test's in-flight request being a handler that takes 300 ms on purpose. A boundary proven by sleeping proves nothing on a loaded machine, and no boundary in this suite is.

The freshest seam tests are the adversarial round's own regressions, and one earns its place as the exemplar, the histogram's first-touch race, closed by eager cells and pinned by assertions on the rendered sums:

```
  @Test
  void theFirstConcurrentObservationsNeverLoseABucketIncrement() throws Exception {
    // the regression the race once allowed: a lazily created cell lost
    // increments when a fresh route's first observations collided
    int threads = 8;
    Metrics m = new Metrics(null);
    CountDownLatch go = new CountDownLatch(1);
    Thread[] ts = new Thread[threads];
    for (int i = 0; i < threads; i++) {
      ts[i] = Thread.ofPlatform().unstarted(() -> {
        try {
          go.await();
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        }
        m.observe("/fresh", 200, MS);
      });
      ts[i].start();
    }
    go.countDown();
    for (Thread t : ts) {
      t.join();
    }
    String out = m.render();
    assertTrue(out.contains(
        "javabook_request_duration_seconds_bucket{route=\"/fresh\",le=\"0.001\"} " + threads + "\n"),
        "8 first observations, all 8 in the first bucket, none lost to the cell's creation");
    assertTrue(out.contains(
```

listing 248: the regression the blind round forced: 8 simultaneous first observations, all 8 counted in the bucket and the total (source: java/api/test/javabook/obs/MetricsTests.java, lines 137-164)

31.5. the adversarial history

Each wave ran its own attackers before landing, and the record is the suite's scar tissue. Wave one, the kernel through the user resource, closed 9 code and 20 prose findings. The store wave closed 17, the wal's length overflow and append-failure poisoning among them, and documented 2 latent items in prose rather than pretending them fixed, the values-shared view rule and the non-ApiError escape at the envelope boundary. The concurrency, cache, and limit wave closed 18, the falsifiable single-flight among them, 6390 double fills measured before the fix and 0 after, then the update scope's own round closed 6 more, the json null member rules and the repeated If-Match lines. This wave's record has two halves. The development half caught its own bugs as they surfaced, an unordered map reaching the report unsorted, a racy counter in a fake, an open loop whose cold connection pool met the listen backlog before the server did, each closed with the test that pinned it. The blind half, 2 independent attackers on the chapters, the code, and the ship lane, returned 19 confirmed findings, and every one is closed in the tree: the histogram's check-then-act bucket cell, the nearest-rank ceil an ulp too far at 2 measured vectors, the open loop's warmup failure settling its await one arrival early, the untimed client that hung against a silent server, the rate argument that accepted NaN, the ops path that accepted percent-encoded spellings, the unsent exchange and the measured window nothing pinned, the prose that claimed a shell-less runtime on top of ubuntu, and the counts and ranges this chapter re-verified.

NOTE: what the rounds are for

The two-attacker round is the mutation-testing minimum this lane currently meets by hand: every surviving finding names a hole in the suite, and each fix lands with the test that would have caught it, so the next round

> starts from a harder suite. The rounds are recorded because an unfixed finding that survives into the book is a lie with a publication date.

31.6. what the suite does not hold

The inventory's gaps are stated as plainly as its contents. No standing property or fuzz targets ride the java lane, the go spine's 6 fuzz engines have no siblings here, so the arithmetic surfaces the lane does hold, the json codec, the bucket refill, the rank, are pinned by boundary vectors and adversarial rounds rather than by a mutation engine, and a corpus-level fuzz minimum is carried by the lanes that built those engines. No coverage percentage is printed anywhere in the book, the figure moves with every edit and a pinned number would be a lie with a date on it. And the capstone module's own suite, the chat hub and the mini apps, is chapter 32′s subject, walked there beside the code it pins.

31.7. what the suite proves

The closer maps each layer to the fact it establishes. The kernel and json tests prove the envelope and the codec, the user and authn tests the validation and the one 401 shape, the store tests the wal's recovery and compaction, the concurrency tests the admission cap, the cache tests the single flight, the limit tests the honest 429, the observability tests the exposition and the exact rank, the load tests the harness itself, the samples hold the language facts, and the docker lane proves all of it again over a real image on a real port with a real clock. That surface is contract, not preference, and the spine's last word is the same as its first: a green lane means the behaviors hold, and every number in this chapter was counted from the tree on the day it was written.

sources: the junit console launcher's 6.1.3 cli, the `execute` subcommand, `--scan-classpath`, and `--fail-if-no-tests`, verified against the vendored jar at tools/junit/vendor/junit-platform-console-standalone-6.1.3.jar, whose sha256 e62b96ac475dbcde8599ea905d088f65d90778f86e259b856a49fa5c4ea256ec matches the vendor manifest, accessed 2026-10-05. The lane anatomy read from tools/verify-java.ps1 and tools/run-java-samples.ps1 as they stand at this commit. The wave histories quoted from the ledger at .claude/plans/briefs/java-TASKS.md and from this wave's own commit messages. Verified live 2026-10-05 on tools/jdk27/build/jdk-27, build 27+35-2325: the module suite 158 green 3 consecutive runs, the integration suite 5 green 2 consecutive runs under `make verify-javaapi-docker`, and the samples runner reporting 58 files and 493 checks, all counted again for this chapter the same day.

32. capstone: line clone chat with mini apps

The capstone is a chat service in the shape of the messenger everyone knows: named users claim a handle, join rooms, and exchange text with history replay, typing indicators, and read receipts, while four mini apps, a poll, a rock paper scissors duel, a shared sketchpad, and a room-wide 2048 board, run as panels over the same wire. It lives in this repo at `java/capstone/`, 26 java source files and 12 junit classes plus 2 client doubles, 131 tests green three consecutive runs, beside a react frontend at `java/capstone/web/` whose 21 node tests include an end to end round that boots the real server and drives it through the same wire and state modules the browser runs. The whole tree is 8,193 lines by count including the stylesheet, no external java dependency anywhere, the jdk only, and one process serves the site and the websocket on one port.

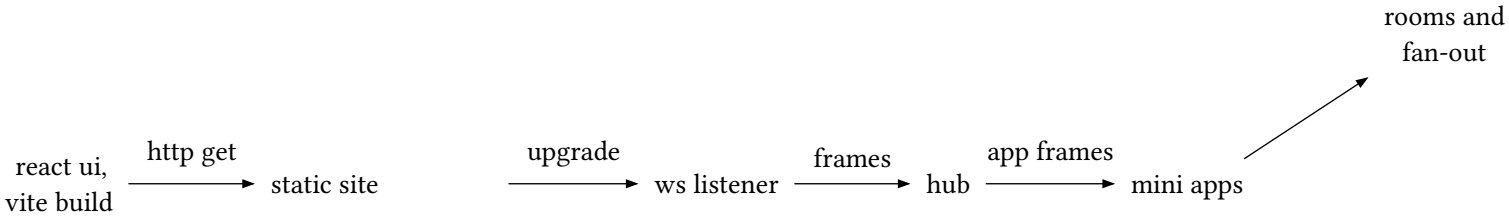


figure 61: the capstone, one process, one port, two protocols

32.1. the shape in thirty seconds

`Main` boots one `WsServer` on a port from argv or 8080, loopback only, and hands it a `Hub` wired to a `MiniApps` runtime holding the 4 registered apps. When the vite build exists the same listener serves it through an `HttpFallback` seam, so one process owns both protocols, and during frontend development vite's dev server proxies the `/chat` upgrade to the server's port instead. Every tunable lives in one record: `Config.defaults()` caps a ws message at 1 MiB, the handshake head at 8,192 bytes, the per-connection outbound queue at 256 frames, the room history at 64 messages, and a chat text at 1,000 characters. The tests shrink those caps to reach every edge with small payloads, which is the whole reason the record exists.

```
  public static void main(String[] args) throws Exception {
    int port = args.length > 0 ? Integer.parseInt(args[0]) : 8080;
    Path webRoot = Path.of(args.length > 1 ? args[1] : "web/dist");
    Config cfg = Config.defaults();
    MiniApps apps = MiniApps.of(new PollApp(), new DuelApp(), new SketchApp(), new G2048App());
    Hub hub = new Hub(cfg, apps);
    HttpFallback web = Files.isDirectory(webRoot) ? new StaticSite(webRoot) : null;
    try (WsServer server = WsServer.start(port, hub, cfg, web)) {
      System.out.println("javabook chat on ws://127.0.0.1:" + server.port() + "/chat (apps: "
          + String.join(" ", apps.ids()) + ")");
      if (web != null) {
        System.out.println("serving " + webRoot + " on http://127.0.0.1:" + server.port() + "/");
      } else {
        System.out.println("web root " + webRoot + " not found, ws only (build it: cd web && npm run
build)");
      }
      Thread.currentThread().join();
    }
  }
```

listing 249: the whole boot: hub, registry, fallback, park (source: java/capstone/src/javabook/Main.java, lines 27-44)

The wire vocabulary is 8 chat frame types, `hello`, `join`, `send`, `typing`, `read`, `leave`, `app.open`, and `app.event`, answered by `hello-ok`, `joined`, `history`, `message`, `typing`, `read`, `left`, and `error`, plus the app envelopes. Errors never close the socket,

they arrive as `error` events carrying a code and a detail, because a bad frame from a client is a client problem and the connection should survive it. Only wire level violations close, with the RFC's own codes.

32.2. the wire we own

The transport is a raw `ServerSocketChannel` in blocking mode with one virtual thread per accepted connection, and the chapter owes the reader the reason this is not `jdk.httpserver` with a 101 handler, because chapter 19 built the whole service spine on that kernel and the capstone walks away from it. The probe ran both paths on the pinned jdk 27 at wave 0:

PITFALL: why not jdk.httpserver 101

`sendResponseHeaders(101, -1)` emits a byte-perfect handshake the jdk's own `java.net.http.WebSocket` client accepts, and then the exchange machinery kills it. A bodyless `sendResponseHeaders` ends with a flush and close, so both exchange streams die before the handler's next line runs. `ExchangeFinished` re-queues the socket to the keep-alive selector, whose next-request parser black-holes every websocket frame: no echo, no error, no FIN, measured past 90 seconds. The reflection escape onto the underlying channel dies on `IllegalBlockingModeException`, a selector-registered channel cannot go back to blocking reads. Passing 0 instead of −1 differs only by a WARNING line. Measured on tools/jdk27/build/jdk-27, 2026-10-04, wave 0.

So the socket channel is owned outright, and the accept loop is the whole listener:

```
  private void acceptLoop() {
    while (!closed) {
      try {
        SocketChannel ch = channel.accept();
        Thread.ofVirtual().start(() -> serve(ch));
      } catch (ClosedChannelException e) {
        return;
      } catch (IOException e) {
        // listener broke or was closed mid-accept, nothing to serve
        return;
      }
    }
  }

  private void serve(SocketChannel ch) {
    if (closed) {
      closeQuietly(ch);
      return;
    }
    try {
      ch.configureBlocking(true);
      ch.socket().setTcpNoDelay(true);
      // false means the fallback answered a plain http request and closed
      if (!Handshake.perform(ch, cfg, fallback)) return;
    } catch (IOException e) {
      closeQuietly(ch);
      return;
    }
    WsConnection c = new WsConnection(ch, handler, cfg, connections::remove);
    // registration fences against close(): either this lands before the close
    // snapshot and the connection gets its 1001, or the guard rejects it
    synchronized (this) {
      if (closed) {
        closeQuietly(ch);
        return;
      }
      connections.add(c);
    }
    c.start();
  }
```

listing 250: accept on a virtual thread, handshake, fenced registration (source: java/capstone/src/javabook/ws/WsServer.java, lines 67-106)

Three details carry weight. `TcpNoDelay` is set on every socket because a chat frame is small and latency is the product. The `HttpFallback` returns false when a plain http request arrived and was answered, and the connection ends there. And registration into the connection set is fenced against `close()`: either the registration lands before the close snapshot and the connection gets its 1001, or the guard inside the synchronized block rejects it.

The opening handshake is RFC 6455 s4.2.1 written by hand, GET with `Upgrade: websocket`, a `Connection` header carrying the upgrade token, version 13, and a key that base64-decodes to exactly 16 bytes, answered by the bare three-header 101 the jdk client accepts. Anything else gets a 400, or a 426 carrying the supported version when only the version header disagrees. Reads stop exactly at the blank line so the connection's stream sees every byte a pipelining client sent past the head:

```
  public static boolean perform(SocketChannel ch, Config cfg, HttpFallback fallback) throws IOException {
    String head = readHead(Channels.newInputStream(ch), cfg.maxHandshakeBytes());
    String[] lines = head.split("\r\n");
    String[] requestLine = lines[0].split(" ");
    if (requestLine.length != 3 || !requestLine[2].startsWith("HTTP/1.")) throw fail(ch, RESPONSE_400);
    TreeMap<String, String> headers = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);
    for (int k = 1; k < lines.length; k++) {
      int colon = lines[k].indexOf(':');
      if (colon <= 0) throw fail(ch, RESPONSE_400);
      headers.merge(lines[k].substring(0, colon).trim(), lines[k].substring(colon + 1).trim(),
          (a, b) -> a + "," + b);
    }
    String upgrade = headers.get("Upgrade");
    if (upgrade == null) {
      if (fallback != null) {
        fallback.serve(ch, requestLine[0], requestLine[1], headers);
        return false;
      }
      throw fail(ch, RESPONSE_400);
    }
    // s4.2.1 allows a list: the handshake is wanted when any token is websocket
    boolean wantsWebSocket = false;
    for (String token : upgrade.split(",")) {
      if (token.trim().equalsIgnoreCase("websocket")) wantsWebSocket = true;
    }
    if (!wantsWebSocket) throw fail(ch, RESPONSE_400);
    if (!requestLine[0].equals("GET")) throw fail(ch, RESPONSE_400);
    boolean connectionUpgrade = false;
    for (String token : headers.getOrDefault("Connection", "").split(",")) {
      if (token.trim().equalsIgnoreCase("upgrade")) connectionUpgrade = true;
    }
    if (!connectionUpgrade) throw fail(ch, RESPONSE_400);
    if (!"13".equals(headers.get("Sec-WebSocket-Version"))) throw fail(ch, RESPONSE_426);
    String key = headers.get("Sec-WebSocket-Key");
    byte[] decoded;
    try {
      decoded = Base64.getDecoder().decode(key == null ? "" : key);
    } catch (IllegalArgumentException e) {
      decoded = null;
    }
    if (decoded == null || decoded.length != 16) throw fail(ch, RESPONSE_400);
    String response = "HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\n"
        + "Connection: Upgrade\r\nSec-WebSocket-Accept: " + acceptFor(key) + "\r\n\r\n";
    Channels.newOutputStream(ch).write(response.getBytes(StandardCharsets.ISO_8859_1));
    return true;
  }
```

listing 251: the upgrade decision tree, header folding, the three-header 101 (source: java/capstone/src/javabook/ws/Handshake.java, lines 53-98)

Headers fold into a case-insensitive `TreeMap` with duplicates comma-joined, the spec allows `Upgrade: h2c, websocket` lists and the handshake is wanted when any token matches, and the accept key is SHA-1 of key plus the RFC's fixed guid, the one place this book touches SHA-1, because the protocol dictates it and nothing else rides on collision resistance here.

32.3. the frame codec's edge table

`Frame.read` pulls one frame off a client stream and enforces the protocol's whole grammar before any payload is buffered:

```
public static Frame read(InputStream in, int maxMessageBytes) throws IOException {
  int b0 = in.read();
  int b1 = in.read();
  if (b0 < 0 || b1 < 0) throw new EOFException("eof in frame header");
  boolean fin = (b0 & 0x80) != 0;
  if ((b0 & 0x70) != 0) throw new WsException(1002, "rsv bits set");
  int opcode = b0 & 0x0F;
  if (opcode >= 0x3 && opcode <= 0x7 || opcode >= 0xB && opcode <= 0xF)
    throw new WsException(1002, "reserved opcode " + opcode);
  if ((b1 & 0x80) == 0) throw new WsException(1002, "client frame not masked");
  long len = switch (b1 & 0x7F) {
    case 126 -> {
      long v = readUint(in, 2);
      if (v < 126) throw new WsException(1002, "non-minimal 16-bit length " + v);
      yield v;
    }
    case 127 -> {
      long v = readUint(in, 8);
      if (v < 65536) throw new WsException(1002, "non-minimal 64-bit length " + v);
      yield v;
    }
    default -> b1 & 0x7F;
  };
  if (opcode >= 0x8) {
    if (len > MAX_CONTROL) throw new WsException(1002, "control frame over 125 bytes");
    if (!fin) throw new WsException(1002, "fragmented control frame");
    if (opcode == CLOSE && len == 1) throw new WsException(1002, "close payload of 1 byte");
  }
  if (len > maxMessageBytes) throw new WsException(1009, "message too big");
  byte[] mask = new byte[4];
  readFully(in, mask);
  byte[] payload = new byte[(int) len];
  readFully(in, payload);
  for (int i = 0; i < payload.length; i++) payload[i] ^= mask[i & 3];
  return new Frame(fin, opcode, payload);
}
```

listing 252: mask required, minimal lengths, control rules, the cap (source: java/capstone/src/javabook/ws/Frame.java, lines 33-68)

Client frames must be masked, rsv bits and reserved opcodes fail with 1002, control frames stay at or under 125 bytes and never fragment, a close payload of exactly 1 byte fails, and the length must use its minimal encoding, 7-bit below 126, 16-bit below 65,536, 64-bit above. An announced or accumulated payload past the cap fails with 1009 before anything is buffered, and text is decoded strict utf-8, malformed or overlong sequences answering 1007. The test tables pin all of it against a live listener, 125, 126, and 127 bytes crossing the length forms both directions, a 300 byte and a 70,000 byte payload through both extended forms, valid utf-8 split mid character across fragments reassembling cleanly, and every malformed row answered with the code the RFC names.

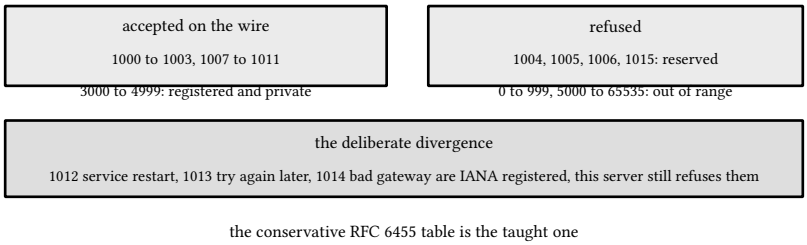


figure 62: the close code table this server enforces

The divergence is stated, not hidden: 1012, 1013, and 1014 sit in the IANA registry, registered through the HYBI mailing list and updated there in 2025, and this server still refuses them because the RFC 6455 table is what the chapter teaches and a toy chat has no restart or gateway story to tell with them.

32.4. the connection, a reader and a writer

One upgraded connection runs 2 virtual threads. The reader owns the frame loop, message assembly across continuations under the size cap, strict utf-8 on complete text messages, ping answered with a matching pong, and the close choreography where a peer close is echoed then tcp closed. The writer drains an outbound queue of pre-encoded frames:

```
  private void readLoop() {
    try {
      ByteArrayOutputStream parts = new ByteArrayOutputStream();
      int assembling = -1;
      while (true) {
        Frame f = Frame.read(in, cfg.maxMessageBytes());
        switch (f.opcode()) {
          case Frame.TEXT, Frame.BINARY -> {
            if (assembling ≥ 0) throw new WsException(1002, "data frame mid-message");
            if (f.fin()) {
              deliver(f.opcode(), f.payload());
            } else {
              assembling = f.opcode();
              parts.reset();
              parts.writeBytes(f.payload());
            }
          }
          case Frame.CONTINUATION -> {
            if (assembling < 0) throw new WsException(1002, "continuation without a leading data frame");
            if (parts.size() + (long) f.payload().length > cfg.maxMessageBytes())
              throw new WsException(1009, "message too big");
            parts.writeBytes(f.payload());
            if (f.fin()) {
              byte[] whole = parts.toByteArray();
              parts.reset();
              int opcode = assembling;
              assembling = -1;
              deliver(opcode, whole);
            }
          }
          case Frame.PING -> enqueue(Frame.encode(Frame.PONG, true, f.payload()));
          case Frame.PONG -> {
            // unsolicited pongs are legal, nothing tracks them
          }
          case Frame.CLOSE -> {
            closeIn(f);
            return;
          }
          default -> throw new WsException(1002, "opcode " + f.opcode() + " rejected here");
        }
      }
    } catch (WsException e) {
      sendClose(e.closeCode, e.getMessage());
    } catch (IOException e) {
      abort(1006);
    } catch (RuntimeException e) {
      // a handler bug must not wedge the connection open
      closeCode = 1011;
      abort(1011);
    }
  }
```

listing 253: the reader loop, opcode dispatch, the close choreography (source: java/capstone/src/javabook/ws/WsConnection.java, lines 98-148)

The queue is the fan-out decision and the capstone's most consequential design choice. A broadcast only enqueues, never writes, so a room send touches 1 socket zero times and N queues once each, and a member whose tcp window stalled fills its own queue and gets dropped at the frame cap instead of stalling the room's sends behind

one slow socket. The room lock never waits on io. Sends from any thread land in fifo order because the enqueue happens under the caller's ordering and the queue serializes the writes.

The drop path is where the locking story earns its care:

```
  private void enqueue(byte[] frame) {
    if (closed) return;
    outbound.add(frame);
    if (outbound.size() > cfg.outboundFramesMax()) {
      // a stalled member gets dropped, never allowed to stall the room. the
      // drop must not run finish() here: the caller can hold a room lock, and
      // finish calls the handler, which reenters the hub. fencing, waking the
      // writer with the sentinel, and killing the socket is enough, the
      // writer thread runs finish() holding no hub state. the socket kill
      // runs on its own thread for the same reason: close() can wait on the
      // writer's blocked write to a never-read tcp, and that wait must never
      // run under the caller's lock either.
      closeCode = 1008;
      closed = true;
      outbound.clear();
      outbound.add(SHUTDOWN);
      Thread.ofVirtual().start(() -> {
        try {
          channel.close();
        } catch (IOException ignored) {
          // already gone
        }
      });
    }
  }
```

listing 254: the overflow drop: fence, sentinel, kill the socket off thread (source: java/capstone/src/javabook/ws/WsConnection.java, lines 185-209)

A caller holding a room lock must never run `finish()` inline, because finish tells the handler, the handler reenters the hub, and the hub wants the same lock. So the drop only fences the closed flag, clears the queue, wakes the writer with the sentinel, and kills the socket on its own virtual thread, because `close()` can wait on the writer's blocked write to a never-read tcp and that wait must never run under the caller's lock either. The writer thread runs `finish()` holding no hub state. This is chapter 25's lesson, no blocking under a lock, paid forward with interest.

32.5. the hub

Above the transport sits the chat core. A `hello` names the connection and claims the user, `join` binds it to a room and replays history, `send` broadcasts a message carrying a globally unique monotonically increasing id from one `AtomicLong`, `typing` and `read` fan out to the others only, `leave` acknowledges, and any disconnect departs every room with a `left` event. Names and rooms match `[A-Za-z0-9_-]{1,32}`, a name already online is refused, and a second hello on the same connection is an error event.

The join path carries the membership and generation invariants:

```
  private void join(WsConnection c, Map<String, Object> m) {
    Session s = session(c);
    if (s == null) {
      error(c, "nologin", "say hello first");
      return;
    }
    String roomName = Json.string(m, "room");
    if (roomName == null || !NAME.matcher(roomName).matches()) {
      error(c, "room", "room must match " + NAME.pattern());
      return;
    }
    while (true) {
      Room room = rooms.computeIfAbsent(roomName, n -> new Room(n, cfg));
      synchronized (room) {
        // a concurrent last-leave may have emptied and unregistered this room
        // between computeIfAbsent and the lock, retry on the live one
        if (rooms.get(roomName) != room) continue;
        if (room.members.putIfAbsent(s.user, c) != null) {
          error(c, "join", "already in " + roomName);
          return;
        }
        // the connection can have died while its reader held this join frame:
        // the drop's close may already have run, taking with it the only
        // depart this member would ever get, and the room would hold a ghost
        // forever. rollback under the same lock keeps dead connections out;
        // a close still pending finds this lock held and departs normally
        // once we release.
        if (!c.isOpen()) {
          room.members.remove(s.user, c);
          return;
        }
        s.rooms.add(roomName);
        List<String> members = new ArrayList<>(room.members.keySet());
        Collections.sort(members);
        c.sendText(Json.obj("type", "joined", "room", roomName, "user", s.user,
            "members", members.toArray(new String[0])));
        String[] messages = room.history.snapshot().stream()
            .map(msg -> msg.toJson(roomName))
            .toArray(String[]::new);
        c.sendText(Json.obj("type", "history", "room", roomName, "messages",
Json.Raw.of(Json.arr(messages))));
        fanout(room, s.user, Json.obj("type", "joined", "room", roomName, "user", s.user));
        return;
      }
    }
  }
```

listing 255: join: the retry on a dead generation, the ghost rollback, the snapshot under the lock (source: java/capstone/src/javabook/hub/Hub.java, lines 115-159)

Four invariants live here. A concurrent last-leave may empty and unregister the room between the `computeIfAbsent` and the lock, so the loop re-checks the live registration and retries. The id joins the history append inside the room lock, so the history snapshot a joiner sees and the live events that follow can neither duplicate nor skip a message. The connection can die while its reader holds this join frame, and the drop's close may already have run the only depart that member would ever get, which would leave the room holding a ghost forever, so the liveness re-check under the lock rolls the registration back. And every room command enters through one guarded path:

```
/**
 * resolves the room and runs one room action under its lock, membership
 * re-checked inside against the live map and the live generation: a
 * frame racing its sender's depart must lose here instead of
 * broadcasting after the left event, and a last-leave that emptied and
 * replaced the room between the lookup and the lock must not let a
 * stale frame act on the dead generation. this is the one guarded path
 * every room command and every app frame enters.
 */
private void inRoom(WsConnection c, Session s, String roomName, Consumer<Room> action) {
  Room room = rooms.get(roomName);
  if (room == null) {
    error(c, "notmember", "join " + roomName + " first");
    return;
  }
  synchronized (room) {
    if (rooms.get(roomName) != room || room.members.get(s.user) != c) {
      error(c, "notmember", "join " + roomName + " first");
      return;
    }
    action.accept(room);
  }
}
```

listing 256: inRoom, the one guarded path: membership and generation re-checked under the lock (source: java/capstone/src/javabook/hub/Hub.java, lines 322-344)

A frame racing its sender's depart must lose here instead of broadcasting after the `left` event, and a last-leave that emptied and replaced the room between the lookup and the lock must not let a stale frame act on the dead generation. Departure is exactly once per connection and room, `room.members.remove(user) == connection` returns silently, because a racing close can run depart from the writer thread while the same connection's reader is parked inside `leave`, and the second run would double the `left` event and the app's `onLeave`. When the last member leaves, the room unregisters by name and value and the runtime hears the room object itself, the generation key, so a stale room-closed from a dying generation can never drop the state of the generation that replaced it:

```
  /** removes the user from the room and announces left, to the others, plus the leaver when ack is set. */
  private void depart(Room room, WsConnection c, Session s, boolean ack) {
    String json = Json.obj("type", "left", "room", room.name, "user", s.user);
    synchronized (room) {
      // exactly once per connection and room: a racing close can run depart
      // from the writer thread while this connection's reader is already
      // parked inside leave, and the second run would double the left event
      // and the app's onLeave
      if (room.members.remove(s.user) != c) return;
      s.rooms.remove(room.name);
      // after the remove, app leave fan-out reaches exactly who stays
      if (apps != null) apps.memberLeft(new AppSocket(room), s.user);
      if (ack) c.sendText(json);
      fanout(room, s.user, json);
      if (room.isEmpty()) {
        rooms.remove(room.name, room);
        // the room object, not the name: a join may already have registered a
        // new generation under the freed name, and the close must not touch it
        if (apps != null) apps.roomClosed(room);
      }
    }
  }

  /** the room as the runtime sees it: name, members, and member-addressed delivery, all under the room
lock. */
  private static final class AppSocket implements RoomSocket {

    private final Room room;

    AppSocket(Room room) {
      this.room = room;
    }

    @Override public String room() {
      return room.name;
    }

    @Override public Object key() {
      return room;  // the room object itself: one identity per registration of the name
    }

    @Override public Collection<String> members() {
      return room.members.keySet();
    }

    @Override public void send(String user, String json) {
      WsConnection c = room.members.get(user);
      if (c != null) c.sendText(json);
    }
  }
```

listing 257: depart exactly once, and the socket the runtime sees, key is the room object (source: java/capstone/src/javabook/hub/Hub.java, lines 265-313)

Two of these invariants were adversarial finds, not first drafts. The orphaned-room race reproduced at an 8.5 percent rate under concurrent join and last-leave churn before the generation re-check closed it, and the ghost member reproduced 12 of 12 rounds before the rollback, its regression now driving a 4-million-element pad array

past the json parser while a 600 message flood drops the connection mid parse, 6 rounds asserting the dropped name never appears in a later member list.

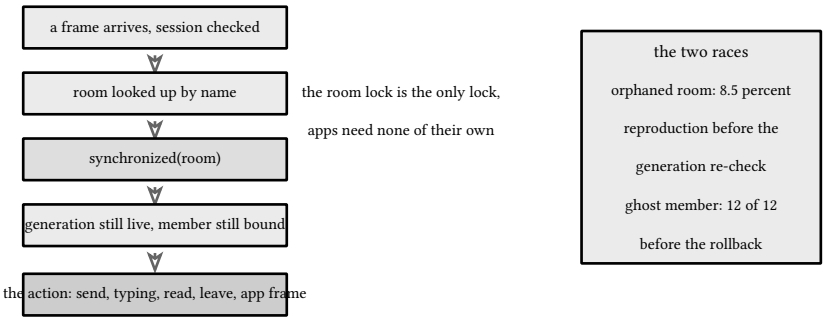


figure 63: the hub's one guarded path, everything re-checked under the room lock

32.6. message history

Each room holds a fixed-capacity ring of its last messages, the oldest dropped first, synchronized on itself so a snapshot copies without holding the room lock across the copy:

```
public synchronized void add(ChatMessage m) {
  ring[head] = m;
  head = (head + 1) % ring.length;
  if (size < ring.length) size++;
}

public synchronized List<ChatMessage> snapshot() {
  List<ChatMessage> out = new ArrayList<>(size);
  int start = (head - size + ring.length) % ring.length;
  for (int k = 0; k < size; k++) out.add(ring[(start + k) % ring.length]);
  return out;
}
```

listing 258: the ring: add, snapshot in order, count (source: java/capstone/src/javabook/msg/History.java, lines 22-33)

A join replays the snapshot inside the room lock as one `history` event, then the live `message` events follow in the same lock's order, which is why the two can never disagree. Ids come from the hub's global counter, unique across rooms and strictly increasing inside each, and the read receipt carries one, so a client can mark exactly the message its member read.

32.7. the mini app runtime

The apps are the capstone's second act. The runtime owns a registry of app kinds by id, `[a-z0-9][a-z0-9-]{0,15}`, and one instance per room generation per app, created lazily on the room's first frame of either kind and dropped when the room empties. The hub forwards `app.open` and `app.event` frames after its own session and membership checks, then hands the room to the runtime as a `RoomSocket`:

```
  public void onFrame(RoomSocket socket, String sender, Map<String, Object> envelope, boolean open) {
    String appId = Json.string(envelope, "app");
    if (appId == null) {
      appError(socket, sender, null, "app", "the app field is missing");
      return;
    }
    App app = registry.get(appId);
    if (app == null) {
      appError(socket, sender, appId, "app", "unknown app " + appId);
      return;
    }
    RoomApp instance = instances(socket.key()).computeIfAbsent(appId, id -> app.newInstance());
    Out out = new AppOut(socket, appId, sender);
    if (open) {
      socket.send(sender, Json.obj("type", "app.opened", "room", socket.room(), "app", appId));
      for (String member : socket.members()) {
        if (member.equals(sender)) continue;
        socket.send(member, Json.obj("type", "app.opened", "room", socket.room(), "app", appId, "by",
sender));
      }
      instance.onOpen(sender, out);
      return;
    }
    String action = Json.string(envelope, "action");
    if (action == null) {
      appError(socket, sender, appId, "action", "the action field is missing");
      return;
    }
    instance.onEvent(sender, action, envelope, out);
  }
```

listing 259: dispatch: lazy instance, open ack and announce, event by action (source: java/capstone/src/javabook/miniapp/MiniApps.java, lines 57-85)

The contract that makes the whole layer lock free is placement: every app callback runs on the caller's thread under the owning room's lock, in arrival order interleaved with the room's chat events. An instance needs no locks of its own, plain fields are the whole state story, and a callback must never block, which every send satisfying because sends only enqueue. The runtime wraps app payloads into envelopes, `app.state` room visible, `app.priv` to one member, `app.error` to the sender, always carrying the room and app id. Opening a panel acks the opener with the current state through `onOpen`, announces the panel to the others, and fires on every open, not just the first, so a late joiner catching up is the same code path as the first panel.

32.8. the four mini apps

Four apps ride the runtime, each a different lesson in shared state under one lock. The poll is the broadcast shape: one active poll per room, `create` with a question up to 200 characters and 2 to 8 options of up to 80 each, every member votes once whether or not their panel is open, each vote fans the live tallies with voter names per option to the whole room, and the creator closes with the winner or a tie: plurality, the option standing alone at the top count wins even below half, the word tie only when the top is level. The creator leaving an open poll closes it, since nobody else could.

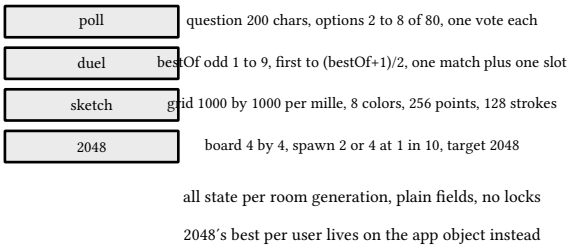


figure 64: the four apps' bounds, every number from the source

The duel is the hidden information shape: one waiting slot and one match at a time per room. `queue` enters matchmaking, a second queue matches the two, and the waiting player's `bestOf` wins, the responder's ignored. Each `choose` is echoed to the chooser alone, and the room sees a round only once both weapons are in, draws replay the same round number, and the series ends at the majority. The validation lesson lives in one line of the queue path:

```
      Long bestOf = Json.longNum(body, "bestOf");
      if (bestOf == null && body.containsKey("bestOf")) {
        out.error("bestof", "bestOf must be odd, 1 to " + MAX_BEST_OF);
        return;
      }
      // the range check runs on the long: narrowing first would wrap 2^32+1 into a legal 1
      long n = bestOf == null ? DEFAULT_BEST_OF : bestOf;
      if (n < 1 || n > MAX_BEST_OF || n % 2 == 0) {
        out.error("bestof", "bestOf must be odd, 1 to " + MAX_BEST_OF);
        return;
      }
      waiting = user;
      waitingBestOf = (int) n;
      out.toSender(Json.obj("kind", "waiting", "by", user, "bestOf", n));
```

listing 260: bestOf validated on the long before the narrowing cast (source: java/capstone/src/javabook/duel/DuelApp.java, lines 106-119)

Narrowing first would wrap 2^32+1 into a legal 1, and the json parser hands the app a `Long`, so the range check runs where the value is still honest. A player leaving mid match forfeits to the opponent, a waiting player leaving clears the slot.

The sketchpad is the coalescing shape. A stroke is one gesture, a color index into a fixed 8 color palette plus its points as integer per-mille coordinates on a 1000 by 1000 grid, so every client renders the same board whatever its canvas size. The client coalesces its pointer samples, one stroke is one `app.event`, never one frame per point, 2 to 512 array entries of 0 to 1000, and the server keeps a bounded ring of 128 strokes, oldest first out:

```
  private void stroke(String user, Map<String, Object> body, Out out) {
    Long color = Json.longNum(body, "color");
    if (color == null || color < 0 || color ≥ COLORS) {
      out.error("color", "color must be 0 to " + (COLORS - 1));
      return;
    }
    if (!(body.get("points") instanceof List<?> raw) || raw.size() < 2 || raw.size() > MAX_POINTS * 2
        || raw.size() % 2 ≠ 0) {
      out.error("points", "points must be 1 to " + MAX_POINTS + " x,y pairs");
      return;
    }
    int[] points = new int[raw.size()];
    for (int i = 0; i < raw.size(); i++) {
      if (!(raw.get(i) instanceof Long v) || v < 0 || v > COORD_MAX) {
        out.error("points", "every coordinate must be 0 to " + COORD_MAX);
        return;
      }
      points[i] = v.intValue();
    }
    Stroke s = new Stroke(user, color.intValue(), points);
    strokes.addLast(s);
    if (strokes.size() > MAX_STROKES) strokes.pollFirst();
    out.toOthers(strokeJson(s));
  }
```

listing 261: one stroke validated whole, then into the ring (source: java/capstone/src/javabook/sketch/SketchApp.java, lines 59-83)

2048 is the shared state machine shape: one board per room, every member moves the same 16 cell array. A move slides all 4 lines in the asked direction, each tile merges at most once per move so 2,2,2,2 becomes 4,4 and never 8, the merge values sum into the score, and a fresh 2 or 4, one in ten, spawns on a random empty cell of the moved board. The move that changes nothing is rejected, so the spawn never rewards a no-op. Reaching 2048 flags the win while play continues, and the game is over when no empty cell and no equal neighbor remain:

```
  static int slide(int[] board, String dir) {
    int gained = 0;
    for (int line = 0; line < 4; line++) {
      int[] out = new int[4];
      int n = 0;
      for (int slot = 0; slot < 4; slot++) {
        int v = board[cell(line, slot, dir)];
        if (v != 0) out[n++] = v;
      }
      int m = 0;
      int[] merged = new int[4];
      for (int i = 0; i < n; i++) {
        if (i + 1 < n && out[i] == out[i + 1]) {
          merged[m] = out[i] * 2;
          gained += merged[m];
          i++;
        } else {
          merged[m] = out[i];
        }
        m++;
      }
      for (int slot = 0; slot < 4; slot++) board[cell(line, slot, dir)] = merged[slot];
    }
    return gained;
  }
```

listing 262: slide: compaction, merge once, sum the gains (source: java/capstone/src/javabook/g2048/G2048App.java, lines 130-154)

The best score is per user but lives on the app object, not the room instance: the registered app survives the room emptying and the name being re-created and dies with the process, so a best is a fact about a player in one server run, and the field is an instance field so tests holding different app objects never bleed records into each other. The mover banks the game's running score against their own best after every legal move, so in a shared game whoever pushed the score highest owns the record. The random source is injectable, a seeded game replays deterministically in tests.

32.9. the react frontend

The frontend is react 19.3.0 with vite 8.3.2 and the react plugin 6.1.1, pinned exact in `package.json`, and its discipline is the one the spike probe fixed before any code was written: the logic lives in plain `.js` modules that node's test runner can load, because `node --test` rejects `.jsx` at the loader level, and the jsx files are components only, covered by the vite build. The two load-bearing modules are `wire.js`, the client, one websocket and the frame vocabulary and nothing else, and `state.js`, the room state as a pure fold over the hub's events, same event in, same state out, no io, no clock. Both run unchanged in the browser and in node, both ship the same `WebSocket` global, which is what lets the e2e suite drive the real server through the exact files the browser runs.

```
  connect() {
    return new Promise((resolve, reject) => {
      this.ws = new WebSocket(this.url)
      this.ws.addEventListener('open', () => {
        this.open = true
        resolve()
      })
      this.ws.addEventListener('message', (m) => this.listener(JSON.parse(m.data)))
      this.ws.addEventListener('error', () => {
        this.errorCount++
        if (!this.open) reject(new Error('could not connect to ' + this.url))
      })
      this.ws.addEventListener('close', (e) => {
        this.open = false
        this.closeInfo = { code: e.code, reason: e.reason }
        this.listener({ type: 'closed', code: e.code })
      })
    })
  }
```

listing 263: connect: open, error, close folded into the same event stream (source: java/capstone/web/src/wire.js, lines 36-54)

The fold's one subtle duty is the sketch cap. The server keeps 128 strokes, so the live fold trims to the same bound, else a resident board and a late opener's replay diverge, a bug both blind attackers of the frontend wave found independently before it ever shipped:

```
    case 'stroke': {
      const strokes = [...((prev && prev.strokes) || []), copyStroke(p)]
      if (strokes.length > SKETCH_MAX_STROKES) strokes.splice(0, strokes.length - SKETCH_MAX_STROKES)
      return { kind: 'board', strokes }
    }
```

listing 264: the stroke fold trimming to the server's bound (source: java/capstone/web/src/state.js, lines 134-138)

The panels are 5, chat, poll, 2048, duel, sketch, selected on a bottom bar, mobile first. The 2048 board reads swipes from a shared `direction` helper with a 24 pixel threshold, a shorter gesture is a tap and answers null. Once a room is joined the app opens every panel, so all 4 app states fold from the same event stream and each panel renders its slice. A dropped connection folds the socket's close event into an offline flag instead of leaving every panel silently stale.

32.10. one process, two protocols

Production is one process: the listener serves the vite build when `web/dist` exists, through the same `Handshake` seam that decides upgrade versus plain request. `StaticSite` is deliberately small, GET only, files that exist inside the root after percent-decoding and normalization, no directory listing, an 8 MiB cap, content types for the shapes a frontend build produces, and `Connection: close` on every response:

```
  /** the target with its query stripped, percent-decoded, and contained in the root, null when it escapes.
*/
  private Path resolve(String target) {
    int query = target.indexOf('?');
    String path = query ≥ 0 ? target.substring(0, query) : target;
    if (path.isEmpty() || path.charAt(0) ≠ '/') return null;
    String decoded = percentDecode(path);
    if (decoded == null) return null;
    // windows strips trailing dots and spaces from any path segment, not
    // just the last, so a mid-segment spelling like /js./main.js would
    // serve js/main.js under a name nobody asked for: reject the char
    // before every slash and the final char
    for (int i = 0; i < decoded.length(); i++) {
      char c = decoded.charAt(i);
      if ((c == '.' || c == ' ') && (i + 1 == decoded.length() || decoded.charAt(i + 1) == '/')) {
        return null;
      }
    }
    if (decoded.equals("/")) decoded = "/index.html";
    try {
      Path resolved = root.resolve(decoded.substring(1)).normalize();
      return resolved.startsWith(root) ? resolved : null;
    } catch (Exception e) {
      return null;
    }
  }

  /** strict %XX decoding: a bare percent, a short escape, or a control or non-ascii char is a rejection.
*/
  private static String percentDecode(String s) {
    StringBuilder b = new StringBuilder(s.length());
    for (int i = 0; i < s.length(); i++) {
      char c = s.charAt(i);
      if (c == '%') {
        if (i + 2 ≥ s.length()) return null;
        int hi = Character.digit(s.charAt(i + 1), 16);
        int lo = Character.digit(s.charAt(i + 2), 16);
        if (hi < 0 || lo < 0) return null;
        char d = (char) ((hi ≪ 4) | lo);
        if (d < 0x21 || d > 0x7e || d == '\\') return null;
        b.append(d);
        i += 2;
      } else {
        if (c < 0x21 || c > 0x7e || c == '\\') return null;
        b.append(c);
      }
    }
    return b.toString();
  }
```

listing 265: resolve and decode: strict, contained, windows aliases refused (source: java/capstone/src/javabook/web/StaticSite.java, lines 67-113)

The containment rules earn their lines. Percent-decoding is strict, a bare percent, a short escape, a control or non-ascii or backslash character is a rejection, never a repair. The decoded path must normalize back inside the root. And windows strips trailing dots and spaces from path segments, so those spellings are refused outright, they would serve a file under a name nobody asked for.

The end to end round is the frontend's real gate. It boots the compiled server on an ephemeral port with a throwaway site root, asserts the boot line lists all 4 apps and serves the site, then drives two clients through chat with typing and a read receipt, a poll from create to a tied close, a shared 2048 move both boards agree on, a best-of-one duel to a series win, and a sketch stroke that survives a leave and rejoin as a replayed board. Its java binary resolution is the lane's one platform concession:

```
function javaBin() {
  // the repo's pinned jdk wins: the machine's PATH java can be a release
  // older than the classes (class file version 71 needs 27)
  const candidates = [
    join(repo, 'tools', 'jdk27', 'build', 'jdk-27'),
    process.env.JAVA_HOME || '',
  ].filter(Boolean)
  for (const home of candidates) {
    const exe = join(home, 'bin', process.platform === 'win32' ? 'java.exe' : 'java')
    if (existsSync(exe)) return exe
  }
  return 'java'
}
```

listing 266: the repo's pinned jdk wins, the machine's java can be a release older than the classes (source: java/capstone/web/test/e2e.test.js, lines 23-35)

Class file version 71 needs 27, and a machine java of 26 dies at `UnsupportedClassVersionError` before the first assertion, so the pinned build is resolved first. The remaining 20 node tests pin the state fold, the wire client, the swipe direction, and the dependency smoke that asserts the react version string itself.

32.11. measured, and why

The capstone carries no benchmark harness, and says so: the load family lives in the service spine, chapter 29 owns measurement, and pasting a microbenchmark of a toy chat would decorate rather than inform. What the tree does carry is the adversarial record, and its numbers are the chapter's honest measurements. The orphaned-room race reproduced in 8.5 percent of rounds before the generation re-check, 0 after, pinned by a churn regression that hammers join against last-leave. The ghost member reproduced 12 of 12 rounds before the rollback, 0 across 6 rounds after, each round a 4-million-element pad holding the reader in the parser while a 600 message flood drops it. The overflow drop was restructured when a room wedge appeared, a blocked peer write holding the room lock through an inline channel close, and the fix moved the kill off thread, pinned by the flood regressions that park readers inside the room lock on purpose. The double-depart guard exists because a stalled action raced a racing close into two left events, its regression resetting `SO_LINGER` and flooding 200 messages. None of these are timings, they are reproduction rates and round counts, and that is the honest shape of the evidence this tree prints.

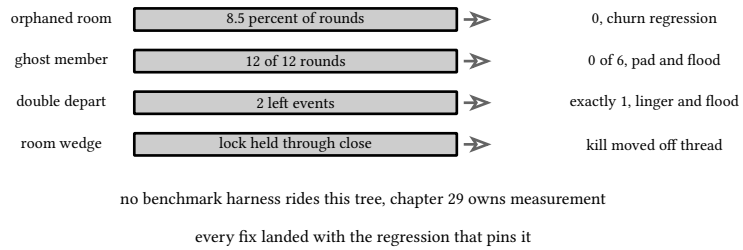


figure 65: the adversarial record, reproduction rates before and after each fix

32.12. accepted design, stated

Five items are design choices this chapter records rather than fixes, and the list is the contract. The poll accepts duplicate option texts, two options spelled the same both collect votes, an ids-not-texts fix would buy nothing a toy poll needs. The per-user 2048 best map grows for the process lifetime, one long per user name ever seen,

unbounded across a server run, and dies with the process, correct for a toy and stated for a real one. Two nanosecond windows remain unconfirmed, the close fence where a reader may observe one frame past a close the writer already flushed, and the file size read in the static site where a file growing past the cap between the check and the read serves the larger file, both judged harmless for a loopback toy and both named here so a real port knows where to look. And the two latent items chapter 24 recorded against the service spine stand unchanged, the values-shared view rule where a store view exposes the same list instance the store may still mutate, and the non-ApiError throw escaping at the chapter 20 envelope boundary where a handler bug surfaces as a 500 with its detail logged but not enveloped.

32.13. what the capstone proves

Every layer of the book cashes once, in place. The toolchain's pinned jdk 27 builds and runs the whole tree. The ladder's features are the shape: records carry the messages, switch expressions dispatch the opcodes, pattern matching guards the frames. The lexical chapter's escape families are the json parser's surrogate discipline. Types and bit arithmetic are the frame codec's length forms and masks. Classes and records are the whole message layer. Generics are the writer's `Object...` pairs and the typed registries. Nested types are the private companions, `Poll`, `Duel`, `Board`, `Game`, `AppSocket`, while enums themselves go unused, opcode dispatch switching on ints, a stated non-use rather than an oversight. The functional chapter's one-method interfaces are every seam, `WsHandler`, `HttpFallback`, `App`, `Out`. Pattern matching is `instanceof Session s` in the guards and record patterns in the parsers. Concurrency is virtual threads per connection, the room lock discipline, and the atomics. The memory chapter's sizing instinct is the bounded fan-out itself, every connection's outbound queue capped at 256 frames, so a loud peer costs flat memory, not growing memory. The collections chapter is the `ArrayDeque` ring, the `LinkedBlockingQueue` fan-out, and the case-insensitive `TreeMap`. Formats are the writer's escape table. The io chapter's socket channels and the jdk client's measured fragmentation are the transport's own facts. Modules and reflection pay negatively and say so: the capstone declares no `module-info`, it compiles and runs on the plain classpath, and nothing in the tree reads a class at runtime, the json codec binding through `instanceof` and explicit constructors, which is why the record write refusals the reflection chapter pins never bite here. Security is SHA-1 exactly where the RFC dictates it and nowhere else. The testing chapter's discipline is the 131 test lane. And the service spine pays off negatively and positively: negatively, the wave 0 probe is why the capstone does not ride `jdk.httpserver`, positively, the spine's hand-rolled json and its no-blocking-under-a-lock lesson are the codec and the drop path verbatim.

chapter	payoff	chapter	payoff
toolchain	the pinned jdk	patterns	instanceof guards, record parsers
ladder	records, switches, virtual threads	concurrency	a thread per connection, the lock
lexical	escapes and surrogate pairing	collections	the ring, the queue, the treemap
types	masks, length forms, long ids	io	channels, the jdk client's frames
classes	records as the message layer	security	sha-1 only where dictated
generics	the Object... writer, registries	testing	the 131 test lane
functional	one-method seams	the spine	the json codec, the lock lesson

131 junit tests and 21 node tests, zero skipped

figure 66: every chapter pays off once, in place

The book's last word on the capstone is the lane's own: `make verify-java` runs the samples, the api module, the capstone module, and the web leg in one gate, the e2e last because it spawns the server from the compiled classes, and the whole chain was green 3 consecutive runs the day this chapter was written.

sources: the capstone tree in this repo, `java/capstone/`, counted from the tree at this commit: 26 source files, 12 junit classes plus the RawClient and JdkClient doubles, 131 junit tests (Frame 16, Handshake 6, Protocol 27, Json 18, History 5, Hub 16, MiniApp 10, Poll 6, Duel 7, Sketch 4, G2048 8, StaticSite 8), 11 web source files, 10 js modules plus the stylesheet, and 5 web test files holding 21 node tests (state 13, wire 2, swipe 4, deps 1, e2e 1), 8,193 lines by wc over java sources, java tests, web sources including the stylesheet, and web tests. Verified live 2026-10-05 on tools/jdk27/build/jdk-27, build 27+35-2325, through `pwsh -NoProfile -File tools/verify-java.ps1`: the junit lane

green 3 consecutive runs, the web leg's `npm run verify` green the same day. RFC 6455 for the handshake, frame, and close rules, read at tools.ietf.org/html/rfc6455, accessed 2026-10-04 during chapter 14 and re-read for this chapter's close code table. The IANA websocket close code registry, www.iana.org/assignments/websocket/websocket.xml, accessed 2026-10-05, for the 1012 to 1014 divergence: all 3 registered through the HYBI mailing list, last updated there 2025-07-30, and refused by this server anyway. The wave 0 jdk.httpserver probe facts quoted from the plan ledger at .claude/plans/briefs/java-TASKS.md, measured 2026-10-04. React 19.3.0, vite 8.3.2, and the vite react plugin 6.1.1 pinned exact in java/capstone/web/package.json. The adversarial history, the 8.5 percent orphaned-room reproduction, the 12 of 12 ghost rounds, the overflow restructure, and the double-depart guard, quoted from the wave ledger and the regression tests named in the tree.

33. appendices: coverage and sources

The coverage matrices are the book's audit trail. Every reserved, contextual, and literal token enumerated from the se 27 spec, every stdlib component family the book touches mapped to the chapter that teaches it, and the official sources pinned with their access dates. The build enforces honesty from the other side: every chapter listing reads a real file the lanes compile and test, 158 junit tests in the api module plus 5 behind the docker gate, 131 in the capstone beside 21 node tests on the frontend, and 493 sample checks across 58 runnable classes, zero skipped anywhere.

33.1. keyword coverage

Chapter 3 owns the census and this table is its accounting: 51 reserved character sequences, the list frozen since 9 when the underscore joined, `enum` the last arrival at 5, `const` and `goto` reserved and unused since 1.0, `strictfp` an obsolete word 17 turned into a no-op. Beside them sit the 3 literal tokens, `true`, `false`, and `null`, reserved like keywords but classified as literals, and the 17 contextual keywords the spec's recognition rule keeps shadowable. Unmapped count: 2, the 2 recorded drops, `assert` and `transient`, the assertion budget riding the ok contract and junit instead of the statement, the io chapter walking the modern path instead of object serialization.

word	kind	since	taught in
abstract	reserved		ch 5
assert	reserved		unmapped
boolean	reserved		ch 4
break	reserved		ch 3
byte	reserved		ch 4
case	reserved		ch 9
catch	reserved		ch 14
char	reserved		ch 4
class	reserved		ch 5
const	reserved		ch 3
continue	reserved		ch 3
default	reserved		ch 9
do	reserved		ch 3
double	reserved		ch 4
else	reserved		ch 3
enum	reserved		ch 7
extends	reserved		ch 5
final	reserved		ch 5
finally	reserved		ch 25
float	reserved		ch 4
for	reserved		ch 3
goto	reserved		ch 3
if	reserved		ch 3
implements	reserved		ch 5
import	reserved		ch 15
instanceof	reserved		ch 9
int	reserved		ch 4
interface	reserved		ch 5
long	reserved		ch 4

word	kind	since	taught in
native	reserved		ch 10
new	reserved		ch 5
package	reserved		ch 15
private	reserved		ch 5
protected	reserved		ch 5
public	reserved		ch 5
return	reserved		ch 3
short	reserved		ch 4
static	reserved		ch 5
strictfp	reserved		ch 3
super	reserved		ch 5
switch	reserved		ch 9
synchronized	reserved		ch 10
this	reserved		ch 5
throw	reserved		ch 14
throws	reserved		ch 14
transient	reserved		unmapped
try	reserved		ch 14
void	reserved		ch 4
volatile	reserved		ch 10
while	reserved		ch 3
_	reserved		ch 9
true	literal		ch 3
false	literal		ch 3
null	literal		ch 3
module	contextual	9	ch 15
open	contextual	9	ch 15
requires	contextual	9	ch 15
transitive	contextual	9	ch 15
exports	contextual	9	ch 15
opens	contextual	9	ch 15
to	contextual	9	ch 15
uses	contextual	9	ch 15
provides	contextual	9	ch 15
with	contextual	9	ch 15
var	contextual	10	ch 4
yield	contextual	14	ch 3
record	contextual	16	ch 5
sealed	contextual	17	ch 4
permits	contextual	17	ch 4
non-sealed	contextual	17	ch 4
when	contextual	21	ch 9

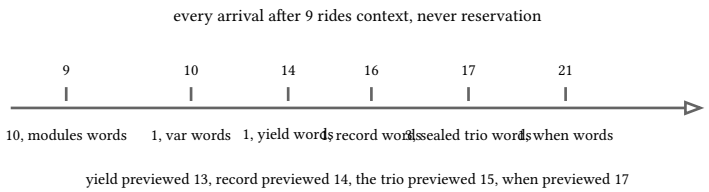


figure 67: contextual arrivals by release, the escape valve that kept the frozen list frozen

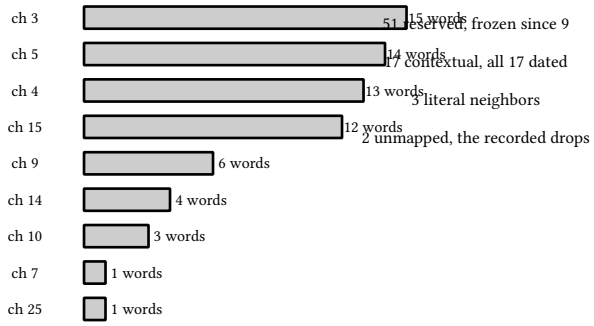


figure 68: the census by teaching chapter, counted from the table above

33.2. stdlib coverage

Component families against chapters, with the honest status column. The jdk's own sources in `tools/jdk27/build/jdk-27/lib/src.zip` answered the since tags, read the way every chapter read them.

component family	chapter	status
String, StringBuilder, text blocks	ch 13	covered
BigInteger and BigDecimal	ch 13	covered
java.time, Instant through zones	ch 13	covered
NumberFormat, compact numbers since 12	ch 13	covered, the non-jep enhancement
Regex	ch 13	mentioned only, not demonstrated
collections hierarchy, SequencedCollection since 21	ch 12	covered
the stream api	ch 12	covered
stream gatherers since 24	ch 12	covered
Arrays and Collections utilities	ch 12	covered
Optional	ch 8	covered, as a return contract
the function package, 43 interfaces	ch 8	covered
Math and the exact arithmetic family	ch 4	covered
boxed types and the Integer cache	ch 4	covered
io streams, try-with-resources	ch 14	covered
nio Path, Files	ch 14	covered
channels and buffers	ch 14	covered, the capstone rides them
tcp sockets, ServerSocketChannel	ch 14	covered
HttpClient, HTTP/3 since 26	ch 14	covered
java.net.http.WebSocket client	ch 14	covered, the fragmentation facts
jdk.httpserver	ch 19	covered, the spine's kernel, refused by the capstone with the wave 0 evidence
System.Logger	ch 20	covered
executors, CompletableFuture	ch 10	covered
monitors, volatile, happens-before	ch 10	covered
atomics, concurrent collections	ch 10	covered

component family	chapter	status
virtual threads, scoped values, structured concurrency preview	ch 10	covered, the 27 preview as snippets
ThreadLocal and its costs	ch 10	covered
gc, collectors, jcmd, jstat, jfr	ch 11	covered
compact object headers, default in 27	ch 11	covered
modules, jlink, jdeps, jar	ch 15	covered
reflection, proxies, method handles	ch 16	covered
annotations at runtime	ch 7	covered
MessageDigest, Mac, Cipher, KDF since 25	ch 17	covered
TLS, DRBG, keytool	ch 17	covered, the post-quantum default in 27
object serialization, ObjectInput/OutputStream	unmapped	not taught, the io chapter walks the modern path, transient unmapped in the keyword matrix for the same reason
json	ch 19	covered, hand-rolled under RFC 8259, the jdk ships no json api
junit console 6.1.3	ch 18	covered, vendored and sha-gated, the book's one external artifact

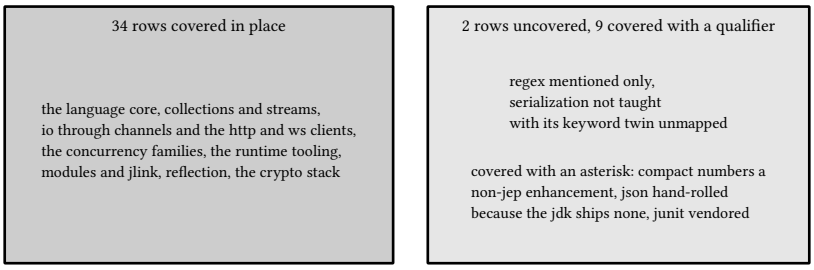


figure 69: stdlib coverage, covered in place against the honest exceptions

33.3. pinned sources

Every chapter ends with its own sources line naming what it consulted and how it verified live. The book level pins:

topic	url	accessed
java language spec, se 27, chapter 3 (lexical, keywords)	https://docs.oracle.com/javase/specs/jls/se27/html/jls-3.html	2026-10-04
openjdk release project pages, jdk 9 through jdk 27	https://openjdk.org/projects/jdk/	2026-10-04
oracle java se support roadmap, the lts cadence	https://www.oracle.com/java/technologies/java-se-support-roadmap.html	2026-10-04
java almanac, the withdrawn string templates lineage	https://javaalmanac.io	2026-10-04
JEP 444, virtual threads	https://openjdk.org/jeps/444	2026-10-04
JEP 491, synchronized no longer pins	https://openjdk.org/jeps/491	2026-10-04
JEP 533, structured concurrency, seventh preview	https://openjdk.org/jeps/533	2026-10-05
JEP 534, compact object headers	https://openjdk.org/jeps/534	2026-10-04
JEP 431, sequenced collections	https://openjdk.org/jeps/431	2026-10-04
JEP 485, stream gatherers	https://openjdk.org/jeps/485	2026-10-04

topic	url	accessed
JEP 378, text blocks	https://openjdk.org/jeps/378	2026-10-04
JEP 400, utf-8 by default	https://openjdk.org/jeps/400	2026-10-04
JEP 511, module import declarations	https://openjdk.org/jeps/511	2026-10-04
JEP 517, HTTP/3	https://openjdk.org/jeps/517	2026-10-04
JEP 510, key derivation function api	https://openjdk.org/jeps/510	2026-10-04
JEP 527, post-quantum key exchange default	https://openjdk.org/jeps/527	2026-10-04
JEP 500, prepare to make final mean final	https://openjdk.org/jeps/500	2026-10-04
JEP 456, unnamed variables and patterns	https://openjdk.org/jeps/456	2026-10-04
RFC 8259, the json grammar	https://www.rfc-editor.org/rfc/rfc8259	2026-10-04
RFC 9110, http semantics, etags and conditionals	https://www.rfc-editor.org/rfc/rfc9110	2026-10-05
RFC 7519, json web token	https://www.rfc-editor.org/rfc/rfc7519	2026-10-05
RFC 8018, pbkdf2	https://www.rfc-editor.org/rfc/rfc8018	2026-10-04
RFC 6585, the 429 and Retry-After	https://www.rfc-editor.org/rfc/rfc6585	2026-10-05
RFC 6455, the websocket protocol	https://www.rfc-editor.org/rfc/rfc6455	2026-10-04
RFC 4231, hmac test vectors	https://www.rfc-editor.org/rfc/rfc4231	2026-10-04
IANA websocket close code registry	https://www.iana.org/assignments/websocket/websocket.xml	2026-10-05
OWASP, IDOR and access control cheat sheets	https://cheatsheetseries.owasp.com	2026-10-05
prometheus exposition format	https://prometheus.io/docs/instrumenting/exposition_formats	2026-10-05
junit 6.1.3 console launcher user guide	https://docs.junit.org/6.1.3/user-guide	2026-10-04
junit console standalone on maven central	https://repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone	2026-08-07
openjdk jcstress	https://github.com/openjdk/jcstress	2026-10-04
docker docs, multi-stage builds and compose up --wait	https://docs.docker.com	2026-10-05
hub.docker.com registry api, temurin and ubuntu digests	https://hub.docker.com	2026-10-05
oracle javadoc, java se 27	https://docs.oracle.com/en/java/javase/27	2026-10-05
oracle jdk 27 ga archive, the pinned windows zip the installer sha256-gates	https://download.oracle.com/java/27/archive/jdk-27_windows-x64_bin.zip	2026-10-04
Java in a Nutshell, 8th edition (local ref/java pdf)	ref/java/Java-nutshell-8th-edition-ebook-Red-Hat-Developer.pdf	2026-10-04

NOTE: version drift rule

35 of these pins were fetched 2026-10-04 and 2026-10-05 against the ga jdk 27, build 27+35-2325, with the 19 release pages re-verified per release and the spec read at se 27. The one older date is the junit maven fetch, 2026-08-07, the day the jar was first vendored, and it stands because the sha gate would catch a moved artifact. If an upstream page changes, the delta gets recorded here rather than silently rewriting chapters, so this table is also the book's changelog for documentation movement.

NOTE: the pinned build

Every lane in this book compiles and runs on the oracle jdk 27 ga build 27+35-2325 installed sha256-gated into tools/jdk27/build by `sh tools/build-jdk27.sh`, because the machine's own java can be a release older than the classes, class file version 71 needs 27, the fact the capstone's e2e resolves first. Under the corpus version currency law the pin rides the newest installable ga build, nightlies excepted, and the next lts is dated september 2027 by the roadmap.

NOTE: the one external artifact

The spine's standing rule is stdlib-exclusive, no gradle, no maven, no spring, and the book makes exactly one vendoring decision: the junit console standalone 6.1.3, sha-gated in the vendor manifest and rehashed by the lane before anything runs. React 19.3.0, vite 8.3.2, and the vite react plugin 6.1.1 are pinned exact in the capstone frontend's package.json, outside the java lane's dependency surface.

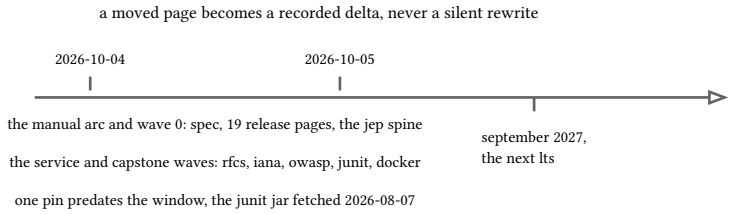


figure 70: the access window, when the pins were read and what moves next

sources: the three coverage files this chapter renders, `books/java/coverage/keywords.typ`, `stdlib.typ`, and `sources.typ`, enumerated 2026-10-05 from the chapters' own recorded sources paragraphs and the se 27 spec census chapter 3 measured live through `javax.lang.model.SourceVersion` on tools/jdk27/build/jdk-27, build 27+35-2325. The test counts quoted in the opening paragraph are the commit's own: 158 api junit tests plus 5 integration, 131 capstone junit tests, 21 node tests, 58 sample files at 493 checks, all green through `pwsh -NoProfile -File tools/verify-java.ps1` on 2026-10-05. The IANA registry row re-fetched 2026-10-05, its 1012 to 1014 entries last updated 2025-07-30 there.